

ShilpoHubBD

.NET / ASP.NET Core Backend — Complete Viva Preparation

124 Controllers · 138 Services · 437 Entities · 821 DTOs · 2623 C# files
Clean Architecture · .NET 8 · PostgreSQL (Supabase) · JWT Auth · EF Core 8

প্রস্তুতকারী: প্রকৃত source code বিশ্লেষণ করে, কোনো assumption ছাড়া।

ShilpoHubBD — .NET Backend Viva Preparation

তৈরি হয়েছে: তোমার নিজের ASP.NET Core backend source code পড়ে, বিশ্লেষণ করে। এখানে কোনো assumption বা fake feature নেই — যা কোড এ আছে শুধু তাই ব্যাখ্যা করা হয়েছে।

এই ডকুমেন্ট কীভাবে ব্যবহার করবে

ShilpoHubBD একটা বিশাল **multi-domain platform** — marketplace, e-learning academy, heritage tourism, government/NGO dashboard, logistics, business-partner/research সব একসাথে। Backend এ আছে:

- **124 Controllers**
- **138 Services**
- **437 Domain Entities**
- **821 DTOs**
- মোট **2623 C# files**

এত বড় codebase এর প্রতিটা লাইন মুখস্থ করা possible না, আর viva examiner সেটা আশাও করে না। যেটা examiner দেখতে চায় তা হলো — তুমি কি **প্যাটার্নটা বোঝো**? একটা Controller দেখলেই কি বলতে পারবে এটা কীভাবে কাজ করে, কেন এভাবে লেখা হয়েছে, ভেতরে কী হচ্ছে?

ভালো খবর হলো — এই পুরো প্রজেক্টের প্রতিটা **module একই architecture pattern follow করে** (Clean Architecture: Controller → Service → Repository → DbContext), একই conventions (async/await, DTO validation, custom exceptions, JWT auth) ব্যবহার করে। তাই একটা module এর pattern গভীরভাবে বুঝলে, বাকি 123 controller-ও একই যুক্তিতে ব্যাখ্যা করতে পারবে।

এই ডকুমেন্টের গঠন:

1. **Part 1 — Architecture Overview:** পুরো প্রজেক্টের ছবি, folder structure, request flow
2. **Part 2 — Core Building Blocks Deep Dive:** Program.cs, Middleware pipeline, Auth/JWT flow, Validation, Exception handling — লাইন বাই লাইন, আসল কোড দিয়ে
3. **Part 3-7 — Fundamentals:** C#, ASP.NET Core, Async/Await, EF Core, Auth/Authorization — theory + তোমার project থেকে real example
4. **Part 8 — Module-by-Module Deep Dive:** প্রতিটা business domain (Marketplace, Academy, Heritage, Government, Logistics, Business Partner/Research) এর জন্য representative controller deep-dive + reference table + question bank
5. **Part 9 — Master Question Bank:** সব মিলিয়ে বড় question bank (short answer + detailed explanation + project example + follow-up)
6. **Part 10 — Tricky & Scenario-Based Questions**
7. **Part 11 — Mock Viva চালানোর নিয়ম**
8. **Part 12 — Preparation Plan & Revision Checklist**

Important note: Item #11 (interactive "Mock Viva — one question at a time, wait for answer") একটা static PDF এ সম্ভব না, কারণ সেটা live conversation দরকার। Part 11-এ বলা আছে কীভাবে Claude এর সাথে চ্যাটে সেটা করবে।

সূচিপত্র (Table of Contents)

ShilpoHubBD — .NET Backend Viva Preparation

- এই ডকুমেন্ট কীভাবে ব্যবহার করবে

Part 1 — Project Architecture Overview

- 1.1 Tech Stack (project থেকে verified)
- 1.2 Clean Architecture — 5 Layers, 5 Projects
 - প্রতিটা layer এর দায়িত্ব
- 1.3 Scale of the Codebase (verified counts)
- 1.4 Request Flow — কীভাবে একটা HTTP Request প্রসেস হয়

Part 2 — Core Building Blocks: Line-by-Line Deep Dive

- 2.1 Program.cs — Application Entry Point (ShilpoHubBD.Api/Program.cs)
 - .env file loading (Development only)
 - Services registration
 - JWT Authentication setup
 - Rate Limiting
 - Middleware Pipeline Order (এটা examiner খুব পছন্দ করে জিজ্ঞেস করতে!)
 - SuperAdmin Seeding (idempotent)
- 2.2 Validation — ValidationFilter.cs
- 2.3 Global Exception Handling — GlobalExceptionHandler.cs
- 2.4 Dependency Injection & Service Lifetimes
- 2.5 Options Pattern — Strongly-Typed Configuration
- 2.6 Role-Based Access Control (RBAC) — RoleNames.cs + RolesController.cs

Part 3 — C# Fundamentals (তোমার project থেকে real example সহ)

- 3.1 OOP Pillars
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Abstraction
- 3.2 Class vs Object
- 3.3 Constructor & Dependency Injection Pattern
- 3.4 Interface vs Abstract Class
- 3.5 Static Members
- 3.6 Access Modifiers
- 3.7 Generics
- 3.8 Exception Handling
- 3.9 Collections & LINQ
- 3.10 IEnumerable vs IQueryable (খুব common viva question)
- 3.11 Lambda Expressions

Part 4 — ASP.NET Core Deep Dive

- 4.1 Web API vs MVC
- 4.2 Controller Structure
- 4.3 Model Binding Attributes
- 4.4 Middleware Pipeline (বিস্তারিত Part 2.1-এ দেওয়া আছে)
- 4.5 CORS (Cross-Origin Resource Sharing)
- 4.6 Swagger / OpenAPI
- 4.7 Health Checks
- 4.8 DTO vs Entity — কেন আলাদা রাখা হয়েছে
- 4.9 Repository + Service Layer Pattern (Clean/Layered Architecture)
- 4.10 SignalR (Realtime)
- 4.11 Rate Limiting Attribute Usage

Part 5 — Async/Await Deep Dive

- 5.1 কেন সব I/O method Async?
 - Synchronous হলে কী হতো
 - Async হলে
- 5.2 Task এবং Task<T>
- 5.3 async / await কীভাবে কাজ করে
- 5.4 Blocking vs Non-Blocking
- 5.5 Deadlock Risk এবং ConfigureAwait
- 5.6 Missing await — সাধারণ ভুল (viva trap!)
- 5.7 async void — কখনো ব্যবহার করা হয়নি (correctly)
- 5.8 নির্দিষ্ট EF Core Async মেথড
- 5.9 CancellationToken — প্রতিটা async method-এ কেন থাকে
- 5.10 Multithreading vs Async — একটা common ভুল ধারণা

Part 6 — Entity Framework Core & Database Deep Dive

- 6.1 DbContext এবং DbSet<T>

- 6.2 Code-First এবং Migrations
- 6.3 Relationships — Foreign Key & Navigation Properties
- 6.4 AsSplitQuery() — কেন দরকার (N+1 এর বিপরীত সমস্যা!)
- 6.5 N+1 Query Problem
- 6.6 Change Tracking vs AsNoTracking()
- 6.7 Transactions & Concurrency
- 6.8 Postgres-specific: EF.Functions.Like
- 6.9 Pagination Pattern
- 6.10 Normalization & Constraints (concept, project-grounded)
- Part 7 — Authentication & Authorization Deep Dive
- 7.1 Authentication vs Authorization (এই প্রজেক্টে concrete উদাহরণ)
- 7.2 কেন Custom JWT, ASP.NET Core Identity না?
- 7.3 JWT Structure ও Claims (JwtTokenService.GenerateAccessToken)
- 7.4 Refresh Token — কেন দরকার, কীভাবে কাজ করে
 - Refresh Flow (AuthService.RefreshTokenAsync)
- 7.5 Password Hashing
- 7.6 Password Reset Flow
- 7.7 Multi-Role & Switch Role
- 7.8 Middleware-এ Role Enforcement — Pipeline Recap
- 7.9 SignalR + JWT (Special Case)
- Part 8 — Module-by-Module Deep Dive
- ShilpoHubBD Viva Prep — Cluster: Marketplace & Commerce
 - 1. Cluster Overview
 - 2. Deep Dive — 4 Representative Controllers
 - 2.1 ProductsController + ProductService + ProductRepository
 - 2.2 OrdersController + OrderService (Checkout flow)
 - 2.3 CartController + CartService
 - 2.4 AuctionsController + AuctionService (Bidding logic)
 - 3. Reference Table — Remaining Controllers
 - 4. Viva Question Bank
 - 5. Tricky / Gotcha Questions
- Academy & Careers Cluster — Viva Prep
 - 1. Cluster Overview
 - 2. Deep Dive — 4 Representative Controllers
 - 2.1 CoursesController + EnrollmentsController — Course Authoring & Progress Flow
 - 2.2 ExamsController (vs QuizzesController) — Grading Engine
 - 2.3 MentorMatchingController + JobMatchingController — Rule-Based Weighted Scoring
 - 2.4 MessagingController + MessagingHub — Real-time Chat (SignalR)
 - 3. Reference Table — Remaining Controllers
 - 4. Viva Question Bank
 - 5. Tricky / Gotcha Questions
- Heritage & Tourism Cluster — Viva Prep
 - 1. Cluster Overview
 - 2. Deep Dive — 4 Representative Controllers
 - 2.1 PassportController (api/passport) — Gamification Engine
 - 2.2 QRVerificationController (api/qr-verification) — Anti-Counterfeit Verification
 - 2.3 HeritagePlacesController (api/heritage-places) — Discovery CRUD + Geo Query
 - 2.4 BookingsController (api/bookings) — Tourist Service Reservation Lifecycle
 - 3. Reference Table — Remaining Controllers
 - 4. Viva Question Bank
 - 5. Tricky / Gotcha Questions
- Government & NGO Cluster — Viva Prep
 - 1. Cluster Overview
 - 2. Deep Dive — 4 Representative Controllers
 - 2.1 PolicySimulatorController — most viva-worthy
 - 2.2 GovForecastsController — forecasting, honest check
 - 2.3 MonitoringController — rule-based fraud/anomaly detection
 - 2.4 HeritageIntelligenceController — the composite-index engine
 - 3. Reference Table — Remaining Controllers
 - 4. Viva Question Bank
 - 5. Tricky / Gotcha Questions
- ShilpoHubBD — Viva Prep: Logistics & Supply Chain Cluster
 - 1. Cluster Overview
 - 2. Deep Dive — 4 Representative Controllers
 - 2.1 ShipmentsController — Delivery Tracking
 - 2.2 WarehouseStockController — Warehouse Stock & Movement Ledger
 - 2.3 ReturnsController — Return Handling (Inspection → Disposition → Restock/Refund)

- 2.4 AiLogisticsController — "AI" Logistics (Rule-Based, Not ML)
- 3. Reference Table — Remaining Controllers
- 4. Viva Question Bank (20 Q&A)
- 5. Tricky/Gotcha Questions (6)
- Cluster: Business Partner & Research/Innovation — Viva Prep
 - 1. Cluster Overview
 - 2. Deep Dive — 4 Representative Controllers
 - 2.1 AllIntelligenceController — "AI" that is really weighted arithmetic
 - 2.2 ResearchAiAssistantController — persisted, project-scoped "AI" analyses
 - 2.3 SupplierMatchingController — weighted-criteria matching engine
 - 2.4 ContractsController — full contract lifecycle + renewal chain
 - 3. Reference Table — Remaining Controllers
 - 4. Viva Question Bank
 - 5. Tricky/Gotcha Questions

Part 9 — Master Question Bank (Core/Cross-Cutting)

- Q1. তোমার প্রজেক্ট কী ধরনের architecture ব্যবহার করে, আর কেন?
- Q2. Controller এত "thin" কেন? সব logic Service-এ কেন রাখা হয়েছে?
- Q3. তুমি কেন AutoMapper ব্যবহার করোনি, ম্যানুয়াল mapping কেন?
- Q4. IEnumerable, ICollection, IQueryable, List<T> — পার্থক্য কী?
- Q5. [ApiController] attribute কী কী automatic behavior দেয়?
- Q6. Middleware pipeline-এ order কেন গুরুত্বপূর্ণ? একটা উদাহরণ দাও যেখানে order ভুল হলে bug হবে।
- Q7. Dependency Injection কী, আর Scoped/Transient/Singleton এর মধ্যে পার্থক্য কী?
- Q8. FluentValidation কীভাবে কাজ করে, ModelState.IsValid-এর চেয়ে আলাদা কীভাবে?
- Q9. Custom Exception (NotFoundException, ConflictException) কেন বানানো হয়েছে, generic Exception কেন যথেষ্ট না?
- Q10. Rate Limiting কীভাবে কাজ করে, কেন login endpoint-এ আলাদা কড়া limit?
- Q11. Task আর Task<T> এর মধ্যে পার্থক্য? Task.Result কেন এড়ানো উচিত?
- Q12. Guid কেন Primary Key হিসেবে ব্যবহার হয়েছে, auto-increment int না কেন?
- Q13. appsettings.json-এ connection string ফাঁকা কেন, .env কীভাবে কাজ করে?
- Q14. PagedResult<T> জেনেরিক ক্লাস কীভাবে কাজ করে?
- Q15. তোমার প্রজেক্টে "AI" নাম দেওয়া features (যেমন AiBusinessAssistant, ResearchAiAssistant, AiLogistics) কি সত্যিকারের Machine Learning?

Part 10 — Mock Viva কীভাবে চালাবে

Part 11 — Preparation Plan (কী আগে পড়বে, কীভাবে revise করবে)

- ধাপ ১ (Day 1-2) — Big Picture আগে বোঝা
- ধাপ ২ (Day 2-4) — Core Building Blocks (সবচেয়ে গুরুত্বপূর্ণ অংশ)
- ধাপ ৩ (Day 4-6) — একটা end-to-end flow সম্পূর্ণ মুখস্থ করো
- ধাপ ৪ (Day 6-9) — Fundamentals (C#, Async, EF Core, Auth)
- ধাপ ৫ (Day 9-12) — নিজের Module + ২-৩টা অন্য Module
- ধাপ ৬ (Day 12-14) — Question Bank + Mock Viva
- Revision Checklist (viva-র ঠিক আগে, দ্রুত দেখে নেওয়ার জন্য)
- শেষ পরামর্শ

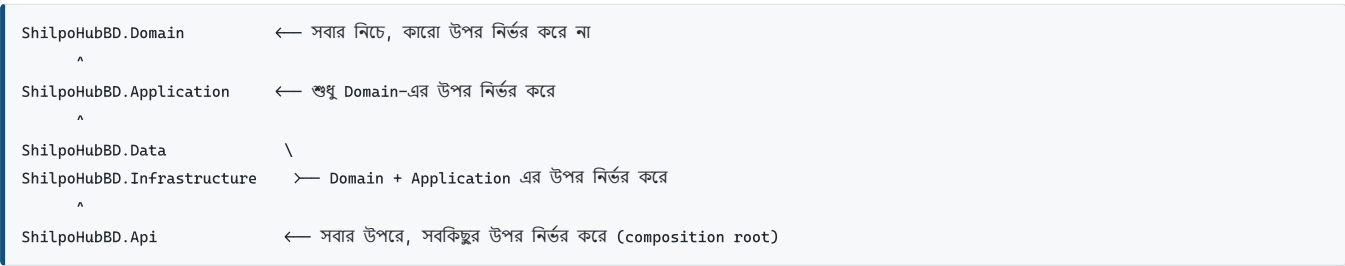
Part 1 — Project Architecture Overview

1.1 Tech Stack (project থেকে verified)

জিনিস	কী ব্যবহার হয়েছে	কোথায় দেখবে
Framework	.NET 8 (ASP.NET Core Web API)	সব .csproj এ Microsoft.NET.Sdk.Web / Microsoft.NET.Sdk
Database	PostgreSQL (Supabase-hosted)	Npgsql.EntityFrameworkCore.PostgreSQL package, ConnectionStrings:DefaultConnection
ORM	Entity Framework Core 8	ShilpoHubDbContext, Code-First Migrations
Auth	JWT Bearer (custom, hand-rolled — no ASP.NET Identity)	Microsoft.AspNetCore.Authentication.JwtBearer, JwtTokenService
Validation	FluentValidation	AbstractValidator<T> classes + custom ValidationFilter
Realtime	SignalR	MessagingHub, LiveEventHub, LiveClassHub
API docs	Swashbuckle (Swagger/OpenAPI)	Dev-only, app.UseSwagger()
Env config	DotNetEnv	loads a local .env file in Development
Object mapping	Manual (private ToDto() / ToListItemDto() helper methods inside each Service)	NOT AutoMapper — see the gotcha box below

⚠ **Viva gotcha:** ShilpoHubBD.Application/Mappings/README.md বলে "AutoMapper profiles and object mapping configuration" — কিন্তু এটা একটা পুরনো/stale scaffold ফাইল মাত্র। প্রজেক্টের কোনো .csproj -এ AutoMapper package reference নেই, আর কোনো Service এ IMapper / Profile class ব্যবহার হয়নি। সব mapping **manual** — প্রতিটা Service এর ভেতরে private static/instance method যেমন ToDto(Product product) । Examiner যদি জিজ্ঞেস করে "তুমি কোন mapping library ব্যবহার করেছ?", honest উত্তর: "কোনো library না, manual mapping — কারণ এতে explicit control থাকে, reflection overhead থাকে না, আর mapping logic এর ভেতরে business rule (যেমন ViewCount++) সহজে বসানো যায়।"

1.2 Clean Architecture — 5 Layers, 5 Projects



এটা .csproj ফাইলগুলোর <ProjectReference> দেখলেই প্রমাণ হয়:

- ShilpoHubBD.Domain.csproj — কোনো ProjectReference নেই। এটাই **Dependency Inversion Principle**-এর কেন্দ্র: সবচেয়ে গুরুত্বপূর্ণ business concept (Entities) কোনো framework/database/web layer-এর উপর নির্ভর করে না।
- ShilpoHubBD.Application.csproj — শুধু Domain reference করে, plus FluentValidation NuGet package।
- ShilpoHubBD.Data.csproj — Domain + Application reference করে, plus Npgsql.EntityFrameworkCore.PostgreSQL । এখানে EF Core-এর সবকিছু থাকে — কারণ database access-কে "detail" হিসেবে বাইরে রাখা হয় Clean Architecture-এ।
- ShilpoHubBD.Api.csproj — সব project reference করে; এটাই composition root, Program.cs -এ সব DI wiring হয়।

প্রতিটা layer এর দায়িত্ব

Layer	কী থাকে	Real example
Api	Controllers, Middlewares, SignalR Hubs, Program.cs (startup wiring)	ProductsController, GlobalExceptionHandler, ValidationFilter
Application	Services (business logic), DTOs, Interfaces (IxxxService , IxxxRepository), FluentValidation Validators, custom Exceptions	ProductService, IProductRepository, CreateProductRequestValidator
Domain	Entities (plain C# classes representing DB tables), Constants	Product, Order, RoleNames
Data	ShilpoHubDbContext, Repository implementations, EF IEntityTypeConfiguration, Migrations	ProductRepository, ShilpoHubDbContext, Migration files
Infrastructure	External/cross-cutting concerns: JWT signing, email sending, payment gateway, "AI" feature logic	JwtTokenService, Infrastructure/Email, Infrastructure/Payments

Interfaces থাকে **Application layer** এ, **implementation** থাকে **Data/Infrastructure** এ — এটাই Dependency Inversion বাস্তবে। যেমন IProductRepository interface টা আছে ShilpoHubBD.Application/Interfaces/Repositories/ -এ, কিন্তু আসল ক্লাস ProductRepository আছে ShilpoHubBD.Data/Repositories/ -এ। Service সবসময় interface-এর সাথে কথা বলে, concrete class-এর সাথে না — এতে Service layer টেস্ট করা সহজ হয় (mock repository বসানো যায়) আর database

technology বদলালেও (Postgres থেকে অন্য কিছু) Service layer পাল্টাতে হয় না।

1.3 Scale of the Codebase (verified counts)

জিনিস	সংখ্যা
Controllers	124
Services	138
Domain Entities	437 (in 46 domain-grouped namespace folders)
DTOs	821
EF Core Migrations	64
Total <code>.cs</code> files	2623

এত বড় হওয়ার কারণ — এটা একটা মাল্টি-ডোমেইন প্র্যাটফর্ম, একটা সিম্পল CRUD app না। মূল domain গুলো:

- Marketplace/Commerce** — Products, Cart, Orders, Payments, Reviews, Auctions, Live-shopping ইত্যাদি
- Academy/Careers** — Courses, Exams, Mentorship, Apprenticeship, Job matching
- Heritage & Tourism** — Heritage places/routes/festivals, Village tourism, Digital passport (gamification), QR verification, AR
- Government & NGO** — Policy simulation, funding, compliance monitoring, heritage-risk dashboards
- Logistics** — Pickup, delivery routes, warehouse/inventory, returns, shipment tracking
- Business Partner / Research & Innovation** — B2B procurement, supplier matching, research project management, innovation prototypes

প্রতিটা domain-এর নিজস্ব role আছে (`RoleNames.cs` দেখো — `Customer`, `Producer`, `BusinessPartner`, `Tourist`, `HeritageAcademyMember`, `HeritageInnovationHub`, `GovernmentNGO`, `LogisticsPartner`, `SuperAdmin`), আর একজন user একসাথে একাধিক role হোল্ড করতে পারে এবং `switch-role` করে সক্রিয় role বদলাতে পারে (details Part 7-এ)।

1.4 Request Flow — কীভাবে একটা HTTP Request প্রসেস হয়



এই flow টা প্রতিটা controller/endpoint-এ একইরকম — তাই একটা endpoint ভালোভাবে বুঝলে বাকি ১২৩টাও একই ভাষায় ব্যাখ্যা করা যায়।

Part 2 — Core Building Blocks: Line-by-Line Deep Dive

এই অংশে project-এর সবচেয়ে গুরুত্বপূর্ণ "shared" ফাইলগুলো — যেগুলো পুরো backend-কে একসাথে ধরে রাখে — একদম লাইন বাই লাইন ব্যাখ্যা করা হলো। এগুলো examiner-এর সবচেয়ে প্রিয় জায়গা, কারণ এখানে বোঝা যায় তুমি framework-টা আসলেই বোঝো কিনা।

2.1 Program.cs — Application Entry Point (ShilpoHubBD.Api/Program.cs)

.NET 8-এ **Minimal Hosting Model** ব্যবহার হয়েছে — কোনো **Startup.cs** নেই, সবকিছু **Program.cs** -এ top-level statements দিয়ে লেখা।

```
var builder = WebApplication.CreateBuilder(args);
```

এটা একটা **WebApplicationBuilder** তৈরি করে — এর ভেতরে থাকে **Services** (DI container), **Configuration** (appsettings.json + env vars), **Environment** (Development/Production), **Logging** ।

.env file loading (Development only)

```
if (builder.Environment.IsDevelopment())
{
    var envCandidates = new[] { ... };
    var envFile = envCandidates.FirstOrDefault(File.Exists);
    if (envFile is not null)
    {
        DotNetEnv.Env.Load(envFile);
        builder.Configuration.AddEnvironmentVariables();
    }
}
```

কেন: Connection string, JWT secret ইত্যাদি sensitive value কখনো **appsettings.json** -এ commit করা ঠিক না (git এ চলে যাবে)। তাই local dev-এ একটা **.env** ফাইল থেকে environment variable হিসেবে load করা হয় (**ConnectionStrings__DefaultConnection** — ডবল আন্ডারস্কোর — .NET configuration-এ nested key represent করে, **ConnectionStrings:DefaultConnection** -এর সমতুল্য)। Production-এ (Azure/Render/Docker) এগুলো সরাসরি platform-এর environment variable হিসেবে সেট করা হবে, **.env** লাগবে না — এজন্যই **IsDevelopment()** চেক।

Services registration

```
builder.Services.AddScoped<ValidationFilter>();
builder.Services.AddControllers(options => options.Filters.Add<ValidationFilter>())
    .AddJsonOptions(options => options.JsonSerializerOptions.Converters.Add(
        new System.Text.Json.Serialization.JsonStringEnumConverter()));
```

- AddControllers** MVC controller support যোগ করে, আর **global action filter** হিসেবে **ValidationFilter** লাগিয়ে দেয় — মানে প্রতিটা controller action-এর জন্য আলাদা করে filter লাগানো লাগবে না, এক জায়গায় লেখা, সব জায়গায় কাজ করে (DRY)।
- JsonStringEnumConverter** — C# enum গুলো (যেমন **OrderStatus.Pending**) default এ JSON এ integer (**0**, **1**, **2**) হিসেবে serialize হয়। এই converter দিয়ে সেগুলো string ("Pending") হিসেবে serialize/deserialize হবে — frontend-এর জন্য readable, আর enum-এ নতুন value ঢুকালে numeric ordering ভেঙে যাওয়ার ঝুঁকি থাকে না।

```
builder.Services.AddApplication(builder.Configuration);
builder.Services.AddData(builder.Configuration);
builder.Services.AddInfrastructure(builder.Configuration);
```

প্রতিটা layer-এর নিজস্ব **DependencyInjection.cs** (extension method: **AddApplication**, **AddData**, **AddInfrastructure**) আছে যেটা সেই layer-এর সব service/repository DI container-এ register করে। এভাবে **Program.cs** পরিষ্কার থাকে, আর প্রতিটা layer নিজেই DI registration নিজে manage করে (Separation of Concerns)।

JWT Authentication setup

```
var jwtKey = builder.Configuration["Jwt:Key"];
if (string.IsNullOrEmpty(jwtKey))
    throw new InvalidOperationException("JWT signing key is not configured ...");

builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.MapInboundClaims = false;
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true, ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidateAudience = true, ValidAudience = builder.Configuration["Jwt:Audience"],
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)),
            ClockSkew = TimeSpan.Zero,
        };
        options.Events = new JwtBearerEvents { OnMessageReceived = context => { ... } };
    });
```

- Missing config হলে **fail-fast** — app শুরুই হবে না একটা clear exception দিয়ে, বরং later একটা confusing 500 error দেওয়ার চেয়ে ভালো।
- MapInboundClaims = false** — by default, .NET কিছু standard JWT claim name (যেমন **sub**) কে .NET-এর নিজস্ব longer claim URI (`http://schemas.xmlsoap.org/ws/2005/05/identity/claims/...`) তে rename করে দেয়। এটা বন্ধ করলে JWT-তে যা লেখা আছে (raw claim name যেমন **sub**, **role**) ঠিক তই পড়া যায়।
- ValidateIssuer / ValidateAudience / ValidateLifetime / ValidateIssuerSigningKey** — সবগুলো **true** মানে টোকেন সম্পূর্ণভাবে verify হচ্ছে: এটা সঠিক issuer থেকে এসেছে, সঠিক audience-এর জন্য, মেয়াদ শেষ হয়নি, আর signature সঠিক key দিয়ে sign করা।
- ClockSkew = TimeSpan.Zero** — default এ .NET expiry check-এ ৫ মিনিট extra grace period দেয় (clock এর সামান্য পার্থক্য সামালানোর জন্য)। এখানে সেটা 0 করা মানে token ঠিক তার **exp** সময়েই expire হবে, কোনো grace period নেই — access token যেহেতু মাত্র ১৫ মিনিটের (**Jwt:AccessTokenMinutes**), strict expiry চাওয়া হয়েছে।
- OnMessageReceived** event — **SignalR-এর জন্য special কেস**। ব্রাউজার WebSocket handshake-এ custom **Authorization** header বসাতে পারে না, তাই SignalR client access token পাঠায় query string-এ (`?access_token=...`)। এই event হ্যান্ডলার চেক করে request path **/hubs** দিয়ে শুরু কিনা, হলে query string থেকে token নিয়ে normal bearer token হিসেবে treat করে।

Rate Limiting

```
static string RatePartitionKey(HttpContext ctx) =>
    userId != null ? $"user:{userId}" : $"ip:{ctx.Connection.RemoteIpAddress}";

builder.Services.AddRateLimiter(options =>
{
    options.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(ctx => ...
        PermitLimit = authenticated ? 300 : 75, Window = 1 minute ...);
    options.AddPolicy("read", ctx => ... PermitLimit = authenticated ? 200 : 40 ...);
    options.AddPolicy("auth", ctx => RateLimitPartition.GetFixedWindowLimiter(RateIpKey(ctx), ... PermitLimit = 10 ...));
});
```

Microsoft.AspNetCore.RateLimiting -এর **Fixed Window Limiter** ব্যবহার হয়েছে। তিন লেভেলের rate limiting:

- Global limiter** — সব endpoint-এ apply হয় (authenticated: প্রতি মিনিটে ৩০০ request, anonymous: ৭৫)।
- "read" policy** — catalog/browse endpoint-এ [**EnableRateLimiting("read")**] লাগিয়ে আরও tight limit (anonymous: ৪০/মিনিট) — anonymous scraper পুরো catalog টেনে নিতে না পারে তাই।
- "auth" policy** — login/register-এ সবসময় IP দিয়ে key (user logged-in না থাকায়), মাত্র ১০/মিনিট — brute-force/credential-stuffing ঠেকাতে।

Partition key authenticated user হলে **user:{id}**, না হলে **ip:{address}** — এতে একজন scraper পুরো সিস্টেম আটকাতে পারে না, আবার একজন legitimate user অন্য user-দের প্রভাবিত করে না।

Middleware Pipeline Order (এটা examiner খুব পছন্দ করে জিজ্ঞেস করতে!)

```
app.UseExceptionHandler(_ => { });
if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(); }
app.UseHttpsRedirection();
app.UseCors("Frontend");
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();
app.MapControllers();
```

Order matters — middleware একটা pipeline, request উপর থেকে নিচে যায়, response নিচ থেকে উপরে ফেরত আসে:

1. **UseExceptionHandler** সবার আগে — যাতে পরের যেকোনো middleware/controller-এ exception হলেও ধরতে পারে।
2. **UseHttpsRedirection** — HTTP কে HTTPS-এ redirect।
3. **UseCors** — Authentication/Authorization-এর **আগে** হওয়া উচিত যাতে preflight (**OPTIONS**) request blocked না হয়।
4. **UseAuthentication** **আগে**, **UseAuthorization** তারপর — কারণ authorization এর জন্য জানতে হবে "কে" (authentication দিয়ে **HttpContext.User** populate হয়), তারপর "এই কে-এর permission আছে কিনা" (authorization)। এই দুইটার ক্রম উল্টালে সব request 401/403 পাবে।
5. **UseRateLimiter** authentication-এর **পরে** — কারণ partition key নির্ধারণ করতে user id লাগে (**ctx.User.FindFirst("sub")**), যেটা authentication-এর পরেই পাওয়া যায়।
6. **MapControllers()** — সবশেষে, actual endpoint routing।

SuperAdmin Seeding (idempotent)

```
if (!string.IsNullOrEmpty(seedEmail) && !string.IsNullOrEmpty(seedPassword))
{
    using var scope = app.Services.CreateScope();
    var userRepository = scope.ServiceProvider.GetRequiredService<IUserRepository>();
    if (!await userRepository.AnyInRoleAsync(RoleNames.SuperAdmin, CancellationToken.None))
    {
        var authService = scope.ServiceProvider.GetRequiredService<IAuthService>();
        await authService.RegisterAsync(new RegisterRequest { ... Roles = [SuperAdmin] }, null, CancellationToken.None);
    }
}
```

- **app.Services.CreateScope()** দরকার কারণ **IUserRepository / IAuthService Scoped** lifetime (নিচে ব্যাখ্যা আছে), কিন্তু **app.Services** root container (Singleton scope) — root container থেকে সরাসরি scoped service resolve করা যায় না, তাই manual scope বানাতে হয়।
- **Idempotent** — প্রতিবার app start হওয়ার সময় চেক করে যে already কোনো SuperAdmin আছে কিনা (**AnyInRoleAsync**); থাকলে আবার বানায় না। ফলে বারবার app restart করলেও duplicate SuperAdmin তৈরি হয় না।
- শুধু **.env** -এ **Seed:SuperAdminEmail / Password** সেট থাকলেই চলে — migration-এর ভেতরে seed data hardcode করা হয়নি, কারণ password migration ফাইলে থাকলে সেটা git history-তে চলে যেত।

2.2 Validation — **ValidationFilter.cs**

```
public class ValidationFilter : IAsyncActionFilter
{
    public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate next)
    {
        foreach (var argument in context.ActionArguments.Values)
        {
            if (argument is null) continue;
            var validatorType = typeof(IValidator<>).MakeGenericType(argument.GetType());
            if (_serviceProvider.GetService(validatorType) is IValidator validator)
            {
                var result = await validator.ValidateAsync(new ValidationContext<object>(argument), ...);
                if (!result.IsValid) throw new ValidationException(result.Errors);
            }
        }
        await next();
    }
}
```

এটা একটা **IAsyncActionFilter** — MVC action filter pipeline-এর অংশ, controller action চলার ঠিক আগে/পরে code চালানোর জন্য (Aspect-Oriented cross-cutting concern)।

লাইন বাই লাইন:

- `context.ActionArguments` — dictionary of parameter name → value, যেগুলো model-binding দিয়ে ইতিমধ্যে populate করা হয়েছে (JSON body থেকে deserialize)।
- `typeof(IValidator<>).MakeGenericType(argument.GetType())` — reflection দিয়ে generic type বানানো। যদি argument-টা `CreateProductRequest` টাইপের হয়, তাহলে এটা রানটাইমে `IValidator<CreateProductRequest>` টাইপ বানায়।
- `_serviceProvider.GetService(validatorType)` — DI container-এ ওই টাইপের validator registered আছে কিনা চেক করে। গুরুত্বপূর্ণ: এটা `GetService` (nullable রিটার্ন করে), `GetRequiredService` না — কারণ সব DTO-এর জন্য validator নাও থাকতে পারে (optional validation)।
- Validator পেলে `ValidateAsync` কল করে; fail হলে `FluentValidation.ValidationException` throw করে — এটা `GlobalExceptionHandler`-এ ধরা পড়ে HTTP 400 এ রূপান্তরিত হয়।

কেন Controller-এ ম্যানুয়ালি `ModelState.IsValid` চেক করা হয়নি? কারণ এভাবে একটা centralized filter দিয়ে সব controller-এ automatically validation চলে আসে — 124টা controller-এর প্রতিটিতে `if (!ModelState.IsValid) return BadRequest(...)` লেখা লাগে না! DRY principle।

`AddValidatorsFromAssembly` (`Application/DependencyInjection.cs` -এ) reflection দিয়ে পুরো assembly স্ক্যান করে সব `AbstractValidator<T>` ক্লাস খুঁজে বের করে এবং DI-তে register করে — তাই নতুন validator বানালে আলাদা করে `services.AddScoped<IValidator<X>, XValidator>()` লেখা লাগে না।

2.3 Global Exception Handling — `GlobalExceptionHandler.cs`

```
public class GlobalExceptionHandler : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext, Exception exception, CancellationToken ct)
    {
        var (statusCode, title, errors) = exception switch
        {
            FluentValidation.ValidationException v => (400, "Validation failed.", v.Errors.GroupBy(...) ...),
            ConflictException c => (409, c.Message, null),
            NotFoundException n => (404, n.Message, null),
            UnauthorizedAccessException u => (401, u.Message, null),
            _ => (500, "An unexpected error occurred.", null),
        };

        if (statusCode == 500) _logger.LogError(exception, "Unhandled exception ...");
        var problemDetails = new ProblemDetails { Status = statusCode, Title = title, Instance = httpContext.Request.Path };
        if (errors is not null) problemDetails.Extensions["errors"] = errors;
        httpContext.Response.StatusCode = statusCode;
        await httpContext.Response.WriteAsJsonAsync(problemDetails, ct);
        return true;
    }
}
```

- .NET 8-এর নতুন `IExceptionHandler` interface implement করা — এটা `app.UseExceptionHandler()` middleware-এর সাথে wired (`Program.cs` -এ `builder.Services.AddExceptionHandler<GlobalExceptionHandler>()`)।
- **Switch expression (pattern matching)** দিয়ে exception-এর টাইপ অনুযায়ী HTTP status code map করা — এটাই কেন Service layer শুধু plain exception throw করে, নিজে HTTP status নিয়ে ভাবে না (Separation of Concerns — business logic HTTP layer থেকে independent থাকে)।
- **RFC 7807 ProblemDetails** ফরম্যাট — এটা standard error response shape (`type`, `title`, `status`, `detail`, `instance`), REST API best practice — frontend সবসময় একই shape-এ error handle করতে পারে, controller ভেদে বদলায় না।
- শুধু 500 (unexpected) এর ক্ষেত্রে `_logger.LogError` — কারণ 400/404/409/401 এগুলো "expected" business errors (ব্যবহারকারীর ভুল input বা normal flow), server-এর bug না, তাই এগুলো error log-এ noise করবে না। কিন্তু 500 মানে সত্যিই কিছু ভেঙে গেছে যেটা developer-কে জানতে হবে।
- `return true` — মানে "আমি এই exception handle করে ফেলেছি, আর কাউকে propagate করার দরকার নেই।"

2.4 Dependency Injection & Service Lifetimes

প্রজেক্টে সব custom service/repository `AddScoped` দিয়ে register করা (`Application/DependencyInjection.cs` -এ ১২৭টা `AddScoped` কল, `Data/DependencyInjection.cs` -এও একই প্যাটার্ন)।

তিনটা lifetime .NET DI তে:

- **Transient** — প্রতিবার inject করার সময় নতুন instance।
- **Scoped** — একটা HTTP request-এর ভেতরে একই instance (নতুন request = নতুন instance)।
- **Singleton** — পুরো app-এর জীবনকালে একটাই instance।

কেন সবকিছু Scoped? কারণ `ShilpoHubDbContext` -ও Scoped (`AddDbContext` by default Scoped lifetime রেজিস্টার করে)। একটা request-এর ভেতরে যদি Service একবার `_context.Products.Add(...)` করে আর পরে `SaveChangesAsync()` কল করে, তাহলে পুরো request জুড়ে একই `DbContext` instance দরকার — নাহলে EF Core-এর change tracking কাজ করবে না। যেহেতু Service→Repository→DbContext একটা chain, পুরো chain-টাকেই Scoped রাখা লাগে; Singleton হলে DbContext (যেটা thread-safe না) সব request-এর মধ্যে shared হয়ে যেত — race condition আর data corruption হতো।

`Program.cs` -এ কয়েকটা Singleton-ধরনের জিনিসও আছে indirectly — `AddSignalR()`, `AddRateLimiter()` এর internal state, কিন্তু business logic service কখনোই Singleton না।

2.5 Options Pattern — Strongly-Typed Configuration

`appsettings.json` -এ:

```
"Jwt": { "Issuer": "", "Audience": "", "Key": "", "AccessTokenMinutes": 15, "RefreshTokenDays": 7 },
"HeritageScore": { "VerifiedBonusPoints": 50, "PointsPerFamilyMember": 5, ... },
"SustainabilityScore": { "PointsPerRecycledMaterial": 5, ..., "BronzeBadgeThreshold": 30, ... }
```

এগুলো raw string দিয়ে (`builder.Configuration["Jwt:Key"]`) কোথাও কোথাও পড়া হয়, আবার কোথাও **strongly-typed Options pattern** দিয়ে — যেমন `JwtTokenService`:

```
public class JwtTokenService : ITokenService
{
    private readonly JwtSettings _settings;
    public JwtTokenService(IOptions<JwtSettings> options) { _settings = options.Value; }
    ...
    public TimeSpan RefreshTokenLifetime => TimeSpan.FromDays(_settings.RefreshTokenDays);
}
```

`IOptions<JwtSettings>` DI দিয়ে inject করা হয় — `JwtSettings` একটা plain C# class যার property গুলো `appsettings.json` -এর "Jwt" section-এর সাথে auto-bind হয় (`Microsoft.Extensions.Options.ConfigurationExtensions` package + `services.Configure<JwtSettings>(configuration.GetSection("Jwt"))`) — Infrastructure-এর DI setup-এ। **সুবিধা:** compile-time type safety (magic string `"Jwt:AccessTokenMinutes"` বারবার লিখতে হয় না, typo হলে compiler ধরবে), আর একটা জায়গায় (constructor) সব config value পাওয়া যায়।

`HeritageScore` / `SustainabilityScore` sections দেখায় যে গেমিফিকেশন/স্কেরিং সিস্টেমগুলো hardcoded magic number না — এগুলো configurable, deployment না বদলিয়ে (শুধু config দিয়ে) স্কেরিং rule টিউন করা যায়।

2.6 Role-Based Access Control (RBAC) — `RoleNames.cs` + `RolesController.cs`

```
public static class RoleNames
{
    public const string Customer = "Customer";
    public const string Producer = "Producer";
    public const string BusinessPartner = "BusinessPartner";
    public const string Tourist = "Tourist";
    public const string HeritageAcademyMember = "HeritageAcademyMember";
    public const string HeritageInnovationHub = "HeritageInnovationHub";
    public const string GovernmentNGO = "GovernmentNGO";
    public const string LogisticsPartner = "LogisticsPartner";
    public const string SuperAdmin = "SuperAdmin";

    public static readonly IReadOnlyList<string> All = new[] { ... };
    public static readonly IReadOnlyList<string> SelfRegisterableRoles = new[] { ... }; // SuperAdmin বাদে সব
}
```

Multi-role user — `User` entity-র সাথে `UserRole` (many-to-many join entity) দিয়ে সম্পর্ক, তাই একজন user একসাথে `Producer` আর `Tourist` দুটোই হতে পারে। JWT-তে সব role claim হিসেবে বসানো হয় (`ClaimTypes.Role` — multiple বার), plus একটা আলাদা `active_role` claim — যেটা বলে দেয় এই মুহূর্তে user কোন role হিসেবে কাজ করছে (frontend সেটা দিয়ে UI ঠিক করে)। `AuthController.SwitchRole` দিয়ে active role বদলানো যায় (নতুন access+refresh token issue হয়)।

`RolesController`:

```
[Route("api/roles")]
[Authorize(Roles = RoleNames.SuperAdmin)]
public class RolesController : ControllerBase
{
    [HttpPost("assign")] public async Task<IActionResult> AssignRole(AssignRoleRequest request, ...)
    [HttpPost("remove")] public async Task<IActionResult> RemoveRole(RemoveRoleRequest request, ...)
}
```

Class-level `[Authorize(Roles = RoleNames.SuperAdmin)]` — পুরো controller-এর সব action-এ apply হয়, individual action-এ আলাদা করে লিখতে হয় না। ASP.NET Core-এর built-in `[Authorize]` attribute JWT-র `ClaimTypes.Role` claim (গুলো) চেক করে — যেকোনো একটা role match করলেই access দেয় (OR logic, একসাথে একাধিক role লিখলে)।

Authentication vs Authorization — এখানে স্পষ্ট:

- **Authentication** ("তুমি কে?") — `[Authorize]` (কোনো role specify ছাড়া) মানে শুধু valid JWT থাকলেই চলবে, role যাই হোক। এটা handle করে `UseAuthentication()` middleware + JWT bearer scheme।

- **Authorization** ("তোমার permission আছে কিনা?" — `[Authorize(Roles = "...")]` মানে valid JWT থাকতে হবে এবং নির্দিষ্ট role claim থাকতে হবে।
`[AllowAnonymous]` মানে কোনো authentication-ই লাগবে না (public endpoint, যেমন `AuthController.Login`)।

Part 3 — C# Fundamentals (তোমার project থেকে real example সহ)

3.1 OOP Pillars

Encapsulation

Product (বা যেকোনো) Entity-তে property গুলো `{ get; set; }` দিয়ে exposed, কিন্তু business rule Service layer-এ রাখা হয়, Entity-তে না — এটা একধরনের **Anemic Domain Model** style (pure encapsulation না, কিন্তু layered architecture-এ common)। উদাহরণ — `AuthService.RegisterAsync` -এ:

```
var user = new User { Id = Guid.NewGuid(), Email = email, PasswordHash = _passwordHasher.Hash(request.Password), ... };
```

Password কখনো plain text রাখা হয় না — `_passwordHasher.Hash()` দিয়ে সবসময় hash করে রাখা হয় (`PasswordHash` property)। এটা data hiding/protection-এর একটা practical উদাহরণ — raw password কখনো persist হয় না।

Inheritance

`GlobalExceptionHandler` implement করে `IExceptionHandler` interface (interface inheritance)। Custom exceptions (`NotFoundException`, `ConflictException`) সরাসরি .NET-এর `Exception` ক্লাস থেকে inherit করে:

```
public class NotFoundException : Exception
{
    public NotFoundException(string message) : base(message) { }
}
```

`: base(message)` — parent class (`Exception`)-এর constructor কল করা, যাতে `.Message` property ঠিকমতো সেট হয়। এটা **method/constructor overriding**-এর একটা রূপ না বরং constructor chaining।

Polymorphism

`GlobalExceptionHandler` -এর switch expression-এ runtime polymorphism দেখা যায়:

```
var (statusCode, title, errors) = exception switch
{
    FluentValidation.ValidationException v => (400, "Validation failed.", ...),
    ConflictException c => (409, c.Message, null),
    NotFoundException n => (404, n.Message, null),
    UnauthorizedAccessException u => (401, u.Message, null),
    _ => (500, "An unexpected error occurred.", null),
};
```

এখানে `exception` প্যারামিটারের static type `Exception`, কিন্তু **runtime-এ actual type** (কোন subclass) অনুযায়ী match হয় — এটা **type pattern matching**, একটা modern C# feature যেটা polymorphic dispatch-এর কাজ করে without a virtual method।

আরেকটা classic polymorphism উদাহরণ — Interface-based: প্রতিটা `IXxxRepository` interface-এর একটাই implementation আছে এখানে, কিন্তু design টা এমন যে চাইলে unit test-এর জন্য একটা `FakeProductRepository : IProductRepository` বানিয়ে বসিয়ে দেওয়া যায়, Service-এর কোড এক লাইনও না বদলিয়ে — কারণ Service সবসময় interface টাইপ দিয়ে কাজ করে (`IProductRepository _productRepository`), concrete class দিয়ে না।

Abstraction

`IAuthService`, `IProductService`, `ITokenService` ইত্যাদি — এগুলো **abstract contract**, বলে দেয় "কী করা যাবে" (method signature), "কীভাবে করা হবে" সেটা implementation class-এ (`AuthService`, `ProductService`, `JwtTokenService`)। Controller বা অন্য Service শুধু interface জানে, implementation detail জানে না।

3.2 Class vs Object

- Class** — blueprint, যেমন `Product` ক্লাস define করে একটা প্রোটোটাইপের shape (Name, Price, Stock, Category...)।
- Object** — সেই blueprint থেকে তৈরি একটা actual instance, যেমন `ProductRepository.GetByIdAsync(id, ct)` থেকে ফেরত আসা একটা specific `Product` object, যার `Name = "হাতে বোনা নকশিকথা"`।

3.3 Constructor & Dependency Injection Pattern

প্রায় প্রতিটা Service/Controller **Constructor Injection** ব্যবহার করে:

```
public class AuthService : IAuthService
{
    private readonly IUserRepository _userRepository;
    ... (আরও ৭টা dependency)
    public AuthService(IUserRepository userRepository, IRoleRepository roleRepository, ...)
    {
        _userRepository = userRepository;
        ...
    }
}
```

সব dependency **private readonly** field-এ রাখা — **readonly** মানে constructor-এর বাইরে এই field আর reassign করা যাবে না (immutability, thread-safety-র জন্য ভালো practice)। DI container (`Program.cs` -এ `AddScoped<IAuthService, AuthService>()`) runtime-এ automatically এই সব dependency resolve করে constructor-এ পাস করে দেয় — কোথাও manual `new AuthService( ... )` লেখা হয় না (Controller-এও না)।

3.4 Interface vs Abstract Class

প্রজেক্টে প্রায় সবখানে **Interface** ব্যবহার হয়েছে (`IProductService` , `IProductRepository` , `ITokenService` ...), Abstract class প্রায় নেই বললেই চলে। কেন:

- C#-এ একটা class **একাধিক interface** implement করতে পারে, কিন্তু মাত্র **একটা class** থেকে inherit করতে পারে — তাই flexible composition-এর জন্য interface বেশি সুবিধাজনক।

- Interface-এ কোনো shared implementation/state থাকে না (শুধু contract) — এখানে সব Service/Repository independent, তাদের মধ্যে shared base behavior দরকার হয়নি।

- (Viva follow-up prep: যদি ভবিষ্যতে multiple repository-তে common CRUD logic শেয়ার করা লাগতো, তখন হয়তো `abstract class BaseRepository<T>` justified হতো — কিন্তু এই প্রজেক্টে প্রতিটা repository নিজের নির্দিষ্ট domain-specific query লিখেছে, generic base class না বানিয়ে!)।

3.5 Static Members

- `RoleNames` — পুরো ক্লাসটাই **static**, শুধু constant string আর readonly list রাখার জন্য (utility/constants holder — instance বানানোর দরকার নেই)।
- `ValidationFilter.OnActionExecutionAsync` -এর ভেতরে ব্যবহৃত `typeof(IValidator<>)` — **typeof** একটা static-context operator, compile-time এ টাইপ information পায়।
- `Program.cs` -এ local `static string RatePartitionKey(HttpContext ctx) { ... }` — এটা একটা **local static function**, **builder** / **app** ভ্যারিয়েবলগুলো capture না করেই independently কাজ করে (কোনো state নেই, pure function)।

3.6 Access Modifiers

- **public** — Controller, Service, Interface, DTO ক্লাস (বাইরে থেকে access দরকার)।
- **private** — Service-এর ভেতরের helper method (`ToDto`), field (`_productRepository`)।
- **internal** (কম ব্যবহৃত এখানে, কিন্তু concept হিসেবে) — একই assembly-র মধ্যে সীমাবদ্ধ।
- **private readonly** — সবচেয়ে বেশি ব্যবহৃত pattern, constructor-injected dependency-র জন্য।

3.7 Generics

- `Task<T>` — সবখানে (`Task<AuthResponse>` , `Task<Product?>`)।
- `IEnumerable<T>` , `List<T>` , `IReadOnlyList<T>` — collection-এর সব জায়গায়।
- `PagedResult<T>` (custom generic class):

```
public class PagedResult<T>
{
    public List<T> Items { get; set; } = new();
    public int TotalCount { get; set; }
    public int Page { get; set; }
    public int PageSize { get; set; }
    public int TotalPages => PageSize > 0 ? 0 : (int)Math.Ceiling(TotalCount / (double)PageSize);
}
```

এটা যেকোনো টাইপের জন্য reusable pagination wrapper — `PagedResult<ProductListItemDto>` , `PagedResult<OrderDto>` ইত্যাদি একই ক্লাস দিয়ে বানানো যায়, কোড duplicate করা লাগে না। `TotalPages` একটা **computed property** (expression-bodied `=>`), ceiling division দিয়ে হিসাব করা।

- `IValidator<>` / `MakeGenericType` (`ValidationFilter` -এ) — **runtime এ generic type নির্মাণ**, reflection দিয়ে।

- `IOptions<JwtSettings>` — generic wrapper pattern, .NET-এর Options framework।

3.8 Exception Handling

- **Custom exceptions** (`NotFoundException` , `ConflictException`) — domain-specific error বোঝানোর জন্য, generic `Exception` throw করার বদলে।

- কোথাও **try/catch** নেই **controller-এ** বা **service-এ** (প্রায়) — কারণ centralized `GlobalExceptionHandler` সব ধরে নেয় (উপরে ব্যাখ্যা করা হয়েছে)। এটা একটা ভালো design decision: প্রতিটা controller action-এ try/catch লিখে repetitive code বাড়ানোর বদলে, throw করে দাও, এক জায়গায় handle হবে।
- `throw new NotFoundException("Product not found.")` — Service-এ, Repository `null` রিটার্ন করলে:

```
var product = await _productRepository.GetByIdAsync(id, cancellationToken)
?? throw new NotFoundException("Product not found.");
```

এটা **null-coalescing operator (??)** + **throw expression** এর সুন্দর combo — C# 7+ ফিচার, এক লাইনে null-check + exception throw।

3.9 Collections & LINQ

Project জুড়ে LINQ প্রচুর ব্যবহার হয়েছে — কিছু উদাহরণ:

```
var roleNames = user.UserRoles.Select(ur => ur.Role.Name).ToList();
var requestedRoleNames = request.Roles.Distinct(StringComparer.OrdinalIgnoreCase).ToList();
if (!roleNames.Contains(role, StringComparer.OrdinalIgnoreCase)) { ... }
products = products.Where(p => p.CategoryId == query.CategoryId.Value);
products = query.SortBy switch { ... => products.OrderBy(p => p.Price), ... };
```

- `.Select()` — projection/transformation (map)।
- `.Where()` — filtering।
- `.Distinct(StringComparer.OrdinalIgnoreCase)` — duplicate role name বাদ দেওয়া, case-insensitive comparer দিয়ে (`"Customer"` আর `"customer"` একই ধরা হবে)।
- `.OrderBy / .OrderByDescending / .ThenByDescending` — sorting, multi-key sort (`.ThenByDescending`)।
- `.Contains()` — membership check।

3.10 IEnumerable vs IQueryable (খুব common viva question)

- `IEnumerable<T>` — একবার data মেমোরিতে চলে এলে (যেমন `List<T>`) তার উপর filter/select করলে সেটা **LINQ-to-Objects**, C# কোড হিসেবে execute হয় in-memory।
- `IQueryable<T>` — EF Core-এর `DbSet<T>` (যেমন `_context.Products`) এই টাইপ রিটার্ন করে। এর উপর `.Where()` / `.OrderBy()` চেইন করলে সেটা সাথে সাথে execute হয় **না** — বরং একটা **Expression Tree** তৈরি হতে থাকে, যেটা EF Core পরে **SQL-এ translate করে** এবং ডাটাবেজে পাঠায়। Query আসলে execute হয় যখন `.ToListAsync()` / `.FirstOrDefaultAsync()` / `.CountAsync()` ইত্যাদি call হয় (deferred execution)।

`ProductRepository.GetPagedAsync` এটার নিখুঁত উদাহরণ:

```
private IQueryable<Product> WithDetails() => _context.Products.Include(...).AsSplitQuery();
...
var products = WithDetails().Where(p => p.IsActive);
if (!string.IsNullOrWhiteSpace(query.Search)) products = products.Where(p => EF.Functions.ILike(...));
if (query.CategoryId.HasValue) products = products.Where(p => p.CategoryId == query.CategoryId.Value);
...
var totalCount = await products.CountAsync(cancellationToken); // ← এখানে execute হয় (SQL: SELECT COUNT(*) ...)
var items = await products.Skip(...).Take(...).ToListAsync(cancellationToken); // ← আবার execute হয় (SQL: SELECT ... LIMIT/OFFSET)
```

এখানে সবগুলো `.Where()` চেইন এক জায়গায় accumulate হয়ে **শেষে একটাই SQL query**-তে translate হয় WHERE clause গুলো দিয়ে — filter সব database-এ গিয়ে apply হয়, পুরো টেবিল memory তে না এনে। এটাই `IQueryable` এর power।

Follow-up trap: যদি ভুল করে মাঝপথে `.ToListAsync()` কল করে ফেলতে (যেমন `WithDetails().ToListAsync().Where(...)`), তাহলে পুরো টেবিল প্রথমে মেমোরিতে চলে আসত, তারপর filter memory-তে (LINQ-to-Objects) হতো — performance disaster বড় টেবিলে। এই প্রজেক্টে এই ভুল হয়নি (filter সব `.ToListAsync()` -এর আগে)।

3.11 Lambda Expressions

`p => p.IsActive, ur => ur.Role.Name, (ctx, cancellationToken) => { ... }` — সবই lambda। এগুলো `Func<T, TResult>` / `Action<T>` / `Predicate<T>` delegate টাইপে assign হয়, LINQ methods-এর argument হিসেবে সবচেয়ে বেশি ব্যবহৃত।

Part 4 — ASP.NET Core Deep Dive

4.1 Web API vs MVC

এই প্রজেক্ট একটা pure **Web API** (JSON only, কোনো Razor View/HTML page নেই)। `[ApiController]` attribute প্রতিটা Controller-এ আছে — এটা কয়েকটা automatic behavior দেয়:

- **Automatic 400 response** — যদি model binding fail করে (invalid JSON), automatically `400 Bad Request` রিটার্ন হয়, ম্যানুয়াল চেক লাগে না।
- **Automatic model binding source inference** — `[FromBody]` অনেক জায়গায় explicit লেখা হয়নি কারণ `[ApiController]` থাকলে complex type (class) parameter automatically body থেকে bind হয়, simple type (int, Guid, string) route/query থেকে।
- **Attribute routing required** — `[Route("api/...")]` বাধ্যতামূলক (conventional routing না)।

4.2 Controller Structure

প্রতিটা controller এই প্যাটার্ন follow করে (উদাহরণ `AuthController`):

```
[ApiController]
[Route("api/auth")]
[EnableRateLimiting("auth")]
public class AuthController : ControllerBase
{
    private readonly IAuthService _authService;
    public AuthController(IAuthService authService) { _authService = authService; }

    [HttpPost("register")]
    public async Task<ActionResult<AuthResponse>> Register(RegisterRequest request, CancellationToken cancellationToken)
    {
        var result = await _authService.RegisterAsync(request, ClientIp, cancellationToken);
        return Ok(result);
    }
}
```

- `ControllerBase` (Web API-র জন্য, `Controller` না — সেটা View support সহ MVC-র জন্য, যা এখানে দরকার নেই)।
- `[HttpPost("register")]` — HTTP verb + sub-route (পুরো route হয় `api/auth/register`)।
- `ActionResult<T>` vs `IActionResult`: `ActionResult<T>` দিলে Swagger response type জানতে পারে (`AuthResponse` টাইপ documented হয়), এবং implicit conversion করা যায় (`return Ok(result)` অথবা সরাসরি `return result`)। `IActionResult` (যেমন `Logout`, যেটা কিছু রিটার্ন করে না, শুধু status) — বেশি generic, কোনো নির্দিষ্ট return type-এর কমিটমেন্ট নেই।
- `Ok(result)` → 200, `NoContent()` → 204, `NotFound()` → 404, `BadRequest()` → 400, `CreatedAtAction( ... )` → 201 + Location header।

4.3 Model Binding Attributes

Attribute	কোথা থেকে data আসে	উদাহরণ
<code>[FromBody]</code>	Request JSON body	<code>RegisterRequest request</code> (POST/PUT-এ implicit, <code>[ApiController]</code> থাকায়)
<code>[FromRoute]</code>	URL path segment	<code>GetById(Guid id)</code> যখন route <code>api/products/{id}</code>
<code>[FromQuery]</code>	URL query string	<code>GetProducts([FromQuery] ProductQueryParameters query)</code> — pagination/filter params
<code>[FromHeader]</code>	HTTP header	কম ব্যবহৃত এখানে (JWT header framework নিজে হ্যান্ডল করে)

4.4 Middleware Pipeline (বিস্তারিত Part 2.1-এ দেওয়া আছে)

মনে রাখার সহজ নিয়ম: **Middleware একটা pipeline, order matters, request উপর থেকে নিচে যায় response নিচ থেকে উপরে ফেরে (like an onion / Russian nesting doll)।**

4.5 CORS (Cross-Origin Resource Sharing)

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("Frontend", policy => policy
        .WithOrigins("http://localhost:5173", "https://localhost:5173")
        .AllowAnyHeader()
        .AllowAnyMethod());
});
...
app.UseCors("Frontend");
```

ব্রাউজার security feature — default এ একটা origin (**example.com**) থেকে JavaScript অন্য origin (**api.example.com**)-এ request পাঠাতে পারে না, যদি না server explicitly অনুমতি দেয় (CORS headers)। **WithOrigins** দিয়ে শুধু frontend dev server (Vite-এর default port **5173**) থেকেই allow করা হয়েছে — wildcard (**AllowAnyOrigin**) ব্যবহার করা হয়নি, যেটা security best practice (production-এ actual frontend domain বসবে)।

4.6 Swagger / OpenAPI

```
builder.Services.AddSwaggerGen(options =>
{
    options.CustomSchemaIds(type => type.FullName);
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme { ... BearerFormat = "JWT" ... });
    options.AddSecurityRequirement(...);
});
if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(); }
```

- শুধু **Development**-এ enable — production-এ API schema/internal structure বাইরের কাউকে দেখানো হয় না (security/information hiding)।
- CustomSchemaIds(type => type.FullName)** — সাধারণত Swagger শুধু class-এর short name (**UpdateStatusRequest**) দিয়ে schema id বানায়, কিন্তু এখানে অনেক module-এ একই নামের DTO আছে (যেমন **Logistics.UpdateMilestoneStatusRequest** vs **Research.UpdateMilestoneStatusRequest**) — **FullName** (namespace সহ) ব্যবহার করলে naming collision হয় না।
- AddSecurityDefinition / AddSecurityRequirement** — Swagger UI-তে "Authorize" বাটন যোগ করে, যাতে developer সরাসরি Swagger থেকে JWT বসিয়ে authenticated request টেস্ট করতে পারে।

4.7 Health Checks

```
builder.Services.AddHealthChecks().AddNpgsql(connectionString, name: "supabase-postgres");
...
app.MapHealthChecks("/health/db");
```

/health/db endpoint hit করলে ASP.NET Core সত্যি সত্যি database-এ কানেক্ট করে দেখে (একটা lightweight query চালায়), আর respond করে database up আছে কিনা — deployment/monitoring tool (uptime checker, Docker healthcheck, load balancer) এটা ব্যবহার করে বুঝতে পারে app সুস্থ আছে কিনা।

4.8 DTO vs Entity — কেন আলাদা রাখা হয়েছে

- Entity** (**Product** , **Order**) — database table represent করে, EF Core দিয়ে persist হয়, internal fields থাকতে পারে যা বাইরে দেখানো উচিত না।
- DTO** (**ProductDto** , **ProductListItemDto** , **CreateProductRequest**) — API contract, শুধু যা client-কে দরকার তাই exposed। উদাহরণ, **ProductListItemDto** (list view-এর জন্য) হালকা — full **ProductDto** (detail view) এর তুলনায় কম field, কারণ list এ সব detail দরকার নেই (performance + payload size)।

সুবিধা:

- Security** — Entity-র sensitive field (যেমন internal audit columns) accidentally leak হয় না।
- Decoupling** — Database schema বদলালে সব API contract বদলাতে হয় না, আর API contract বদলালে DB schema বদলাতে হয় না।
- Validation surface** — **CreateProductRequest** -এর মতো "Request" DTO-তে FluentValidation rule বসানো যায়, যেটা Entity-তে বসানো হলে EF Core-এর সাথে conflict করতে পারে।
- Over-posting protection** — client যদি extra field পাঠায় (যেমন **IsActive** বা **Id** নিজে থেকে সেট করার চেষ্টা করে) সেটা DTO-তে না থাকলে ignored হয়ে যায় (mass-assignment vulnerability prevention)।

4.9 Repository + Service Layer Pattern (Clean/Layered Architecture)

```
Controller → Service (business logic, orchestration, validation rules, DTO mapping)
    → Repository (data access abstraction – EF Core query)
        → DbContext → Database
```

- Repository** শুধু data access নিয়ে ভাবে — "কীভাবে data আনা/সেভ করা হবে।" কোনো business rule নেই এখানে।

- **Service** business logic নিয়ে ভাবে — "কী শর্তে কী হবে" (যেমন stock না থাকলে order করা যাবে না), একাধিক Repository কে coordinate করে (Order তৈরি করতে Product Repository আর Order Repository দুটোই লাগতে পারে)।

কেন আলাদা? Testability (Service টেস্ট করার সময় fake/mock Repository বসানো যায়, real DB লাগে না), Separation of Concerns, আর যদি কখনো EF Core থেকে অন্য কোনো data-access technology-তে সরতে হয়, শুধু Repository layer বদলালেই হবে, Service layer অক্ষত থাকবে।

4.10 SignalR (Realtime)

```
builder.Services.AddSignalR();
builder.Services.AddScoped<IMessageNotifier, SignalRMessageNotifier>();
...
app.MapHub<MessagingHub>("/hubs/messaging");
app.MapHub<LiveEventHub>("/hubs/live-events");
app.MapHub<LiveClassHub>("/hubs/live-classes");
```

SignalR হলো ASP.NET Core-এর realtime communication library — WebSocket (fallback: Server-Sent Events / Long Polling) ব্যবহার করে server থেকে client-কে push notification পাঠানো যায়, client বারবার poll না করেও। তিনটা Hub আছে: chat messaging, live shopping/cultural events, live classes — এই তিনটাতেই realtime update দরকার (নতুন message, নতুন bid/comment, ক্লাসে নতুন কিছু ঘটলে) সাথে সাথে সব connected client-কে জানানোর জন্য।

`IMessageNotifier` ইন্টারফেসের implementation `SignalRMessageNotifier` — এটাও Dependency Inversion-এর উদাহরণ: `MessagingService` (business logic) সরাসরি SignalR জানে না, শুধু `IMessageNotifier.NotifyAsync(...)` কল করে — SignalR-specific কোড আলাদা `SignalRMessageNotifier` ক্লাসে isolated।

4.11 Rate Limiting Attribute Usage

`[EnableRateLimiting("auth")]` (class-level, `AuthController` -এ), `[EnableRateLimiting("read")]` (নির্দিষ্ট browse/catalog endpoint-এ) — `Program.cs` -এ define করা named policy গুলো নির্দিষ্ট controller/action-এ apply করার জন্য attribute ব্যবহার।

Part 5 — Async/Await Deep Dive

5.1 কেন সব I/O method Async?

Project জুড়ে প্রায় প্রতিটা Controller action, Service method, Repository method `async` / `await` দিয়ে লেখা, এবং প্রায় সবগুলোর নামের শেষে `Async` suffix (নিয়ম মেনে — C# convention)। উদাহরণ: `RegisterAsync`, `GetByIdAsync`, `SaveChangesAsync`, `ToListAsync`।

কারণ: এই backend-এর প্রতিটা meaningful operation একটা **I/O-bound** কাজ — database-এ query পাঠানো (network call, EF Core → PostgreSQL over TCP), অথবা email পাঠানো (`ISender`)। I/O operation-এ CPU আসলে কিছু করে না, শুধু **wait** করে network/disk response-এর জন্য।

Synchronous হলে কী হতো

```
var product = _productRepository.GetById(id); // ধরি synchronous ভার্সন
```

এই কল চলাকালীন যে thread টা এটা রান করছে (ASP.NET Core-এর thread pool থেকে একটা thread), সেটা সম্পূর্ণ **blocked** থাকবে — database respond না করা পর্যন্ত কিছুই করতে পারবে না, অন্য কোনো incoming HTTP request handle করতে পারবে না। হাজার হাজার concurrent user থাকলে thread pool exhaust হয়ে যাবে, নতুন request queue-তে আটকে থাকবে — throughput ভয়াবহভাবে কমে যাবে।

Async হলে

```
var product = await _productRepository.GetByIdAsync(id, cancellationToken);
```

await করার সময়, thread টা **database response-এর জন্য বসে থাকে না** — সেটা thread pool-এ ফেরত যায়, ততক্ষণে অন্য incoming HTTP request handle করতে পারে। যখন database response আসে, .NET runtime একটা (যেকোনো) available thread নিয়ে বাকি method continue করে। এভাবে **কম সংখ্যক thread দিয়ে অনেক বেশি concurrent request handle করা যায় (higher throughput, better scalability)** — এটাই `async/await`-এর মূল সুবিধা। ASP.NET Core Web API-তে, বিশেষভাবে "non-blocking I/O" বলা হয়।

5.2 Task এবং Task<T>

- Task** — represents একটা asynchronous operation যেটা কোনো value রিটার্ন করে না (যেমন `void` এর `async` ভার্সন)। উদাহরণ: `Task LogoutAsync(...)`, `Task ResetPasswordAsync(...)`।
- Task<T>** — একটা asynchronous operation যেটা **T** টাইপের value রিটার্ন করবে (ভবিষ্যতে)। উদাহরণ: `Task<AuthResponse> RegisterAsync(...)`, `Task<Product?> GetByIdAsync(...)`।

`Task` / `Task<T>` আসলে একটা **"promise"/"future"** — একটা object যেটা বলে "এই কাজটা চলছে, শেষ হলে এই এর ভেতর result/exception থাকবে।"

5.3 async / await কীভাবে কাজ করে

```
public async Task<AuthResponse> LoginAsync(LoginRequest request, string? ipAddress, CancellationToken cancellationToken)
{
    var email = request.Email.Trim().ToLowerInvariant();
    var user = await _userRepository.GetByEmailWithRolesAsync(email, cancellationToken);
    if (user is null || !_passwordHasher.Verify(request.Password, user.PasswordHash))
        throw new UnauthorizedAccessException("Invalid email or password.");
    ...
    return await IssueTokensAsync(user, roleNames, activeRole, ipAddress, cancellationToken);
}
```

- `async` keyword method-কে বলে দেয় যে এর ভেতরে `await` ব্যবহার হবে, আর compiler এটাকে একটা **state machine**-এ রূপান্তর করে (কম্পাইলার magic — মানুষ যা `await`-এর জায়গায় লেখে, তার পেছনে callback/continuation তৈরি হয়ে যায়)।
- `await` মানে: "এই `async` operation শেষ না হওয়া পর্যন্ত এই method-এর execution suspend করো, কিন্তু thread-টা ব্লক করে রেখো না — ছেড়ে দাও। কাজ শেষ হলে এখান থেকেই চালিয়ে যাও।"
- একটা method-এর ভেতরে multiple `await` sequential ভাবে চলে (যেমন প্রথমে `GetByEmailWithRolesAsync`, তারপর সফল হলে `IssueTokensAsync`) — কিন্তু প্রতিটা `await`-এর সময় thread **blocked** না থেকে "released" থাকে।

5.4 Blocking vs Non-Blocking

- Blocking call** — `Thread.Sleep(1000)`, বা `.Result` / `.Wait()` দিয়ে একটা `Task`-কে synchronously অপেক্ষা করানো — thread কিছু না করাই বসে থাকে।
- Non-blocking call** — `await Task.Delay(1000)`, বা এই প্রজেক্টের সব `await someAsyncMethod()` — thread ছেড়ে দেয়, অন্য কাজ করে, পরে ফিরে আসে।

⚠ কখনো `.Result` বা `.Wait()` ব্যবহার করা উচিত না `async method`-এর উপর — এতে deadlock হতে পারে (নিচে ব্যাখ্যা) এবং thread pool starvation-ও।

5.5 Deadlock Risk এবং ConfigureAwait

ASP.NET Core-এ (Framework-এর পুরনো ভার্সনের তুলনায়) আসলে কোনো `SynchronizationContext` capture হয় না by default — তাই এই প্রজেক্টে `ConfigureAwait(false)` কোথাও ব্যবহার করা হয়নি, এবং সেটা **ঠিক আছে** — ASP.NET Core Web API-তে এটা optional/কম গুরুত্বপূর্ণ (WPF/WinForms-এর মতো UI context নেই এখানে)। কিন্তু viva-তে concept জানা জরুরি:

- পুরনো ASP.NET (.NET Framework)-এ একটা `SynchronizationContext` থাকতো যেটা নিশ্চিত করতো `await`-এর পরের কোড একই "context"-এ (যেমন same request thread-like context) চলবে। যদি কেউ async method-কে `.Result` / `.Wait()` দিয়ে synchronously কল করতে, সেই context-টা block হয়ে যেত, আর ভেতরের `await` যখন সেই একই context-এ ফিরে আসার চেষ্টা করতো — **deadlock!**
- `ConfigureAwait(false)` বলে দেয় "await-এর পরের কোড original context-এ ফিরে যাওয়ার দরকার নেই, যেকোনো thread pool thread-এ চালাও" — library code-এ (যেখানে caller-এর context জানা নেই) এটা best practice ছিল।
- ASP.NET Core-এ** কোনো `SynchronizationContext` নেই by default, তাই এই ক্লাসিক deadlock সমস্যা এখানে হয় না — কিন্তু তবুও `.Result` / `.Wait()` avoid করা ভালো, কারণ এটা thread pool থেকে একটা thread কে অকারণে blocked রাখে (thread pool starvation, throughput কমে)।

5.6 Missing await — সাধারণ ভুল (viva trap!)

```
// ভুল উদাহরণ (এই প্রজেক্টে এই ভুল নেই, কিন্তু concept জানা জরুরি):
public Task<Product?> GetByIdAsync(Guid id, CancellationToken ct)
{
    _productRepository.GetByIdAsync(id, ct); // ← await ভুলে গেলে
    return null; // compiler warning, runtime এ ভুল ফলাফল
}
```

যদি `await` ভুলে বাদ পড়ে (বা `async void` ব্যবহার হয়), তাহলে:

- exception silently swallow হয়ে যেতে পারে (কেউ ধরতে পারবে না)।
- method আসল কাজ শেষ হওয়ার আগেই রিটার্ন করে ফেলবে (fire-and-forget হয়ে যাবে, কিন্তু caller ভাবছে সব শেষ)।
- Compiler warning দেয় (CS4014) "because this call is not awaited..." — এই প্রজেক্টে `dotnet build` করলে এরকম কোনো warning থাকা উচিত না, কারণ প্রতিটা async কল সঠিকভাবে `await` করা হয়েছে।

5.7 async void — কখনো ব্যবহার করা হয়নি (correctly)

প্রজেক্টে কোনো `async void` method নেই (`ApiController`, `AuthController` — সব `async Task` / `async Task<T>`)। `async void` শুধু event handler-এর জন্য উপযুক্ত (UI framework-এ), কারণ এটা:

- Caller কে কোনো `Task` রিটার্ন করে না, তাই caller `await` করতে পারে না, exception ধরতে পারে না — exception সরাসরি thread-এ throw হয়ে app crash করতে পারে।
- Web API-তে controller action `async void` হলে ASP.NET Core-ই ঠিকমতো response পাঠাতে পারবে না।

5.8 নির্দিষ্ট EF Core Async মেথড

Method	কী করে
<code>ToListAsync()</code>	Query execute করে সব result <code>List<T></code> -এ আনে (async)
<code>FirstOrDefaultAsync()</code>	প্রথম match আনে, না পেলে <code>null</code> / default (async)
<code>SingleOrDefaultAsync()</code>	ঠিক একটা match আশা করে (একাধিক পেলে exception)
<code>CountAsync()</code>	শুধু count (async), পুরো row আনে না
<code>AnyAsync()</code>	কোনো row match করে কিনা (boolean, async) — <code>ExistsByEmailAsync</code> -এ ব্যবহার হয়েছে
<code>SaveChangesAsync()</code>	DbContext-এর tracked change গুলো database-এ commit করে (INSERT/UPDATE/DELETE, একটা DB round-trip)

`AuthService.RegisterAsync` -এ:

```
if (await _userRepository.ExistsByEmailAsync(email, cancellationTokens)) { throw new ConflictException(...); }
...
await _userRepository.AddAsync(user, cancellationTokens);
await _userRepository.SaveChangesAsync(cancellationTokens);
```

লক্ষ্য করো — `AddAsync` শুধু EF Core-এর in-memory change tracker-এ entity টা "Added" state এ mark করে (এখনো database-এ কিছু যায়নি), আসল INSERT SQL চলে `SaveChangesAsync()` কল হওয়ার সময়। এটা একটা common viva confusion পয়েন্ট — `Add` / `AddAsync` আর `SaveChanges` / `SaveChangesAsync` দুটো আলাদা ধাপ।

5.9 CancellationToken — প্রতিটা async method-এ কেন থাকে

প্রায় প্রতিটা Controller action আর Service/Repository method-এর শেষ প্যারামিটার `CancellationToken cancellationToken`। ASP.NET Core প্রতিটা request-এর জন্য একটা `CancellationToken` তৈরি করে যেটা **automatically cancel হয়ে যায় client disconnect করলে** (ব্রাউজার ট্যাব বন্ধ, নেটওয়ার্ক কাটা, request timeout)। এই token EF Core-এর সব async call-এ পাস করা হয় (`ToListAsync(cancellationToken)`), যাতে client চলে গেলে server অকারণে ভারী query চালিয়ে যেতে না থাকে — resource বাঁচে।

5.10 Multithreading vs Async — একটা common ভুল ধারণা

`async / await` **নতুন thread তৈরি করে না**। এটা মাল্টিথ্রেডিং না, এটা **concurrency** (একটা thread একসাথে অনেকগুলো pending operation manage করতে পারা), মাল্টিথ্রেডিং না (একাধিক thread parallel-এ CPU কাজ করা)। এই প্রজেক্টের async কোড মূলত I/O-bound wait handle করছে, **নতুন thread spawn করে parallel computation করছে না** (যেমন `Parallel.ForEach` বা `Task.Run` কোথাও নেই কারণ CPU-bound কাজ নেই এখানে)।

Part 6 — Entity Framework Core & Database Deep Dive

6.1 DbContext এবং DbSet<T>

```
public class ShilpoHubDbContext : DbContext
{
    public ShilpoHubDbContext(DbContextOptions<ShilpoHubDbContext> options) : base(options) { }

    public DbSet<User> Users => Set<User>();
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Order> Orders => Set<Order>();
    ... (২০০+ DbSet, প্রতিটা Entity-র জন্য একটা)

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(ShilpoHubDbContext).Assembly);
        base.OnModelCreating(modelBuilder);
    }
}
```

- **DbContext** — EF Core-এর কেন্দ্রীয় ক্লাস, database-এর সাথে একটা "session" represent করে। এটাই **Unit of Work** এবং **Repository** pattern-এর (EF Core built-in আকারে) সমন্বয় — change tracking, query translation, transaction management সব এখান থেকে হয়।
- **DbSet<T>** — একটা টেবিলের representation, **IQueryable<T>** implement করে, তাই এর উপর LINQ query চালানো যায় যেটা SQL-এ translate হবে।
- **Set<T>()** ব্যবহার করা হয়েছে property getter-এ (**=> Set<T>()**) generic method দিয়ে — এটা EF Core-এর নতুন recommended style, ম্যানুয়াল field ছাড়াই।
- **ApplyConfigurationsFromAssembly** — পুরো assembly স্ক্যান করে সব **IEntityTypeConfiguration<T>** ক্লাস (Fluent API configuration, **ShilpoHubBD.Data/Configurations/** ফোল্ডারে) খুঁজে বের করে apply করে — Entity-র schema definition (column type, relationship, constraint) প্রতিটা টেবিলের জন্য আলাদা ফাইলে রাখা হয় (Single Responsibility), **OnModelCreating** -এ সব একসাথে পাদাগাদি করা হয়নি।

6.2 Code-First এবং Migrations

এই প্রজেক্ট **Code-First** approach ব্যবহার করে — মানে প্রথমে C# Entity ক্লাস লেখা হয়, তারপর EF Core সেই ক্লাসগুলো থেকে database schema বানায় (উন্টোটা — Database-First — যেখানে existing database থেকে C# ক্লাস generate করা হয়, এখানে হয়নি)।

- **৬৪টা migration file** আছে (**ShilpoHubBD.Data/Migrations/**) — প্রতিটা migration একটা incremental schema change represent করে (নতুন টেবিল, নতুন কলাম, constraint বদল ইত্যাদি)। প্রতিটা migration-এর দুটো method: **Up()** (change apply করা) আর **Down()** (rollback করার জন্য undo করা)।
- Command: **dotnet ef migrations add <Name>** (নতুন migration বানায়, current model vs আগের snapshot compare করে) এবং **dotnet ef database update** (pending migration গুলো database-এ apply করে)।
- **ModelSnapshot** ফাইল — পুরো model-এর current state-এর snapshot রাখে, পরের migration বানানোর সময় এটার সাথে compare করা হয়।

Code-First vs Database-First:

- Code-First: C# entity → migration → DB schema। সুবিধা: schema change version-controlled (git-এ migration ফাইল কমিট হয়), পুরো টিম একই schema history শেয়ার করে।
- Database-First: existing DB → scaffold করে C# class জেনারেট। সাধারণত legacy database-এর সাথে কাজ করার সময় ব্যবহার হয়।

এই প্রজেক্টে Code-First স্বাভাবিক পছন্দ, কারণ পুরো database schema-টাই প্রজেক্টের সাথে develop হয়েছে, নতুন feature (নতুন controller) মানেই সাধারণত নতুন entity + migration।

6.3 Relationships — Foreign Key & Navigation Properties

ProductRepository.WithDetails() :

```
private IQueryable<Product> WithDetails() => _context.Products
    .Include(p => p.Category)
    .Include(p => p.District)
    .Include(p => p.Producer)
    .Include(p => p.HandmadeVerifiedBy)
    .Include(p => p.Images)
    .Include(p => p.Variants)
    .Include(p => p.Videos)
    .AsSplitQuery();
```

- **Navigation Property** — **p.Category**, **p.Images** ইত্যাদি Entity-র property, যেগুলো related entity (বা collection) point করে (raw foreign key column না, actual related object)। EF Core এগুলো configuration (Fluent API বা convention) দেখে বোঝে কোন Foreign Key দিয়ে সম্পর্কটা তৈরি।
- **.Include()** — **eager loading**। Default এ EF Core একটা **Product** আনলে তার related **Category / Images** আনে না (lazy/no loading by default এখানে, কারণ lazy-loading proxy configure করা হয়নি) — **.Include()** explicit ভাবে বলে দেয় "এই related data-ও একসাথে আনো।"

- `.ThenInclude()` — nested relationship-এর জন্য (এই স্নিপেটে দেখা যাচ্ছে না কিন্তু প্রজেক্টে অন্য জায়গায় আছে, যেমন `Order → OrderItems → Product`)।
- **One-to-Many:** `Category` → many `Product` (একটা `Category`-র অনেক `Product`)।
- **Many-to-One:** `Product` → একটা `Category` (বিপরীত দিক থেকে দেখলে)।
- **Many-to-Many:** `User` ↔ `Role`, `UserRole` join entity দিয়ে (explicit join entity ব্যবহার করা হয়েছে, implicit skip-navigation না — কারণ join entity-তে extra field দরকার ছিল: `AssignedAt`, `AssignedByUserId`)।

6.4 AsSplitQuery() — কেন দরকার (N+1 এর বিপরীত সমস্যা!)

`Product` -এর সাথে ৭টা `.Include()` — একসাথে সবগুলো single SQL query-তে (default `AsSingleQuery`) আনতে গেলে multiple `LEFT JOIN` লাগবে, আর একাধিক one-to-many relationship (`Images`, `Variants`, `Videos`) একসাথে join করলে **Cartesian Explosion** হয় — যদি একটা `Product`-এর ৫টা `Image` আর ৩টা `Variant` থাকে, single query তে ১৫টা duplicate row আসবে (৫×৩), যেগুলো EF Core পরে client-side এ deduplicate করে সাজায়। ছোট ডেটাবেইজ এটা অপ্রয়োজনীয় network traffic আর processing।

`.AsSplitQuery()` দিয়ে EF Core কে বলা হয় — একটা query-র বদলে **একাধিক আলাদা SQL query** পাঠাও (একটা `Product`-এর জন্য, একটা `Images`-এর জন্য, একটা `Variants`-এর জন্য...), প্রতিটা আলাদা ভাবে filtered/joined, কোনো cartesian duplicate ছাড়াই। **Trade-off:** একাধিক round-trip (network latency বাড়তে পারে) কিন্তু ডেটা transfer অনেক কম এবং কোনো duplicate row processing লাগে না — multiple one-to-many collection include করার সময় এটাই recommended approach।

6.5 N+1 Query Problem

N+1 সমস্যা — যদি কেউ `.Include()` ছাড়াই একটা loop-এ related data access করে:

```
var products = await _context.Products.ToListAsync(); // 1 query
foreach (var p in products)
{
    var categoryName = p.Category.Name; // প্রতিটা product-এর জন্য আলাদা query! (N queries)
}
```

এভাবে ১টা query (products আনতে) + N টা query (প্রতিটা product-এর category আনতে, lazy loading enabled থাকলে) = N+1 query — ভয়াবহ performance killer।

এই প্রজেক্টে N+1 কীভাবে এড়ানো হয়েছে: সব repository method-এ প্রয়োজনীয় relationship আগে থেকেই `.Include()` দিয়ে eager load করা হয় (`WithDetails()` প্যাটার্ন), আর lazy-loading proxies enable করা হয়নি (`UseLazyLoadingProxies()` কোথাও কল হয়নি `Data/DependencyInjection.cs`-এ) — তাই accidental lazy N+1 হওয়ার সুযোগই নেই, ভুলে `.Include()` বাদ পড়লে navigation property `null` থাকবে (exception বা bug হিসেবে ধরা পড়বে), silent N+1 হবে না।

6.6 Change Tracking vs AsNoTracking()

Default এ EF Core যেকোনো query থেকে আসা entity **track** করে — মানে `DbContext` মনে রাখে entity-টার original state কী ছিল, আর পরে কোনো property বদলালে (যেমন `product.ViewCount++`) সেটা track করে, `SaveChangesAsync()` কল হলে automatically সঠিক `UPDATE` SQL জেনারেট করে — কোনো `_context.Update(product)` ম্যানুয়ালি কল করতে হয় না।

`ProductService.GetByIdAsync`:

```
var product = await _productRepository.GetByIdAsync(id, cancellationToken) ?? throw new NotFoundException(...);
product.ViewCount++;
await _productRepository.SaveChangesAsync(cancellationToken);
```

এখানে `GetByIdAsync` কোনো `.AsNoTracking()` ব্যবহার করেনি (দেখা যায়নি রিপোর মধ্যে) — **ইচ্ছাকৃতভাবে**, কারণ পরের লাইনে `product.ViewCount++` করে সেভ করা হচ্ছে — যদি tracked না থাকতো, তাহলে `SaveChangesAsync` কিছুই detect করতে পারতো না, manual `_context.Update(product)` লাগতো।

`AsNoTracking()` — যখন শুধু read করা হচ্ছে, কোনো update করা হবে না (যেমন list/browse endpoint), তখন change tracking overhead বাদ দেওয়ার জন্য ব্যবহার হয় (performance optimization — EF Core কে original value ম্যাপশট রাখতে হয় না)। **Viva note:** এই প্রজেক্টে দেখতে হবে নির্দিষ্ট কোন repository গুলো এটা ব্যবহার করেছে বনাম করেনি — যেখানে শুধু pure listing/browsing হচ্ছে (কোনো update হবে না), সেখানে `AsNoTracking()` না থাকলে সেটা একটা optimization opportunity/gotcha হিসেবে examiner পয়েন্ট আউট করতে পারে।

6.7 Transactions & Concurrency

- **Implicit transaction:** EF Core-এর `SaveChangesAsync()` নিজে থেকেই একটা transaction-এর ভেতরে চলে যদি একাধিক change (Insert+Update+Delete) একসাথে থাকে single `SaveChangesAsync()` কলে — সব হয় অথবা কিছুই হয় না (atomicity), automatic rollback হয় exception হলে।
- **Explicit transaction** (`_context.Database.BeginTransactionAsync()`) — দরকার হয় যখন একাধিক `SaveChangesAsync()` কল (আলাদা আলাদা repository-তে) একসাথে atomic হতে হবে। Cluster research-এ (যেমন Marketplace checkout flow) কিছু জায়গায় এই ধরনের multi-step operation single `SaveChangesAsync()` -এর উপর নির্ভর করে (সব change একসাথে track হয়ে একবারে সেভ হয়) — সেক্ষেত্রে আলাদা explicit transaction দরকার হয় না, কারণ EF Core-এর একটা `SaveChangesAsync()` কল নিজেই atomic।
- **Concurrency:** এই প্রজেক্টে optimistic concurrency token (`[Timestamp]` / `RowVersion`) ব্যবহার করে না বেশিরভাগ জায়গায় — মানে দুইজন user একসাথে একই row আপডেট করলে "last write wins" (কোনো conflict detection নেই)। এটা একটা সম্ভাব্য gotcha/improvement point — stock/inventory-এর মতো জায়গায় race condition হতে পারে (module-specific cluster ডকুমেন্টে detail আছে)।

6.8 Postgres-specific: `EF.Functions.ILike`

```
products = products.Where(p => EF.Functions.ILike(p.Name, term) || EF.Functions.ILike(p.Description, term));
```

ILIKE PostgreSQL-এর **case-insensitive pattern matching** operator (standard SQL **LIKE** case-sensitive, Postgres-এর **ILIKE** না)। `EF.Functions.ILike(...)` হলো EF Core-এর Npgsql provider-এর একটা extension method যেটা database-specific function ব্যবহার করতে দেয় LINQ-এর ভেতর থেকেই, C# এ প্লেইন `.Contains()` লিখলে সেটা generic SQL **LIKE** (case-sensitive হতে পারে database collation অনুযায়ী) এ translate হতো, **ILIKE** না।

6.9 Pagination Pattern

```
var totalCount = await products.CountAsync(cancellationToken);  
var items = await products.Skip((query.Page - 1) * query.PageSize).Take(query.PageSize).ToListAsync(cancellationToken);
```

দুইটা আলাদা query — একটা total count-এর জন্য (`COUNT(*)`), একটা actual page-এর data-র জন্য (`LIMIT / OFFSET` , Postgres-এ EF Core `.Skip()` / `.Take()` কে এভাবে translate করে)। দুটোই একই **WHERE** filter শেয়ার করে (একই `products` IQueryable variable থেকে branch করা)। **Trade-off**: দুই query মানে দুই round-trip, কিন্তু simple এবং readable — একটা single query দিয়ে `COUNT` + page data একসাথে আনা সম্ভব (window function দিয়ে) কিন্তু জটিলতা বাড়ে।

6.10 Normalization & Constraints (concept, project-grounded)

- **Primary Key**: প্রতিটা Entity-তে `Guid Id` (GUID primary key, auto-increment integer না) — distributed system-এ ভালো (client-side ID generation সম্ভব, merge conflict এড়ানো যায়), আর security দিক থেকে sequential integer ID guess করা কঠিন করে তোলে (`/api/orders/1` , `/api/orders/2` — enumeration attack প্রতিরোধ)।
- **Foreign Key**: navigation property গুলোর পেছনে (`CategoryId` , `ProducerId`) FK constraint database লেভেলে enforce হয় (Fluent API configuration দিয়ে, `ShilpoHubBD.Data/Configurations/`)।
- **Normalization**: Entity ভাগ করা হয়েছে (`Product` / `ProductImage` / `ProductVariant` / `ProductVideo`) আলাদা টেবিল, একটা Product-এর একাধিক Image রাখার জন্য — এটা 1NF/2NF মেনে ডেটা duplication এড়ানো (repeating group একটা কলামে না রেখে আলাদা child টেবিলে রাখা)।
- **Indexes**: search/filter column-এ (যেমন `Product.Slug` , `Product.CategoryId`) সাধারণত index লাগানো থাকে schema configuration-এ পারফরম্যান্সের জন্য (`ExistsBySlugAsync` এর মতো query দ্রুত চালানোর জন্য)।

Part 7 — Authentication & Authorization Deep Dive

7.1 Authentication vs Authorization (এই প্রজেক্টে concrete উদাহরণ)

- **Authentication** ("তুমি কে?") — JWT দিয়ে হয়। `[Authorize]` (কোনো role ছাড়া) শুধু চেক করে valid, non-expired, সঠিক signature-এর টোকেন আছে কিনা।
- **Authorization** ("তোমার permission আছে কিনা?") — `[Authorize(Roles = "SuperAdmin")]` (`RolesController`), বা `[Authorize(Roles = RoleNames.GovernmentNGO)]` (Governance cluster) — নির্দিষ্ট role claim চেক করে।
- `[AllowAnonymous]` — কোনো authentication লাগবে না, সম্পূর্ণ public (`AuthController.Register/Login`, `ShipmentsController.Track` (public tracking))।

7.2 কেন Custom JWT, ASP.NET Core Identity না?

প্রজেক্টে **ASP.NET Core Identity** (Microsoft-এর built-in user management framework) ব্যবহার করা হয়নি — বরং হাতে বানানো (`User`, `Role`, `UserRole` Entity + `AuthService` + `JwtTokenService`)। কারণ হতে পারে:

- Full control — Identity একটা ভারী framework অনেক default assumption সহ (যেমন `IdentityUser<TKey>` inheritance), যেখানে এই প্রজেক্টের multi-role/role-switch এর মতো custom business logic দরকার ছিল যেটা plain custom code দিয়ে সহজে করা যায়।
- সহজ testing/understanding — student-level/portfolio প্রজেক্টে auth flow-টা নিজে হাতে বানিয়ে পুরোপুরি বোঝার সুযোগ থাকে।

7.3 JWT Structure ও Claims (`JwtTokenService.GenerateAccessToken`)

```
var claims = new List<Claim>
{
    new(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
    new(JwtRegisteredClaimNames.Email, user.Email),
    new("name", user.FullName),
    new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
};
claims.AddRange(roles.Select(role => new Claim(ClaimTypes.Role, role)));
if (!string.IsNullOrEmpty(activeRole)) claims.Add(new Claim("active_role", activeRole));

var signingKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
var credentials = new SigningCredentials(signingKey, SecurityAlgorithms.HmacSha256);
var token = new JwtSecurityToken(issuer: ..., audience: ..., claims: claims, expires: expiresAt, signingCredentials: credentials);
var accessToken = new JwtSecurityTokenHandler().WriteToken(token);
```

একটা **JWT (JSON Web Token)**-এর ৩টা অংশ (. দিয়ে আলাদা, Base64Url-encoded): **Header** (algorithm/type info) . **Payload** (claims) . **Signature**।

- **sub** (Subject) — user-এর unique ID (`Guid`)।
- **email**, **name** — human-readable info।
- **jti** (JWT ID) — এই নির্দিষ্ট token instance-এর unique ID, token replay/tracking-এর জন্য কাজে লাগতে পারে।
- **role** (multiple হতে পারে) — user-এর সব role, `[Authorize(Roles= ... )]` এই claim-ই চেক করে (`ClaimTypes.Role` .NET-এর well-known claim type URI)।
- **active_role** — custom claim, বলে দেয় এই মুহূর্তে user কোন role হিসেবে সক্রিয় (multi-role user একবারে একটা role-এর ড্যাশবোর্ড দেখে, `SwitchRole` দিয়ে বদলায়)।
- **SymmetricSecurityKey** + **HmacSha256** — **symmetric signing**, মানে একই secret key (`Jwt:Key`) দিয়ে sign আর verify দুটোই হয় (asymmetric/RSA না, যেখানে আলাদা private/public key থাকতো)। সুবিধা: simple, single-server backend-এর জন্য যথেষ্ট। সীমাবদ্ধতা: এই secret key অন্য কোনো service-এর সাথে শেয়ার করলে সেই service token forge-ও করতে পারবে (asymmetric হলে শুধু public key দিয়ে verify করতে পারতো, sign করতে পারতো না)।
- **Signature** নিশ্চিত করে token-টা tamper হয়নি — client যদি payload-এর ভেতর **role: SuperAdmin** বসিয়ে দেয় নিজে নিজে, signature আর match করবে না, `ValidateIssuerSigningKey` সেটা ধরে ফেলবে।

Access Token একটা stateless credential — server কোথাও এটা store করে না, শুধু validate করে (signature + expiry check) প্রতিটা request-এ। এইজন্যই lifetime ছোট রাখা হয় (`AccessTokenMinutes: 15`) — কারণ কোনোভাবে leak হলেও বেশিক্ষণ valid থাকবে না।

7.4 Refresh Token — কেন দরকার, কীভাবে কাজ করে

Access token মাত্র ১৫ মিনিট — প্রতি ১৫ মিনিটে user কে আবার password দিয়ে login করানো UX-এর জন্য খারাপ। তাই **Refresh Token** — দীর্ঘমেয়াদী (৭ দিন, `RefreshTokenDays`), যেটা দিয়ে নতুন access token নেওয়া যায়, password ছাড়াই।

```

public string GenerateRefreshTokenValue()
{
    var bytes = RandomNumberGenerator.GetBytes(64);
    return Convert.ToBase64String(bytes);
}

public string HashToken(string rawToken)
{
    var bytes = Encoding.UTF8.GetBytes(rawToken);
    var hash = SHA256.HashData(bytes);
    return Convert.ToHexString(hash);
}

```

- `RandomNumberGenerator.GetBytes(64)` — **cryptographically secure random bytes** (৫১২ বিট এনট্রপি) — `Random` ক্রাস না, কারণ `Random` predictable, security-sensitive token বানানোর জন্য অনুপযুক্ত।
- Refresh token **raw** (plain) আকারে শুধু একবার client-কে পাঠানো হয় — database-এ কখনো raw token save হয় না, শুধু তার **SHA-256 hash** সেভ হয় (`RefreshToken.TokenHash`)। এটা ঠিক password hashing-এর মতো যুক্তি — যদি কখনো database leak হয়, attacker raw refresh token গুলো পাবে না, শুধু hash পাবে (যেখান থেকে reverse করে original বের করা প্রায় অসম্ভব)।

Refresh Flow (`AuthService.RefreshTokenAsync`)

```

var hash = _tokenService.HashToken(refreshToken);
var stored = await _refreshTokenRepository.GetByTokenHashAsync(hash, cancellationTokens);
if (stored is null) throw new UnauthorizedAccessException( ... );
if (stored.IsRevoked)
{
    await _refreshTokenRepository.RevokeAllActiveForUserAsync(stored.UserId, ipAddress, "Reuse of a revoked refresh token was detected.", cancella
    throw new UnauthorizedAccessException( ... );
}
if (stored.IsExpired) throw new UnauthorizedAccessException( ... );
...
// নতুন refresh token ইস্যু, পুরনোটা revoke করে "replaced by" লিংক রাখে
stored.RevokedAt = DateTime.UtcNow;
stored.ReplacedByTokenHash = newHash;
stored.ReasonRevoked = "Rotated on refresh.";

```

Refresh Token Rotation + Reuse Detection — একটি advanced/security-conscious pattern:

1. Client raw refresh token পাঠায়, server hash করে DB-তে match খোঁজে (raw token কখনো DB-তে store না থাকায় direct lookup সম্ভব না, hash দিয়েই মিলাতে হয়)।
2. প্রতিবার refresh করার সময়, **পুরনো token revoke করে একটি নতুন token issue করা হয়** ("rotation") — মানে একটি refresh token একবারই ব্যবহারযোগ্য।
3. **Reuse Detection:** যদি কেউ একটা **already-revoked** refresh token আবার ব্যবহারের চেষ্টা করে — এটা একটা strong সংকেত যে token টা চুরি হয়ে গেছে (আসল owner তো নতুনটা ব্যবহার করবে, পুরনোটা না — তাহলে এই request নিশ্চয়ই attacker-এর, যে চুরি করা পুরনো token ব্যবহার করছে) — তখন সিস্টেম **ওই user-এর সব active refresh token একসাথে revoke** করে দেয় (`RevokeAllActiveForUserAsync`), মানে legitimate user সহ সবাইকে আবার login করতে হবে — কিন্তু এতে attacker-এর access সাথে সাথে বন্ধ হয়ে যায়।

7.5 Password Hashing

```

PasswordHash = _passwordHasher.Hash(request.Password),
...
if (user is null || !_passwordHasher.Verify(request.Password, user.PasswordHash)) throw new UnauthorizedAccessException( ... );

```

`IPasswordHasher` interface (implementation সাধারণত ASP.NET Core-এর `Rfc2898DeriveBytes` /BCrypt-স্টাইল salted hash ব্যবহার করে — Infrastructure layer-এ)। **কখনো plain text password store হয় না**। Login-এর সময় ইউজারের দেওয়া password আবার hash করে **stored hash-এর সাথে compare** হয় (`Verify`) — reverse করে original password বের করার চেষ্টা হয় না (one-way hashing এর মূল ধারণা)। Salt ব্যবহার করার ফলে (আধুনিক hasher-এ built-in) একই password থাকা দুইজন user-এর hash আলাদা হবে (rainbow-table attack প্রতিরোধ)।

7.6 Password Reset Flow

```
public async Task ForgotPasswordAsync(ForgotPasswordRequest request, CancellationToken cancellationToken)
{
    var user = await _userRepository.GetByEmailWithRolesAsync(email, cancellationToken);
    if (user is null) return; // ← email exist করে কিনা বলা হয় না!
    var rawToken = _tokenService.GenerateRefreshTokenValue();
    var hash = _tokenService.HashToken(rawToken);
    ... resetToken সেভ, ExpiresAt = now + 1 hour ...
    var resetLink = $"{baseUrl}?email=...&token={Uri.EscapeDataString(rawToken)}";
    await _emailSender.SendPasswordResetEmailAsync(user.Email, resetLink, cancellationToken);
}
```

Security-conscious design choices:

- Email registered না থাকলেও **silently return** করে (কোনো error/different response দেয় না) — **user enumeration attack প্রতিরোধ**: attacker যদি বারবার আলাদা email দিয়ে forgot-password কল করে বোঝার চেষ্টা করে কোন কোন email registered, response সবসময় same হওয়ায় বুঝতে পারবে না।
- Reset token-ও refresh token-এর মতোই hash করে store হয়, ১ ঘণ্টার মধ্যে expire।
- Password reset হলে (**ResetPasswordAsync**) সেই user-এর **সব active refresh token revoke** করা হয় (**RevokeAllActiveForUserAsync(..., "Password was reset.", ...)**)— যুক্তি: password change মানে সম্ভবত account compromise হয়েছিল বা user সাবধানতা নিচ্ছে, তাই আগের সব session invalidate করে দেওয়া নিরাপদ।

7.7 Multi-Role & Switch Role

AuthController.SwitchRole → **AuthService.SwitchRoleAsync**:

```
var roleNames = user.UserRoles.Select(ur => ur.Role.Name).ToList();
if (!roleNames.Contains(role, StringComparer.Ordinal.IgnoreCase))
    throw new UnauthorizedAccessException("You do not hold the requested role.");
return await IssueTokensAsync(user, roleNames, role, ipAddress: null, cancellationToken);
```

User একাধিক role হোল্ড করতে পারে (**Producer** + **Tourist** একসাথে), কিন্তু নির্দিষ্ট সময়ে একটাই **active role** (**active_role** claim)। **SwitchRole** চেক করে requested role আদৌ user-এর আছে কিনা, থাকলে **নতুন access+refresh token** issue করে যার **active_role** claim বদলানো — পুরনো token invalidate হয় না automatically এখানে (নতুন refresh token যোগ হয়, পুরনোটো থেকে যায় unless explicitly revoke করা হয়) — frontend **নতুন** token সেট করে দেয়, এরপরের request গুলোতে **নতুন** active_role দেখা যাবে।

7.8 Middleware-এ Role Enforcement — Pipeline Recap

UseAuthentication() → JWT পড়ে, valid হলে **HttpContext.User** (a **ClaimsPrincipal**) populate করে সব claim দিয়ে। **UseAuthorization()** → এরপর, প্রতিটা endpoint-এর **[Authorize]** / **[Authorize(Roles=...)]** attribute চেক করে **HttpContext.User**-এর claims-এর বিপরীতে — মিললে request পাস, না মিললে যথাক্রমে **401 Unauthorized** (authentication fail) বা **403 Forbidden** (authenticated কিন্তু authorized না) রিটার্ন করে।

7.9 SignalR + JWT (Special Case)

```
options.Events = new JwtBearerEvents
{
    OnMessageReceived = context =>
    {
        var accessToken = context.Request.Query["access_token"];
        if (!string.IsNullOrEmpty(accessToken) && context.HttpContext.Request.Path.StartsWithSegments("/hubs"))
            context.Token = accessToken;
        return Task.CompletedTask;
    },
};
```

ব্রাউজারের WebSocket API **Authorization** header সাপোর্ট করে না handshake-এর সময়, তাই SignalR client library টোকেন **query string**-এ পাঠায় (**wss://.../hubs/messaging?access_token=xxx**)। এই event handler intercept করে সেই query param থেকে টোকেন তুলে normal bearer token হিসেবে সেট করে দেয় — কিন্তু শুধু **/hubs** পাথের জন্য (অন্য কোনো normal REST endpoint-এ কেউ query string দিয়ে token পাঠালে সেটা গ্রহণ করা হয় না, কারণ URL query string browser history/server log-এ থেকে যেতে পারে — REST এন্ডপয়েন্টের জন্য এটা এড়ানো নিরাপদ, কিন্তু SignalR-এর জন্য এটাই একমাত্র practical উপায়)।

Part 8 — Module-by-Module Deep Dive

প্রতিটা business domain / cluster-এর জন্য: overview, ৪টা representative controller-এর deep dive (real code সহ), বাকি controller-দের reference table, আর একটা module-specific viva question bank + tricky questions। মনে রাখবে — প্রতিটা module একই core pattern (Part 1-7) follow করে, তাই এখানে বারবার basic pattern ব্যাখ্যা না করে সরাসরি ওই module-এর business logic আর interesting design decision নিয়ে কথা বলা হয়েছে।

ShilpoHubBD Viva Prep — Cluster: Marketplace & Commerce

1. Cluster Overview

Ei cluster ta ShilpoHubBD er "core business" — je part ta actual taka-poysha wala transaction handle kore. Ekhane ache product browsing (ProductsController, CategoriesController, DistrictsController), shopping flow (CartController → OrdersController → PaymentsController), post-purchase feedback (ReviewsController, WishlistController), producer-side selling tools (ProducerOrdersController, InventoryController, ProducerStoriesController, ProducerFollowsController, ProducerComparisonController, WorkshopGalleryController, PortfoliosController), alternative selling channels (AuctionsController, LiveShoppingController, CustomOrdersController, QuotationsController for B2B bulk quotes), support/dispute handling (ComplaintsController), and discovery/AI-assist layers (SearchController, RecommendationsController, AIShoppingController, AnalyticsController, CraftStoriesController). Total 24 ta controller — ekta e-commerce marketplace er full lifecycle: browse → cart → checkout → payment → fulfillment → review/return/refund, plus producer-side order management ebong niche-market selling tools (auction, live-shopping, custom order, B2B quotation).

Typical request flow shob jay○ঐ same pattern follow kore, jeta already onno cluster gulote o dekha gese: Client → Controller (thin, shudhu DTO নেয়, [Authorize] / [EnableRateLimiting] attribute check kore, ekta Service method call kore) → before that, ValidationFilter (global IAsyncActionFilter) automatically FluentValidation validator run kore action argument er upor (CreateProductRequest , CheckoutRequest , etc.) — fail korle exception throw kore, controller-e kono ModelState.IsValid check likhte hoy na → Service (business logic layer — entity fetch kore Repository theke, business rule check kore, throw kore NotFoundException / ConflictException jodi rule break hoy, entity mutate kore, manually Entity → DTO map kore private ToDto / ToListItemDto helper diye — NO AutoMapper) → Repository (EF Core wrap kore — .Include()/ .ThenInclude() , .AsSplitQuery() , EF.Functions.ILike() case-insensitive search-er jonno, pagination .Skip().Take() + .CountAsync()) → ShilpoHubDbContext → Postgres (Supabase hosted) → result DTO hishebe Ok(...) / CreatedAtAction(...) return hoy. Exception hole GlobalExceptionHandler (global IExceptionHandler) segula RFC7807 ProblemDetails e map kore (400/404/409/401/500).

Ei cluster er modhhe deep interconnection ache: Product ekta central entity — Cart, Order, Review, Wishlist, Auction, Inventory, ProducerOrders shobkichu Product ke reference kore (ProductId FK). Order creation (checkout) ekta perfect example — ekhane Cart (source of items), Product/ProductVariant (stock check + decrement + price snapshot), District (shipping address validation), sob ekshathe touch hoy — CartService ebong OrderService dujon e IProductRepository/ICartRepository use kore. Payment abar Order-er upor depend kore (OrderId diye initiate hoy). Ekhane ekta important design decision — order er moddhe je OrderItem.ProducerStatus ache seta order-level Order.Status theke আলদা track hoy, karon ekta order-e multiple producer-er product thakte pare — tai each producer nijer part ta independently accept/ship/deliver korte pare (ProducerOrdersController diye), kintu overall order-level status (Pending → Processing → Shipped → Delivered) SuperAdmin manually drive kore. Erom "polyglot" ownership model — customer + admin + individual producers shobai eki Order-ke different angle theke dekhe — ei cluster er sobcheye interesting architectural decision.

2. Deep Dive — 4 Representative Controllers

2.1 ProductsController + ProductService + ProductRepository

File: ShilpoHubBD.Api/Controllers/ProductsController.cs

Route table:

Verb	Route	Auth	Roles
GET	/api/products	AllowAnonymous	— (rate-limited "read")
GET	/api/products/featured	AllowAnonymous	—
GET	/api/products/trending	AllowAnonymous	—
GET	/api/products/mine	Authorize	Producer, SuperAdmin
GET	/api/products/slug/{slug}	AllowAnonymous	—
GET	/api/products/{id}	AllowAnonymous	—
POST	/api/products	Authorize	Producer, SuperAdmin
PUT	/api/products/{id}	Authorize	Producer, SuperAdmin
DELETE	/api/products/{id}	Authorize	Producer, SuperAdmin
PATCH	/api/products/{id}/featured	Authorize	SuperAdmin
POST	/api/products/{productId}/variants	Authorize	Producer, SuperAdmin
PUT	/api/products/{productId}/variants/{variantId}	Authorize	Producer, SuperAdmin
DELETE	/api/products/{productId}/variants/{variantId}	Authorize	Producer, SuperAdmin
POST	/api/products/bulk	Authorize	Producer, SuperAdmin
POST/PUT/DELETE	/api/products/{productId}/videos[/{videoId}]	Authorize	Producer, SuperAdmin
PATCH	/api/products/{id}/handmade-verification	Authorize	SuperAdmin

Controller class [EnableRateLimiting("read")] diye annotate kora — mane shob GET-heavy public endpoint global "read" rate-limit policy-r under e pore, scraping/abuse thekano jay.

Pagination + filter + sort (Repository, `ProductRepository.GetPagedAsync`):

```
// ShilpoHubBD.Data/Repositories/ProductRepository.cs
var products = WithDetails().Where(p => p.IsActive);

if (!string.IsNullOrEmpty(query.Search))
{
    var term = $"{query.Search.Trim()}%";
    products = products.Where(p => EF.Functions.ILike(p.Name, term) || EF.Functions.ILike(p.Description, term));
}
...
products = query.SortBy switch
{
    ProductSortOption.PriceLowToHigh => products.OrderBy(p => p.Price),
    ProductSortOption.Popular => products.OrderByDescending(p => p.ViewCount + p.SalesCount * 5),
    ProductSortOption.TopRated => products.OrderByDescending(p => p.AverageRating).ThenByDescending(p => p.ReviewCount),
    _ => products.OrderByDescending(p => p.CreatedAt),
};

var totalCount = await products.CountAsync(cancellationToken);
var items = await products.Skip((query.Page - 1) * query.PageSize).Take(query.PageSize).ToListAsync(cancellationToken);
```

Explanation: `WithDetails()` ekta private helper jeta

`Include(Category)/Include(District)/Include(Producer)/Include/Images)/Include(Variants)/Include(Videos)` shob eksathe kore, ebong `.AsSplitQuery()` call kore — eta important, karon 5-6 ta collection navigation (`Images`, `Variants`, `Videos`) একসাথে single query-te `Include` korle SQL-e cartesian product/row explosion hoy (each combination of image×variant×video ekta row banay) — `AsSplitQuery()` EF Core ke bole je eta ekta single JOIN query na kore, multiple separate SELECT query চালাক (ekta base query + ekta per collection), fole row-count bloat hoy na. `EF.Functions.ILike` — Postgres-specific case-insensitive `LIKE`, `%term%` wildcard diye substring search. Filter chain-e each `if` condition query-ke progressively narrow kore (deferred execution — LINQ query builder; DB e hit hoy na jotokhon na `ToListAsync` / `CountAsync` call hoy). `CountAsync` + `Skip().Take()` — classic offset pagination, kintu note: eta 2 ta separate DB round-trip (count + page fetch).

`GetByIdAsync` — view count increment (Service):

```
// ProductService.cs
public async Task<ProductDto> GetByIdAsync(Guid id, CancellationToken cancellationToken)
{
    var product = await _productRepository.GetByIdAsync(id, cancellationToken)
        ?? throw new NotFoundException("Product not found.");

    product.ViewCount++;
    await _productRepository.SaveChangesAsync(cancellationToken);

    return await ToDtoAsync(product, cancellationToken);
}
```

Explanation: Product na paille `NotFoundException` throw hoy → `GlobalExceptionHandler` eta 404 e map kore. Product paille simple `ViewCount++` (in-memory mutate, EF change tracker automatically eta "Modified" hishebe detect kore কারণ entity already tracked) তাৎক্ষণিক `SaveChangesAsync` call kore UPDATE query fire kore. Ei pattern-er ekta viva-worthy gotcha: প্রতিবার GET call-e ekta extra UPDATE query hocche — high traffic e eta N+1-er moto na hoile o unnecessary write load create korte pare, ebong race condition (nicher tricky section dekho).

Manual DTO mapping (`ToDtoAsync`) — extra cross-repository query kore `HasCraftStory` / `HasProducerStory` flag set korar jonno:

```
private async Task<ProductDto> ToDtoAsync(Product product, CancellationToken cancellationToken)
{
    var hasCraftStory = await _craftStoryRepository.ExistsByCategoryIdAsync(product.CategoryId, cancellationToken);
    var hasProducerStory = await _producerStoryRepository.ExistsByProducerIdAsync(product.ProducerId, cancellationToken);
    return new ProductDto { ... HasCraftStory = hasCraftStory, HasProducerStory = hasProducerStory, ... };
}
```

Eta AutoMapper na thakar concrete proof — mapping method-e business/query logic (extra repository call) mix kora ache, jeta declarative AutoMapper profile diye express kora kothin/awkward hoy.

Business rules noted: slug uniqueness (`GenerateUniqueSlugAsync` loop kore `-2`, `-3` suffix try kore jotokhon slug unique na hoy), ownership check (`EnsureOwnership` — non-admin producer nijer product chara kichu update/delete korte pare na), soft-delete (`DeleteAsync` `IsActive = false` set kore, row actually delete kore na), `BulkCreateAsync` — per-item try/catch diye partial success allow kore (kono ekta product invalid category dile shudhu shei ekta fail hobe, baki gulo create hobe — result-e `Created` list o `Errors` list dujon e thake).

2.2 OrdersController + OrderService (Checkout flow)

File: `ShilpoHubBD.Api/Controllers/OrdersController.cs`

Route table:

Verb	Route	Auth	Roles
GET	<code>/api/orders</code>	Authorize	any logged-in
GET	<code>/api/orders/{id}</code>	Authorize	owner or SuperAdmin
GET	<code>/api/orders/{id}/tracking</code>	Authorize	owner or SuperAdmin
POST	<code>/api/orders/checkout</code>	Authorize	any logged-in
POST	<code>/api/orders/{id}/cancel</code>	Authorize	owner or SuperAdmin
POST	<code>/api/orders/{id}/return</code>	Authorize	owner or SuperAdmin
POST	<code>/api/orders/{id}/confirm</code>	Authorize	SuperAdmin
POST	<code>/api/orders/{id}/ship</code>	Authorize	SuperAdmin
POST	<code>/api/orders/{id}/deliver</code>	Authorize	SuperAdmin
POST	<code>/api/orders/{id}/return/approve</code>	Authorize	SuperAdmin
POST	<code>/api/orders/{id}/return/reject</code>	Authorize	SuperAdmin
POST	<code>/api/orders/{id}/refund</code>	Authorize	SuperAdmin

Checkout — the most important method in the whole cluster:

```
// OrderService.CheckoutAsync
var cartItems = await _cartRepository.GetByUserIdAsync(userId, cancellationToken);
if (cartItems.Count == 0) throw new ConflictException("Your cart is empty.");

foreach (var cartItem in cartItems)
{
    if (!cartItem.Product.IsActive)
        throw new ConflictException($"'{cartItem.Product.Name}' is no longer available.");

    var availableStock = cartItem.ProductVariant?.Stock ?? cartItem.Product.Stock;
    if (cartItem.Quantity > availableStock)
        throw new ConflictException($"Only {availableStock} unit(s) of '{cartItem.Product.Name}' are available.");
}

var subtotal = cartItems.Sum(c => (c.ProductVariant?.Price ?? c.Product.Price) * c.Quantity);
var order = new Order { ... Status = OrderStatus.Pending, Subtotal = subtotal, Total = subtotal, ... };

foreach (var cartItem in cartItems)
{
    var unitPrice = cartItem.ProductVariant?.Price ?? cartItem.Product.Price;
    order.Items.Add(new OrderItem
    {
        ProductId = cartItem.ProductId, ProductName = cartItem.Product.Name,           // ← SNAPSHOT
        UnitPrice = unitPrice, Quantity = cartItem.Quantity, LineTotal = unitPrice * cartItem.Quantity,
    });

    if (cartItem.ProductVariant is not null) cartItem.ProductVariant.Stock -= cartItem.Quantity;
    else cartItem.Product.Stock -= cartItem.Quantity;

    cartItem.Product.SalesCount += cartItem.Quantity;
}

AddStatusEvent(order, OrderStatus.Pending, "Order placed.");
await _orderRepository.AddAsync(order, cancellationToken);
await _orderRepository.SaveChangesAsync(cancellationToken); // ← single SaveChanges = implicit transaction
await _cartRepository.ClearAsync(userId, cancellationToken);
```

Line-by-line explanation:

- Cart empty check → `ConflictException` → 409.
- Prottek cartItem-er jonno `IsActive` (product delete/deactivate hoye jaite pare cart e rakha obosthay) ebong stock sufficiency check — dujon check pass na hole

loop-e immediately exception throw hoy, order create shuru e hoyna.

- `subtotal` calculate: variant price thakle variant price, na thakle product base price — `??` null-coalescing.
- Notice `order.Items.Add(new OrderItem { ProductName = cartItem.Product.Name, UnitPrice = unitPrice, ... })` — eta **price/name snapshotting**. Product-er `Name` / `Price` future e change hoye gele o purono order-er `OrderItem.ProductName` / `UnitPrice` change hobe na, karon eta already order creation somoy e ekta independent copy hishebe store hoye geche, FK relation diye live lookup na kore.
- Stock decrement — variant thakle variant.Stock, na thakle product.Stock direct minus kora hoy — sob in-memory, EF change tracker eta dhore rakhe.
- `AddStatusEvent` — comment onujayi eta shudhu **new (untracked) Order**-er jonno safe, karon navigation collection-e item add korle already-tracked entity-r khete EF eta UPDATE hishebe misdetect korte pare INSERT er bodole; already-tracked order-er jonno `RecordStatusEventAsync` (direct repository add) use hoy.
- Sobshesh e **ekbar shudhu `SaveChangesAsync()` call hoy** — EF Core internally eta shob change (new Order insert, new OrderItems insert, Product/Variant stock update, new OrderStatusEvent insert) **ekta single DB transaction**-e wrap kore পাঠায়. Explicit `BeginTransaction()` / `Commit()` kono jaygay likha nei — EF-er built-in per-`SaveChanges` atomic transaction e bhorosha kora hoyeche. Shobshesh e cart clear kora hoy — checkout success hole customer-er cart khali hoye jay.

Status-machine style transitions (Confirm/Ship/Deliver/ApproveReturn/RejectReturn/Refund) — shobgulo eki pattern follow kore: `EnsureStatus(order, expectedStatus)` guard call kore (order thik state-e ache kina check), tarpor status change kore, `RecordStatusEventAsync` diye audit trail (`OrderStatusEvent`) add kore, `SaveChangesAsync`.

```
private static void EnsureStatus(Order order, OrderStatus expected)
{
    if (order.Status != expected)
        throw new ConflictException($"Order must be in '{expected}' status for this action (current: '{order.Status}').");
}
```

Eta ekta simple **finite-state-machine guard**: Pending → Processing (Confirm) → Shipped (Ship) → Delivered (Deliver) → ReturnRequested (RequestReturn, customer-side) → Returned (ApproveReturn, stock restore) or back to Delivered (RejectReturn) → Refunded (Refund). Cancel shudhu Pending/Processing obosthay allowed.

Cancel — stock restoration:

```
RestoreStock(order); // item.Product.Stock += item.Quantity (or variant)
CancelProducerFulfillment(order); // per-producer OrderItem.ProducerStatus → Cancelled, jodi already Accepted/Processing na hoy
order.Status = OrderStatus.Cancelled;
```

Business rule: cancel korle stock ফেরত jog hoy, ebং `OrderItem.ProducerStatus` (Pending/Accepted/Processing obosthay thakle) `Cancelled` -e set hoy — jate producer-side dashboard e stale "action needed" na dekhay.

2.3 CartController + CartService

File: `ShilpoHubBD.Api/Controllers/CartController.cs` — pura controller `[Authorize]` (class-level), mane cart shudhu logged-in user-er.

Route table:

Verb	Route	Auth
GET	<code>/api/cart</code>	any logged-in
GET	<code>/api/cart/summary</code>	any logged-in
POST	<code>/api/cart</code>	any logged-in
PUT	<code>/api/cart/{itemId}</code>	owner only
DELETE	<code>/api/cart/{itemId}</code>	owner only
DELETE	<code>/api/cart</code>	any logged-in (clears own cart)

AddOrIncrementAsync — "add to cart" ki real e kore:

```

public async Task<CartItemDto> AddOrIncrementAsync(Guid userId, Guid productId, Guid? productVariantId, int quantity, CancellationToken cancellati
{
    var product = await _productRepository.GetByIdAsync(productId, cancellationTokens)
        ?? throw new NotFoundException("Product not found.");
    if (!product.IsActive) throw new ConflictException("This product is no longer available.");

    var variant = productVariantId.HasValue
        ? product.Variants.FirstOrDefault(v => v.Id == productVariantId.Value) ?? throw new NotFoundException("Variant not found for this product.
        : null;

    var existing = await _cartRepository.GetAsync(userId, productId, productVariantId, cancellationTokens);
    EnsureStockAvailable(product, variant, (existing?.Quantity ?? 0) + quantity);

    if (existing is not null)
    {
        existing.Quantity += quantity; // increment, not duplicate row
        await _cartRepository.SaveChangesAsync(cancellationTokens);
        return ToDto(existing);
    }

    var item = new CartItem { UserId = userId, ProductId = productId, ProductVariantId = productVariantId, Quantity = quantity, ... };
    await _cartRepository.AddAsync(item, cancellationTokens);
    await _cartRepository.SaveChangesAsync(cancellationTokens);
    return ToDto(item);
}

```

Explanation: "AddOrIncrement" naam theke bujha jay — eta idempotent-ish upsert: same (user, product, variant) combination ager cart-e thakle new row create na kore purono row-er **Quantity** barano hoy (duplicate cart line item thake na). Stock check **existing.Quantity + new quantity** diye hoy — mane "already 3 ta cart-e ache, aro 2 ta add korte chaile total 5 ta available stock-er against check hobe"; 5 ta na thakle **ConflictException**. Variant lookup product-er already-loaded **Variants** navigation theke in-memory **FirstOrDefault** diye kora hoy (karo extra query lage na, karon **ProductRepository.GetByIdAsync** already **.Include(Variants)** kore niye ase).

UpdateQuantityAsync / RemoveItemAsync — **EnsureOwnership(item, userId)** diye check kore je cart item shei user-erই, na hole **UnauthorizedAccessException** (401 mapped).

GetSummaryAsync — full cart list fetch kore, ত্রিপুর in-memory **Sum()** diye **ItemCount**, **TotalQuantity**, **Subtotal** calculate kore — **kono separate SQL aggregate query na**, shob application-level LINQ-to-Objects.

2.4 AuctionsController + AuctionService (Bidding logic)

File: **ShilpoHubBD.Api/Controllers/AuctionsController.cs**

Route table:

Verb	Route	Auth	Roles
GET	/api/auctions	AllowAnonymous	—
GET	/api/auctions/{id}	AllowAnonymous	—
POST	/api/auctions	Authorize	Producer, SuperAdmin
POST	/api/auctions/{id}/bids	Authorize	any logged-in
POST	/api/auctions/{id}/cancel	Authorize	owner Producer or SuperAdmin

PlaceBidAsync:

```

public async Task<AuctionDto> PlaceBidAsync(Guid id, Guid bidderId, PlaceBidRequest request, CancellationToken cancellationToken)
{
    var auction = await _auctionRepository.GetByIdAsync(id, cancellationToken) ?? throw new NotFoundException("Auction not found.");
    SyncStatus(auction);
    // lazily advance Scheduled→Active→Ended based on wall-clock

    if (auction.ProducerId == bidderId) throw new ConflictException("You cannot bid on your own auction.");

    switch (auction.Status)
    {
        case AuctionStatus.Scheduled: throw new ConflictException("This auction has not started yet.");
        case AuctionStatus.Ended:      throw new ConflictException("This auction has already ended.");
        case AuctionStatus.Cancelled:  throw new ConflictException("This auction has been cancelled.");
    }

    var minimumAmount = auction.CurrentPrice + auction.MinBidIncrement;
    if (request.Amount < minimumAmount) throw new ConflictException($"Your bid must be at least {minimumAmount}.");

    var bid = new AuctionBid { AuctionId = id, BidderId = bidderId, Amount = request.Amount, CreatedAt = DateTime.UtcNow };
    await _auctionRepository.AddBidAsync(bid, cancellationToken);
    auction.CurrentPrice = request.Amount;
    await _auctionRepository.SaveChangesAsync(cancellationToken);
    ...
}

```

Explanation: Ekhane kono cron job/background scheduler nei je auction-er status automatically Scheduled→Active→Ended kore. Bodole eta **lazy/on-demand sync** — `SyncStatus(auction)` protyekbar `GetByIdAsync` / `PlaceBidAsync` / `CancelAsync` call howar shomoy chalano hoy, current UTC time compare kore `StartAt` / `EndAt` -er shathe, দরকার hole status update kore ebং `SaveChangesAsync` kore. Eta interesting design trade-off: simple (no background worker dependency), kintu jodi kono API call e na ashe (low-traffic auction), auction "Ended" state-e formally transition e hobe na jotokhon na keu eta hit kore — যদিও `TimeRemainingSeconds()` calculation always real wall-clock use kore tai UI-te already ended dekhabe.

```

private static bool SyncStatus(Domain.Entities.Auction.Auction auction)
{
    var now = DateTime.UtcNow;
    if (auction.Status == AuctionStatus.Scheduled && now ≥ auction.StartAt && now < auction.EndAt)
    {
        auction.Status = AuctionStatus.Active; changed = true;
    }

    if ((auction.Status == AuctionStatus.Scheduled || auction.Status == AuctionStatus.Active) && now ≥ auction.EndAt)
    {
        var topBid = auction.Bids.OrderByDescending(b => b.Amount).ThenBy(b => b.CreatedAt).FirstOrDefault();
        auction.Status = AuctionStatus.Ended;
        auction.WinnerId = topBid?.BidderId;
        changed = true;
    }

    return changed;
}

```

Winner determine kora hoy highest `Amount`, tie hole earliest `CreatedAt` (first bidder wins tie) — in-memory sort of already-loaded `Bids` collection (repository `GetByIdAsync` te `.Include(Bids)` kora ache dhore newa jai).

Business rules: seller nijer product-e bid dite pare na (self-bid guard), bid minimum `CurrentPrice + MinBidIncrement` -er niche hote parbe na, ekbar bid place hoye gele seta immutable (kono "retract bid" endpoint nei), `CurrentPrice` protibar bid-e update hoy tai next minimum bid always latest highest bid-er upor base kore.

3. Reference Table — Remaining Controllers

Controller	Route prefix	Key endpoints	Auth/Roles
CategoriesController	<code>/api/categories</code>	GET, GET <code>{id}</code> , POST/PUT/DELETE	Public read; SuperAdmin write
PaymentsController	<code>/api/payments</code>	POST, GET <code>order/{orderId}</code> , GET <code>{id}</code> , POST <code>{id}/verify</code> , POST <code>{id}/callback</code> (AllowAnonymous, gateway webhook), POST <code>{id}/refund</code> (SuperAdmin)	Authorize (mostly), callback anonymous
ReviewsController	<code>/api/reviews</code>	GET <code>product/{id}</code> , <code>heritage-place/{id}</code> , <code>service/{id}</code> , POST/PUT (Authorize), DELETE (owner/admin)	Public read; Authorize write

Controller	Route prefix	Key endpoints	Auth/Roles
WishlistController	/api/wishlist	GET, POST, DELETE {productId} , POST {productId}/move-to-cart	Authorize (class-level)
LiveShoppingController	/api/live-events	GET, GET {id} , POST (Producer/Admin), {id}/start , {id}/end , {id}/comments , {id}/reactions , {id}/buy	Public read; Producer/Admin manage; Authorize for interaction
CustomOrdersController	/api/custom-orders	GET {id} , mine/customer , mine/producer (Producer), POST, {id}/respond (Producer), {id}/cancel	Authorize (class-level)
QuotationsController	/api/quotations	POST, GET (BusinessPartner), GET received (Producer), GET {id} , GET {id}/compare , POST {id}/responses (Producer), POST {id}/responses/{id}/decision , POST {id}/cancel	BusinessPartner/Producer/SuperAdmin split by role
InventoryController	/api/inventory	POST products/{id}/adjust , GET products/{id}/history , GET low-stock	Producer/SuperAdmin (class-level)
ProducerOrdersController	/api/producer/orders	GET, GET {orderId} , accept / reject / processing / ship / deliver , customers , analytics/*	Producer only (class-level)
ProducerFollowsController	/api/follows/producers	GET, POST {producerId} , DELETE {producerId}	Authorize
ProducerComparisonController	/api/producer-comparison	POST compare	BusinessPartner/SuperAdmin
ProducerStoriesController	/api/producer-stories	GET {producerId} , PUT {producerId} (upsert), DELETE (Admin)	Public read; Producer/Admin write
CraftStoriesController	/api/craft-stories	GET category/{id} , POST/PUT/DELETE (SuperAdmin)	Public read; Admin write
ComplaintsController	/api/governance/complaints	GET, GET {id} , POST, PUT, {id}/updates , {id}/assign , {id}/resolve , {id}/link-flag , DELETE	GovernmentNGO/SuperAdmin only
SearchController	/api/search	GET ?q=	Public, rate-limited "read"
RecommendationsController	/api/recommendations	GET (personalized), GET similar/{productId}	Public (optional auth for personalization)
WorkshopGalleryController	/api/producers/{producerId}/workshop-gallery	GET, POST (Producer/Admin), DELETE {itemId}	Public read; Producer/Admin write
PortfoliosController	/api/portfolios	GET me , GET {academyMemberProfileId} , PUT me , PUT me/visibility , me/projects CRUD	Authorize for own; public for viewing others
DistrictsController	/api/districts	GET	Public

Controller	Route prefix	Key endpoints	Auth/Roles
AIShoppingController	/api/ai-shopping	POST <code>gift-recommendations</code> , <code>fashion-matches</code> , <code>interior-preview</code> , <code>translate</code>	Public
AnalyticsController	/api/analytics	GET <code>purchases</code> , <code>spending</code> , <code>favorite-categories</code>	Authorize

4. Viva Question Bank

Q1. Cart-এ item add korle backend e কি হয়, step by step bolo.

- **Short answer:** `CartController.AddItem` → `CartService.AddOrIncrementAsync` product fetch kore, active/stock check kore, already same product+variant cart-e thakle quantity increment kore noile new `CartItem` row create kore, `SaveChangesAsync` diye persist kore.
- **Detailed explanation:** Product na paoya gele `NotFoundException` (404), inactive hole `ConflictException` (409). Variant specify korle segula product-er already-loaded `Variants` navigation theke validate hoy. Existing cart row-er khoje `_cartRepository.GetAsync(userId, productId, productVariantId, ...)` call hoy — eta (user, product, variant) composite match kore. Match paille row-er `Quantity` barano hoy (duplicate row toiri hoy na), na paille notun `CartItem` insert.
- **Project example:** `CartService.cs` — `AddOrIncrementAsync` method, `EnsureStockAvailable` helper.
- **Follow-up:** *Ekই product 2 bar different variant diye add korle ki 2 ta row hobe?* — Ha, karon `(userId, productId, productVariantId)` composite key-er upor match kora hoy, tai different variant = different `CartItem` row.

Q2. Checkout kora hole order create howar age কোন কোন validation hoy?

- **Short answer:** Cart khali kina, product active kina, এবং প্রতিটি item-er quantity available stock-er beshi kina — ei tinta check pass korle tobe order create hoy.
- **Detailed explanation:** `OrderService.CheckoutAsync` -e first `cartItems.Count == 0` check, তারপর loop diye protyek cart item-er `Product.IsActive` এবং `cartItem.Quantity > availableStock` check kora hoy. Kono ekta fail korle loop-er modheই `ConflictException` throw hoy — mane order creation kono onnayo shurui hoyna, DB-te kono partial write hoyna.
- **Project example:** `OrderService.CheckoutAsync`, first 15 lines.
- **Follow-up:** *Ei validation loop-e jodi 5 nombor item invalid hoy, tahole 1-4 nombor item ki process hoye jabe?* — Na, loop shudhu validate kore, kono mutation kore na; mutation (order/orderItem create, stock decrement) alada পরবর্তী loop-e hoy, jeta first loop pura pass korার পরই shuru hoy.

Q3. Order-e product-er price ki live product table theke ashe, naki snapshot?

- **Short answer:** Snapshot — `OrderItem.UnitPrice`, `ProductName`, `ProductImageUrl`, `VariantName` sob order creation-er shomoy e copy kore rakha hoy, seta pore product-er price change hole o change hoy na.
- **Detailed explanation:** `OrderItem` entity-te `ProductId` FK thakle o `ProductName` / `UnitPrice` -er নিজস্ব column ache. Checkout korার shomoy `CheckoutAsync` e `var unitPrice = cartItem.ProductVariant?.Price ?? cartItem.Product.Price;` diye current price snapshot kore `OrderItem.UnitPrice` e set kora hoy. Eta historical accuracy-r jonno joruri — customer je দামে kine sheita order history-te thik seim thakte hobe, producer price পরে barale/komale purono order affected hobe na.
- **Project example:** `OrderService.CheckoutAsync` — `order.Items.Add(new OrderItem { ProductName = cartItem.Product.Name, UnitPrice = unitPrice, ... });`; `Domain.Entities.Commerce.OrderItem.cs`.
- **Follow-up:** *Tahole order history-te product delete hoye gele ki dekhabe?* — `OrderItem` er own `ProductName` / `ProductImageUrl` field theke dekhabe, `Product` navigation null/broken hoyeo order display thik থাকবে (though `OrderItem.Product` FK relation exists — soft-delete pattern e product row thake, `IsActive=false` hoy, hard delete kono controller-e nei).

Q4. Order checkout-e explicit database transaction (`BeginTransaction`) ki use kora hoyeche?

- **Short answer:** Na, explicit transaction nei — ekta single `SaveChangesAsync()` call-er upor bhorosha kora hoyeche, jeta EF Core internally atomic DB transaction hishebe wrap kore pathay.
- **Detailed explanation:** `CheckoutAsync` -e Order, shobgulo `OrderItem`, `StatusEvent`, এবং Product/Variant stock update — shob eksathe `DbContext`-e track hoy, তারপর একবারই `SaveChangesAsync` call hoy. EF Core by default protyekta `SaveChanges` call-ke nijeই ekta transaction-e wrap kore (BEGIN...COMMIT), tai multiple entity change atomic-i hoy — either shob succeed noile shob rollback. Kintu eta shudhu single `SaveChanges` call-er moddhei kaj kore; jodi kokhono duita আলাদা `SaveChangesAsync` call thake (jemon `CheckoutAsync` -er por `_cartRepository.ClearAsync`), segula আলাদা transaction, tai between-them failure hole partial state thakte pare (nicher gotcha section dekho).
- **Project example:** `OrderService.CheckoutAsync` — `await _orderRepository.SaveChangesAsync(cancellationToken); await _cartRepository.ClearAsync(userId, cancellationToken);`
- **Follow-up:** *Jodi `ClearAsync` fail kore `SaveChangesAsync`-er por, tahole ki hobe?* — Order already committed thakbe DB-te, kintu cart clear na hoye purono item soho thake jabe — customer abar order kore felte pare shei cart theke (duplicate order risk), eta ekta real gap.

Q5. Duita user ekই shomoy last stock item order korle ki race condition hote pare?

- **Short answer:** Ha, code-e explicit locking/optimistic concurrency token nei — `Product.Stock -= quantity` in-memory mutate kore, tai high-concurrency-te overselling (negative stock) hote pare.
- **Detailed explanation:** `CheckoutAsync` -e stock check hoy (`cartItem.Quantity > availableStock`) load kora entity-r upor, kintu ei check এবং porborti `SaveChangesAsync` (jekhane actual UPDATE query pathano hoy)-er modhhe kono DB-level lock nei. Duita concurrent request dujon e same product row read korte pare (stock=1), dujon e check pass korte pare, dujon e stock decrement kore save korte pare — jodi EF-e `RowVersion` /concurrency token na thake, second `SaveChanges` first-take overwrite kore dite pare (last-write-wins), fole stock -1 hoye jete pare.
- **Project example:** `OrderService.CheckoutAsync` stock check + `cartItem.Product.Stock -= cartItem.Quantity;`; `Product` entity-te kono `[ConcurrencyCheck]` / `RowVersion` column dekha jayni.

- **Follow-up:** *Eta kivabe fix kora jete pare?* — Postgres-e `xmin` system column ke EF Core concurrency token banaye, ba `Product` -e explicit `RowVersion / byte[]` timestamp column add kore optimistic concurrency enable kore (DbUpdateConcurrencyException catch kore retry), ba database-level `SELECT ... FOR UPDATE` pessimistic lock, ba atomic `UPDATE Products SET Stock = Stock - @qty WHERE Stock ≥ @qty` diye row-count check kora.

Q6. Auction-e bid place korle minimum bid amount kivabe validate hoy?

- **Short answer:** `minimumAmount = auction.CurrentPrice + auction.MinBidIncrement`; new bid seta theke kom hole `ConflictException`.

- **Detailed explanation:** Auction-er `CurrentPrice` protyektbar successful bid-e update hoy (highest bid = current price), tai next bid always latest highest bid + fixed increment-er upor base kore. Eta prevent kore je keu ektom same ba shamanno beshi bid diye "sniping" na korte pare small increment-e.

- **Project example:** `AuctionService.PlaceBidAsync` — `var minimumAmount = auction.CurrentPrice + auction.MinBidIncrement; if (request.Amount < minimumAmount) throw new ConflictException(...)`.

- **Follow-up:** *Duijon user ek᳚ shomoy bid dile, race condition e ki hote pare?* — Duijon e same `CurrentPrice` read korte pare, dujon᳚ validate pass korte pare against shei purono price, ত᳚ঁরপর dujon e nijer bid save korte cheshta korbe — jei pore `SaveChangesAsync` kore shei "jite" jabe DB-te, kintu first bidder-er thekeo kom bid hote pare jodi first bidder-er bid actually higher chilo but processing order ulta hoy. Eta o ekta unguarded race — no row-level lock/concurrency token.

Q7. Auction status (Scheduled/Active/Ended) kivabe update hoy — background job ache?

- **Short answer:** Na, kono background scheduler/cron nei — status **lazily** sync hoy protyektbar kono API call (GetById/PlaceBid/Cancel) hit howar shomoy, current time compare kore.

- **Detailed explanation:** `SyncStatus(auction)` private static method wall-clock `DateTime.UtcNow` ke `StartAt / EndAt` -er shathe compare kore, dorkar hole status flip kore eb᳚ `SaveChangesAsync` call kore save kore. Eta "on-read" state transition pattern.

- **Project example:** `AuctionService.SyncStatus`, called from `GetByIdAsync`, `PlaceBidAsync`, `CancelAsync`.

- **Follow-up:** *Ekta auction ended hoye gele kintu keu ota kokhono hit na kore, tahole ki winner determine hobe?* — Na, jotokhon na keu shei auction-e GET/bid/cancel hit kore, DB-te `Status` field purono e thak᳚ (`Active / Scheduled`). Eta ekta gap — production e ei jonno ekta background worker (Hangfire/hosted service) proper hoto, kintu erokom kichu nei ei codebase-e.

Q8. Order cancel korle stock ki abar ফেরত jog hoy?

- **Short answer:** Ha, `CancelAsync` e `RestoreStock(order)` call hoy je each `OrderItem` -er `Quantity` abar `Product / ProductVariant.Stock` -e add kore dey.

- **Detailed explanation:** Order shudhu `Pending` ba `Processing` obosthay cancel kora jay (`EnsureStatus` guard-er moto explicit if-check). `Cancel-e RestoreStock` loop kore protyekt item-er quantity stock-e ferot dey, eb᳚ `Product.SalesCount` o komano hoy (`Math.Max(0, ...)`) diye negative na hote deya hoy). Shathe `CancelProducerFulfillment` per-producer `OrderItem.ProducerStatus` ke `Cancelled` -e set kore jodi seta already Pending/Accepted/Processing thake.

- **Project example:** `OrderService.CancelAsync`, `RestoreStock`, `CancelProducerFulfillment`.

- **Follow-up:** *Delivered order cancel kora jabe?* — Na, `if (order.Status ≠ OrderStatus.Pending && order.Status ≠ OrderStatus.Processing) throw ConflictException` — Delivered/Shipped order-er jonno customer-ke "Return" flow use korte hobe (`RequestReturnAsync`), cancel na.

Q9. Ekta order-e multiple producer-er product thakle fulfillment track kivabe hoy?

- **Short answer:** Order-wide `Order.Status` (Admin/logistics driven — Pending→Processing→Shipped→Delivered) eb᳚ per-item `OrderItem.ProducerStatus` (each producer nijer item accept/ship/deliver) — duita alada tracking dimension.

- **Detailed explanation:** `ProducerOrdersController / IProducerOrderService` diye each producer nijer `OrderItem` -gulo dekhte pare (`GetOrders`), accept/reject/processing/ship/deliver korte pare — eta `OrderItem.ProducerStatus` enum update kore, order-wide `Order.Status` na. `OrdersController` -er Confirm/Ship/Deliver hocche Admin-driven overall order status, jeta logistics/shipping-er jonno (mone hoy ekta consolidated shipment).

- **Project example:** `OrderItem.cs` — comment "Per-producer fulfillment tracking, independent of the order-wide Status/tracking ... since a single order can contain items from multiple producers."; `ProducerOrdersController.Accept/Reject/StartProcessing/Ship/MarkDelivered`.

- **Follow-up:** *Ekta order-e 2 ta producer-er item thakle, ekjon ship korle baki jon na korle overall Order.Status ki hobe?* — `Order.Status` change hobe na automatically — seta shudhu SuperAdmin manually `Ship / Deliver` endpoint call korle change hoy, tai ei duita status "desync" hote pare jodi admin-er process producer-level status-er shathe synced na thake — code-e kono automatic aggregation/sync logic nei.

Q10. Review kon kon entity-r upor dewa jay?

- **Short answer:** Product, HeritagePlace, eb᳚ TouristService — tinta আলাদা GET endpoint (`product/{id}`, `heritage-place/{id}`, `service/{id}`) ache, kintu shob᳚ ek᳚ `IReviewService / Review` entity use kore, DTO-te nullable `ProductId / HeritagePlaceId` field diye distinguish hoy.

- **Detailed explanation:** `Create` action-e response-er `ProductId / HeritagePlaceId` kon ta populated seita dekhe `CreatedAtAction` decide kore kon GET route-e redirect kor᳚ — eta ekta polymorphic-ish review entity design.

- **Project example:** `ReviewsController.Create` — `if (result.ProductId.HasValue) return CreatedAtAction(nameof(GetByProduct), ...)`.

- **Follow-up:** *Ek᳚ user ek᳚ product-e duibar review dite parbe?* — Controller/Service level e eta dekha jayni ei read-e explicit unique constraint — DB-level unique index thakle thakte pare, na hole duplicate review allow hote pare (validator level check kora dorkar chilo).

Q11. Wishlist theke direct cart-e move kor᳚ logic ki?

- **Short answer:** `WishlistController.MoveToCart` → `IWishlistService.MoveToCartAsync` — product wishlist theke remove kore, cart-e add kore, ekta atomic-feeling operation hishebe ek᳚ API call-e.

- **Detailed explanation:** Eta convenience wrapper — internally hoyto wishlist item delete + cart add duita step, kintu client-er jonno single POST call.

- **Project example:** `WishlistController.cs` — `POST {productId}/move-to-cart`.

- **Follow-up:** *Move-to-cart kor᳚ shomoy jodi product-e stock na thake, ki hobe?* — `IWishlistService` internally `ICartService` -er moto stock check use korte pare (implementation `IWishlistService` -e, ei research-e file read hoyini deeply, kintu pattern onujayi `ConflictException` expect kora jai jodi stock na thake).

Q12. Custom order (bespoke product) eb᳚ Quotation (B2B RFQ)-er modhhe parthokko ki?

- **Short answer:** CustomOrder ekta individual customer→producer 1-to-1 bespoke request (ekta design/photo diye "amake erokom banai dao"), Quotation hocche BusinessPartner (bulk buyer) একসাথে multiple producer-ke RFQ pathay, quote gula compare kore ekjonke accept kore.

- **Detailed explanation:** `QuotationsController` -e `GetForBusinessPartnerAsync / GetForProducerAsync` duita আলাদা role-based view ache, plus `Compare` endpoint jeta multiple producer-er `QuotationResponse` pashapashi dekhay. CustomOrder-e এককম compare/multi-producer concept nei — seta 1-to-1.

- **Project example:** `QuotationsController.Compare`, `CustomOrdersController.Respond`.
- **Follow-up:** *Quotation-e "Decide" action ki kore?* — `DecideResponseAsync` — BusinessPartner ekta producer-er quotation response accept/reject kore, likely accept korle order/custom-fulfillment trigger hoy (exact logic `QuotationService` -e, deep-dive na kora hoyleo pattern onujayi).

Q13. Payment callback endpoint `AllowAnonymous` keno?

- **Short answer:** Karon eta payment gateway (jemon bKash/SSLCommerz/Stripe) nije server-to-server call kore, gateway-r kache JWT token thake na — tai logged-in user auth bypass kore, gateway ke direct hit korar permission deya hoyeche.
- **Detailed explanation:** `[AllowAnonymous]` `[HttpPost("{id:guid}/callback")]` — payload `JsonElement` hishebe accept kore (raw/flexible JSON, karon gateway-specific schema), `payload.GetRawText()` diye raw string `HandleCallbackAsync` -e pass hoy verification/parsing-er jonno.
- **Project example:** `PaymentsController.Callback`.
- **Follow-up:** *Tahole eta security risk na? Jekeu random `POST /api/payments/{id}/callback` call korle payment verify kore felte pare?* — Eta ekta legitimate gotcha — `HandleCallbackAsync` -er internal implementation e gateway signature/hash verification thaka uchit (jemon gateway-provided secret/signature check) noile eta spoof-able; ei level-er controller code-e seita dekha jayni, tai eta ekta viva-worthy security gap point out korar moto jayga.

Q14. Inventory adjustment (`InventoryController.AdjustStock`) ki `Product.Stock` direct update kore, naki আলাদা ledger?

- **Short answer:** Duita e — `AdjustStockAsync` likely `Product.Stock` update kore ebong ekta `InventoryTransaction` audit row create kore (`GetHistory` endpoint theke bujha jay ekta transaction history ache).
- **Detailed explanation:** `GetHistory` endpoint return kore `List<InventoryTransactionDto>` — mane প্রতিটি adjustment (reason soho, jemon "restock", "damage", "correction") ekta immutable log row hishebe store hoy, just current `Stock` number overwrite na kore.
- **Project example:** `InventoryController.AdjustStock`, `GetHistory`.
- **Follow-up:** *Order checkout-er stock decrement ki eki `InventoryTransaction` ledger-e log hoy?* — `OrderService.CheckoutAsync` code (jeta dekha hoyeche) shudhu direct `Product.Stock -= quantity` kore, kono `InventoryTransaction` row create korar call nei — mane checkout-flow-er stock change ebong manual inventory-adjustment flow duita আলাদা code path, audit ledger-e checkout-triggered stock changes track hoy na — eta ekta consistency gap point out korar moto jayga.

Q15. `ProductsController` ke `[EnableRateLimiting("read")]` diye annotate kora hoyeche keno?

- **Short answer:** Product listing/search endpoints high-traffic, publicly-anonymous hit hote pare (bots, scrapers soho) — rate limiting diye abuse/scraping thekano hoy.
- **Detailed explanation:** `SearchController` -e o eki policy. Global rate limiter policies ("read", "auth" etc.) `Program.cs` -e `AddRateLimiter` diye configure kora — controller-level attribute diye per-controller apply kora hoy.
- **Project example:** `ProductsController` class attribute, `SearchController` class attribute.
- **Follow-up:** *Cart/Orders controller-e keno rate limiting nei?* — Segula already `[Authorize]` -protected, per-user action, high-volume anonymous scraping risk kom, tai "read" policy লাগানো হয়নি — যদিও general auth-related rate-limit policy alada jaygay thakte pare.

Q16. `GetMineAsync` (producer's own products) ebong `GetByIdAsync` (public product view)-er modhhe DTO-r difference ki?

- **Short answer:** Onno kono file na dekhei bola jai — `GetMine` / `GetById` duitai `ProductDto` return kore, kono আলাদা "owner view" DTO nei — same shape shobar jonno.
- **Detailed explanation:** Eta ekta simplification — normally owner-view-e hoyto extra internal field (jemon cost price, margin) thakte pare jeta public-e dekhano thik na, kintu ei codebase-e single `ProductDto` shared, tai jodi kono sensitive field thake bhoবিশেষতঃ add kora hoy, seta accidentally public `GetById`-teo expose hoye jabe — design gotcha.
- **Project example:** `ProductService.GetMineAsync` / `GetByIdAsync`, both call `ToDtoAsync`.
- **Follow-up:** *Eta kivabe better design kora jete pare?* — Alada `ProducerProductDto` / `ownerView` flag diye conditional field, ba separate mapping method.

Q17. Producer follow (`ProducerFollowsController`) ki notification trigger kore?

- **Short answer:** Ei controller/route theke direct dekha jay na — shudhu Follow/Unfollow toggle kore, notification pathano hocche kina seta `IProducerFollowService` -er internal implementation-er upor depend kore (ei research scope-e deep-dive kora hoyni).
- **Detailed explanation:** Route simple: `POST {producerId} follow`, `DELETE {producerId} unfollow`, `GET` own followed-list — CRUD-moto simple join-table pattern (UserId, ProducerId composite).
- **Project example:** `ProducerFollowsController.cs`.
- **Follow-up:** *Ei follow feature Auction/LiveShopping-er shathe kivabe connect hote pare?* — Logically, followed producer notun auction/live-event shuru korle follower-der notify kora uchit — কিন্তু ei controller-level code-e emon linkage dekha jayni.

Q18. `RecommendationsController.GetForMe` -e user logged-in na thakleo kaj kore keno?

- **Short answer:** `CurrentUserIdOrNull` ekta nullable Guid property jeta claim na paille `null` return kore (exception throw kore na, `Authorize` attribute o nei) — mane guest user-der jonyo o "for you" recommendation (probably generic/popular fallback) dekhano hoy.
- **Detailed explanation:** `Guid.TryParse` pattern diye safe-parse kora hoy, `Authorize` chara controller-level — eta deliberate design: personalized recommendation logged-in user pele better, na thakle service internally trending/generic fallback dite pare.
- **Project example:** `RecommendationsController.CurrentUserIdOrNull` property, `GetForMe` action.
- **Follow-up:** *Eta `ProductsController.CurrentUserId` -er shathe pattern-e ki difference?* — `ProductsController` (ebong onno protected controller) `Guid.Parse(...)`! use kore (non-null assert, karon `[Authorize]` guarantee kore claim থাকবে), kintu `RecommendationsController` `TryParse` diye null-safe rakhe karon endpoint anonymous-accessible.

Q19. `ComplaintsController` ei "Marketplace" cluster-e keno, eta to Governance route-e?

- **Short answer:** Route `api/governance/complaints`, role gate `GovernmentNGO` / `SuperAdmin` — mane eta actually consumer-protection/regulatory complaint handling (Gov/NGO monitor kore), customer nije direct file kore na ei controller diye.
- **Detailed explanation:** Eta ekta cross-cluster overlap — assignment-e "Marketplace & Commerce" list-e thakleo, implementation-e eta Governance domain-er under-e pore (namespace `DTOs.Governance`), likely customer-side complaint filing kono onno controller (hoyto `AuthController` / `CustomerSupport` type kichu) theke hoy ba eita shudhu Gov/NGO-facing moderation view.
- **Project example:** `ComplaintsController.cs` — `[Route("api/governance/complaints")]`, `[Authorize(Roles = $"{RoleNames.GovernmentNGO},`

{RoleNames.SuperAdmin}"]}] .

- **Follow-up:** *Tahole customer kivabe complaint file kore?* — Ei research scope-e shudhu ei controller dekha hoyeche; `CreateAsync` action ache eikhanei (`POST /api/governance/complaints`), tai hoyto complaint filing-o e **খানেক** hoy kintu role-gate থাকায় eta customer-direct na, kono intermediary/support-agent flow hote pare — eta ekta flag-worthy inconsistency.

Q20. AIShoppingController-er shob endpoint POST keno, GET na?

- **Short answer:** Karon protyekta endpoint-e complex request body lage (jemon `GiftRecommendationRequest` , `FashionMatchRequest` , `InteriorPreviewRequest`) — user preference/image/context data, jeta GET query string-e pathano awkward/size-limited hoy.
- **Detailed explanation:** AI-feature gulo typically structured input (occasion, budget, style tags, **একটা** image URL etc.) lage, tai REST convention onujayi "action-like" POST use kora hoyeche read-only hoyeo.
- **Project example:** `AIShoppingController` — sob 4 ta action [HttpPost] .
- **Follow-up:** *Eta REST-purist view theke thik ki?* — Kichuta debatable — pure REST-e GET হওয়া উচিত chilo (idempotent, cacheable), kintu complex-payload practical reason-e POST common pattern e-commerce AI feature-e.

Q21. AnalyticsController এবং ProducerOrdersController -er analytics endpoint-er modhhe difference ki?

- **Short answer:** `AnalyticsController` (`/api/analytics/*`) customer-er **নিজ** kenakata analytics (purchase history, spending by month, favorite categories) dekhায়; `ProducerOrdersController` -er `analytics/*` (`/api/producer/orders/analytics/*`) producer-er **selling-side** analytics (revenue, sales, visitor, income report, product performance) dekhায়.
- **Detailed explanation:** Duitai [Authorize] (ekta any-logged-in customer, arekta Producer-only), duitai `CurrentUserId` diye filtered — customer nijer data dekhe, producer nijer store-er data dekhe. Dujonই "self-scoped" — kono admin-wide analytics ei duita controller-e nei (seta hoyto Governance/Admin cluster-e).
- **Project example:** `AnalyticsController.GetPurchaseAnalytics` ; `ProducerOrdersController.GetRevenueDashboard` .
- **Follow-up:** *Admin shob producer/customer-er combined analytics kothay dekhe?* — Ei research scope-e dekha jayni, hoyto onno cluster (Government & NGO ba Admin dashboard)-e.

Q22. Category delete korle shei category-r product-gula ki hoy?

- **Short answer:** `CategoriesController.Delete` → `ICategoryService.DeleteAsync` — controller-level shudhu `NoContent()` return kore, cascade/orphan-handling logic `CategoryService` -er internal implementation-e (ei research-e deep-dive kora hoynei), kintu `Product.CategoryId` required FK (non-nullable Guid dekha geche `ProductQueryParameters / Product` entity-te) tai naive DB-level delete korle FK constraint violation hote pare jodi product thake.
- **Detailed explanation:** Safe assumption: category delete-er age "product ache kina" check থাকে উচিত (`ConflictException` throw kore delete block kora), noile orphan reference/DB error. Exact implementation `CategoryService.cs` -e (ei report-e file read hoynei deeply).
- **Project example:** `CategoriesController.Delete` route.
- **Follow-up:** *Erokom "referenced entity delete" case-e shadharon prottasha ki?* — Hoy soft-delete (`IsActive=false` , jemon Product-e kora hoyeche), noile hard-delete-er age reference-count check kore `ConflictException` throw kora — Product-e ei pattern ("EnsureCategoryAndDistrictExistAsync" er ulta") already Category/District delete-er age check korte dekha jayni explicitly ei research-e, tai eta ekta follow-up-worthy open question exam-e.

5. Tricky / Gotcha Questions

1. "Tomar `ProductService.GetByIdAsync` -e protibar GET call-e `ViewCount++` kore `SaveChangesAsync` call hocche. Eta ki race-condition-prone? 1000 user ekshathe **একই** product dekhle ki hobe?" — Ha, lost-update problem: shob request same `ViewCount` value read korte pare (e.g. 100), each ekta ekta kore বাড়িয়ে 101 banay, last-write-wins hoye actual increment onek kom hote pare je হওয়া উচিত chilo (1000 view-e হয়তো 101-ই থাকবে, 1100 না). Proper fix: DB-level atomic `UPDATE Products SET ViewCount = ViewCount + 1` (raw SQL/ `ExecuteUpdateAsync`), read-modify-write in-memory na kore.
2. "AutoMapper-er README bole `Mappings/` folder-e AutoMapper use hocche, kintu tumi bolcho manual mapping. Ei discrepancy explain koro." — Codebase-e `Mappings/` folder-e ekta stale/outdated README ache jeta claim kore AutoMapper wired up ache, kintu actual e kono controller/service-e `IMapper / .Map<T>(C)` call kono jaygay dekha jayni — shob jayগায় `ProductService.ToDtoAsync` , `OrderService.ToDto` , `AuctionService.ToDto` er moto private static/instance helper method diye manual property-by-property mapping hocche. Eta ekta documentation-vs-reality mismatch, viva-te specifically flag করায় moto point.
3. " `CheckoutAsync` -e `SaveChangesAsync` er por আলাদা kore `_cartRepository.ClearAsync(userId, ...)` call hocche. Eta same transaction-er modhhe na?" — Na. `SaveChangesAsync` (Order persist) এবং `ClearAsync` (cart empty kora) duita আলাদা DB round-trip/transaction. Jodi `SaveChangesAsync` succeed kore kintu process crash/ `ClearAsync` fail kore (network blip, timeout), order successfully create hoye jabe DB-te kintu cart-e puro no item থাকে jabe — customer confuse hote pare "ami to already order korlam, cart-e keno ache" — eta ekta partial-failure/atomicity gap.
4. " `ProductRepository.WithDetails()` -e 5-6 ta `.Include()` ekshathe ache — `Category` , `District` , `Producer` , `HandmadeVerifiedBy` , `Images` , `Variants` , `Videos` . Eta ki N+1 problem create kore, naki solve kore?" — Eta N+1 solve kore (single query-e shob related data fetch), kintu multiple collection navigation (`Images` , `Variants` , `Videos`) ekshathe include korle without `.AsSplitQuery()` cartesian explosion hoy (row count = images × variants × videos), tai `.AsSplitQuery()` add kora hoyeche — jeta multiple SELECT-e break kore kore proper. Kintu follow-up: `AsSplitQuery` nijeই ekta trade-off — multiple round-trip lage, এবং jodi concurrent write hoy dui query-r modhhe, data inconsistency (non-repeatable read) hote pare যদিও default isolation level-e eta commonly acceptable.
5. " `OrderRepository.GetPagedByUserAsync` -e `CountAsync` এবং `ToListAsync` — duita আলাদা query. Eta optimize kora jay?" — Ha, eta classic "count query + page query" pattern, duita আলাদা DB round-trip lage. High-traffic list endpoint-e eta ekta perf concern hote pare. Alternative: window function (`COUNT(*) OVER()`) diye single query-te count+data dujon paoya, kintu EF Core-e eta manually raw SQL/ `FromSqlRaw` chara shohoje expressible na — tai current simple-kintu-2-query approach ekta deliberate trade-off (simplicity vs perf).
6. "Auction-e bid place করার por keu jodi order na kore (winner declare hoyar por), ki hoy? Kono automatic Order create hoy kina?" — Kono jayগায় dekha jayni je `AuctionService` -e winner declare howar por automatically ekta `Order` create hocche — `SyncStatus` shudhu `WinnerId / Status=Ended` set kore, kono cross-service call `IOrderRepository` -er dike nei `AuctionService` -e (constructor-e shudhu `IAuctionRepository` , `IProductRepository` inject kora, `IOrderRepository` na). Mane winner-ke হয়তো manually checkout korte hoy (alada flow, hoyto `CustomOrdersController` -er moto kichu diye, ba eta ekta incomplete feature) — eta ekta gap point out করার moto sharp question.

Academy & Careers Cluster — Viva Prep

Scope note (viva gotcha #0): assignment-এ **QuestionsController** চাওয়া হয়েছিল Academy cluster-এর অংশ হিসেবে, কিন্তু বাস্তবে ওই controller **api/questions** route-এ বসে আছে এবং এটা **product Q&A** (marketplace-এর জন্য, "product-টা কি organic কাপড়ে বানানো?" টাইপ প্রশ্ন) — **ShilpoHubBD.Application.DTOs.Community** namespace-এ, Academy-র সাথে কোনো সম্পর্ক নেই। একইভাবে **CertificatesController** (**api/certificates**) আসলে **product authenticity certificate** (marketplace craft-এর জন্য, QR verification), আর কোর্স-শেষে যে certificate issue হয় সেটা আলাদা controller — **TrainingCertificatesController** (**api/training-certificates**)। Viva-তে examiner যদি "certificate কই issue হয়" জিজ্ঞেস করে, দুইটা আলাদা system আছে সেটা বলতে হবে। এই দুটো controller-ই নিচে cover করা হয়েছে যথাযথ জায়গায়।

1. Cluster Overview

ShilpoHubBD-র "Academy & Careers" cluster আসলে তিনটা sub-domain নিয়ে গঠিত যেগুলো একসাথে মিলে একটা heritage-skill-to-employment pipeline তৈরি করে। প্রথমটা হলো **e-learning ("Academy")** অংশ — **CoursesController**, **CourseCategoriesController**, **EnrollmentsController**, **ExamsController**, **QuizzesController**, **AssignmentsController**, **LiveClassesController**, **LearningRoadmapsController** — এখানে একজন Mentor (producer-role) বা Trainer (academy-member-role) কোর্স বানায়, module/lesson/material যোগ করে, publish করে, আর student (apprentice) সেই কোর্সে enroll করে, lesson progress track করে, exam/quiz/assignment দেয়, এবং শেষে certificate পায় (**TrainingCertificatesController**)। দ্বিতীয়টা হলো **mentoring marketplace** — **MentorsController** (কে mentor হতে পারে, কী skill শেখায়), **MentorMatchingController** (rule-based কোরিং দিয়ে সঠিক mentor খুঁজে দেয়), **MentorshipRequestsController** (learner request পাঠায়, mentor accept/reject করে, তারপর complete করে), **MentorFeedbackController** (mentorship শেষে learner rating/review দেয়)। তৃতীয়টা হলো **careers/hiring** — **JobListingsController** (business partner job পোস্ট করে), **JobApplicationsController** (apply → shortlist → hire lifecycle), **JobMatchingController** (candidate-এর heritage skill অনুযায়ী recommended job বের করে)। এর বাইরে **ApprenticeshipProgramsController** + **ApprenticeEnrollmentsController** হলো longer-duration, milestone-based hands-on training (কোর্সের চেয়ে বেশি formal, physical apprenticeship-এর মতো), আর **SkillAssessmentsController** + **AchievementsController** হলো gamification layer (XP, badge, skill-level tracking)। কমিউনিটি অংশে আছে **DiscussionsController** (public forum threads) আর **MessagingController** + SignalR **MessagingHub** (1:1 real-time private chat, e.g. student ↔ mentor)।

একটা typical request flow দেখা যাক: একজন user প্রথমে **AcademyMemberProfilesController**-এ profile খোলে (student/trainer role বেছে), **POST /api/academy-profiles**। তারপর **CoursesController** থেকে published কোর্স ব্রাউজ করে, **EnrollmentsController**-এ **POST /api/enrollments/courses/{courseId}/enroll** দিয়ে enroll করে। কোর্স চলাকালীন **MarkLessonProgressAsync** দিয়ে lesson-ভিত্তিক progress আপডেট হয় (percentage auto-calculate হয় completed/total lessons থেকে)। কোর্সের ভেতরে থাকা **ExamsController** / **QuizzesController**-এ attempt শুরু করে (**StartAttempt**), উত্তর জমা দেয় (**SubmitAttempt**) — MCQ auto-grade হয়, essay হলে trainer manually **EvaluateAnswerAsync** দিয়ে নম্বর দেয় এবং **FinalizeEvaluationAsync** দিয়ে চূড়ান্ত করে। কোর্স **CompleteEnrollmentAsync** কল হলে **EnrollmentService** নিজে থেকেই একটা **TrainingCertificate** (Type=Course) বানিয়ে ফেলে — এটাই "finishing a Course → Certificate issuance" সংযোগ। আবার **SkillAssessmentsController.Run** দিয়ে কোনো heritage skill-এ assessment দিলে academy profile-এর **SkillLevel** আপডেট হয়, আর **JobMatchingController.GetRecommendedJobs** সেই skill-level দিয়েই score করে job সাজেস্ট করে — এইটাই "passing SkillAssessment → JobMatching" সংযোগ। Mentor side-এ **TrainingCertificatesController.IssueSkillCertificateAsync** একজন learner-এর skill level **Advanced**-এর নিচে থাকলে certificate ইস্যু করতে refuse করে — অর্থাৎ SkillAssessment → SkillLevel → Certificate eligibility একটা chain। পুরো cluster জুড়েই messaging আর discussions community glue হিসেবে কাজ করে — mentor-student বা employer-candidate সরাসরি চ্যাট করতে পারে **MessagingController** / **MessagingHub** দিয়ে।

2. Deep Dive — 4 Representative Controllers

2.1 CoursesController + EnrollmentsController — Course Authoring & Progress Flow

Files:

ShilpoHubBD.Api/Controllers/CoursesController.cs, **ShilpoHubBD.Api/Controllers/EnrollmentsController.cs**,
ShilpoHubBD.Application/Services/Learning/CourseService.cs, **ShilpoHubBD.Application/Services/Learning/EnrollmentService.cs**

Route table — CoursesController (**api/courses**)

Verb	Route	Auth
GET	/	Anonymous (published only)
GET	/ {id}	Anonymous (owner sees drafts too)
GET	/mine	[Authorize]
POST	/	[Authorize]
PUT	/ {id}	[Authorize] (owner only)
POST	/ {id}/publish	[Authorize] (owner only)
POST	/ {id}/archive	[Authorize] (owner only)
DELETE	/ {id}	[Authorize] (owner, Draft only)
POST/PUT/DELETE	/ {id}/lessons[/...]	[Authorize] (owner only)
POST/PUT/DELETE	/ {id}/modules[/...]	[Authorize] (owner only)
POST/DELETE	/ {id}/materials[/...]	[Authorize] (owner only)

Route table — EnrollmentsController (**api/enrollments**, class-level [Authorize])

Verb	Route	Auth
POST	<code>/courses/{courseId}/enroll</code>	Any authenticated user
GET	<code>/mine</code>	Any authenticated user
GET	<code>/id</code>	owner apprentice, course author, or SuperAdmin
GET	<code>/courses/{courseId}</code>	course owner
POST	<code>/id/progress</code>	course owner (mentor marks progress on behalf of student — see gotcha below)
POST	<code>/id/complete</code>	course owner

গুরুত্বপূর্ণ কোড ১ — Publish rule (`CourseService.cs`):

```
public async Task<CourseDto> PublishAsync(Guid userId, Guid courseId, CancellationToken cancellationToken)
{
    var course = await GetOwnedCourseAsync(userId, courseId, cancellationToken);

    if (course.Status == CourseStatus.Published)
        throw new ConflictException("This course is already published.");

    if (course.Lessons.Count == 0)
        throw new ConflictException("Add at least one lesson before publishing this course.");

    var now = DateTime.UtcNow;
    course.Status = CourseStatus.Published;
    course.PublishedAt = now;
    course.UpdatedAt = now;

    await _courseRepository.SaveChangesAsync(cancellationToken);
    return ToDto(course);
}
```

Banglish explanation: `GetOwnedCourseAsync` প্রথমে course খুঁজে বের করে, তারপর check করে course-এর `Mentor?.UserId` অথবা `TrainerProfile?.UserId` current user-এর সাথে match করে কিনা — না হলে `UnauthorizedAccessException` (global handler এটাকে 401-এ ম্যাপ করে)। এখানে interesting design decision হলো একটা `Course` entity-র author হয় **Mentor**, নাহলে **TrainerProfile** — দুইটা আলাদা role-এর user কোর্স বানাতে পারে (producer যে mentor হয়েছে, অথবা academy-member যে trainer role নিয়েছে), তাই সব জায়গায় `AuthorUserId(course) => course.Mentor?.UserId ?? course.TrainerProfile?.UserId` এই null-coalescing pattern repeat হয়। Publish করার আগে business rule হলো — কমপক্ষে একটা lesson লাগবে, ফাঁকা কোর্স publish করা যায় না। Status একটা simple state machine: `Draft → Published → Archived`, আর `Delete` শুধু Draft অবস্থায় allowed (published কোর্স delete না করে archive করতে বলা হয়, কারণ enrollment থাকলে data integrity নষ্ট হবে)।

গুরুত্বপূর্ণ কোড ২ — Enroll business rules (`EnrollmentService.cs`):

```
public async Task<CourseEnrollmentDto> EnrollAsync(Guid apprenticeUserId, Guid courseId, CancellationToken cancellationToken)
{
    var course = await _courseRepository.GetByIdAsync(courseId, cancellationToken);
    ?? throw new NotFoundException("Course not found.");

    if (course.Status != CourseStatus.Published)
        throw new ConflictException("You can only enroll in published courses.");

    if (AuthorUserId(course) == apprenticeUserId)
        throw new ConflictException("You cannot enroll in your own course.");

    var existing = await _enrollmentRepository.GetByCourseAndApprenticeAsync(courseId, apprenticeUserId, cancellationToken);
    if (existing is not null)
        return ToDto(existing); // idempotent – double-enroll silently returns existing row

    if (course.MaxApprentices.HasValue)
    {
        var activeCount = await _enrollmentRepository.GetActiveCountByCourseAsync(courseId, cancellationToken);
        if (activeCount ≥ course.MaxApprentices.Value)
            throw new ConflictException("This course has reached its maximum number of apprentices.");
    }

    // ... create CourseEnrollment with Status = Active
}
```

Banglish explanation: Enroll করার তিনটা guard আছে — (1) unpublished কোর্সে enroll করা যাবে না, (2) নিজের বানানো কোর্সে নিজে enroll করা যাবে না (self-enrollment block), (3) seat capacity (`MaxApprentices`) থাকলে সেটা respect করা হয় — শুধু `Active` enrollment count against করা হয় (dropped/completed বাদ)। খেয়াল করার মতো জিনিস: এই endpoint idempotent বানানো হয়েছে — যদি already enrolled থাকে, নতুন করে error না দিয়ে existing enrollment-ই ফেরত দেওয়া হয়, এটা REST-এর সাধারণ "double POST" সমস্যার একটা graceful সমাধান।

গুরুত্বপূর্ণ কোড ৩ — Completion → auto certificate issuance:

```
public async Task<CourseEnrollmentDto> CompleteEnrollmentAsync(Guid mentorUserId, Guid enrollmentId, CancellationToken cancellationToken)
{
    var enrollment = await GetEnrollmentOwnedByMentorAsync(mentorUserId, enrollmentId, cancellationToken);
    if (enrollment.Status == EnrollmentStatus.Completed)
        throw new ConflictException("This enrollment has already been marked complete.");

    enrollment.Status = EnrollmentStatus.Completed;
    enrollment.CompletedAt = DateTime.UtcNow;

    var existingCertificate = await _trainingCertificateRepository.GetByEnrollmentIdAsync(enrollment.Id, cancellationToken);
    if (existingCertificate is null)
    {
        var certificate = new TrainingCertificate
        {
            Id = Guid.NewGuid(),
            Type = CertificateType.Course,
            RecipientUserId = enrollment.ApprenticeId,
            IssuerUserId = AuthorUserId(enrollment.Course),
            EnrollmentId = enrollment.Id,
            CertificateNumber = GenerateCertificateNumber(now), // "SH-TRN-yyyyMMdd-XXXXXXX"
            ...
        };
        await _trainingCertificateRepository.AddAsync(certificate, cancellationToken);
    }

    await _enrollmentRepository.SaveChangesAsync(cancellationToken);
    return ToDto(enrollment);
}
```

Banglish: এইটাই cluster-এর সবচেয়ে গুরুত্বপূর্ণ cross-domain সংযোগ — Enrollment complete হওয়া মাত্র `EnrollmentService` নিজেই একটা `TrainingCertificate` তৈরি করে ফেলে, alada কোনো event/message queue ছাড়াই (synchronous, same DbContext transaction-এর মধ্যে — `SaveChangesAsync` একবারই কল হয় শেষে)। Certificate number generate হয় `SH-TRN-{date}-{random 8 hex}` ফরম্যাটে যেটা human-readable আর globally-unique! `GetByEnrollmentIdAsync` চেক দিয়ে duplicate certificate আটকানো হয় (idempotent completion)।

Business rules সারসংক্ষেপ:

- কোর্স publish করতে ≥ 1 lesson লাগবে; Draft ছাড়া delete করা যায় না।
- Enrollment idempotent; নিজের কোর্সে enroll করা যায় না; seat-limit respect হয়।
- Progress percentage সবসময় $\frac{\text{completedLessons}}{\text{totalLessons}} * 100$ থেকে **on-the-fly calculate** হয় (কোথাও persist করা নেই), তাই lesson সংখ্যা বদলালেও progress সবসময় সঠিক থাকে।
- Complete করলে auto certificate issue হয়, duplicate-proof।
- লক্ষণীয় gotcha: `MarkLessonProgressAsync` আর `CompleteEnrollmentAsync` — দুটোই `mentorUserId` parameter নেয় আর `GetEnrollmentOwnedByMentorAsync` দিয়ে authorize হয়, অর্থাৎ **student নিজে তার progress mark করতে পারে না** — এটা mentor/trainer-driven progress tracking (viva-তে জিজ্ঞেস করতে পারে "student নিজে lesson complete মার্ক করতে পারে কি?" — উত্তর: না, এই কোডবেসে trainer/mentor করে)।

2.2 ExamsController (vs QuizzesController) — Grading Engine

Files: `ShilpoHubBD.Api/Controllers/ExamsController.cs`, `ShilpoHubBD.Api/Controllers/QuizzesController.cs`, `ShilpoHubBD.Application/Services/Learning/ExamService.cs`, `ShilpoHubBD.Application/Services/Learning/QuizService.cs`

Route table — ExamsController (`api/exams`, class-level `[Authorize]`)

Verb	Route	Purpose
POST	<code>/courses/{courseId}</code>	create exam (course owner)
GET	<code>/courses/{courseId}</code>	list exams (owner or enrolled student)
GET/PUT/DELETE	<code>/[{id}]</code>	exam CRUD (owner)
POST/PUT/DELETE	<code>/[{id}]/questions[/...]</code>	question CRUD, blocked once attempts exist
POST	<code>/[{id}]/attempts/start</code>	student starts attempt
POST	<code>/attempts/{attemptId}/submit</code>	student submits answers

Verb	Route	Purpose
GET	/attempts/{attemptId}	view result
GET	/[{id}]/attempts/mine /[{id}]/attempts	student's own vs trainer's view of all attempts
POST	/attempts/{attemptId}/questions/{questionId}/evaluate	trainer manually grades one essay answer
POST	/attempts/{attemptId}/finalize	trainer finalizes score after all essays graded

`QuizzesController` (`api/quizzes`) প্রায় identical route table, কিন্তু **no evaluate / finalize endpoints** — কারণ Quiz-এ শুধু multiple-choice question থাকে (`QuizQuestion` entity-তে `QuestionType` field-ই নেই, সরাসরি Options list), তাই manual grading দরকার হয় না, submit করা মাত্রই পুরো attempt auto-evaluated হয়ে `Status = Submitted` (Exam-এ essay থাকলে `Submitted` অর্থ "awaiting evaluation", কিন্তু Quiz-এ `Submitted` মানেই final)।

গুরুত্বপূর্ণ কোড ১ — Attempt limits & logout (`ExamService.StartAttemptAsync`):

```
var priorAttempts = await _examRepository.GetAttemptsByStudentAsync(examId, studentUserId, cancellationToken);
if (priorAttempts.Any(a => a.Status == AttemptStatus.InProgress))
    throw new ConflictException("You already have an attempt in progress for this exam.");

if (exam.MaxAttempts.HasValue && priorAttempts.Count >= exam.MaxAttempts.Value)
    throw new ConflictException("You have reached the maximum number of attempts for this exam.");

var attempt = new ExamAttempt
{
    ...
    AttemptNumber = priorAttempts.Count + 1,
    Status = AttemptStatus.InProgress,
    MaxScore = exam.Questions.Sum(q => q.Points),
};
```

Banglish: Re-take করা যায় কিনা তার উত্তর — যদি `Exam.MaxAttempts` null হয়, unlimited retake allowed; নাহলে limit পর্যন্ত। একসাথে দুইটা attempt `InProgress` অবস্থায় থাকতে পারবে না (student একটা শেষ না করে নতুন শুরু করতে পারবে না) — এটা concurrency abuse (multiple tabs খুলে answer কপি করা) কিছুটা প্রতিরোধ করে, যদিও পুরোপুরি race-condition-proof না (নিচে "Gotcha" সেকশনে বিস্তারিত)।

গুরুত্বপূর্ণ কোড ২ — Auto-grading + essay branch (`ExamService.SubmitAttemptAsync`):

```

var autoScore = 0;
var hasEssayQuestions = exam.Questions.Any(q => q.QuestionType == QuestionType.Essay);

foreach (var question in exam.Questions)
{
    var submitted = request.Answers.FirstOrDefault(a => a.QuestionId == question.Id);
    var answer = new ExamAttemptAnswer { ... };

    if (question.QuestionType == QuestionType.MultipleChoice)
    {
        var selectedOption = submitted?.SelectedOptionId is not null
            ? question.Options.FirstOrDefault(o => o.Id == submitted.SelectedOptionId.Value)
            ?? throw new ConflictException("One of the submitted options does not belong to this exam.")
            : null;
        var isCorrect = selectedOption?.IsCorrect ?? false;
        var pointsAwarded = isCorrect ? question.Points : 0;
        autoScore += pointsAwarded;
        answer.SelectedOptionId = selectedOption?.Id;
        answer.IsCorrect = isCorrect;
        answer.PointsAwarded = pointsAwarded;
    }
    else { answer.EssayAnswerText = submitted?.EssayAnswerText?.Trim(); } // no score yet
    ...
}

attempt.Score = autoScore;
if (hasEssayQuestions)
{
    attempt.Status = AttemptStatus.Submitted; // waits for trainer
}
else
{
    attempt.Status = AttemptStatus.Evaluated;
    attempt.PercentageScore = attempt.MaxScore > 0 ? Math.Round(autoScore / (decimal)attempt.MaxScore * 100m, 2) : 0;
    attempt.IsPassed = attempt.PercentageScore ≥ exam.PassingScorePercentage;
    attempt.EvaluatedAt = attempt.SubmittedAt;
}

```

Banglish: MCQ auto-grade হয় সরাসরি option-এর **IsCorrect** flag চেক করে — কোনো separate answer-key table নেই, correct answer option-এর মধ্যেই encode করা। একটা security check আছে: submitted **SelectedOptionId** যদি ওই question-এর real option না হয় (client tampering বা অন্য exam-এর option id পাঠানো), **ConflictException** throw হয়। মূল branching logic: যদি exam-এ কোনো Essay question **না থাকে**, submit করা মাত্রই percentage/pass-fail calculate করে **Evaluated** স্ট্যাটাসে চলে যায় (fully automatic)। কিন্তু essay থাকলে status **Submitted**-এ আটকে থাকে (percentage/IsPassed তখনো null) — trainer পরে আলাদাভাবে প্রতিটা essay answer-এ **EvaluateAnswerAsync** কল করে **PointsAwarded** বসায়, এবং সব essay গ্রেড হয়ে গেলে **FinalizeEvaluationAsync** কল করে চূড়ান্ত percentage/pass-fail বসায়:

```

var essayAnswers = attempt.Answers.Where(a => a.ExamQuestion.QuestionType == QuestionType.Essay).ToList();
if (essayAnswers.Any(a => a.PointsAwarded is null))
    throw new ConflictException("All essay questions must be evaluated before finalizing this attempt.");

var totalScore = attempt.Answers.Sum(a => a.PointsAwarded ?? 0);
attempt.PercentageScore = attempt.MaxScore > 0 ? Math.Round(totalScore / (decimal)attempt.MaxScore * 100m, 2) : 0;
attempt.IsPassed = attempt.PercentageScore ≥ attempt.Exam.PassingScorePercentage;
attempt.Status = AttemptStatus.Evaluated;

```

Finalize guard করে দেয় যে প্রতিটা essay answer-এর **PointsAwarded** null না — অর্থাৎ, trainer সব essay না দেখলে finalize করতে পারবে না, half-graded attempt evaluated হয়ে যেতে পারবে না।

Business rules সারসংক্ষেপ:

- Exam/Quiz আছে attempts হয়ে গেলে question add/edit/delete করা যায় না (**EnsureExamNotAttempted**) — grading fairness বজায় রাখতে (মাঝপথে question বদলালে আগের students-দের সাথে অন্যায্য হতো)।
- Exam-এ attempts থাকলে exam-ই delete করা যায় না।
- Enrollment **Active** না থাকলে attempt শুরু করা যায় না (**EnsureEnrolledAsync**)।
- MCQ auto-grade instant; Essay → **Submitted** (pending) → trainer evaluate প্রতিটা answer → **Finalize** → **Evaluated** ।
- Quiz-এ শুধু MCQ, তাই সবসময় submit করামাত্র fully auto-evaluated (কোনো pending state নেই)।

2.3 MentorMatchingController + JobMatchingController — Rule-Based Weighted Scoring

Files: ShilpoHubBD.Api/Controllers/MentorMatchingController.cs , ShilpoHubBD.Api/Controllers/JobMatchingController.cs , ShilpoHubBD.Application/Services/MentorMatching/MentorMatchingService.cs , ShilpoHubBD.Application/Services/Employment/JobMatchingService.cs

Route table

Controller	Verb	Route	Auth
MentorMatchingController	POST	api/mentor-matching/match	[Authorize]
JobMatchingController	POST	api/job-matching/recommended	[Authorize]

দুইটাই single-endpoint controller — thin wrapper, সব logic service-এ। Source code comment-এ explicit বলা আছে:

```
// Rule-based (not AI) matching engine: scores each mentor against the criteria the caller actually
// specified, so an unspecified filter neither helps nor hurts a candidate's score.
```

এটা viva-তে গুরুত্বপূর্ণ — যদিও folder/namespace-এর নাম "AI"-adjacent মনে হতে পারে (AIIntelligenceController আলাদা জায়গায় আছে), এই matching pure weighted-sum heuristic, কোনো ML model বা external AI call না।

গুরুত্বপূর্ণ কোড — MentorMatchingService weighted scoring:

```
private const decimal BaselineExperienceWeight = 5m;
private const decimal SkillWeight = 25m;
private const decimal SkillLevelWeight = 15m;
private const decimal LocationWeight = 15m;
private const decimal ExperienceWeight = 15m;
private const decimal AvailabilityWeight = 10m;
private const decimal CategoryWeight = 10m;
private const decimal GoalKeywordWeight = 10m;

private static MentorMatchResultDto Score(MentorMatchCandidateDto candidate, MentorMatchRequest request)
{
    var totalWeight = BaselineExperienceWeight;
    var achievedWeight = Math.Min(1m, candidate.YearsOfExperience / 10m) * BaselineExperienceWeight;
    var reasons = new List<string>();

    if (request.HeritageSkillId.HasValue)
    {
        totalWeight += SkillWeight;
        if (candidate.HasMatchingSkill)
        {
            achievedWeight += SkillWeight;
            reasons.Add("Teaches the requested heritage skill.");
            if (request.MinSkillLevel.HasValue)
            {
                totalWeight += SkillLevelWeight;
                if (candidate.MatchingSkillLevel.HasValue && candidate.MatchingSkillLevel.Value ≥ request.MinSkillLevel.Value)
                {
                    achievedWeight += SkillLevelWeight;
                    reasons.Add($"Skill level ({candidate.MatchingSkillLevel}) meets the requested {request.MinSkillLevel} level.");
                }
            }
        }
    }

    // ... Location, MinYearsOfExperience, AvailabilityKeyword, PreferredCategory, LearningGoalKeyword - same pattern

    var score = totalWeight == 0m ? 0m : Math.Round(achievedWeight / totalWeight * 100m, 1);
    return new MentorMatchResultDto { ... , MatchScore = score, MatchReasons = reasons };
}
```

Banglish explanation — এই algorithm-এর মূল trick: এটা fixed weight-sum না, বরং adaptive denominator — totalWeight শুধু তখনই বাড়ে যখন request -এ সেই criterion আসলে specify করা হয়েছে (if (request.HeritageSkillId.HasValue) ইত্যাদি guard)। মানে ধরা যাক caller শুধু Location দিয়েছে, বাকি সব ফিল্ড null — তাহলে totalWeight = BaselineExperienceWeight + LocationWeight (শুধু ২০), আর matching মেলাতে পারলে achievedWeight সেই একই ২০-এর কাছাকাছি হয়ে score ~১০০% এর কাছে যাবে। এইভাবে "unspecified filter neither helps nor hurts" — যে criterion caller দেয়নি সেটা score-কে কৃত্রিমভাবে কমিয়ে দেয় না। প্রতিটা

matched-criterion একটা human-readable `reasons` string যোগ করে (explainability — কেন এই mentor সাজেস্ট হলো সেটা UI-তে দেখানো যায়)। Baseline experience-এর জন্য একটা সবসময়-active ৫-পয়েন্ট weight আছে (experience zero থেকে ১০ বছর পর্যন্ত linear scale, ১০+ বছর হলে cap করে ১০০%-এ)। শেষে candidates `MatchScore` descending, tie হলে `YearsOfExperience` descending দিয়ে sort হয়ে `Take(request.MaxResults)`।

`JobMatchingService` structurally প্রায় identical কিন্তু চারটা weight নিয়ে কাজ করে (`RequiredSkillWeight=45`, `PreferredSkillWeight=20`, `LocationWeight=20`, `ExperienceWeight=15`) এবং সোর্স ভিন্ন — এখানে caller নিজে criteria দেয় না, বরং candidate-এর নিজের `AcademyMemberProfile.Skills` (heritage skill → `SkillLevel` dictionary) automatically job-এর `SkillRequirements` -এর সাথে match করা হয়:

```
var requiredSkills = job.SkillRequirements.Where(r => r.IsRequired).ToList();
if (requiredSkills.Count > 0)
{
    totalWeight += RequiredSkillWeight;
    var met = requiredSkills.Count(r => memberSkills.TryGetValue(r.HeritageSkillId, out var level) && level >= r.MinLevel);
    achievedWeight += RequiredSkillWeight * (met / (decimal)requiredSkills.Count);    // partial credit
}
```

Banglish: এখানে partial-credit ধারণা যোগ হয়েছে — required skill-এর ৩টার মধ্যে ২টা মিললে পুরো weight না দিয়ে `2/3` fraction দেওয়া হয়। এটাই skill-assessment → job-matching সংযোগ: `SkillAssessmentsController.Run` করলে academy profile-এর skill level বাড়ে, সেটাই এখানে `memberSkills` dictionary হিসেবে lookup হয়।

- Business rules:**
- দুইটাই stateless, pure-function scoring — কোনো DB write নেই, শুধু read + calculate।
 - Score সবসময় ০-১০০%, কোনো criterion না দিলে candidate বাদ পড়ে না, শুধু score গঠনের ভিত্তি বদলায়।
 - `MaxResults` দিয়ে caller কতগুলো result চায় সেটা bound করা যায় (pagination না, hard cap)।

2.4 MessagingController + MessagingHub — Real-time Chat (SignalR)

Files: `ShilpoHubBD.Api/Controllers/MessagingController.cs`, `ShilpoHubBD.Api/Hubs/MessagingHub.cs`, `ShilpoHubBD.Api/Realtime/SignalRMessageNotifier.cs`, `ShilpoHubBD.Application/Services/Messaging/MessagingService.cs`, `ShilpoHubBD.Api/Program.cs`

Route table — MessagingController (`api/messaging` , class-level `[Authorize]`)

Verb	Route	Purpose
GET	<code>/conversations</code>	paged list of my conversations (with unread count)
GET	<code>/conversations/{id}</code>	full thread incl. all messages
POST	<code>/conversations</code>	start new (or reuse existing) 1:1 conversation + first message
POST	<code>/conversations/{id}/messages</code>	send message in existing conversation
POST	<code>/conversations/{id}/read</code>	mark my read-cursor

Hub: `MessagingHub` mounted at `/hubs/messaging` (`app.MapHub<MessagingHub>("/hubs/messaging")` — `Program.cs:237`), `[Authorize]` .

গুরুত্বপূর্ণ কোড ১ — REST + Hub hybrid architecture:

`MessagingController`-এ কোনো `SignalR` code নেই — message পাঠানো এখনো plain `POST` REST call (persist reliably, client offline থাকলেও পরে load হবে)। Hub শুধু **server → client push notification**-এর জন্য ব্যবহার হয়, client → server direction-এ কিছু পাঠাতে হয় না (Hub class-এ কোনো public method নেই যা message পাঠায়, শুধু connect/disconnect handle করে):

```
[Authorize]
public class MessagingHub : Hub
{
    public override async Task OnConnectedAsync()
    {
        var userId = GetUserId();
        if (userId is not null)
            await Groups.AddToGroupAsync(Context.ConnectionId, GroupName(userId.Value));
        await base.OnConnectedAsync();
    }

    public override async Task OnDisconnectedAsync(Exception? exception)
    {
        var userId = GetUserId();
        if (userId is not null)
            await Groups.RemoveFromGroupAsync(Context.ConnectionId, GroupName(userId.Value));
        await base.OnDisconnectedAsync(exception);
    }

    internal static string GroupName(Guid userId) => $"user:{userId}";
}
```

Banglish: কানেক্ট হওয়া মাত্র ওই connection একটা per-user group-এ যোগ হয় (`user:{guid}`), disconnect হলে বের হয়ে যায়। একটা user multiple device/tab থেকে কানেক্ট করলে multiple connectionId একই group-এ থাকবে, তাই সব device-এই push পৌঁছাবে। এই pattern-এর সুবিধা: broadcast করার সময় connectionId ট্র্যাক করতে হয় না, শুধু `user:{id}` group-এ পাঠালেই হয়।

গুরুত্বপূর্ণ কোড ২ — Service → Notifier → Hub push (`MessagingService.SendMessageAsync`):

```
public async Task<MessageDto> SendMessageAsync(Guid conversationId, Guid userId, SendMessageRequest request, CancellationToken cancellationToken)
{
    var conversation = await _messagingRepository.GetByIdAsync(conversationId, cancellationToken)
        ?? throw new NotFoundException("Conversation not found.");
    EnsureParticipant(conversation, userId);

    var message = new Message { ..., SenderId = userId, Body = request.Body.Trim(), CreatedAt = DateTime.UtcNow };
    await _messagingRepository.AddMessageAsync(message, cancellationToken);
    conversation.UpdatedAt = message.CreatedAt;
    await _messagingRepository.SaveChangesAsync(cancellationToken); // 1) persist first

    var dto = ToMessageDto(message, senderName, isRead: false);
    foreach (var recipient in conversation.Participants.Where(p => p.UserId != userId))
        await _messageNotifier.NotifyMessageAsync(recipient.UserId, dto, cancellationToken); // 2) then push

    return dto;
}
```

```
// SignalRMessageNotifier.cs – Api layer, implements Application-layer IMessageNotifier interface
public Task NotifyMessageAsync(Guid recipientUserId, MessageDto message, CancellationToken cancellationToken)
    => _hubContext.Clients.Group(MessagingHub.GroupName(recipientUserId)).SendAsync("ReceiveMessage", message, cancellationToken);
```

Banglish: এইটা Clean Architecture-এর dependency-inversion-এর সুন্দর উদাহরণ — `MessagingService` (Application layer) সরাসরি SignalR জানে না, শুধু `IMessageNotifier` interface কল করে। Actual implementation `SignalRMessageNotifier` থাকে Api layer-এ (যেখানে SignalR package reference করা আছে), DI দিয়ে wire হয়। Order-টা লক্ষ্য করার মতো: **আগে DB-তে persist, পরে push** — যদি push fail-ও করে (client অফলাইন), message হারায় না, database-এ থেকে যায়, client পরে REST দিয়ে fetch করে দেখতে পাবে। `IHubContext<MessagingHub>` inject করে `Clients.Group( ... ).SendAsync("ReceiveMessage", dto)` কল করা হয় — client-side listener `connection.on("ReceiveMessage", handler)` দিয়ে শুনবে।

গুরুত্বপূর্ণ কোড ৩ — JWT-over-query-string for WebSocket handshake (`Program.cs`):

```
// Browsers cannot set an Authorization header on the WebSocket handshake, so SignalR
// clients pass the access token as a query parameter instead.
options.Events = new JwtBearerEvents
{
    OnMessageReceived = context =>
    {
        var accessToken = context.Request.Query["access_token"];
        if (!string.IsNullOrEmpty(accessToken) && context.HttpContext.Request.Path.StartsWithSegments("/hubs"))
        {
            context.Token = accessToken;
        }
        return Task.CompletedTask;
    },
};
```

Banglish: Normal REST call-এ JWT **Authorization: Bearer <token>** header দিয়ে যায়, কিন্তু browser-এর native **WebSocket** API handshake request-এ custom header সেট করতে দেয় না। তাই SignalR client connect করার সময় URL-এ **?access_token=<jwt>** হিসেবে পাঠায় (**HubConnectionBuilder().withUrl(url, { accessTokenFactory: () => token })** — client SDK নিজেই এটা করে)। Server-side **JwtBearerEvents.OnMessageReceived** hook চেক করে path **/hubs** দিয়ে শুরু কিনা (safety: শুধু hub route-এই query-string token accept করা হয়, বাকি সব normal API route-এ header-ই লাগবে), থাকলে সেই token-কেই normal validation pipeline-এ পাঠায় (**context.Token = accessToken**)। এইটা industry-standard SignalR pattern, কিন্তু এর security trade-off নিচে Gotcha সেকশনে আলোচনা করা হয়েছে।

LiveClassHub-এর সাথে তুলনা (**LiveClassesController + **LiveClassHub**):** ডিজাইন প্যাটার্ন ভিন্ন — **MessagingHub** -এ per-user group auto-join হয় connect করলেই, কিন্তু **LiveClassHub** -এ client নিজে explicit **JoinClass(LiveClassId)** / **LeaveClass(LiveClassId)** method কল করে group-এ ঢোকে (per-class group, **live-class:{id}**), কারণ একজন user একসাথে একাধিক live class-এর room-এ থাকতে পারবে না বা চায় না — client UI নিয়ন্ত্রণ করে কখন কোন class-room-এ join করবে। **SignalRLiveClassNotifier** চারটা event পাঠায়: **StatusChanged** , **ParticipantJoined** , **QuestionAsked** , **QuestionAnswered** — যেমন mentor **Start** / **End** করলে সব participant-কে broadcast হয়।

Business rules:

- Conversation সবসময় exactly ২ participant (1:1), group chat নেই।
- একই দুই user-এর মধ্যে দ্বিতীয়বার **StartConversation** করলে নতুন conversation না বানিয়ে existing-টাই reuse হয় (**GetBetweenUsersAsync**)।
- **MarkAsReadAsync** participant-এর **LastReadAt** timestamp আপডেট করে; unread count calculate হয় **m.CreatedAt > me.LastReadAt** দিয়ে on-the-fly, persisted counter না।
- নিজেকে নিজে message করা যায় না (**RecipientId == userId** → **ConflictException**)।

3. Reference Table — Remaining Controllers

Controller	Route prefix	Key endpoints	Auth/Roles	Purpose
CourseCategoriesController	api/course-categories	GET / (anon); POST /	POST: SuperAdmin	Admin-managed taxonomy for courses
QuestionsController	api/questions	GET /product/{id} ; POST /product/{id} ; POST / {id}/answers ; DELETE / {id} , / {id}/answers/{aid}	Ask/Answer/Delete require [Authorize] ; delete allows author or Admin	Not Academy — product Q&A for marketplace listings
MentorsController	api/mentors	GET / , / {id} (public); GET /me ; POST / (BecomeMentor); PUT /me ; POST/DELETE /me/skills	Mutations: Producer role	Producer opts in to become a paid/volunteer mentor, lists taught heritage skills
MentorFeedbackController	api/mentor-feedback	POST / ; GET /mine	[Authorize]	Learner submits rating/review after mentorship completes
MentorshipRequestsController	api/mentorship-requests	POST / ; GET / {id} ; GET /mine/as-learner , /mine/as-mentor ; POST / {id}/accept , /reject , /complete	[Authorize]	Learner requests a mentor; accept/reject/complete lifecycle
ApprenticeshipProgramsController	api/apprenticeship-programs	GET / (published, anon); GET / {id} ; GET /mine ; POST / ; PUT / {id} ; POST / {id}/publish , /close ; DELETE / {id} ; POST/PUT/DELETE / {id}/milestones[/.]	Mutations: [Authorize] (program owner)	Longer, milestone-based craft apprenticeships (vs. short online Courses)
ApprenticeEnrollmentsController	api/apprentice-enrollments	GET /mine ; GET / {id} ; GET /programs/{pid} ; POST / {id}/milestones/{mid}/progress ; POST / {id}/complete	[Authorize]	Tracks a learner's milestone-by-milestone progress in an apprenticeship
CertificatesController	api/certificates	GET / {id} , / {id}/download ; GET /mine ; POST /generate ; POST / {id}/revoke ; POST /verify	mine/generate/revoke: Producer , SuperAdmin ; verify: public	Not course certs — product authenticity

Controller	Route prefix	Key endpoints	Auth/Roles	Purpose
				certificate for marketplace craft items
TrainingCertificatesController	api/training-certificates	GET /{id}, /{id}/download; GET /mine; POST /verify (public); POST /skill/issue; POST /{id}/revoke	mine/issue/revoke: [Authorize]	Actual Academy certificates: Course completion, Skill mastery, Apprenticeship
LearningRoadmapsController	api/learning-roadmaps	POST /; GET /active, /{id}, /history; POST /{id}/refresh; POST /{id}/milestones/{mid}/complete	class-level [Authorize]	Personal learning-path recommendation with milestones, refreshable progress
LiveClassesController	api/live-classes	GET /, /{id} (anon); GET /mine, /registered; POST /; PUT /{id}; POST /{id}/start, /end, /cancel; DELETE /{id}; POST /{id}/register, /join, /leave; GET /{id}/attendance; POST /{id}/questions[/...]	Mutations: [Authorize]	Scheduled live sessions; realtime via LiveClassHub (/hubs/live-classes)
SkillAssessmentsController	api/skill-assessments	POST /skills/{heritageSkillId}/run; GET /{id}; GET /history	class-level [Authorize]	Runs an assessment against one heritage skill, updates member's SkillLevel
JobListingsController	api/job-listings	GET /, /{id} (anon); GET /mine; POST /; PUT /{id}; POST /{id}/publish, /close; DELETE /{id}; POST/DELETE /{id}/skill-requirements[/...]	Mutations: [Authorize] (employer)	Business partner posts/manages job openings + required heritage skills
JobApplicationsController	api/job-applications	POST / (Apply); GET /{id}, /mine, /job-listings/{jId}; GET /{id}/candidate-portfolio; POST /{id}/shortlist, /reject, /hire, /withdraw	class-level [Authorize]	Candidate applies; employer shortlist/reject/hire workflow
AcademyMemberProfilesController	api/academy-profiles	POST /; GET /me; PUT /me; GET /{id} (anon); POST/DELETE /me/skills[/...]; GET /me/learning-history	Mutations: [Authorize]	Student/Trainer role profile; skill inventory; enrollment history
AchievementsController	api/achievements	GET /xp/mine, /xp/mine/history; GET / (all achievements, anon); GET /mine; POST / (create), /xp/award; POST /evaluate	create/award: SuperAdmin; rest: [Authorize]	XP economy + badge/achievement gamification layer
AssignmentsController	api/assignments	POST/GET /courses/{cid}; GET/PUT/DELETE /{id}; POST /{id}/submit; GET /{id}/my-submission, /submissions; POST /submissions/{sid}/grade	class-level [Authorize]	Homework/project assignments per course, manual grading
DiscussionsController	api/discussions	GET /, /{id} (anon); POST /; POST /{id}/replies; DELETE /{id}, /{id}/replies/{rid}	Mutations: [Authorize]; delete: author or SuperAdmin	Public community forum threads

4. Viva Question Bank

Q1. Course-এর author দুই রকমের হতে পারে কেন — Mentor এবং TrainerProfile দুইটাই কেন আছে?

- **Short answer:** কারণ দুই ধরনের user কোর্স বানাতে পারে — একজন Producer যে mentor হয়েছে (craft-skill শেখানোর জন্য), অথবা একজন Academy-member যে Trainer হিসেবে (formal instructor)।

- **Detailed:** Course entity-তে MentorId আর TrainerProfileId দুটোই nullable FK, exactly একটা populated থাকে। AuthorUserId(course) => course.Mentor?.UserId ?? course.TrainerProfile?.UserId — null-coalescing দিয়ে যেকোনো একটা source থেকে owner বের করা হয়। ResolveAuthorAsync কোর্স তৈরির সময় দুইটা repository চেক করে, কোনোটাই না থাকলে ConflictException।

- **Project example:** CourseService.cs — ResolveAuthorAsync, AuthorUserId, AuthorNameOf তিনটা helper method।

- **Follow-up:** "এটা কি একটা design smell না — polymorphic author কেন একটা shared AuthorProfile base class দিয়ে করা হলো না?" → Reasonable critique; current design nullable-FK দিয়ে দুইটা আলাদা domain concept (marketplace producer বনাম pure-academy trainer) জোড়া দিয়ে রেখেছে, cleaner would be an interface/base-type বা polymorphic association, কিন্তু EF Core-এ inheritance mapping জটিলতা এড়াতে এই shortcut নেওয়া হয়েছে।

Q2. Exam এবং Quiz-এর মধ্যে মূল পার্থক্য কী?

- **Short answer:** Exam-এ Essay question থাকতে পারে (manual grading দরকার), Quiz-এ শুধু MCQ, তাই fully auto-graded।

- **Detailed:** ExamQuestion.QuestionType enum (MultipleChoice / Essay), QuizQuestion -এ এই field-ই নেই। এর ফলে ExamService.SubmitAttemptAsync -এ branching আছে (hasEssayQuestions চেক করে status Submitted বা Evaluated), কিন্তু QuizService.SubmitAttemptAsync -এ সবসময় সরাসরি Status = AttemptStatus.Submitted with score calculated (কোনো conditional evaluated-state নেই — যদিও naming একটু বিভ্রান্তিকর, কারণ Quiz-এ Submitted কার্যত final)।

- **Project example:** ExamService.cs:256-308 vs QuizService.cs:249-284।

- **Follow-up:** "Quiz-এর AttemptStatus.Submitted আর Evaluated — Quiz কখনো Evaluated status পায় কি?" → কোডে QuizService -এ attempt.Status = AttemptStatus.Submitted সরাসরি সেট হয়, Evaluated কখনো সেট হয় না Quiz flow-তে — shared enum AttemptStatus দুটো entity-তে reuse হচ্ছে কিন্তু semantics কিছুটা আলাদা।

Q3. একজন student একই Exam কতবার দিতে পারে?

- **Short answer:** `Exam.MaxAttempts` null হলে unlimited, নাহলে সেই সংখ্যা পর্যন্ত; এবং একসাথে একটার বেশি `InProgress` attempt রাখা যায় না।
- **Detailed:** `StartAttemptAsync` -এ `priorAttempts.Count ≥ exam.MaxAttempts.Value` চেক আছে, আর `priorAttempts.Any(a => a.Status == AttemptStatus.InProgress)` চেক দিয়ে parallel-attempt block করা হয়।
- **Project example:** `ExamService.StartAttemptAsync`, লাইন ~200-210।
- **Follow-up:** "এটা কি abandoned attempt (user browser বন্ধ করে চলে গেছে, submit করেনি) permanently lock করে দেয় না?" → হ্যাঁ, এটা একটা real gap — কোনো timeout/auto-expire logic নেই `InProgress` attempt-এর জন্য, `TimeLimitMinutes` শুধু client-side display-এর জন্য ব্যবহৃত হয় (দেখুন Gotcha #2)।

Q4. Essay question কিভাবে গ্রেড হয়?

- **Short answer:** Auto-grade হয় না; trainer প্রতিটা essay answer-এ `EvaluateAnswerAsync` দিয়ে points বসায়, তারপর `FinalizeEvaluationAsync` কল করে final percentage/pass-fail নির্ধারণ করে।
- **Detailed:** `EvaluateAnswerAsync` -এ guard আছে — attempt status `Submitted` হতে হবে, question type `Essay` হতে হবে, আর `PointsAwarded` অবশ্যই `0..question.Points` range-এর মধ্যে। `FinalizeEvaluationAsync` check করে সব essay answer-এ `PointsAwarded is not null`, নাহলে `ConflictException`।
- **Project example:** `ExamService.cs:348-416`।
- **Follow-up:** "একই attempt-এর দুইটা essay question থাকলে trainer একটাই evaluate করে finalize করার চেষ্টা করলে কী হবে?" → `essayAnswers.Any(a => a.PointsAwarded is null)` true হবে, `ConflictException("All essay questions must be evaluated before finalizing ...")` throw হবে, 409 রিটার্ন।

Q5. Enrollment progress percentage কোথায় store হয়?

- **Short answer:** কোথাও persist করা হয় না — প্রতিবার request-এ `CompletedLessons/TotalLessons*100` ক্যালকুলেট করে দেখানো হয়।
- **Detailed:** `CalculateProgressPercent(CourseEnrollment enrollment)` static helper, `enrollment.LessonProgress.Count(p => p.IsCompleted)` divide `enrollment.Course.Lessons.Count`। Lesson add/delete হলেও progress সবসময় consistent থাকে কারণ derived value, stale cache নেই।
- **Project example:** `EnrollmentService.cs:253-263`।
- **Follow-up:** "Course-এ যদি কোনো lesson না থাকে (totalLessons=0), কী হবে?" → Division-by-zero এড়াতে explicit `if (totalLessons == 0) return 0;` guard আছে।

Q6. Course-এর "self-enrollment" prevent কিভাবে করা হয়?

- **Short answer:** `EnrollAsync` -এ `AuthorUserId(course) == apprenticeUserId` চেক করে `ConflictException` throw করা হয়।
- **Detailed:** এটা একটা সহজ business-rule guard, HTTP 409 হিসেবে রিটার্ন হয় `GlobalExceptionHandler` দিয়ে।
- **Project example:** `EnrollmentService.EnrollAsync`, লাইন ৩৫-৩৮।
- **Follow-up:** "যদি একজন user Mentor হিসেবে course বানায়, তারপর ওই একই user Trainer role-ও নেয়, তখন কি অন্য কারো TrainerProfile-এর course-এ enroll করতে পারবে?" → হ্যাঁ, কারণ `AuthorUserId` চেক শুধু সেই specific course-এর author-এর সাথে current user মিলছে কিনা দেখে, role-independent।

Q7. MentorMatchingService-এর scoring algorithm-টা কি AI/ML?

- **Short answer:** না, এটা pure rule-based weighted-sum heuristic — কোনো model training বা external AI call নেই।
- **Detailed:** Source-এ comment স্পষ্ট লেখা "Rule-based (not AI) matching engine"। প্রতিটা criterion-এর একটা fixed constant weight আছে (`SkillWeight=25`, `LocationWeight=15` ইত্যাদি), caller যা specify করে শুধু সেগুলোর weight-ই denominator-এ যোগ হয়।
- **Project example:** `MentorMatchingService.cs:1-10` (comment), `Score()` method।
- **Follow-up:** "নাম-এ 'matching' থাকলেও কেন এটা ML না?" → কোনো training data নেই, weight গুলো hardcoded constant (domain expert-defined), model inference call নেই — এটা classic expert-system-style scoring, "AI" folder-এর controller (যেমন `AIIntelligenceController`) থেকে সম্পূর্ণ আলাদা layer-এ থাকে।

Q8. totalWeight কেন request অনুযায়ী dynamic — কেন fixed ১০০ না?

- **Short answer:** যাতে caller যে criteria দেয়নি সেটা score কমিয়ে না দেয় — শুধু specify করা criteria-ই ভাগে অংশ নেয়।
- **Detailed:** যদি fixed ১০০ ব্যবহার করা হতো, আর caller শুধু Location দিত, বাকি সব unmatched হিসেবে count হয়ে score কৃত্রিমভাবে কমে যেত। Dynamic denominator দিয়ে "percentage of what was actually asked" বোঝানো হয়।
- **Project example:** `MentorMatchingService.Score()`, প্রতিটা `if` block-এ `totalWeight += X` conditionally।
- **Follow-up:** "দুইজন mentor — একজন সব criteria match করে score ৮০%, আরেকজন খালি skill criterion match করে (caller শুধু skill চেয়েছিল) score ১০০% — কোনটা ranked আগে আসবে?" → দ্বিতীয়জন (higher MatchScore প্রথমে), যেটা counter-intuitive মনে হতে পারে কিন্তু algorithm অনুযায়ী সঠিক কারণ সে যা চাওয়া হয়েছিল তার সবটাই পূরণ করেছে।

Q9. JobMatchingService-এ "partial credit" concept কী?

- **Short answer:** Required skills-এর সবগুলো না মিললেও, যতগুলো মিলেছে সেই fraction অনুযায়ী আংশিক weight পাওয়া যায়।
- **Detailed:** `achievedWeight += RequiredSkillWeight * (met / (decimal)requiredSkills.Count)` — যেমন ৩টার মধ্যে ২টা মিললে `RequiredSkillWeight * 2/3` পাওয়া যায়, all-or-nothing না।
- **Project example:** `JobMatchingService.cs:50-60`।
- **Follow-up:** "Required আর Preferred skill-এর weight আলাদা কেন (45 বনাম 20)?" → Required skill job-এর জন্য বাধ্যতামূলক তাই বেশি গুরুত্ব পায়, Preferred শুধু bonus-এর মতো।

Q10. SkillAssessment পাস করলে ঠিক কী পরিবর্তন হয়, এবং সেটা Job Matching-এ কিভাবে প্রভাব ফেলে?

- **Short answer:** Assessment run করলে `AcademyMemberProfile` -এর সংশ্লিষ্ট heritage skill-এর `SkillLevel` আপডেট হয়; `JobMatchingService` সেই profile-এর `Skills` dictionary থেকেই match করে।
- **Detailed:** `JobMatchingService.GetRecommendedJobsAsync` -এ `profile.Skills.ToDictionary(s => s.HeritageSkillId, s => s.Level)` — এই dictionary সরাসরি academy profile থেকে read হয়, matching-এর সময় fresh lookup, কোনো caching না।
- **Project example:** `SkillAssessmentsController.Run` → `JobMatchingService.cs:32-33`।
- **Follow-up:** "TrainingCertificatesController.IssueSkillCertificateAsync-এ কী guard আছে যেটা এই chain confirm করে?" → `recipientSkill.Level < SkillLevel.Advanced` হলে `ConflictException` — অর্থাৎ certificate পেতে হলে প্রথমে skill assessment দিয়ে Advanced level পৌঁছাতে হবে।

Q11. একটা conversation-এ কয়জন participant থাকতে পারে?

- **Short answer:** এই codebase-এ exactly ২ জন (1:1 messaging), group chat সাপোর্ট নেই।
- **Detailed:** `StartConversationAsync` -এ `Participants` list এ ঠিক দুইটা entry তৈরি হয় (sender + recipient), আর `GetBetweenUsersAsync` দুই user-এর মধ্যে

existing conversation খোঁজে।

- **Project example:** `MessagingService.StartConversationAsync`।

- **Follow-up:** "যদি group chat দরকার হতো, কী বদলাতে হতো?" → `ConversationParticipant` list-এ ২-এর বেশি entry allow করতে হবে, group-name/identification logic যোগ করতে হবে, `GetBetweenUsersAsync` -এর pairwise lookup পুরোপুরি redesign করতে হবে।

Q12. Message পাঠানোর সময় Push notification কিভাবে কাজ করে — REST নাকি SignalR?

- **Short answer:** দুটোই — message পাঠানো (persist) REST দিয়ে হয়, কিন্তু recipient-কে instant notify করা হয় SignalR দিয়ে।

- **Detailed:** `MessagingController.SendMessage` → `MessagingService.SendMessageAsync` প্রথমে DB-তে save করে, তারপর

`IMessageNotifier.NotifyMessageAsync` কল করে যেটা `SignalRMessageNotifier` implementation দিয়ে

`IHubContext<MessagingHub>.Clients.Group(...).SendAsync("ReceiveMessage", dto)` করে।

- **Project example:** `MessagingService.cs:104-133`, `SignalRMessageNotifier.cs`।

- **Follow-up:** "কেন সরাসরি Hub থেকে message পাঠানো হয় না, REST দরকার কেন?" → REST নিশ্চিত করে reliable persistence (ACK/retry সহজ, offline client পরে fetch করতে পারবে), SignalR শুধু "instant UX" এর জন্য push — যদি SignalR fail করে তাও message হারায় না।

Q13. IMessageNotifier interface Application layer-এ, implementation Api layer-এ কেন?

- **Short answer:** Dependency Inversion Principle মেনে চলার জন্য — Application layer SignalR-এর মতো infrastructure concern সম্পর্কে জানবে না।

- **Detailed:** Clean Architecture rule — inner layer (Application) বাইরের layer-কে depend করবে না। `IMessageNotifier` interface Application-এ define, `SignalRMessageNotifier` (আসল SignalR/Hub reference-সহ) Api layer-এ থাকে, Program.cs-এ DI দিয়ে wire হয়।

- **Project example:** `ShilpoHubBD.Application/Interfaces/Services/IMessageNotifier.cs` (interface) vs

`ShilpoHubBD.Api/Realtime/SignalRMessageNotifier.cs` (impl)।

- **Follow-up:** "এই pattern-এর সুবিধা কী — যদি ভবিষ্যতে SignalR-এর বদলে অন্য কোনো push mechanism (যেমন Firebase) ব্যবহার করতে হয়?" → শুধু নতুন `IMessageNotifier` implementation লিখে DI registration বদলালেই হবে, `MessagingService` -এ কোনো কোড বদলাতে হবে না।

Q14. SignalR-এ JWT authentication কিভাবে হয়, REST-এর থেকে আলাদা কেন?

- **Short answer:** Browser-এর WebSocket handshake-এ custom header (Authorization) সেট করা যায় না, তাই token query-string parameter (`?access_token=`) দিয়ে পাঠানো হয়, `JwtBearerEvents.OnMessageReceived` -এ সেটা intercept করে normal validation-এ পাঠানো হয়।

- **Detailed:** `Program.cs` -এ `OnMessageReceived` handler `context.Request.Query["access_token"]` check করে, path `/hubs` -দিয়ে শুরু হলেই সেটা accept করে (`context.Token = accessToken`)।

- **Project example:** `Program.cs`, লাইন ১২১-১৩৫।

- **Follow-up:** দেখুন Gotcha #1 নিচে।

Q15. Exam/Quiz-এর question option list .Options.Clear() করে আবার নতুন করে বানানো হয় Update-এর সময় কেন — কেন existing option গুলো modify করা হয় না?

- **Short answer:** Simplicity-র জন্য — পুরো option set replace করা হয়, incremental diff/merge logic এড়ানো হয়েছে।

- **Detailed:** `UpdateQuestionAsync` -এ `question.Options.Clear(); foreach (var option in request.Options) { question.Options.Add(new ...) }` — EF Core owned/dependent collection হিসেবে cascade delete+insert হয়ে যায় `SaveChangesAsync` -এ।

- **Project example:** `ExamService.UpdateQuestionAsync`, `QuizService.UpdateQuestionAsync`।

- **Follow-up:** "এই pattern-এর downside কী?" → প্রতিবার নতুন `Id` (`Guid.NewGuid()`) generate হয় option-গুলোর জন্য, তাই client-side আগের option id reference রাখলে সেটা invalid হয়ে যায়; কিন্তু যেহেতু `EnsureExamNotAttempted` / `EnsureQuizNotAttempted` guard আছে (attempt হয়ে গেলে edit-ই করা যায় না), তাই practically এটা সমস্যা তৈরি করে না।

Q16. Achievement/XP system-এ EvaluateAchievementsAsync কেন আলাদা endpoint — কেন সরাসরি automatic না?

- **Short answer:** Explicit trigger pattern — client (frontend) নির্দিষ্ট action (যেমন course complete, exam pass) হওয়ার পর নিজে থেকে `POST /api/achievements/evaluate` কল করে যাতে server user-এর current progress অনুযায়ী achievement unlock করে দেয়।

- **Detailed:** কোনো event-driven/domain-event system এই codebase-এ নেই (কোনো MediatR বা message bus দেখা যায়নি), তাই achievement check pull-based — client trigger করলে তবেই evaluate হয়।

- **Project example:** `AchievementsController.Evaluate` → `IAchievementService.EvaluateAchievementsAsync`।

- **Follow-up:** "এর downside কী?" → যদি client evaluate কল করতে ভুলে যায় (বা bug থাকে), achievement unlock নাও হতে পারে যদিও user criteria পূরণ করেছে — এটা event-driven approach-এর তুলনায় কম robust।

Q17. ApprenticeshipProgram আর Course-এর মধ্যে পার্থক্য কী?

- **Short answer:** Course হলো lesson/module-ভিত্তিক short-form online learning; ApprenticeshipProgram হলো milestone-ভিত্তিক longer-duration hands-on physical craft training।

- **Detailed:** Course-এ `Lessons`, `Modules`, `Materials` — read/watch-based; ApprenticeshipProgram-এ `TrainingMilestone` list, আর progress track হয় `ApprenticeEnrollment.UpdateMilestoneProgressAsync` দিয়ে প্রতিটা milestone-ভিত্তিক (probably percentage বা status per milestone)।

- **Project example:** `ApprenticeshipProgramsController.AddMilestone`, `ApprenticeEnrollmentsController.UpdateMilestoneProgress`।

- **Follow-up:** "দুটোই কি একই CertificateType ব্যবহার করে?" → না, `TrainingCertificate.Type` enum-এ আলাদা ভ্যালু আছে (`Course`, `Skill`, `Apprenticeship`) — certificate number prefix-ও আলাদা (`TRN`, `SKL`, `APR`)।

Q18. Certificate verification public কেন — কোনো auth লাগে না কেন?

- **Short answer:** Certificate verification-এর মূল উদ্দেশ্যই হলো third-party (employer, government body) যেন কোনো account ছাড়াই একটা certificate number দিয়ে authenticity check করতে পারে।

- **Detailed:** `CertificatesController.Verify` এবং `TrainingCertificatesController.Verify` — দুইটাতেই কোনো `[Authorize]` attribute নেই, `[ApiController]` -লেভেলেও class-wide `[Authorize]` নেই এই দুই controller-এ। Response-এ `IsValid`, revoked status, issuer/recipient name দেখানো হয়।

- **Project example:** `CertificateService.VerifyAsync`, `TrainingCertificateService.VerifyAsync`।

- **Follow-up:** "Revoked certificate verify করলে কী হয়?" → `IsValid = false` কিন্তু certificate-এর বাকি তথ্য (product name, issuer ইত্যাদি) তবুও রিটার্ন হয়, message-এ "This certificate has been revoked and is no longer valid." বলা হয় — অর্থাৎ পুরো তথ্য গোপন করা হয় না, শুধু validity flag false করা হয়।

Q19. Global `ValidationFilter` কিভাবে এই cluster-এ কাজ করে — উদাহরণ দাও।

- **Short answer:** কোনো controller-এ manual `ModelState.IsValid` চেক নেই; global `IAsyncActionFilter` প্রতিটা action argument-এ reflection দিয়ে matching `IValidator<T>` খুঁজে বের করে validate করে, fail করলে `FluentValidation.ValidationException` throw করে যেটা `GlobalExceptionHandler` 400-এ ম্যাপ করে।
- **Detailed:** যেমন `CreateExamRequest` -এর জন্য যদি `CreateExamRequestValidator : AbstractValidator<CreateExamRequest>` থাকে, সেটা DI-তে registered থাকলে automatically চলে আসবে, `ExamsController.Create` নিজে কিছু চেক করে না।
- **Project example:** `ShilpoHubBD.Api/Middlewares/ValidationFilter.cs` (cross-cluster shared pattern)।
- **Follow-up:** "Controller-এ কেন কোনো `try/catch` নেই?" → এটাই design decision — সব exception `GlobalExceptionHandler` (IExceptionHandler) দিয়ে centrally handle হয়, controller শুধু happy-path কোড লেখে।

Q20. `AutoMapper` ব্যবহার করা হয়েছে কি DTO mapping-এর জন্য?

- **Short answer:** না — যদিও README-তে দাবি থাকতে পারে, বাস্তবে সব mapping manual, private static `ToDto()` / `ToListItemDto()` helper method দিয়ে।
- **Detailed:** প্রতিটা service (`CourseService.ToDto`, `ExamService.ToDto`, `MessagingService.ToDto` ইত্যাদি) নিজের static mapping method লেখে, কোনো `IMapper` inject করা হয় না।
- **Project example:** সব সার্ভিস ফাইলের নিচের অংশ — যেমন `ExamService.cs:491-590`।
- **Follow-up (viva gotcha):** "তাহলে README-তে AutoMapper-এর কথা কেন আছে?" → এটা stale documentation, বাস্তব implementation-এর সাথে মেলে না — viva-তে honest থাকা ভালো, "আমরা manual mapping করি, README updated না" বলে দেওয়া উচিত।

Q21. `EnsureExamNotAttempted` / `EnsureQuizNotAttempted` — এই guard না থাকলে কী সমস্যা হতে পারত?

- **Short answer:** Trainer যদি attempt শুরু হয়ে যাওয়ার পর question বদলে ফেলত, তাহলে আগে যারা দিয়েছে আর পরে যারা দেবে তাদের মধ্যে unfair grading হতো (different question set, কিন্তু একই exam attempt হিসেবে count হতো)।
- **Detailed:** `if (exam.Attempts.Count > 0) throw new ConflictException(...)` — যেকোনো attempt (এমনকি InProgress) থাকলেই question-set locked হয়ে যায়।
- **Project example:** `ExamService.cs:444-450`।
- **Follow-up:** "এটা কি delete-এর ক্ষেত্রেও same guard?" → হ্যাঁ, `DeleteAsync` -এও `exam.Attempts.Count > 0` চেক করে exam delete block করে।

Q22. Course-এর `MaxApprentices` seat-limit কিভাবে enforce হয় — active নাকি সব enrollment count হয়?

- **Short answer:** শুধু `Status == Active` enrollment count হয়, completed/dropped বাদ যায়।
- **Detailed:** `_enrollmentRepository.GetActiveCountByCourseAsync(courseId, ...)` — এটা `EnrollmentStatus.Active` filter দিয়ে কাউন্ট করে।
- **Project example:** `EnrollmentService.EnrollAsync`, লাইন ৪৬-৫৩।
- **Follow-up:** "এই কাউন্ট আর concurrent enroll request একসাথে এলে race condition হবে কি?" → হ্যাঁ, সম্ভাব্য — দেখুন Gotcha #3।

5. Tricky / Gotcha Questions

G1. SignalR-এ JWT query-string দিয়ে পাঠানোর security trade-off কী?

Query string-এ token থাকলে সেটা সহজে log হয়ে যেতে পারে — web server access log, browser history, proxy log, referrer header (যদি hub URL থেকে কোনো external link click করা হয়)। এই codebase-এ mitigation যা আছে: (1) token শুধু `/hubs`-prefixed path-এ accept হয় (`context.HttpContext.Request.Path.StartsWithSegments("/hubs")`), বাকি সব সাধারণ API route header-only থাকে; (2) JWT-এর নিজস্ব short lifetime আছে (access token expiry), আর refresh token আলাদাভাবে HASHED (SHA256) স্টোর হয়, তাই leaked access token-এর damage window সীমিত। কিন্তু এখনো WSS (secure websocket, HTTPS-based) না হলে network sniffing দিয়ে token চুরি সম্ভব — production-এ TLS বাধ্যতামূলক হওয়া উচিত (যেটা এই appsettings-এ enforced কিনা আলাদাভাবে verify করা দরকার)। এটা industry-wide known trade-off, SignalR ডকুমেন্টেশনেই officially recommended pattern।

G2. Exam attempt-এ `TimeLimitMinutes` আছে, কিন্তু server-side এটা কি enforce হয়?

কোডে `TimeLimitMinutes` শুধু `ExamAttemptStartDto`-তে রিটার্ন হয় client-কে জানানোর জন্য, কিন্তু `SubmitAttemptAsync`-এ কোথাও `attempt.StartedAt` আর current time compare করে "সময় শেষ হয়ে গেছে" চেক নেই। অর্থাৎ এটা **client-side enforced only** — frontend টাইমার শেষ হলে auto-submit করতে পারে, কিন্তু কেউ যদি directly API কল করে (Postman দিয়ে), time-limit বাইপাস করে যত ইচ্ছা সময় নিয়ে submit করতে পারবে। এটা একটা real security/fairness gap যা viva-তে critical thinking demonstrate করতে ব্যবহার করা যায়।

G3. Course seat-limit (`MaxApprentices`) check এবং enroll — এর মধ্যে কি race condition সম্ভব?

হ্যাঁ। `GetActiveCountByCourseAsync` read আর নতুন `CourseEnrollment` insert — এই দুইটা operation একটা single DB transaction-এ atomic না (কোনো explicit transaction/lock দেখা যায়নি, শুধু `SaveChangesAsync` কল হয় শেষে)। যদি দুইজন student একই মুহুর্তে শেষ available seat-এর জন্য enroll করার চেষ্টা করে, দুইজনই count check-এ "seat আছে" দেখতে পারে (TOCTOU — time-of-check to time-of-use), এবং দুইজনই enroll হয়ে যেতে পারে, `MaxApprentices` overshoot করে। Fix করতে হলে DB-level unique constraint বা `SERIALIZABLE` isolation বা optimistic concurrency token লাগবে — বর্তমান কোডে সেটা নেই।

G4. Messaging কেন REST polling-এর বদলে SignalR Hub ব্যবহার করে?

REST polling (প্রতি কয়েক সেকেন্ডে `GET /conversations/{id}` কল করা) কাজ করত, কিন্তু: (1) latency — নতুন message আসতে polling interval পর্যন্ত দেরি হবে; (2) server load — হাজার হাজার active client প্রতি সেকেন্ডে polling করলে unnecessary DB hit বহুগুণ বেড়ে যায়, বেশিরভাগ polling-ই "no new data" রিটার্ন করবে; (3) SignalR push-based — শুধু actual event ঘটলেই (message পাঠানো হলে) server client-কে জানায়, idle সময়ে কোনো traffic নেই (WebSocket persistent connection ছাড়া)। Trade-off হলো complexity বাড়ে (Hub, group management, connection lifecycle handle করা লাগে) এবং horizontal scaling-এর জন্য backplane (Redis ইত্যাদি) দরকার হতে পারে মাল্টি-সার্ভার deployment-এ (এই codebase-এ কোনো SignalR backplane configured দেখা যায়নি — single-instance deployment assume করা হচ্ছে, এটা আরেকটা viva-worthy observation)।

G5. Certificate-এর authenticity/verification design কতটা tamper-proof?

`CertificateNumber` generate হয় `SH-{date}-{8 random hex}` ফরম্যাটে (`Guid.NewGuid().ToString("N")[..8]`) — এটা essentially random, কিন্তু কোনো cryptographic signature (HMAC/digital signature) certificate content-এর উপর নেই। Verification শুধু DB lookup করে (`GetByCertificateNumberAsync`) — অর্থাৎ security পুরোপুরি "certificate number secrecy না, বরং DB-র integrity"-র উপর নির্ভরশীল। যদি কেউ valid-লাগা একটা certificate number guess করতে পারে (৮ hex char = 32-bit space, brute-forceable যদিও practically কঠিন), সে অন্য কারো certificate verify করতে পারবে (data leak, কিন্তু data নিজে বদলাতে পারবে না, শুধু read)।

প্রকৃত tamper-proofing (যেমন QR code-এ embedded digital signature, blockchain anchor) এই ডিজাইনে নেই — এটা "trust the database" মডেল, enterprise-grade PKI-based verification না। যদিও `QRVerificationController` আলাদা আছে codebase-এ (marketplace product-এর জন্য), সেটাও Academy cluster-এর বাইরে, আলাদা patterns থাকতে পারে।

G6. Essay evaluation-এ `EvaluateAnswerAsync` বারবার কল করলে (একই answer-এ দুইবার grade দিলে) কী হয়?

কোডে কোনো "already evaluated" guard নেই এই specific answer-level-এ (শুধু attempt-level status `Submitted` চেক হয়) — অর্থাৎ trainer ভুলবশত একই essay answer-এ দুইবার আলাদা points দিতে পারবে, দ্বিতীয়বার শুধু overwrite হবে (`answer.PointsAwarded = request.PointsAwarded;`), কোনো audit trail বা history রাখা হয় না কে কখন কত দিয়েছিল। এটা acceptable behavior হতে পারে (correction allow করার জন্য ইচ্ছাকৃত), কিন্তু viva-তে জিজ্ঞেস করলে বলা উচিত এটা design choice, কোনো explicit "immutable grade" invariant নেই।

Heritage & Tourism Cluster — Viva Prep

1. Cluster Overview

ShilpoHubBD-এর "Heritage & Tourism" cluster হলো সেই অংশ যেখানে দেশের ঐতিহ্য (heritage sites, festivals, cultural events, local cuisine, museum artifacts) discover করা যায়, village-tourism বুক করা যায়, আর user-দের একটা gamified "digital heritage passport" থাকে যেখানে check-in করলে XP এবং badge পাওয়া যায়। এর সাথে জড়িয়ে আছে একটা physical-item authenticity verification system (QR code দিয়ে handicraft product আসল না নকল যাচাই করা) এবং তার উপর বসানো একটা AR layer (craft item scan করলে producer-এর heritage story দেখানো)। ডোমেইনটা মূলত ৬টা namespace-এ ভাগ করা: **HeritageDiscovery** (Places/Routes/Festivals/CulturalEvents/LocalCuisine), **Community** (Village), **ArVr** (VillageTourStop/MuseumItem/CulturalStory/CraftScan), **Passport** (Badge/HeritageCheckIn/TravelJournalEntry), **QRVerification** (QRCode/QRVerificationRecord), এবং **TouristBooking** (TouristService/Booking)। Achievement/XP system টা আসলে একটা আলাদা cross-cutting module (**Achievement** namespace, **AchievementsController**) যেটাকে Passport module reuse করে।

Typical request flow ধরা যাক এভাবে: একজন user **GET /api/heritage-places/nearby?lat=..&lng=..** দিয়ে কাছাকাছি heritage place খুঁজে পায় (anonymous access — কোনো auth লাগে না, কারণ discovery public)। জায়গায় গিয়ে সে **POST /api/passport/checkins** কল করে, নিজের GPS coordinate পাঠায়। **PassportService.CheckInAsync** Haversine formula দিয়ে distance যাচাই করে (500 metre-এর মধ্যে থাকতে হবে), দিনে একবারই check-in অনুমোদিত, আর success হলে সরাসরি **IAchievementService.AwardXpAsync** কল করে ২০ XP দেয় এবং automatically **EvaluateCheckInBadgesAsync** চালিয়ে দেখে কোনো CheckIn-badge (যেমন "5 জায়গায় visit") unlock হলো কিনা। এভাবে **HeritagePlace → HeritageCheckIn → XpTransaction + UserBadge** — একটা single action তিনটা ভিন্ন entity/table touch করে, এটাই এই cluster-এর সবচেয়ে interesting business-rule chain।

দ্বিতীয় flow: একজন Producer তার handicraft product-এর জন্য **POST /api/qr-verification/generate** কল করে একটা unique **QRCode** (GUID-based string) বানায়, সেটা প্রোডাক্টের গায়ে প্রিন্ট/লেবেল করে দেয়। Customer কেনার আগে সেই কোড scan করে **POST /api/qr-verification/verify** কল করে — কোনো auth লাগে না (anonymous scan-ও কাজ করে, **CurrentUserIdOrNull** nullable), backend প্রতিটা scan-কে **QRVerificationRecord**-এ log করে (audit trail — genuine বা fake দুটোই log হয়), আর producer/district সহ "verified authentic" বা "revoked/not found" message ফেরত দেয়। এই একই **QRCode / QRVerificationRecord** টেবিল আবার **ArCraftScanController** দিয়ে reuse হয় — AR marker scan মানেই আসলে QR verify, শুধু response-এ producer-এর craft story, traceability, certificate ডেটা জুড়ে দেওয়া হয় AR overlay-র জন্য (কোডের কमेंটেই লেখা আছে এই design decision)। Booking flow-টা আলাদা, marketplace booking-এর মতোই: **TouristService → AvailabilitySlot (capacity-checked) → Booking (Pending→Confirmed/Rejected→Completed/Cancelled state machine)**।

2. Deep Dive — 4 Representative Controllers

2.1 PassportController (**api/passport**) — Gamification Engine

File: ShilpoHubBD.Api/Controllers/PassportController.cs

Service: ShilpoHubBD.Application/Services/Passport/PassportService.cs

Verb	Route	Auth
GET	/api/passport/badges	Anonymous
GET	/api/passport/badges/mine	[Authorize]
POST	/api/passport/badges	SuperAdmin
POST	/api/passport/badges/claim/district	[Authorize]
POST	/api/passport/badges/claim/festival	[Authorize]
POST	/api/passport/badges/evaluate-purchases	[Authorize]
POST	/api/passport/checkins	[Authorize]
GET	/api/passport/checkins/mine	[Authorize]
POST	/api/passport/journal	[Authorize]
GET	/api/passport/journal/mine	[Authorize]
PUT	/api/passport/journal/{id:guid}	[Authorize]
DELETE	/api/passport/journal/{id:guid}	[Authorize]

Most important method — **CheckInAsync (**PassportService.cs** lines 166–214):**

```

public async Task<CheckInDto> CheckInAsync(Guid userId, CreateCheckInRequest request, CancellationToken cancellationToken)
{
    var place = await _heritagePlaceRepository.GetByIdAsync(request.HeritagePlaceId, cancellationToken)
        ?? throw new NotFoundException("Heritage place not found.");

    if (!place.IsActive)
        throw new ConflictException("This heritage place is not active.");

    if (request.Latitude.HasValue && request.Longitude.HasValue)
    {
        var distanceKm = HaversineDistanceKm(request.Latitude.Value, request.Longitude.Value, place.Latitude, place.Longitude);
        if (distanceKm > CheckInMaxDistanceKm)
            throw new ConflictException("You must be near the heritage place to check in.");
    }

    var checkInDate = DateOnly.FromDateTime(now);
    if (await _checkInRepository.ExistsForDateAsync(userId, place.Id, checkInDate, cancellationToken))
        throw new ConflictException("You have already checked in to this place today.");

    var checkIn = new HeritageCheckIn { Id = Guid.NewGuid(), UserId = userId, HeritagePlaceId = place.Id, ... };
    await _checkInRepository.AddAsync(checkIn, cancellationToken);
    await _checkInRepository.SaveChangesAsync(cancellationToken);

    await EvaluateCheckInBadgesAsync(userId, cancellationToken);
    await _achievementService.AwardXpAsync(
        new AwardXpRequest { UserId = userId, Amount = CheckInXpReward, Reason = $"Heritage check-in: {place.Name}" },
        cancellationToken);

    checkIn.HeritagePlace = place;
    return ToCheckInDto(checkIn);
}

```

Banglish explanation:

- প্রথমে **HeritagePlace** fetch করে দেখে exist করে কিনা, তারপর **IsActive** চেক করে — inactive/archived জায়গায় check-in বন্ধ।
- Location spoofing ঠেকাতে **Latitude / Longitude** optional (client না পাঠালে skip হয়ে যায় — এটা একটা gotcha, নিচে দেখুন), পাঠালে **Haversine distance formula** দিয়ে user আর heritage place-এর মধ্যে দূরত্ব km-এ বের করা হয় (constants: **EarthRadiusKm = 6371.0**, **CheckInMaxDistanceKm = 0.5** অর্থাৎ ৫০০ মিটার)।
- **DateOnly.FromDateTime(now)** দিয়ে আজকের তারিখ বের করে **ExistsForDateAsync** চেক করে — একই জায়গায় একই দিনে দ্বিতীয়বার check-in আটকায় (day-based dedup, timestamp-based না)।
- Entity বানিয়ে DB-তে save করার পর দুইটা side-effect চালায়: (1) **EvaluateCheckInBadgesAsync** — user-এর total check-in count নিয়ে সব **CheckIn** -type badge-এর **RequiredCheckInCount** এর সাথে তুলনা করে, threshold পার হলে award করে; (2) **IAchievementService.AwardXpAsync** — ফিক্সড ২০ XP দেয় (**CheckInXpReward = 20**), Reason string-এ heritage place-এর নাম বসে যায়।
- লক্ষণীয়: XP award আর badge evaluation-এর মধ্যে কোনো DB transaction wrap করা নেই — প্রতিটা **SaveChangesAsync** আলাদা আলাদা commit।

Haversine helper:

```

private static double HaversineDistanceKm(double lat1, double lon1, double lat2, double lon2)
{
    var dLat = ToRadians(lat2 - lat1);
    var dLon = ToRadians(lon2 - lon1);
    var a = Math.Sin(dLat / 2) * Math.Sin(dLat / 2)
        + Math.Cos(ToRadians(lat1)) * Math.Cos(ToRadians(lat2)) * Math.Sin(dLon / 2) * Math.Sin(dLon / 2);
    var c = 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));
    return EarthRadiusKm * c;
}

```

Great-circle distance বের করার standard spherical trigonometry formula — সরাসরি Euclidean distance (Pythagoras) ব্যবহার করা হয়নি কারণ lat/long degrees-এ, পৃথিবীর curvature ignore করলে ভুল distance আসবে (বিশেষ করে বড় distance-এ)।

Badge system business rules:

- BadgeType** enum-এ ৪ ধরনের badge: **District**, **Festival**, **Purchase**, **CheckIn**। প্রতিটার claim-rule আলাদা:
- **District badge** — **ClaimDistrictBadgeAsync**: user-কে সেই district থেকে product-এর একটা completed order থাকতে হবে (**IOrderRepository.HasCompletedOrderFromDistrictAsync**) — অর্থাৎ tourism module marketplace-এর সাথে cross-check করে।
 - **Festival badge** — **ClaimFestivalBadgeAsync**: **BadgeId** দিয়ে সরাসরি claim, কিন্তু **badge.Type != BadgeType.Festival** হলে reject — self-service claim (কোনো verification নেই যে user আসলেই festival attend করেছে কিনা — এটাও একটা gotcha)।

- **Purchase badge** — `EvaluatePurchaseBadgesAsync`: `RequiredPurchaseCount` threshold-এর সাথে completed order count তুলনা।
 - **CheckIn badge** — automatically `CheckInAsync` -এর ভেতর থেকেই trigger হয়।
- সবগুলোতেই duplicate award ঠেকাতে `HasUserBadgeAsync(userId, badge.Id)` চেক থাকে — `ConflictException("You have already earned this badge.")`।

XP/Level system (`AchievementService.cs` , cross-referenced service):

```
private const int XpPerLevel = 500;
...
var xpIntoLevel = totalXp % XpPerLevel;
return new XpSummaryDto
{
    TotalXp = totalXp,
    Level = 1 + totalXp / XpPerLevel,
    XpIntoCurrentLevel = xpIntoLevel,
    XpForNextLevel = XpPerLevel,
    XpToNextLevel = XpPerLevel - xpIntoLevel,
};
```

Simple linear leveling: প্রতি ৫০০ XP-তে এক level up, `TotalXp` কোনো cached column না — প্রতিবার সব `XpTransaction` sum করে (`GetTotalXpAsync`) live calculate হয়। এর মানে XP ledger (append-only transaction log) থেকে derive করা হয়, direct mutable counter না — data integrity-র জন্য ভালো design (audit trail থাকে, কিন্তু history বড় হলে query খরচ বাড়ে)।

2.2 QRVerificationController (`api/qr-verification`) — Anti-Counterfeit Verification

File: `ShilpoHubBD.Api/Controllers/QRVerificationController.cs`

Service: `ShilpoHubBD.Application/Services/QRVerification/QRVerificationService.cs`

Verb	Route	Auth
POST	<code>/api/qr-verification/verify</code>	Anonymous
POST	<code>/api/qr-verification/generate</code>	Producer, SuperAdmin
POST	<code>/api/qr-verification/{id:guid}/revoke</code>	Producer, SuperAdmin
GET	<code>/api/qr-verification/history/mine</code>	[Authorize]
GET	<code>/api/qr-verification/products/{productId:guid}/history</code>	Producer, SuperAdmin

GenerateAsync — QR code creation (lines 21–51):

```
public async Task<QRCodeDto> GenerateAsync(Guid producerId, GenerateQRCodeRequest request, CancellationToken cancellationToken)
{
    var product = await _productRepository.GetByIdAsync(request.ProductId, cancellationToken)
        ?? throw new NotFoundException("Product not found.");

    if (product.ProducerId != producerId)
        throw new UnauthorizedAccessException("You can only generate QR codes for your own products.");

    var existing = await _qrVerificationRepository.GetActiveByProductIdAsync(request.ProductId, cancellationToken);
    if (existing is not null)
        return ToDto(existing); // idempotent: same active code returned again

    var qrCode = new QRCode
    {
        Id = Guid.NewGuid(),
        ProductId = request.ProductId,
        Code = Guid.NewGuid().ToString("N"), // ← the actual scannable string
        IsActive = true,
        CreatedAt = DateTime.UtcNow,
    };
    await _qrVerificationRepository.AddQRCodeAsync(qrCode, cancellationToken);
    await _qrVerificationRepository.SaveChangesAsync(cancellationToken);
    ...
}
```

Banglish: `Code` field আসলে একটি random 32-char hex string — `Guid.NewGuid().ToString("N")` (dashবিহীন GUID)। এটাই আসল "QR code content" — frontend এই string-কে QR image-এ encode করে product-এর গায়ে বসিয়ে দেয়। Ownership check strict — শুধু নিজের product-এর জন্যই কোড বানানো যায়। Idempotency ভালো: একই product-এর জন্য বারবার generate call করলে নতুন code না বানিয়ে existing active code-ই ফেরত দেয় — duplicate active QR code তৈরি

আটকায়।

VerifyAsync — the actual anti-counterfeit check (lines 72–124):

```
public async Task<QRVerificationResultDto> VerifyAsync(Guid? userId, VerifyQRRequest request, CancellationToken cancellationToken)
{
    var code = request.Code.Trim();
    var qrCode = await _qrVerificationRepository.GetByCodeAsync(code, cancellationToken);
    var isValid = qrCode is { IsActive: true };

    var record = new QRVerificationRecord
    {
        Id = Guid.NewGuid(), ScannedCode = code, QRCodeId = qrCode?.Id,
        VerifiedByUserId = userId, IsValid = isValid, VerifiedAt = now,
    };
    await _qrVerificationRepository.AddVerificationRecordAsync(record, cancellationToken);
    await _qrVerificationRepository.SaveChangesAsync(cancellationToken);

    if (qrCode is null)
        return new QRVerificationResultDto { IsValid = false, Message = "No product was found for this QR code.", ... };

    if (!qrCode.IsActive)
        return new QRVerificationResultDto { IsValid = false, ProductId = ..., Message = "This QR code has been revoked and is no longer valid.", ... };

    return new QRVerificationResultDto { IsValid = true, ProducerName = ..., District = ..., Message = "This product is verified authentic.", ... };
}
```

Banglish: এখানেই আসল "authentic vs fake" decision হয়। তিনটা outcome সম্ভব:

1. `qrCode == null` → scan করা code system-এ কখনোই তৈরি হয়নি → fake/unrecognized ("No product was found") — অর্থাৎ counterfeit item-এ product-এর নাম মিলিয়ে একটা QR বসিয়ে দিলেও, তার `Code` string আমাদের DB-তে না থাকায় ধরা পড়ে যাবে।
2. `qrCode != null` কিন্তু `IsActive == false` → producer নিজেই revoke করেছে (হয়ত সেই batch-এ defect পাওয়া গেছে, বা product delist হয়েছে) → "revoked" message।
3. `qrCode != null && IsActive == true` → genuine, `ProducerName` + `District` সহ পূর্ণ তথ্য দেখানো হয়।

সবচেয়ে গুরুত্বপূর্ণ: verify anonymous endpoint (`CurrentUserIdOrNull`), কিন্তু success/fail নির্বিশেষে প্রতিটা `scan attempt` `QRVerificationRecord` -এ log হয় (audit trail) — কেউ যদি বারবার fake code try করে সেটাও ধরা পড়বে history-তে।

Business rule (how authenticity is technically anchored): Authenticity মূলত database-level trust — QR code-টা নিজে cryptographically signed না, শুধু একটা random unguessable GUID যেটা producer-এর নিজের product-এর সাথে DB-তে linked। "Fake" মানে হলো: attacker যদি নিজে কোনো random code বানিয়ে বসিয়ে দেয়, সেটা DB match না করায় ধরা পড়বে (GUID-space এত বড় যে brute-force guess practically অসম্ভব)। কিন্তু legit code copy করে (cloning) অন্য fake product-এ বসিয়ে দিলে সিস্টেম সেটা ধরতে পারবে না — কারণ verify শুধু code string match করে, physical item-এর সাথে bind করা কোনো extra secret নেই (নিচে Tricky Questions দেখুন)।

2.3 HeritagePlacesController (`api/heritage-places`) — Discovery CRUD + Geo Query

File: `ShilpoHubBD.Api/Controllers/HeritagePlacesController.cs`

Service: `ShilpoHubBD.Application/Services/HeritageDiscovery/HeritagePlaceService.cs`

Verb	Route	Auth
GET	<code>/api/heritage-places</code>	Anonymous
GET	<code>/api/heritage-places/nearby</code>	Anonymous
GET	<code>/api/heritage-places/{id:guid}</code>	Anonymous
POST	<code>/api/heritage-places</code>	SuperAdmin
PUT	<code>/api/heritage-places/{id:guid}</code>	SuperAdmin
DELETE	<code>/api/heritage-places/{id:guid}</code>	SuperAdmin

Controller একদম thin — প্রতিটা action এক লাইনে expression-bodied ⇒ `Ok(await _placeService.XAsync( ... ))` । এই pattern-টা platform-জুড়ে বেশিরভাগ discovery controller-এ (`HeritageRoutesController`, `HeritageFestivalsController`, `CulturalEventsController`, `LocalCuisinesController`, `CulturalStoriesController`, `MuseumItemsController`, `VillageTourController`) দেখা যায় — সব CRUD মূলত read-open + admin-write pattern অনুসরণ করে।

`GetNearby` interesting কারণ `NearbyHeritagePlaceQueryParameters` -এ lat/long/radius থাকে এবং এটাই `PassportController.CheckInAsync` -এর সাথে conceptually জড়িত — user আগে nearby দিয়ে জায়গা খুঁজে পায়, তারপর check-in করে। `HeritagePlace` entity-তে `Latitude` / `Longitude` (double), `IsFeatured`, `IsActive`, `AverageRating`, `ReviewCount` fields আছে, আর `Festivals` / `Events` / `CuisineItems` navigation collections — অর্থাৎ একটা place-এর ভেতরেই তার festivals, cultural events, local cuisine সব nested হয়ে আছে (one-to-many)।

Business rule: শুধু `SuperAdmin` নতুন heritage place add/edit/delete করতে পারে — content moderation কেন্দ্রীভূত, producer/tourist কেউ নিজে heritage place বানাতে পারে না (তথ্যের সঠিকতা নিশ্চিত করতে)।

2.4 BookingsController (`api/bookings`) — Tourist Service Reservation Lifecycle

File: `ShilpoHubBD.Api/Controllers/BookingsController.cs`
Service: `ShilpoHubBD.Application/Services/TouristBooking/BookingService.cs`

Verb	Route	Auth
POST	<code>/api/bookings</code>	<code>[Authorize]</code> (class-level)
GET	<code>/api/bookings/mine</code>	<code>[Authorize]</code>
GET	<code>/api/bookings/provider</code>	<code>Producer, SuperAdmin</code>
GET	<code>/api/bookings/{id:guid}</code>	<code>[Authorize]</code>
PUT	<code>/api/bookings/{id:guid}/confirm</code>	<code>Producer, SuperAdmin</code>
PUT	<code>/api/bookings/{id:guid}/reject</code>	<code>Producer, SuperAdmin</code>
PUT	<code>/api/bookings/{id:guid}/cancel</code>	<code>[Authorize]</code>
PUT	<code>/api/bookings/{id:guid}/complete</code>	<code>Producer, SuperAdmin</code>

CreateAsync — capacity-checked booking (lines 26–86):

```
var slot = await _slotRepository.GetByIdAsync(request.AvailabilitySlotId, cancellationTokens)
    ?? throw new NotFoundException("Availability slot not found.");
if (slot.ServiceId != service.Id) throw new ConflictException("The selected slot does not belong to this service.");
if (!slot.IsActive) throw new ConflictException("This slot is no longer available.");
if (slot.StartAt <= DateTime.UtcNow) throw new ConflictException("You cannot book a slot that has already started.");

var bookedCount = await _bookingRepository.GetActiveCountForSlotAsync(slot.Id, cancellationTokens);
if (bookedCount + request.PartySize > slot.Capacity)
    throw new ConflictException("This slot does not have enough remaining capacity.");

var booking = new Booking { ... , TotalPrice = service.Price * request.PartySize, Status = BookingStatus.Pending, ... };
```

Banglish: Overbooking ঠেকাতে প্রতিবার নতুন booking তৈরির আগে সেই slot-এর current active booked-count query করে (`GetActiveCountForSlotAsync`) + নতুন `PartySize` যোগ করে `slot.Capacity`-র সাথে তুলনা করা হয়। `service.ProducerId == touristId` হলে "নিজের সার্ভিস নিজে বুক" করা আটকায়। `TotalPrice` server-side calculate হয় (`Price * PartySize`) — client থেকে price পাঠানো যায় না, tampering আটকায়।

State machine (`BookingStatus` enum): `Pending` → `Confirmed` → `Completed` (happy path, producer action লাগে প্রতি ধাপে) অথবা `Pending` → `Rejected` / `Pending|Confirmed` → `Cancelled` / `NoShow` (defined কিন্তু কোনো endpoint থেকে set হতে দেখা যায়নি — future use)। প্রতিটা transition guard করা strict status check দিয়ে — যেমন `ConfirmAsync`-এ `if (booking.Status != BookingStatus.Pending) throw ConflictException`। Authorization তিন স্তরের: booking owner (tourist) নিজের booking দেখতে/cancel করতে পারে, producer (owner of service) confirm/reject/complete করতে পারে, `isAdmin` সব bypass করতে পারে।

3. Reference Table — Remaining Controllers

Controller	Route prefix	Key endpoints	Auth/Roles	Purpose
HeritageRoutesController	<code>/api/heritage-routes</code>	GET (paged), GET <code>/recommended</code> , GET <code>/id</code> , POST/PUT/DELETE, POST <code>/id/stops</code> , DELETE <code>/id/stops/{stopId}</code> , PUT <code>/id/stops/reorder</code>	Read anonymous; write <code>SuperAdmin</code>	Multi-stop heritage tour routes with orderable stops
HeritageFestivalsController	<code>/api/heritage-festivals</code>	GET (paged), GET <code>/id</code> , POST/PUT/DELETE	Read anonymous; write <code>SuperAdmin</code>	Festival catalog tied to heritage places
HeritageSkillsController	<code>/api/heritage-skills</code>	GET (<code>activeOnly</code> query), POST	Read anonymous; write <code>SuperAdmin</code>	Traditional-skill directory (for learning/academy linkage)
HeritageIdentityController	<code>/api/heritage-identity</code>	GET <code>/verified</code> , GET <code>/producerId</code> , PUT <code>/producerId</code> , POST <code>/producerId/verify</code> , DELETE, GET <code>/producerId/score</code> , GET <code>/producerId/score/history</code> , POST <code>/producerId/score/recalculate</code>	Read anonymous; upsert <code>Producer, SuperAdmin</code> ; verify+recalc restricted	Producer's family/heritage lineage profile + "Legacy Score"
VillagesController	<code>/api/villages</code>	GET, GET <code>/favorites</code> , GET <code>/id</code> , POST/PUT/DELETE, POST/DELETE <code>/id/favorite</code>	Read anonymous; write <code>SuperAdmin</code> ; favorite <code>[Authorize]</code>	Village-tourism destination catalog + user favorites

Controller	Route prefix	Key endpoints	Auth/Roles	Purpose
CulturalEventsController	/api/cultural-events	GET (paged), GET /{id}, POST/PUT/DELETE	Read anonymous; write SuperAdmin	Event calendar tied to heritage places
CulturalStoriesController	/api/cultural-stories	GET (paged), GET /{id}, POST/PUT/DELETE	Read anonymous; write SuperAdmin	Multi-chapter narrative content (for AR/digital storytelling)
LocalCuisinesController	/api/local-cuisines	GET (paged), GET /{id}, POST/PUT/DELETE	Read anonymous; write SuperAdmin	Regional food catalog
MuseumItemsController	/api/museum-items	GET (paged), GET /{id}, POST/PUT/DELETE	Read anonymous; write SuperAdmin	Digital museum artifact catalog with media
TouristServicesController	/api/tourist-services	GET (paged), GET /mine, GET /{id}, POST/PUT/DELETE, GET/POST /{serviceId}/availability-slots, PUT/DELETE /availability-slots/{slotId}	Read anonymous; manage Producer, SuperAdmin	Bookable tour/service listings + capacity slots
TouristAnalyticsController	/api/tourist-analytics	GET /visited-locations, /popular-destinations (anon), /bookings, /spending, /favorite-categories, /festival-participation, /district-coverage, /achievements	Class-level [Authorize], one [AllowAnonymous] override	Personal + aggregate tourism analytics dashboards
AITourismController	/api/ai-tourism	POST /tour-plan, /budget-plan, /route-optimization, /translate, /cultural-recommendations	Anonymous (no [Authorize])	AI-assisted trip planning (rule-based DummyAITourismProvider in Infrastructure)
KnowledgeGraphController	/api/knowledge-graph	CRUD /nodes, /relationships, GET /nodes/{id}/neighbors, /nodes/{id}/traverse, /paths, /networks/{network}	Class-level HeritageInnovationHub, GovernmentNGO, SuperAdmin	Graph of heritage entities/relationships for research traversal queries
ArCraftScanController	/api/ar-vr/craft-scans	POST /	Anonymous	Reuses QRVerification tables; decorates result with craft/producer story + certificate for AR overlay

4. Viva Question Bank

Q1. PassportController-এ check-in করার সময় location validation কেন optional করা হলো — যদি client Latitude / Longitude না পাঠায়?

- **Short answer:** কারণ সব client (বিশেষ করে পুরনো মোবাইল ব্রাউজার/desktop) থেকে GPS পাওয়া যায় না; backend সেটাকে hard requirement না করে soft-validate করে।

- **Detailed:** CreateCheckInRequest.Latitude / Longitude nullable double? | CheckInAsync -এ if (request.Latitude.HasValue && request.Longitude.HasValue) — তবেই distance check চলে; না দিলে সরাসরি check-in accept হয়ে যায়।

- **Project example:** PassportService.cs লাইন ১৭৬-১৮৩।

- **Follow-up:** এটা কি security gap? — হ্যাঁ, এটাই gotcha #1 (নিচে দেখুন): client location না পাঠালেই geofencing পুরোপুরি bypass হয়ে যায়।

Q2. Haversine formula কী এবং কেন সরাসরি Euclidean distance ব্যবহার করা হলো না?

- **Short answer:** Haversine একটা great-circle distance formula, পৃথিবীর গোলাকার surface-এর উপর দুই lat/long point-এর প্রকৃত দূরত্ব বের করে।

- **Detailed:** lat/long degree-ভিত্তিক coordinate, সরাসরি Pythagoras (Euclidean) দিয়ে distance নিলে ভুল ফল আসবে কারণ longitude-এর প্রতি ডিগ্রি দূরত্ব latitude-এর উপর নির্ভর করে (equator vs pole)। Haversine $\sin / \cos / \atan2$ ব্যবহার করে সঠিক km distance বের করে, EarthRadiusKm = 6371.0 দিয়ে scale করে।

- **Project example:** PassportService.HaversineDistanceKm |

- **Follow-up:** ৫০০ মিটার threshold hardcoded কেন, config করা যেত না? — হ্যাঁ, const double CheckInMaxDistanceKm = 0.5 — configurable করলে flexibility বাড়ত (IOption pattern দিয়ে, যেমন HeritageScoreOptions -এ করা আছে অন্য জায়গায়)।

Q3. একই দিনে দুইবার একই জায়গায় check-in আটকানো কীভাবে হয়?

- **Short answer:** HeritageCheckIn.CheckInDate (a DateOnly) দিয়ে ExistsForDateAsync(userId, placeId, checkInDate) চেক করে।

- **Detailed:** DateOnly.FromDateTime(DateTime.UtcNow) দিয়ে শুধু date part (time বাদ) বের করা হয়, তাই একই UTC ক্যালেন্ডার দিনে দ্বিতীয়বার চেষ্টা করলে ConflictException (HTTP 409)।

- **Project example:** PassportService.cs লাইন ১৮৮-১৯১, IHeritageCheckInRepository.ExistsForDateAsync |

- **Follow-up:** UTC midnight boundary-তে timezone সমস্যা হতে পারে কিনা? — হ্যাঁ, Bangladesh (UTC+6) local midnight আর UTC midnight আলাদা, তাই ঠিক মধ্যরাতে ২ বার check-in সম্ভব — এটা আরেকটা gotcha।

Q4. XP এবং Badge award-এর মধ্যে কোনো database transaction আছে কি? না থাকলে কী রিস্ক?

- **Short answer:** না, নেই — CheckInAsync -এ check-in save, badge evaluation, XP award — তিনটাই আলাদা SaveChangesAsync call, কোনো IDbContextTransaction / TransactionScope wrap করা নেই।

- **Detailed:** যদি XP award করার সময় exception হয় (network/DB issue), check-in আর badge ইতিমধ্যে commit হয়ে গেছে, কিন্তু XP miss হয়ে যাবে — data inconsistency।

- **Project example:** PassportService.CheckInAsync লাইন ২০৪-২১০।

- **Follow-up:** কীভাবে ঠিক করবে? — EF Core-এর ExecutionStrategy /explicit transaction দিয়ে সব operation একসাথে atomic করা যেত, অথবা event-driven/outbox pattern।

Q5. QR code-এর Code field কীভাবে generate হয় এবং এটা কতটা unguessable?

- **Short answer:** `Guid.NewGuid().ToString("N")` — ১২৮-বিট random GUID, dash ছাড়া ৩২-character hex string।
- **Detailed:** GUID version 4 randomness এত বেশি (2^{122} effective entropy) যে brute-force guess practically impossible। কিন্তু এটা cryptographic signature না — শুধু "hard to guess" random ID, cryptographically-signed token (HMAC ইত্যাদি) না।
- **Project example:** `QRVerificationService.GenerateAsync` লাইন ৪১।
- **Follow-up:** DB-তে Code column-এ কোনো unique constraint/index আছে কি? — **না!** `QRVerificationRecordConfiguration.cs` -এ শুধু `QRCodeId`, `VerifiedByUserId` -এর উপর index আছে, `QRCode.Code` -এর কোনো `IEntityTypeConfiguration<QRCode>` file-ই পাওয়া যায়নি (EF convention default দিয়ে চলছে, explicit unique index নেই) — এটা একটা বড় gotcha।

Q6. QR verify করার সময় fake code scan করলে কী হয়, আর সেটা কীভাবে "fake" ধরা পড়ে?

- **Short answer:** `GetByCodeAsync(code)` কোনো match না পেলে `qrCode == null`, response `IsValid = false`, `Message = "No product was found for this QR code."`
- **Detailed:** এই attempt-ও `QRVerificationRecord` -এ log হয় (`ScannedCode = code`, `QRCodeId = null`, `IsValid = false`) — তাই fake scan-ও audit trail-এ থেকে যায়, admin/producer পরে দেখতে পারে কতবার fake code try হয়েছে।
- **Project example:** `QRVerificationService.VerifyAsync` লাইন ৯২-১০০।
- **Follow-up:** legit QR code-এর ছবি তুলে/copy করে অন্য নকল product-এ বসালে ধরা যাবে কি? — **না** (Tricky Q দেখুন)।

Q7. Revoke করার পর সেই QR code স্ক্যান করলে কী রেসপন্স আসে, এবং কেন `NotFoundException` না দিয়ে আলাদা branch আছে?

- **Short answer:** `qrCode.IsActive == false` হলে "This QR code has been revoked and is no longer valid." সহ `ProductId / ProductName` -ও ফেরত দেওয়া হয় (পুরোপুরি hide করা হয় না)।
- **Detailed:** এটা UX decision — revoked হওয়া মানে কোড valid ছিল একসময়, এখন producer/admin নিজে বন্ধ করেছে (হয়তো defect batch), তাই user-কে বলা দরকার এটা কোন product ছিল, "not found" বলে দিলে confusing হতো।
- **Project example:** `QRVerificationService.VerifyAsync` লাইন ১০২-১১২।
- **Follow-up:** Revoke করার permission কার? — `[Authorize(Roles = "Producer, SuperAdmin")]` + service-এ `qrCode.Product.ProducerId != producerId && !isAdmin` চেক — শুধু নিজের product-এর owner অথবা admin।

Q8. `GenerateAsync` -এ idempotency (একই product-এর জন্য বারবার call করলে duplicate code তৈরি না হওয়া) কীভাবে নিশ্চিত করা হয়েছে?

- **Short answer:** Generate করার আগে `GetActiveByProductIdAsync(productId)` চেক করে, active code থাকলে সেটাই আবার ফেরত দেয়, নতুন বানায় না।
- **Detailed:** এতে ২টা active QR code একই product-এর জন্য কখনো তৈরি হয় না normal flow-তে। কিন্তু race condition-এ (দুইটা simultaneous request) DB-level unique constraint না থাকায় ছোট window-তে duplicate হতে পারে।
- **Project example:** `QRVerificationService.GenerateAsync` লাইন ৩১-৩৫।
- **Follow-up:** Product revoke হয়ে নতুন code generate করলে পুরনো code-এর কী হয়? — পুরনোটা `IsActive=false` obviously থাকে (revoke করা ছিল), নতুনটা আলাদা row, একই product-এর একাধিক `QRCode` row থাকতে পারে ইতিহাসে (শুধু একটাই active)।

Q9. `ArCraftScanController` আর `QRVerificationController`-এর মধ্যে সম্পর্ক কী — আলাদা করে "AR marker registry" বানানো হলো না কেন?

- **Short answer:** কোডের কমেণ্ট অনুযায়ী, physical AR marker scan আর QR code scan একই action, তাই আলাদা table না বানিয়ে existing `QRCode / QRVerificationRecord` reuse করা হয়েছে।
- **Detailed:** `ArCraftScanService.ScanAsync` একই `_qrVerificationRepository.GetByCodeAsync / AddVerificationRecordAsync` কল করে, শুধু response-এ extra data (CraftStory, ProducerStory, Traceability, Certificate) জুড়ে দেয় AR overlay-র জন্য।
- **Project example:** `ArCraftScanService.cs` লাইন ৯-১২ (comment), ৪১-৫৬ (reused logic)।
- **Follow-up:** এতে কি duplicate verification history হয়? — হ্যাঁ, AR scan আর normal QR verify দুটোই একই `QRVerificationRecord` table-এ যায়, আলাদা করা যায় না কোনটা AR app থেকে এসেছে আর কোনটা plain QR scanner থেকে (কোনো `Source / Channel` field নেই)।

Q10. `HeritagePlacesController`-এ `Create/Update/Delete` শুধু `SuperAdmin` -এর জন্য কেন, `Producer/GovernmentNGO` কেন না?

- **Short answer:** Heritage place একটা authoritative/curated dataset (জাতীয় ঐতিহ্য তথ্য), তাই content moderation কেন্দ্রীভূত রাখা হয়েছে যাতে ভুল/spam তথ্য না ঢোকে।
- **Detailed:** GovernmentNGO cluster-এ আলাদা `HeritageIndex` /Policy module আছে read-only aggregation-এর জন্য, কিন্তু raw `HeritagePlace` data লেখা শুধু SuperAdmin-এর।
- **Project example:** `HeritagePlacesController.cs` লাইন ৩৫, ৪৩, ৪৮।
- **Follow-up:** তাহলে নতুন heritage place add করার actual workflow কী? — admin panel/backoffice থেকে SuperAdmin manually add করে (platform-এ কোনো crowd-sourced submission flow `HeritagePlaces`-এর জন্য নেই, তবে `HeritageInnovationSubmissionsController` নামে আলাদা module আছে innovation submissions-এর জন্য)।

Q11. `BookingsController`-এ overbooking কীভাবে prevent করা হয়?

- **Short answer:** প্রতিটা নতুন booking তৈরির আগে ওই slot-এর current active booking-count (`GetActiveCountForSlotAsync`) + নতুন `PartySize` মিলিয়ে `slot.Capacity` -র সাথে তুলনা করা হয়।
- **Detailed:** `if (bookedCount + request.PartySize > slot.Capacity) throw new ConflictException(...)`। কিন্তু এটা application-level check, কোনো DB-level pessimistic lock/serializable isolation নেই — race condition সম্ভব দুইটা simultaneous booking request একই সময়ে এলে।
- **Project example:** `BookingService.CreateAsync` লাইন ৫৯-৬৩।
- **Follow-up:** race-condition ঠিক করতে কী করা যেত? — DB transaction + `SELECT ... FOR UPDATE` -এর মতো row-lock, অথবা unique constraint + retry, অথবা optimistic concurrency token।

Q12. Booking-এর TotalPrice client পাঠায় না কেন?

- **Short answer:** Price tampering আটকাতে server নিজে হিসাব করে: `TotalPrice = service.Price * request.PartySize`।
- **Detailed:** `CreateBookingRequest` -এ শুধু `ServiceId`, `AvailabilitySlotId`, `PartySize`, `Notes` থাকে — কোনো price field client থেকে আসে না, এটা platform-জুড়ে common security pattern (marketplace order-এও একই নীতি)।
- **Project example:** `BookingService.CreateAsync` লাইন ৭৪।
- **Follow-up:** Slot-এর সাথে Service মিলছে কিনা কেন আলাদা করে চেক করা হয় (`slot.ServiceId != service.Id`)? — কারণ client `serviceId` আর `availabilitySlotId` আলাদা আলাদা পাঠায়, malicious/buggy client অন্য service-এর slot ID পাঠাতে পারে — তাই cross-check বাধ্যতামূলক।

Q13. Booking state machine-এ কারা কোন transition করতে পারে?

- **Short answer:** Tourist: create, cancel (own booking); Producer/SuperAdmin: confirm, reject, complete; both/admin: cancel; role + ownership দুটোই চেক হয়।
- **Detailed:** `ConfirmAsync / RejectAsync / CompleteAsync` — `GetOwnedByProducerAsync` helper দিয়ে `booking.ProducerId == producerId || isAdmin` যাচাই হয়। `CancelAsync` উভয় পক্ষকে (tourist অথবা producer) অনুমতি দেয় কারণ যেকোনো পক্ষ বাতিল করতে পারে। প্রতিটাতে status-guard আছে — যেমন শুধু `Pending` booking confirm/reject করা যায়, শুধু `Confirmed` booking complete করা যায়।
- **Project example:** `BookingService.cs` লাইন ১২৭-২০৩।
- **Follow-up:** `NoShow` status কোথা থেকে set হয়? — enum-এ define করা আছে (`BookingStatus.NoShow`), কিন্তু কোনো controller action এটাকে set করে না — অসম্পূর্ণ/future feature (gotcha হিসেবে ধরা যায়)।

Q14. `IPassportRepository`, `IHeritageCheckInRepository`, `ITravelJournalRepository` — এতগুলো আলাদা repository কেন, একটাই

`IPassportRepository` দিয়ে সব handle করা যেত না?

- **Short answer:** Single Responsibility মেনে চলা — `Badge/UserBadge` management, `HeritageCheckIn`, `TravelJournalEntry` তিনটা আলাদা aggregate/entity family, তাই আলাদা repository।
- **Detailed:** `PassportService` -এ constructor injection-এ ৭টা dependency (`_passportRepository`, `_districtRepository`, `_orderRepository`, `_checkInRepository`, `_journalRepository`, `_heritagePlaceRepository`, `_achievementService`) — প্রতিটা narrow-focused interface, testability আর decoupling ভালো হয়।
- **Project example:** `PassportService.cs` লাইন ১৬-২২।
- **Follow-up:** এত dependency মানে কি Single Responsibility Principle ভাঙছে service level-এ? — আলোচনার বিষয়; `PassportService` আসলে "gamification orchestrator" — অনেকগুলো bounded context একত্র করে, যা কিছুটা God-Service pattern-এর দিকে ঝুঁকছে, কিন্তু প্রতিটা repository নিজে single-responsibility।

Q15. `KnowledgeGraphController`-এর role-restriction (`HeritageInnovationHub`, `GovernmentNGO`, `SuperAdmin`) কেন বাকি heritage controller-গুলোর মতো public read না?

- **Short answer:** এটা research/analytics tool — general public/tourist-এর জন্য না, শুধু গবেষণা-প্রতিষ্ঠান আর সরকারি সংস্থার জন্য উন্মুক্ত।
- **Detailed:** class-level `[Authorize(Roles = ...)]` পুরো controller-কেই আটকে দেয়, individual action-এ কোনো override নেই — GET হলেও login+role লাগবে, discovery controller-গুলোর মতো anonymous read না।
- **Project example:** `KnowledgeGraphController.cs` লাইন ১৩-১৬।
- **Follow-up:** এই pattern কি platform-এ অন্য কোথাও দেখা যায়? — হ্যাঁ, Government & NGO cluster-এর প্রায় সব controller একই pattern অনুসরণ করে (memory অনুযায়ী `api/governance/*` সব gov-only)।

Q16. `AITourismController` -এর কোনো endpoint-এ `[Authorize]` নেই কেন — security risk না?

- **Short answer:** সব ৫টা endpoint (`tour-plan`, `budget-plan`, `route-optimization`, `translate`, `cultural-recommendations`) সম্পূর্ণ anonymous — কারণ এগুলো stateless AI-suggestion calls, কোনো user-specific data touch/mutate করে না।
- **Detailed:** পেছনে `IAITourismProvider` -এর implementation `DummyAITourismProvider` (rule-based, না আসল LLM call) — তাই actual "AI" শুধু rule-based logic, ChatGPT-এর মতো external paid API call না (ব্যয় নিয়ন্ত্রণেও কারণ থাকতে পারে anonymous রাখার)।
- **Project example:** `AITourismController.cs`, `ShilpoHubBD.Infrastructure/AITourism/DummyAITourismProvider.cs`।
- **Follow-up:** rate-limiting নেই এই controller-এ? — কোডে দেখা `[EnableRateLimiting]` attribute নেই এই controller-এ — abuse-এর ঝুঁকি (gotcha)।

Q17. `HeritageSkillsController.Create` -এ `CreatedAtAction(nameof(GetAll), result)` — এটা কি সঠিকভাবে ব্যবহৃত হয়েছে?

- **Short answer:** টেকনিক্যালি কাজ করলেও এটা RESTful convention অনুযায়ী ideal না — `GetAll` কোনো single-resource-by-id action না, তাই `Location` header ভুল/অর্থহীন URL বানাতে পারে।
- **Detailed:** সাধারণত `CreatedAtAction` একটা `GetById` -জাতীয় action-কে point করে route parameter সহ। এখানে `GetAll(bool activeOnly)` -কে point করছে আর দ্বিতীয় parameter-এ পুরো `result` (DTO object) পাঠাচ্ছে যেটা route-value হিসেবে ব্যবহারযোগ্য না ঠিকভাবে।
- **Project example:** `HeritageSkillsController.cs` লাইন ৩৩।
- **Follow-up:** ঠিক করলে কীভাবে করতে হতো? — `HeritageSkill` -এর জন্য একটা `GetById(Guid id)` action বানিয়ে তাকে point করা উচিত ছিল, platform-এর বাকি সব controller-ই এই pattern মানে (`HeritagePlacesController.Create` দেখুন `nameof(GetById)`)।

Q18. `QRVerification` আর `ArCraftScan` দুটোই কেন `AllowAnonymous` -ভিত্তিতে কাজ করে (scan করতে login লাগে না)?

- **Short answer:** Product কেনার আগে যে কেউ (এমনকি account না থাকা customer-ও) দোকানে দাঁড়িয়ে সাথে সাথে verify করতে পারবে — এটাই anti-counterfeit ফিচারের মূল উদ্দেশ্য, তাই login-gate করলে usability নষ্ট হতো।
- **Detailed:** `CurrentUserIdOrNull` pattern ব্যবহার করে — logged-in হলে `VerifiedByUserId` populate হয় (history track করা যায়), না হলে `null` থাকে কিন্তু verify তবুও কাজ করে।
- **Project example:** `QRVerificationController.cs` লাইন ২৭-৩৪, `ArCraftScanController.cs` লাইন ২০-২৭।
- **Follow-up:** Anonymous verify হলে rate-limiting দরকার কেন? — bot দিয়ে GUID brute-force বা DoS আটকাতে (নিচে Tricky Q দেখুন)।

5. Tricky / Gotcha Questions

G1. QR code "cloning" attack — legit QR code-এর ছবি তুলে অন্য fake product-এ বসিয়ে দিলে সিস্টেম কি ধরতে পারবে?

না। `VerifyAsync` শুধু `Code` string DB-তে আছে কিনা এবং `IsActive` কিনা চেক করে — physical item-এর সাথে কোনো extra binding (যেমন tamper-evident sticker, one-time-scan invalidation, বা location/photo verification) নেই। একই QR code হাজারবার scan করলেও প্রতিবারই "verified authentic" রেসপন্স আসবে (কারণ scan মানেই code invalidate হয়ে যায় না)। তাই attacker legit product-এর QR code copy করে নিজের নকল product-এ বসালে system সেটাকে "authentic" বলবে — এটাই সবচেয়ে বড় real-world weakness।

G2. Check-in geofencing bypass — client নিজে lat/long spoof করলে কী হয়?

`Latitude / Longitude` client থেকে আসা raw input, কোনো device-attestation বা server-side GPS verification নেই। মোবাইল browser dev-tools/rooted device দিয়ে coordinate spoof করে "কাছে আছি" দেখানো সম্ভব — Haversine check শুধু client-প্রদত্ত data trust করে। এটা প্রায় সব location-based check-in system-এর common gotchal।

G3. QRCode.Code-এর উপর কোনো DB unique constraint নেই — race condition risk I

QRVerificationRecordConfiguration -এ শুধু QRCodeId, VerifiedByUserId -এর index দেখা যায়; QRCode entity-র নিজের কোনো IEntityConfiguration<QRCode> file পাওয়া যায়নি — মানে Code column-এ EF convention default প্রযোজ্য, explicit HasIndex(...).IsUnique() নেই। যদিও practically GUID collision প্রায় অসম্ভব, তবু DB-level guarantee না থাকাটা data-integrity নীতির লঙ্ঘন (defense-in-depth নীতি অনুযায়ী থাকা উচিত ছিল)।

G4. Gamification points/XP-এর data integrity — replay বা double-award সম্ভব কিনা?

CheckIn badge/Purchase badge award করার আগে HasUserBadgeAsync চেক থাকলেও, XP award (AwardXpAsync) প্রতিটা successful check-in-এই আলাদা XpTransaction row তৈরি করে — অর্থাৎ badge-এর মতো "one-time" গার্ড নেই XP-র জন্য, কারণ এটা intentional (প্রতিটা নতুন check-in-ই নতুন XP পাওয়ার কথা, যেহেতু day-based dedup আগেই আটকায়)। কিন্তু CheckInAsync আর AwardXpAsync আলাদা transaction হওয়ায় (G-এর মতো partial failure হলে) badge granted কিন্তু XP miss, বা উল্টো — inconsistent state সম্ভব।

G5. Festival badge self-claim — কোনো attendance-proof verification নেই।

ClaimFestivalBadgeAsync -এ BadgeId পাঠালেই badge claim হয়ে যায় (শুধু duplicate-check), user আসলেই সেই festival-এ participate করেছে কিনা তার কোনো cross-verification (যেমন check-in বা ticket-purchase link) নেই — যেকোনো authenticated user যেকোনো festival badge নিজে claim করে নিতে পারে যদি সে শুধু BadgeId জানে (আর GET /api/passport/badges anonymous, তাই সব badge ID discoverable)।

G6. Anonymous QR-verify/scan endpoint-এ কোনো rate-limiting দেখা যায়নি — brute-force/DoS ঝুঁকি।

Verify / Scan উভয় action-এই কোনো [EnableRateLimiting] attribute নেই (যদিও platform-এ globally rate-limiter policy আছে বলে memory-তে উল্লেখ আছে অন্য domain-এর জন্য, এই দুই controller-এ explicit attribute পাওয়া যায়নি)। প্রতিটা verify attempt DB-write করে (QRVerificationRecord insert), তাই attacker script দিয়ে লাখ লাখ random GUID scan করলে টেবিল দ্রুত ফুলে উঠতে পারে (storage-exhaustion/log-flooding DoS)।

গবেষণা covered ১৮টি controller: HeritagePlacesController, HeritageRoutesController, HeritageFestivalsController, HeritageSkillsController, HeritageIdentityController, VillageTourController, VillagesController, CulturalEventsController, CulturalStoriesController, LocalCuisinesController, PassportController, QRVerificationController, ArCraftScanController, MuseumItemsController, TouristServicesController, TouristAnalyticsController, ATourismController, KnowledgeGraphController, BookingsController — সবগুলো actual file থেকে read করা হয়েছে (কোনো guess নেই)।

Government & NGO Cluster — Viva Prep

Source: `ShilpoHubBD.Api/Controllers/*`, `ShilpoHubBD.Application/Services/Governance/*`,
`ShilpoHubBD.Application/Services/HeritageDatabase/*`, `ShilpoHubBD.Application/Services/FieldResearch/*`,
`ShilpoHubBD.Infrastructure/{GovForecasting,PolicySimulation,HeritageIntelligence}/*`,
`ShilpoHubBD.Data/Repositories/*`. All code read directly from the repo on 2026-09-13, not guessed.

1. Cluster Overview

ShilpoHubBD-er "Government & NGO" cluster ta muloto ekটা **read-heavy analytics + oversight layer** যা platform-er existing marketplace, employment, tourism, ebong heritage data-r upor boshe থাকে। Eita kono notun "source of truth" data create kore na — instead, ei cluster **existing tables theke live figures aggregate kore**, segulo ke composite indices/scores-e convert kore, aar policy/funding decision-গুলোর জন্য একটা audit trail toiri kore। Core sub-domains:

- **National Dashboard** (`NationalDashboardController`) — live overview + historical snapshots + district rankings + trend charts, sob kichu raw marketplace/employment/tourism tables theke aggregate koral
- **Heritage Intelligence** (`HeritageIntelligenceController`) — 6-ta composite index (Risk, Living Heritage, Craft Health, Village Survival, Youth Participation, Climate Risk) compute kore, jegulo "AI Predictions" bola holeo আসলে **rule-based weighted scoring**।
- **Policy Simulator** (`PolicySimulatorController`) — "what-if" scenario runner (grant, training, tourism campaign, export strategy, employment prediction), একটা live baseline er upor elasticity formula apply kore।
- **Gov Forecasts** (`GovForecastsController`) — national metric-গুলোকে future-e project kore, using একটা replaceable provider (currently OLS/linear regression, "AI" na)।
- **Monitoring** (`MonitoringController`) — fraud/fake-product/review-abuse/QR-anomaly-r জন্য rule-based scan engine, flags raise kore, triage workflow (assign/status/notes)।
- **Funding** (`FundingProgramsController` + `FundingApplicationsController`) — grant/loan programs, applications, review→decision→disbursement→repayment lifecycle।
- **Compliance** (`ComplianceController`) — regulatory compliance records + requirements checklist।
- **Heritage Database** (`HeritageDataController`, `HeritageDatasetsController`, `HeritageRiskController`) — research-grade read access to live data, curated exportable datasets with access grants, ebong manually-entered heritage risk assessments।
- **Field Research** (`SurveysController`, `SurveyResponsesController`, `FieldEvidenceController`) — survey design, field-collected responses, ebং raw evidence (photo/audio/transcript/GPS) storage।

Typical request flow: Government/NGO officer login kore, JWT-e `GovernmentNGO` (or `SuperAdmin`) role claim thake। Dashboard overview dekhe (`GET /api/governance/dashboard/overview`), kono district-e risk বেশি mone hole Heritage Risk record dekhe/create kore (`api/heritage-database/risk`), Heritage Intelligence index compute kore ei risk records ke input hishebe niye (`POST .../heritage-intelligence/compute`), tarpor Policy Simulator diye একটা grant/training scenario run kore dekhe economic impact ki hote pare, budget approve hole Funding module-e application create hoy, ebং shesh-e একটা period report generate kore shob module theke data pull kore (`GovReportsController`)।

Interconnection — important nuance for viva: Assignment-e mentioned "FieldEvidence feeds HeritageRisk feeds NationalDashboard" pipeline ta code-e **automatic na** — eta 3-ta separate, manually-operated module:

1. `FieldEvidenceController` — researchers raw evidence (photo/GPS/transcript) upload kore, survey-r shathe linked, kono automatic processing hoy na।
2. `HeritageRiskController` — steward (Government/NGO/SuperAdmin) **manually** একটা `HeritageRiskRecord` create kore (title/category/level), field evidence dekhe informed hote pare kintu code-level e kono FK/reference nei FieldEvidence theke HeritageRiskRecord-e।
3. `HeritageIntelligenceService.ComputeAsync` → `HeritageIntelligenceRepository.GatherSignalsAsync` ei `HeritageRiskRecord` table theke `RiskLow/Moderate/High/Critical/Safeguarded` count kore, index score-er একটা component banay।
4. `NationalDashboardService` `HeritageIndexRecord` ba `HeritageRiskRecord` kono ta e **directly touch kore na** — eta shudhu raw marketplace/employment/tourism/producer tables theke live aggregate kore। Dashboard-e risk index dekhte hole segulo separately `HeritageIntelligenceController` theke query korte hobe।

So "pipeline" ta একটা conceptual/workflow chain — human decides kokhon evidence dekhe risk record banabe, kokhon risk record dekhe index compute korbe। Kono background job/event/trigger nei ei tin-tar majhe automatic connect korar jonno।

2. Deep Dive — 4 Representative Controllers

2.1 PolicySimulatorController — most viva-worthy

File: ShilpoHubBD.Api/Controllers/PolicySimulatorController.cs

Route table:

Verb	Route	Roles	Purpose
POST	api/governance/policy-simulator/simulations	GovernmentNGO, SuperAdmin	Run a scenario against live baseline
GET	api/governance/policy-simulator/simulations	GovernmentNGO, SuperAdmin	Paged list
GET	api/governance/policy-simulator/simulations/{id}	GovernmentNGO, SuperAdmin	Get one simulation
DELETE	api/governance/policy-simulator/simulations/{id}	GovernmentNGO, SuperAdmin	Delete

Controller nije thin — just injects `IPolicySimulationService`. Real logic 2 layer-e split:

PolicySimulationService (baseline gather + persistence + orchestration) eb09

RuleBasedPolicySimulationProvider (actual math, in Infrastructure layer — replaceable).

Key method 1 — `PolicySimulationService.RunAsync`

([ShilpoHubBD.Application/Services/Governance/PolicySimulationService.cs](#), lines 34-168):

```
var simulationType = ParseEnum<PolicySimulationType>(request.SimulationType, ...);
var scope = ParseEnum<HeritageIndexScope>(request.Scope, ...);
var horizonMonths = Math.Clamp(request.HorizonMonths ?? DefaultHorizonMonths, MinHorizonMonths, MaxHorizonMonths);
if (request.Budget is < 0) throw new ConflictException("Budget cannot be negative.");
if (request.IntensityPercent is < 0 or > 100) throw new ConflictException(...);

var (scopeId, scopeLabel) = await ResolveScopeAsync(scope, request.ScopeId, cancellationToken);
var to = DateTime.UtcNow;
var from = to.AddMonths(-BaselineWindowMonths); // last 12 months
var baseline = await _repository.GatherBaselineAsync(scope, scopeId, from, to, cancellationToken);
```

Line-by-line (Banglish): string enum values ("GrantProgram", "District" etc.) validate kore proper C# enum-e convert kora hocche — invalid string dile **ConflictException** (→ HTTP 409, global exception handler mapping onujayi) | Budget negative hole ba intensity 0-100 range-er baire gele business-rule violation dhore 409 throw kora hocche (NOT 400 — eta FluentValidation na, service-layer manual check, tai **ConflictException**) | Horizon **Math.Clamp** diye 3-120 mash-er modhe force kora hoy | Baseline gather kora hoy — always last 12 months-er live data (**BaselineWindowMonths = 12**), request এ দেয়া নয়, hardcoded |

```
try {
    var result = _provider.Simulate(input);
    simulation.Status = PolicySimulationStatus.Completed;
    ... // map projections & recommendations into entity
} catch (Exception ex) when (ex is not ConflictException and not NotFoundException) {
    simulation.Status = PolicySimulationStatus.Failed;
    simulation.FailureReason = ex.Message;
}
```

Eita ekটা notable pattern — simulation-er math kono karone fail korle (e.g. unknown enum in provider switch), pura request 500 e crash na kore, simulation entity-ke **Failed** status diye graceful-e persist kora hoy (jodi **Persist=true** hoy)! **ConflictException** / **NotFoundException** re-throw kora hoy jate validation errors normally bubble up kore!

Key method 2 — `RuleBasedPolicySimulationProvider.Grant`

([ShilpoHubBD.Infrastructure/PolicySimulation/RuleBasedPolicySimulationProvider.cs](#), lines 38-94):

```

const decimal grantPerProducer = 25_000m;
const double outputUplift = 0.18;
const double workersPerProducer = 0.4;

var funded = input.TargetBeneficiaries
    ?? (input.Budget.HasValue ? (int)(input.Budget.Value / grantPerProducer) : 0);
var fundedShare = b.ActiveProducers > 0 ? Math.Min(1.0, (double)funded / b.ActiveProducers) : 0;
var horizonFactor = Math.Min(1.0, years);

var economyProjected = Scale(b.EconomyValue, 1 + fundedShare * outputUplift * intensity * horizonFactor);

```

Explanation: Budget diye funded producer count ber kora hoy (BDT 25,000 fixed cost per producer — literally hardcoded assumption, kono historical calibration/ML nei)। **fundedShare** = koto percent-er active producer-der cover kore। Economy projection = baseline × (1 + fundedShare × 0.18 uplift × user-given intensity × horizon-factor) — eita ekটা **linear elasticity formula**, kono regression ba trained model na। Confidence level-ta o simple heuristic (**Confidence()** helper) — always at most "Moderate", kokhono "High" dey na (hardcoded design choice, over-claiming avoid korte)।

5-ta simulation type-er (Grant, Training, TourismCampaign, ExportStrategy, EmploymentPrediction) protijeta ei style-er — different hardcoded constants (cost-per-trainee = BDT 12,000, ROI-per-export-BDT = 3.5x, etc.) diye ekটা transparent multiplier apply kore। Class-er XML doc comment nijei explicit bole: *"Every projection is a transparent elasticity applied to the live baseline... no external calls, no model weights."*

Business rules worth remembering: Scope resolution (**ResolveScopeAsync**) District/Village-er jonno **ScopeId** mandatory kore (na dile 409); National/Craft scope-e **ScopeId** lage na। **Persist=false** dile DB-e kichu save hoy na, just computed result ফেরত দেয় (id = **Guid.Empty**)।

2.2 GovForecastsController — forecasting, honest check

File: **ShilpoHubBD.Api/Controllers/GovForecastsController.cs**

Route table:

Verb	Route	Roles	Purpose
GET	api/governance/forecasts	GovernmentNGO, SuperAdmin	Paged list
GET	api/governance/forecasts/{id}	GovernmentNGO, SuperAdmin	Get one
POST	api/governance/forecasts/generate	GovernmentNGO, SuperAdmin	Project 6 national metrics forward
DELETE	api/governance/forecasts/{id}	GovernmentNGO, SuperAdmin	Delete

Key method — **RuleBasedGovForecastProvider.ProjectTrend**

(**ShilpoHubBD.Infrastructure/GovForecasting/RuleBasedGovForecastProvider.cs** , lines 67-111):

```

var xs = obs.Select(o => (o.T - t0).TotalDays / 30.0).ToArray(); // months since first observation
var ys = obs.Select(o => o.V).ToArray();
var meanX = xs.Average(); var meanY = ys.Average();
var sxx = xs.Sum(x => (x - meanX) * (x - meanX));
var sxy = xs.Zip(ys, (x, y) => (x - meanX) * (y - meanY)).Sum();
var slope = sxx == 0 ? 0 : sxy / sxx;
var intercept = meanY - slope * meanX;

```

Eita **honest-to-god Ordinary Least Squares (OLS) linear regression** — $\text{slope} = \frac{\sum(x-\bar{x})(y-\bar{y})}{\sum(x-\bar{x})^2}$, standard formula, statistics course-e ja shekhano hoy। R^2 o calculate kora hoy (**totalSs / residuals**) confidence rating-er jonno:

```

var confidence = (n, r2) switch {
    ( ≥ 6, ≥ 0.75 ) => GovForecastConfidence.High,
    ( ≥ 3, ≥ 0.4 ) => GovForecastConfidence.Moderate,
    _ => GovForecastConfidence.Low,
};

```

Kom-history (<2 points) thakle **ProjectFlat** fallback hoy — flat line + widening uncertainty band, kono trend estimate kora hoy na।

Honest assessment for viva: Eita "AI" na — eita ekটা classic **statistical linear regression**, scikit-learn/ML library kono kichu use hoy nai, nijer haate OLS formula likha হয়েছে। Class comment nijei bole: "No external calls, no model weights." Confidence band width formula-o simple heuristic: `max(residualStdDev, |predicted| * 8%) * (1 + 0.6 * monthOffset/horizon)` — statistically-motivated (residual std dev use kora hoyeche) kintu ei widening-factor (0.6, 8%) hardcoded magic numbers, kono formal prediction-interval derivation na।

Key method 2 — `GovForecastService.GenerateAsync`
(`ShilpoHubBD.Application/Services/Governance/GovForecastService.cs`, lines 28-89): controller theke call hoye repository theke `GatherForecastInputAsync` diye snapshot history + current values ana hoy (NationalDashboardSnapshot table theke — tai snapshot capture na korle history khali থাকবে ebং forecast always flat hobe), তারপর provider-e pass kore, result-er projections-guloke `GovForecastPoint` entity row হিসেবে flatten kore save kore (each metric × each month = একটা row)।

Metrics forecast kora hoy: `TotalProducers`, `JobsFilled`, `MarketplaceSalesValue`, `ExportSalesValue`, `TourismRevenue`, `HeritageEconomyValue` — fixed 6-ta, hardcoded array (`Metrics` static field), request-e customize kora jay na।

2.3 **MonitoringController** — rule-based fraud/anomaly detection

File: `ShilpoHubBD.Api/Controllers/MonitoringController.cs`

Route table:

Verb	Route	Roles	Purpose
POST	<code>api/governance/monitoring/scans</code>	GovernmentNGO, SuperAdmin	Run automated scan
GET	<code>api/governance/monitoring/flags</code>	GovernmentNGO, SuperAdmin	Paged flag list
GET	<code>api/governance/monitoring/flags/{id}</code>	GovernmentNGO, SuperAdmin	Get one flag
POST	<code>api/governance/monitoring/flags</code>	GovernmentNGO, SuperAdmin	Manual flag creation
POST	<code>api/governance/monitoring/flags/{id}/status</code>	GovernmentNGO, SuperAdmin	Change status
POST	<code>api/governance/monitoring/flags/{id}/assign</code>	GovernmentNGO, SuperAdmin	Assign to officer
POST	<code>api/governance/monitoring/flags/{id}/notes</code>	GovernmentNGO, SuperAdmin	Add note
DELETE	<code>api/governance/monitoring/flags/{id}</code>	GovernmentNGO, SuperAdmin	Delete
GET	<code>api/governance/monitoring/qr/overview</code>	GovernmentNGO, SuperAdmin	QR verification stats

Key method 1 — `MonitoringService.RunScanAsync`
(`ShilpoHubBD.Application/Services/Governance/MonitoringService.cs`, lines 28-130): controller theke `ScanType` (`All` / `Fraud` / `FakeProduct` / `ReviewAbuse` / `QrAnomaly`), optional `Since` date ebং `MinRiskScore` (default 40) neya hoy। Repository theke 4-type-er candidate list ana hoy, তারপর **dedup logic**:

```
var belowThreshold = candidates.RemoveAll(c => c.RiskScore < minScore);
var existingKeys = await _repository.GetOpenFlagDedupeKeysAsync(candidates.Select(c => c.DedupeKey), ...);
foreach (var c in candidates) {
    if (existingKeys.Contains(c.DedupeKey) || !seen.Add(c.DedupeKey)) { duplicates++; continue; }
    // create MonitoringFlag ...
}
```

`DedupeKey` (e.g. `"fraud:buyer:{userId}"`, `"fake:product:{productId}"`) already-open flag-er shathe match korle notun flag toiri hoy na — scan **repeatedly safely re-run kora jay** (idempotent-ish behavior), duplicate spam hoy na।

Key method 2 — `MonitoringRepository.FindFraudCandidatesAsync`
(`ShilpoHubBD.Data/Repositories/MonitoringRepository.cs`, lines 115-211) — pure SQL-style aggregation, kono ML/AI na:

```
var buyerRows = await _context.Orders
    .Where(o => o.CreatedAt >= since)
    .GroupBy(o => o.UserId)
    .Select(g => new { UserId = g.Key, Total = g.Count(),
        Bad = g.Count(o => BadOrderStatuses.Contains(o.Status)), Refunded = g.Sum(o => o.RefundAmount ?? 0m) })
    .Where(x => x.Total >= 5 && x.Bad * 100 >= x.Total * 40) // >=40% bad-rate, min 5 orders
    .ToListAsync(cancellationToken);
```

Fraud rule: kono buyer-er last N din-e kom-se-kom 5-ta order thakle ebong 40%+ cancel/return/refund hole flag ৩৩; risk score = $40 + \text{rate} \times 55$ (i.e. 40-95 range), severity High jodi rate $\geq 70\%$ naile Medium। Erokom-i rule 3-ta ache — producer-side bad-order-rate ($\geq 35\%$ threshold), fake-product (rejected handmade-verification status active thaka, ba ≥ 3 rejected products by same producer), review-abuse (≥ 8 reviews since window, average rating ≥ 4.5), QR-anomaly ($\geq 40\%$ invalid scan rate on a product, ba unknown code scanned ≥ 5 times)। **Sob-i simple threshold + linear scoring rule — kono statistical model, ML classifier, ba anomaly-detection algorithm na।**

Business rule: Flag assign korle status auto `Open` → `UnderReview` hoye jay
(`AssignFlagAsync`), resolve/dismiss korle `ResolvedAt` / `ResolvedByUserId` set hoy, event-log
(`MonitoringFlagEvent`) protita action-e append hoy — audit trail।

2.4 HeritageIntelligenceController — the composite-index engine

File: `ShilpoHubBD.Api/Controllers/HeritageIntelligenceController.cs`

Route table:

Verb	Route	Roles	Purpose
POST	<code>api/governance/heritage-intelligence/compute</code>	GovernmentNGO, SuperAdmin	Compute an index
GET	<code>api/governance/heritage-intelligence/records</code>	GovernmentNGO, SuperAdmin	Paged list
GET	<code>api/governance/heritage-intelligence/records/{id}</code>	GovernmentNGO, SuperAdmin	Get one
DELETE	<code>api/governance/heritage-intelligence/records/{id}</code>	GovernmentNGO, SuperAdmin	Delete
GET	<code>api/governance/heritage-intelligence/trends</code>	GovernmentNGO, SuperAdmin	Score history for a scope

6 index types: `HeritageRiskIndex`, `LivingHeritageIndex`, `CraftHealthScore`, `VillageSurvivalIndex`, `YouthParticipation`, `ClimateRiskAnalysis`. Scope: `National` / `District` / `Village` / `Craft`.

Key method 1 — `RuleBasedHeritageIntelligenceProvider.Risk`
(`ShilpoHubBD.Infrastructure/HeritageIntelligence/RuleBasedHeritageIntelligenceProvider.cs`, lines 38-62):

```
var weightedRisk = (s.RiskLow * 1) + (s.RiskModerate * 2) + (s.RiskHigh * 4)
    + (s.RiskCritical * 7) - (s.RiskSafeguarded * 3);
var pressure = Saturate(Math.Max(0, weightedRisk), 12);
...
var components = new List<HeritageIndexComponentResult> {
    Comp("risk_pressure", ..., weightedRisk, 0.45m, pressure, ...),
    Comp("affected_artisans", ..., s.AffectedArtisans, 0.20m, artisanExposure, null),
    Comp("market_thinness", ..., s.ActiveProducts + s.Orders, 0.20m, marketThinness, ...),
    Comp("successor_gap", ..., s.ApprenticeEnrollments + s.ProgramApplications, 0.15m, successorGap, null),
};
var score = WeightedScore(components); // weighted average, clamped 0-100
```

Explanation: `HeritageRiskRecord` (manually entered by steward via `HeritageRiskController`) count by level, weighted (Critical $\times 7$, High $\times 4$, Moderate $\times 2$, Low $\times 1$, minus Safeguarded $\times 3$) — ekta arbitrary but transparent point system, kono learned weight na, developer nijei decide korche kon factor koto important। Etarpor `Saturate` function diye normalize kora hoy:

```
private static decimal Saturate(double x, double k)
{
    if (x <= 0 || k <= 0) return 0;
    return (decimal)Math.Round(100.0 * x / (x + k), 2); // x=0→0, x=k→50, x→∞→100
}
```

Eita একটা **saturation/diminishing-returns curve** (like Michaelis-Menten in biology, or a simple logistic-ish shape) — x = k hole score thik 50 pay, beshi x-e slowly 100-er dike approach kore kintu kokhono 100 chharay na! 4-ta component-er weighted-average (weights sum kore 1.0 = 0.45+0.20+0.20+0.15) final 0-100 score dey, tarpor rating bucket-e map hoy (**RiskDirectionRating** : ≥80 Critical, ≥60 Poor, ≥40 Fair, ≥20 Good, else Strong)!

Key method 2 — HeritageIntelligenceRepository.GatherSignalsAsync
(**ShilpoHubBD.Data/Repositories/HeritageIntelligenceRepository.cs** , lines 41-228): scope onujayi (National/District/Village/Craft) ekaddhik table theke counts/sums collect kore — producers, verified-heritage identities, active products/orders/sales, villages, cultural events, risk records by level/category, apprentice/course/mentorship enrollments! Important comment (line 186-188): *"Apprenticeship / academy records carry no reliable district link, so these are platform-wide counts within the window regardless of scope"* — mane District/Village scope-e o youth-pipeline numbers national-level e thake, eta একটা known data-modeling limitation, examiner jiggesh korle honestly bolte hobe!

Persist behavior: **ComputeAsync** -e **Persist=false** dile DB-e save hoy na, **Id = Guid.Empty** shoho result ferot dey — "preview mode" for what-if exploration before committing.

3. Reference Table — Remaining Controllers

Controller	Route prefix	Key endpoints	Auth / Role
GovAnalyticsController	api/governance/analytics	gis/districts , exports CRUD	Government
GovReportsController	api/governance/reports	GET/paged, POST generate , PUT, DELETE	Government
FundingProgramsController	api/governance/funding/programs	CRUD	Government
FundingApplicationsController	api/governance/funding/applications	apply, reviews, decision, withdraw, notes, disbursements, disbursements/{id}/status, repayments	Government
ComplianceController	api/governance/compliance	records CRUD, records/{id}/requirements upsert/remove	Government
NationalDashboardController	api/governance/dashboard	overview, district-rankings, snapshots CRUD, trends	Government
HeritageRiskController	api/heritage-database/risk	CRUD	Read: HeritageInn Write: Govei (StewardR
HeritageDataController	api/heritage-database/live	locations/villages/producers/products/tourism, demographics, summary	HeritageInn
HeritageDatasetsController	api/heritage-database/datasets (+ api/heritage-database/exports/mine)	CRUD, versions, access-grants, exports, export-analytics	[Authori: ResearcherR
SurveysController	api/field-research/surveys	CRUD, questions, field-researchers, history	[Authori:
SurveyResponsesController	api/field-research/surveys/{surveyId}/responses	CRUD, submit, review	[Authori:
FieldEvidenceController	api/field-research/surveys/{surveyId}/evidence	CRUD	[Authori:
HeritageInnovationSubmissionsController	api/innovation-lab/submissions	CRUD, submit, withdraw, team-members, reviews, history	[Authori: Government
PreservationStrategiesController	api/innovation-lab/preservation-strategies	CRUD, objectives, actions	[Authori:

Controller	Route prefix	Key endpoints	Auth / Role
ProgramApplicationsController	api/program-applications	apply, mine, programs/{id}, accept, reject, withdraw	[Authorize(Roles = "SuperAdmin")]

Note: GovReportsController and GovAnalyticsController both inject the same IGovReportService — controller-e alada namespace/route thakleo backend service ekta-i, so viva-te jodi jiggesh kore "kotogulo distinct service class ache ei cluster-e" — সৎখ্যাটি controller-count theke kom hobe।

4. Viva Question Bank

Q1. Ei cluster-er controller-guli kon role-e gated? [Authorize(Roles= ...)] kivabe kaj kore?

- **Short answer:** Beshirbhag controller GovernmentNGO + SuperAdmin role-e gated; kichu shared research-endpoint (HeritageDataController, HeritageRiskController GET) HeritageInnovationHub role-oo allow kore।
- **Detailed explanation:** ASP.NET Core-er [Authorize(Roles="A,B")] attribute comma-separated list-e OR logic apply kore — user-er kono ekta role match korle access pay। Route-level attribute controller class-e thake, kichu action (e.g. HeritageRiskController.Create/Update/Delete)-e additional [Authorize(Roles = StewardRoles)] override kore beshi restrictive kore — GET open thake research role-der jonno, write shudhu steward-der jonno।
- **Project example:** GovAnalyticsController — [Authorize(Roles = \$"{RoleNames.GovernmentNGO},{RoleNames.SuperAdmin}")] class-level e; HeritageRiskController — class-level e 3 role (research+gov+admin) GET-er jonno, kintu Create / Update / Delete -e action-level override diye shudhu 2-ta role (StewardRoles const)।
- **Follow-up:** "SuperAdmin sob jaygay keno add kora?" → Platform-wide override role, kono specific domain-permission na thakleo SuperAdmin sob access pay — emergency/admin override pattern।

Q2. Policy Simulator "AI" bole dabi kora hoyeche — eta ki real machine learning?

- **Short answer:** Na, eta rule-based deterministic elasticity formula, kono trained model nei।
- **Detailed explanation:** RuleBasedPolicySimulationProvider class-er naming ebং XML doc comment nijei explicit bole — "Rule-based stand-in for a future ML/forecasting backend... no external calls, no model weights." Protita simulation type-er (Grant/Training/Tourism/Export/Employment) ekটি hardcoded constant (e.g. grantPerProducer = 25_000m, outputUplift = 0.18) diye linear multiplier apply kore baseline-er upor। Interface (IPolicySimulationProvider) design kora hoyeche jate future-e real ML model diye replace kora jay controller/service na change kore — Strategy pattern।
- **Project example:** Grant simulation — economyProjected = Scale(b.EconomyValue, 1 + fundedShare * outputUplift * intensity * horizonFactor) — pure arithmetic, kono statistical fitting na।
- **Follow-up:** "Tahole eta kaje lagbe kivabe?" → Transparent assumption-based projection dise officer-der ekটি starting-point estimate dite, "AssumptionsJson" field-e shob constant explicitly store hoy jate keu audit korte pare kon number diye ki calculate hoise।

Q3. Gov Forecasts-e ki real statistics use hoyeche?

- **Short answer:** Ha, OLS (Ordinary Least Squares) linear regression — eta real statistics, kintu ML/AI na।
- **Detailed explanation:** RuleBasedGovForecastProvider.ProjectTrend method-e standard slope/intercept formula ($slope = \frac{\sum(x-\bar{x})(y-\bar{y})}{\sum(x-\bar{x})^2}$) calculate kora hoy history points theke, R^2 compute kore confidence bucket decide kora hoy (High/Moderate/Low)। History 2-tar kom hole flat projection fallback hoy widening uncertainty band shoho।
- **Project example:** residualStd diye confidence band width calculate kora, $band = max(residualStd, |predicted|*8\%) * (1 + 0.6*m/horizon)$ — statistically-informed but band-width-er 8%/0.6 constant gulo hardcoded heuristic, kono formal prediction-interval theorem theke ashe nai।
- **Follow-up:** "Eta ML na hoile ML add korte gele ki change lagbe?" → Shudhu IGovForecastProvider interface implement kore notun class banai DI-te register korle hobe, service/controller kono kichu change lagbe na (Dependency Inversion via RuleBasedGovForecastProvider : IGovForecastProvider, DependencyInjection.cs -e AddScoped diye wired)।

Q4. Heritage Risk Index-er weighted score kivabe calculate hoy? "Saturate" function ta ki kaj kore?

- **Short answer:** Risk-level count-guloke weighted-sum kore, ekটি diminishing-returns curve (Saturate) diye 0-100-e normalize kore, tarpor component-weight diye weighted-average nay।
- **Detailed explanation:** $weightedRisk = Low*1 + Moderate*2 + High*4 + Critical*7 - Safeguarded*3$ — severity onujayi point dewa। $Saturate(x, k) = 100*x/(x+k)$ — $x=0$ -e 0, $x=k$ -e exactly 50, $x \rightarrow \infty$ -e 100-er kache pouche kintu never exceeds। 4-ta component (risk_pressure 45%, affected_artisans 20%, market_thinness 20%, successor_gap 15%) — weight sum = 1.0, WeightedScore function

$\Sigma(\text{contribution})/\Sigma(\text{weight})$ kore final score dey l

- **Project example:** Component-wise breakdown `HeritageIndexComponent` entity-te stored thake — tai DTO-te "kon factor koto contribute korlo" dekhano jay, full transparency (no black-box) l
- **Follow-up:** "Weight-gulo kothāI theke ashlo?" → Developer-assigned business judgment, kono statistical calibration/training data theke derive kora hoy nai — honest answer dite hobe l

Q5. HeritageRiskController-e access role different keno GET vs write-e?

- **Short answer:** GET (read) 3-role open (HeritageInnovationHub + GovernmentNGO + SuperAdmin), kintu Create/Update/Delete shudhu 2-role (`StewardRoles` — GovernmentNGO + SuperAdmin) l
- **Detailed explanation:** Researcher-der (HeritageInnovationHub) risk data dekhte dorkar (research purpose), kintu official risk assessment record banano/edit kora shudhu "government-level steward"-er kaj — data integrity/authority control l `const string StewardRoles` field-level constant diye DRY-ভাবে defined, action-level `[Authorize(Roles = StewardRoles)]` override kore class-level attribute ke l
- **Follow-up:** "Eta kivabe enforce hoy runtime-e?" → ASP.NET middleware pipeline-e Authorization middleware controller action invoke korar age attribute check kore, fail hole 403 Forbidden (na je 401 — 401 hocche unauthenticated, 403 role thakleo access nai emon case-e) l

Q6. Monitoring scan "duplicate" flags create kena? Dedup logic kivabe kaj kore?

- **Short answer:** `DedupeKey` string (e.g. `"fraud:buyer:{userId}"`) diye already-open flag exist kore kina check kore, thakle skip kore l
- **Detailed explanation:** Scan repeat-run-e same subject-er jonno same problem barbar detect korte pare (e.g. daily scheduled scan) — jodi already ekta "Open"/"UnderReview" flag thake shei subject-er jonno, notun duplicate flag banano hoy na, `duplicates++` count kore l `GetOpenFlagDedupeKeysAsync` query kore `Status ≠ Dismissed && Status ≠ Resolved` filter diye — mane resolved/dissmissed flag thakle abar notun flag uthte pare (re-flag allowed after resolution) l
- **Project example:** `belowThreshold` (risk-score < minScore) ebō `duplicates` (already flagged) dutoi `MonitoringScanResultDto` -te separately report hoy, tai officer bujhte pare koto candidate evaluate hoise vs koto notun flag uthlo l
- **Follow-up:** "Scan ki scheduled/automatic?" → Code-e kono background job/hosted-service scheduler dekha jay nai ei controller-er jonno — eta manually `POST scans` call korei trigger hoy (viva-te bolar age double-check kora bhalo, kintu ei cluster-e ekhono explicit cron/hosted-service paoya jayni ei feature-er jonno) l

Q7. Fraud detection-e kon threshold-guli hardcoded?

- **Short answer:** Buyer: ≥5 orders + ≥40% bad-status → flag; Producer: ≥8 orders + ≥35% bad-status; Fake-product: rejected-verification + active listing, ba ≥3 rejected products/producer; Review-abuse: ≥8 reviews + avg rating ≥4.5; QR-anomaly: ≥10 scans + ≥40% invalid, ba unknown code ≥5 scans l
- **Detailed explanation:** Shob threshold `MonitoringRepository` -er method-e literal constant hishebe embedded (e.g. `x.Total ≥ 5 && x.Bad * 100 ≥ x.Total * 40`), kono config file/admin setting theke ashe na — code change korle hard-deploy lagbe l
- **Follow-up:** "Eta improve korte hole ki korben?" → Threshold-gulo configuration-e move kora (appsettings/DB), ba proper anomaly-detection statistical model diye replace kora — bolar moto valid improvement point l

Q8. `Persist=false` request-e ki hoy? Keno eta dorkar?

- **Short answer:** Compute/simulate hoy, DB-e save hoy na, `Id = Guid.Empty` shoho result direct ferot dey l
- **Detailed explanation:** Policy Simulator, Gov Forecast, Heritage Intelligence — tinTa-i ei pattern follow kore l Controller-e check: `result.Id == Guid.Empty ? Ok(result) : CreatedAtAction(...)` — 200 vs 201 response code-o eta onujayi decide hoy l "What-if preview" use-case — officer experiment korte pare multiple scenario niye, shudhu final decision-ta persist korā l
- **Follow-up:** "REST semantics thik ache ei design-e?" → Ha — POST always resource creation na hote pare jodi side-effect-free preview mode thake, kintu jodi persist=true hoy tahole 201 + Location header thik REST pattern follow kore (`CreatedAtAction`) l

Q9. `ConflictException` ebō `NotFoundException` global exception handler-e kivabe map hoy?

- **Short answer:** `ConflictException` → 409, `NotFoundException` → 404, `ValidationException` → 400, `UnauthorizedAccessException` → 401, onno shob → 500 — sob RFC7807 ProblemDetails format-e l
- **Detailed explanation:** `GlobalExceptionHandler` middleware pipeline-er last-e boshe, controller/service theke thrown exception catch kore proper HTTP status + structured JSON error response-e convert kore l Ei cluster-e beshirbhag business-rule violation (invalid enum string, negative budget, missing ScopeId) `ConflictException` diye 409 dey — `FluentValidation`-er 400 theke deliberately alada, karon eta "request shape thik ache kintu business context-e invalid" case l
- **Follow-up:** "400 ar 409-er difference ki?" → 400 = malformed/invalid input shape (`FluentValidation`

catches — missing required field, wrong format); 409 = well-formed input kintu current server state/business-rule-er shathe conflict kortese (e.g. budget negative, already-resolved flag re-resolve kora)।

Q10. National Dashboard "Live overview" ebং "Snapshot" — difference ki?

- **Short answer:** Overview real-time query (always fresh), Snapshot ekটা point-in-time capture persisted DB-e, pore trend chart-er jonno use hoy।
- **Detailed explanation:** `GetOverviewAsync` protibar call-e shob repository method fresh query kore (no caching) — always current data। `CaptureSnapshotAsync` shei-i data compute kore kintu `NationalDashboardSnapshot` entity te save kore with `DistrictStats` child rows — pore `GetTrendAsync` ei saved snapshots theke historical series ber kore, live query na kore (fast, ebং historical point capture kore rakhe jeta overview always-latest data theke recover kora jabe na)।
- **Follow-up:** "Snapshot na thakle trend/forecast e ki hobe?" → `GovForecast`-o snapshot history-r upor depend kore (`GatherForecastInputAsync`) — history empty hole forecast always flat projection dey, tai regular snapshot capture practically important।

Q11. `HeritageDatasetsController` -e `[Authorize]` (bare) kintu andar-e role-check kivabe hoy?

- **Short answer:** Class-level `[Authorize]` shudhu "logged in thakte hobe" enforce kore; fine-grained access `HeritageDbAccessContext` diye service-layer-e (`IsSuperAdmin` , `IsResearcher`) manually check hoy, ebং `[Authorize(Roles = ResearcherRoles)]` shudhu `Create` action-e ache।
- **Detailed explanation:** GET endpoints (`GetAccessible` , `GetById`) shob authenticated user access korte pare kintu service internally `AccessContext` (userId + role flags) diye filter kore je kon dataset ei user dekhte parbe (grant-based access control, per-dataset)। Eta attribute-level RBAC na, eta **row-level authorization logic service-e**।
- **Follow-up:** "Eta keno controller-e na kore service-e kora holo?" → Karon access dataset-specific grants-er upor depend kore, jeta static role diye express kora jay na — dynamic, data-driven permission checking dorkar।

Q12. Funding Application lifecycle-er states/transitions ki?

- **Short answer:** Create → Reviews (multiple) → Decision (approve/reject) → Disbursement schedule → Disbursement status update → (loan hole) Repayment record; ya Withdraw jekono somoy।
- **Detailed explanation:** `FundingApplicationsController` -e alada endpoint protita step-er jonno — `POST reviews` (multiple reviewer comment add), `POST decision` (final approve/reject), `POST disbursements` (tranches schedule kora), `POST disbursements/{id}/status` (paid/failed track kora), `POST repayments` (jodi loan-type hoy)। Each action controller-e ekta thin wrapper, `IFundingService` -e business logic।
- **Follow-up:** "Ekbar Decide kora hoye gele abar decide kora jabe?" (Service code na dekhle honestly bolte hobe "state-guard check korte hobe service-e giye dekhte, likely `ConflictException` thakbe already-decided application re-decide korle" — dekhe confirm kore bola bhalo)।

Q13. Compliance module-er "Requirements" ki? Parent-child relation ta kivabe model kora?

- **Short answer:** `ComplianceRecord` -er ekটা checklist of `ComplianceRequirement` child items, `UpsertRequirementRequest` diye add/update একসাথে একই endpoint-e।
- **Detailed explanation:** `PUT records/{id}/requirements` — upsert pattern, ID dile update noile create (typical upsert semantics), `DELETE records/{id}/requirements/{requirementId}` diye individual requirement remove kora jay full record delete na kore।
- **Follow-up:** "Keno PUT use hoyeche POST-er bodole requirement add-e?" → Upsert idempotent semantics-er jonno PUT beshi appropriate (same request barbar call korle same result), যদিও strict REST purist-der mote create-action-e POST expected — practical trade-off।

Q14. Survey → SurveyResponse → FieldEvidence — data flow ta kivabe kaj kore?

- **Short answer:** Survey (template/questions) → field researcher assign hoy → SurveyResponse (answers, submit→review workflow) → FieldEvidence (raw media, survey ba specific response-r shathe optionally linked)।
- **Detailed explanation:** `SurveysController.AssignFieldResearcher` diye kon researcher kon survey-te kaj korbe ta assign hoy। `SurveyResponsesController` — response create/update/submit (draft→submitted state), tarpor `Review` action-e reviewer approve/reject kore। `FieldEvidenceController` route `{surveyId}/evidence` — survey-level e evidence thake, optionally `SurveyResponseId` diye specific response-r shathe link kora jay (nullable FK)।
- **Follow-up:** "Evidence ki automatically process hoy (OCR/transcription)?" → Na — entity comment explicit bole "backend stores metadata and file references only — no audio/video is processed," transcript thakle seta manually field researcher nijei type kore dey (`TranscriptText` — "not machine-generated" comment ache code-e)।

Q15. `ScopeLabel` resolution kivabe hoy Policy Simulator ebং Heritage Intelligence-e?

- **Short answer:** National/Craft-er jonno static label ("National"/craft name); District/Village-er jonno DB theke actual name lookup kore, na paille `NotFoundException`।
- **Detailed explanation:** `ResolveScopeAsync` (`PolicySimulationService`) ebং similar switch-case

(HeritageIntelligenceService.ComputeAsync) — `ScopeId` diye `GetDistrictAsync / GetVillageAsync` call kore, village-er khete label-e district name-o append kore (`"{village.Name} ({village.DistrictName})"`).

- **Follow-up:** "Scopeld na dile ki hobe District scope-e?" → `ConflictException("ScopeId (district id) is required for District scope.")` — 4091

Q16. GovAnalyticsController-er "exports" workflow ta ki?

- **Short answer:** Metadata-only export request tracking — actual file kono offline worker generate kore, backend shudhu request/status/metadata track kore!
- **Detailed explanation:** `POST exports` — export request create (Pending status likely), `POST exports/{id}/complete` — "called by the export worker" (doc comment) file metadata diye Completed/Failed mark kore! Eita async job pattern — actual heavy file-generation kaj backend request-cycle-e na kore background-e hoy, ei API shudhu orchestration/status-tracking layer!
- **Follow-up:** "Worker ta ki code-e ache?" → Ei cluster-er controller/service-e worker implementation dekha jayni — eta ekta external/future process hishebe design kora, current codebase-e shudhu "complete" callback endpoint ache!

Q17. HeritageIndexScope.Craft -er producer-set kivabe resolve hoy?

- **Short answer:** `ProducerHeritageIdentity` table theke `PrimaryCraft` match kore producer-id set ber kora hoy!
- **Detailed explanation:** `GatherSignalsAsync` -e — District scope hole `Products.DistrictId` -r upor filter, Craft scope hole `ProducerHeritageIdentities.PrimaryCraft` case-insensitive match (`.ToLower() == craft.ToLower()`), naile default shob active `Producer` -role user! Eta ekta `IQueryable<Guid>` hishebe rakha hoy (materialize kora hoy na) — comment-e explicit bola ache: "Keep the producer set as an IQueryable so EF emits an IN (SELECT ...) sub-query rather than a literal id list."
- **Follow-up:** "Keno IQueryable rakha holo list-e convert na kore?" → Performance — National scope-e producer count onek boro hote pare, shob id memory-te list banaye pathale query slow/heavy hobe; sub-query hishebe rakhle DB-i optimize kore!

Q18. Youth Participation index-e "learners per producer" ratio-r formula ki?

- **Short answer:** `(Apprentices + course-enrollments) / activeProducers * 100`, Saturate(k=60) diye normalize!
- **Detailed explanation:** Ekta relative/density metric — total learner na dekhe, producer-base-er tulonay koto learner ache seta dekhe, jate boro district-er absolute number chhoto district-ke unfairly beat na kore! `ActiveProducers == 0` hole divide-by-zero avoid korte explicit `0` return kore!
- **Follow-up:** "Eta keno dorkar, absolute count thakleo to hoy?" → Fair comparison across differently-sized districts/villages — proportional metric na thakle boro area always "better" dekhabe youth-participation-e, jeta misleading hote pare policy-decision-er jonno!

Q19. MonitoringFlagEvent audit trail-er design pattern ta ki?

- **Short answer:** Event-sourcing-style append-only log — protita status change/assign/note-e ekta `MonitoringFlagEvent` row add hoy, flag-er current state field-o update hoy (dual: state + history)!
- **Detailed explanation:** `flag.Events.Add(new MonitoringFlagEvent { Type = ..., FromStatus = ..., ToStatus = ..., ActorUserId = userId, CreatedAt = now })` — ei pattern protita mutating action-e (`UpdateFlagStatusAsync`, `AssignFlagAsync`, `AddFlagNoteAsync`) repeat hoy! EF Core-e navigation collection-e add kore, `SaveChangesAsync` e cascade insert hoy!
- **Follow-up:** "Full event-sourcing na kena — current state field-o rakha hoise?" → Eta "hybrid" — pure event-sourcing hole current state always events theke replay/derive korte hobe (expensive query), ei design-e denormalized current-state column (fast read) + audit-log (history/compliance) duitai rakha hoise, practical trade-off!

Q20. RoleNames.GovernmentNGO constant kothay defined, ebong kivabe \$"{...}" string interpolation role-list banay?

- **Short answer:** `ShilpoHubBD.Domain/Constants/RoleNames.cs` -e const string; `[Authorize(Roles = $"{RoleNames.GovernmentNGO}, {RoleNames.SuperAdmin}")]` compile-time-e ekta comma-separated string literal-e resolve hoy (`"GovernmentNGO,SuperAdmin"`)!
- **Detailed explanation:** Attribute constructor argument-e compile-time constant lagbe (attributes runtime reflection-e read hoy, tai value must be `const` -compatible) — `const string` diye interpolated string-o compile-time constant hoy jodi shob part const hoy, tai eta legally attribute-e use kora jay!
- **Follow-up:** "Typo hole ki hobe role name-e?" → Silent bug — role string match na korle simply shei role-er user-ke access deny kore dibe, kono compile-time error dhorbe na (magic string weakness), tai centralized `RoleNames` class use kora important best-practice!

Q21. PagedResult<T> pattern ta kivabe consistent shob controller-e?

- **Short answer:** Common DTO (`Items`, `TotalCount`, `Page`, `PageSize`) shob `GetPaged` -style endpoint-e reuse hoy, service-layer-e `Page / PageSize` clamp kora hoy (1-100 typical range)!
- **Detailed explanation:** Protita service method-e boilerplate: `query.Page = query.Page < 1 ? 1 : query.Page; query.PageSize = query.PageSize is < 1 or > 100 ? 20 : query.PageSize;` তারপর

`.Skip().Take()` diye repository-e page kora hoy! Eita cluster jurê 100% consistent pattern (12+ jaygay dekha gelo)।

- **Follow-up:** "Frontend-e default PageSize na dile ki hobe?" → Service-e default 20 fallback hoy, tai frontend-er query param na pathaleo safe।

Q22. Heritage Intelligence eb̃ Heritage Risk — same domain mone hoy, kintu code-e ki different?

- **Short answer:** Ha — `HeritageRiskRecord` (`ShilpoHubBD.Domain.Entities.HeritageDatabase` namespace) ek̃ manually-created assessment entity, `HeritageIndexRecord` (`...Entities.Governance` namespace) ek̃ computed/derived score — completely separate table, alada namespace, alada controller/service, but risk records index-computation-e ek̃ **input signal** hishebe use hoy (one-way dependency: Risk → Intelligence, reverse na)।
- **Detailed explanation:** Namespace-level separation-o architecturally meaningful — HeritageDatabase "raw curated data" domain, Governance "derived analytics" domain। Ei separation examiner-ke bola jete pare Clean Architecture-er "bounded context"-er moto ek̃ practical example, jodio eki Application layer-e ache।
- **Follow-up:** "Eki entity Governance-e keno rakha holo na?" → Karon HeritageRisk data heritage-database domain-er core dataset (research/steward-o access kore), kintu Intelligence index purely government/NGO analytics-specific derived output — domain-boundary maintain kora hoyeche।

5. Tricky / Gotcha Questions

5.1 — "AI Predictions" ki shotti AI?

Na। `GovForecastsController`, `PolicySimulatorController`, eb̃ `HeritageIntelligenceController` — tin-tar backend-i `Infrastructure` layer-e alada `RuleBased*Provider` class hishebe implemented, protita class-er naming (`RuleBasedGovForecastProvider`, `RuleBasedPolicySimulationProvider`, `RuleBasedHeritageIntelligenceProvider`) eb̃ XML doc-comment nijei explicitly confess kore je eta "stand-in for a future ML model," "no external calls, no model weights।" Forecast-e real OLS linear regression ache (genuine statistics), kintu Policy Simulator eb̃ Heritage Intelligence purely hardcoded multiplier/weighted-average formula — kono training data, kono learned parameter nei। Interface (`IGovForecastProvider` etc.) design deliberately future-proof kora — DI-te ekta line change kore real ML backend/external API call diye replace kora jabe (Strategy pattern + Dependency Inversion), but **currently** eta shudhu transparent rule-engine।

5.2 — Role-gating correctness: shob jaygay ki thik consistent?

Beshirbhag "governance" controller uniform (`GovernmentNGO`, `SuperAdmin`), kintu `HeritageDatasetsController` -e class-level shudhu bare `[Authorize]` (any logged-in user), fine-grained control service-layer-e (`HeritageDbAccessContext`)। Eita examiner-er jonno ek̃ valid probe point — "keno consistent attribute-based RBAC na?" Answer: karon dataset access **grant-based, per-resource** dynamic permission (kon user kon specific dataset-e access pay), static role-attribute diye eta express kora possible na — tai deliberately service-e push kora hoyeche। Kintu eita **defense-in-depth weaker** — jodi service-layer-e bug thake (access-check bhule jawa), controller-level attribute eta catch korte parbe na, karon controller shudhu "authenticated" check kore। Interview-e ei trade-off honestly explain korte hobe।

5.3 — Forecasting/Simulation-er "Confidence" rating ki statistically rigorous?

Part-e ha, part-e na। Gov Forecast-er confidence bucket (`High` / `Moderate` / `Low`) genuinely R^2 eb̃ sample-count thresholds theke ashe — statistically motivated। Kintu Policy Simulation-er confidence (`Confidence()` helper) shudhu "hasInput && baselineRealistic" — duita boolean condition, kokhono "High" return kore na (max "Moderate"), eta ek̃ **deliberate conservative design choice** na je real statistical confidence-interval calculation। Band-width formula-o (`8%`, `0.6` widening factor) ek̃ hardcoded magic number, kono empirical calibration/backtesting dekh̃। `জাতি` nai code-e।

5.4 — Data trustworthiness: FieldEvidence (self-reported) vs verified data — ki difference?

`FieldEvidence` entity comment explicit: "no audio/video is processed... Transcript... not machine-generated" — mane eta 100% field-researcher-er self-reported/manually-typed data, kono verification/fact-check layer nei code-e (SurveyResponse-r `Review` action ache bote, kintu eta shudhu reviewer approve/reject binary, deep verification na)। Onno dike, `ProducerHeritageIdentity` -er `VerificationStatus == Verified` field — eta ek̃ separate, presumably admin-verified flag (kon service verify kore seta ei research scope-e dekha hoy nai kintu HeritageIntelligenceRepository e "VerifiedHeritageProducers" ek̃ distinct signal hishebe count hoy, non-verified theke)। So Heritage Intelligence score-e "verified" vs "total" producer duitai alada component — kono explicit down-weighting kora hoy na unverified/self-reported data-r jonno shob jaygay (e.g. `RiskLow/Moderate/High` counts-e kono "confidence"/"source-reliability" weight nei — shob risk record shoman weight pay, shei ta manually-created ba deeply-researched howa nirbishesheh)। Eita ek̃ genuine gap: **data provenance/reliability track kora hoy na scoring-e**, examiner ei point tule dhorle "eta ek̃ known limitation" bole honestly answer dewa uchit।

5.5 — NationalDashboard ki HeritageIndexRecord/HeritageRiskRecord use kore?

Na — ei point ta common misconception hote pare! `NationalDashboardService` -er 6-ta repository call (`GetProducerMetricsAsync`, `GetEmploymentMetricsAsync`, ইত্যাদি) shob **raw marketplace/employment/tourism table** theke directly aggregate kore, kono jaygay `HeritageIndexRecord` ba `HeritageRiskRecord` table touch kora hoy na! Heritage Risk/Intelligence completely আলাদা module — officer-ke manually dutoi আলাদা kore dekhte হবে (dashboard এবং heritage-intelligence, দুইটা separate API call)। Kono cross-linking/foreign-key/join nei duitar modhe। Assignment brief-e ei "pipeline" ta suggest kora hoyechilo kintu code-level e eta shudhu **conceptual workflow**, automatic data-flow na — eta ekটা important gotcha specifically dhora dorkar viva-te।

5.6 — Monitoring scan ki scheduled/automated, na manual trigger?

Code-e `POST /api/governance/monitoring/scans` — ekটা manual, on-demand API endpoint. Kono `IHostedService` /background-worker/cron-scheduler ei repo-r cluster-e ei feature specifically run korar jonno paoya jায়নি। Mane "automated scan" naam thakলেও eta actually **on-demand-triggered rule evaluation**, truly-automated (scheduled, no human action) na — officer-ke nijei "Run Scan" button click korte হবে (ba external cron/scheduler thakলে seta ei controller-er baire, e.g. hangfire/quartz kono setup ei file-guloতে dekha jায়নি, so viva-te over-claim na kore "on-demand" bola safe)।

Controllers covered in this document: 19 (all 18 requested + `HeritageDatasetsController` which was the closest match already present under the exact requested name; all requested controller filenames matched exactly except none needed substitution).

ShilpoHubBD — Viva Prep: Logistics & Supply Chain Cluster

Covers: `PickupRequestsController`, `DeliveryRoutesController`, `ShipmentsController`, `WarehousesController`, `WarehouseStockController`, `ReturnsController`, `AiLogisticsController`, `LogisticsPartnersController`, `TraceabilityController`, `ImpactController`, `SustainabilityController` — all 11 confirmed present, no renames needed. Entities live in `ShilpoHubBD.Domain.Entities.Logistics` (confirmed by directory listing), routes under `api/logistics/*` for the core 7 and separate top-level prefixes (`api/traceability`, `api/impact`, `api/sustainability`) for the 3 "adjacent" controllers that ship with this cluster's theme but sit outside the `logistics/` route group.

1. Cluster Overview

ShilpoHubBD-এর Logistics & Supply Chain cluster হলো "User 9 — Logistics Partner"-এর backend, যেখানে একজন verified Logistics Partner (role: `LogisticsPartner`) তার নিজের pickup, shipment, warehouse, stock, এবং return operations manage করে। প্রতিটা controller-ই thin — single service inject করে, business logic পুরোটাই Application layer-এর Service class-এ, আর manual `ToDo()` extension method দিয়ে Entity → DTO mapping হয় (AutoMapper নেই, এটা একটা common viva gotcha)। Authorization pattern consistent: বেশিরভাগ endpoint `[Authorize(Roles = "LogisticsPartner, SuperAdmin")]`, আর প্রতিটা service call-এ `CurrentUserId` + `IsAdmin` পাঠানো হয় যাতে service নিজে data-ownership filter করতে পারে (a partner শুধু নিজের profile-এর অধীনে থাকা resource দেখতে পারে; SuperAdmin সব দেখে)।

Typical request flow এরকম একটা chain follows করে: প্রথমে customer/producer একটা `PickupRequest` তৈরি করে (পণ্য collect করার জন্য) → Logistics Partner সেটা `schedule` করে driver assign করে → পিক-আপ হয়ে গেলে একটা `Shipment` তৈরি হয় (`TrackingNumber` সহ, format `SHP-yyyyMM-####`) যেটা ঐ `PickupRequestId` reference করতে পারে → Shipment একটা `DeliveryRoute`-এর `DeliveryRouteStop`-এ যুক্ত হয়ে move করে (route নিজেও একটা state machine: Draft→Planned→Dispatched→InProgress→Completed) → শিপমেন্ট চলাকালীন বারবার `ShipmentTrackingEvent` (append-only timeline) যোগ হয় status change/location update-এর মাধ্যমে → শেষে হয় `DeliveryAttempt` রেকর্ড করে Delivered/Failed মার্ক হয়। কাস্টমার নিজে কোনো login ছাড়াই `GET api/logistics/shipments/track/{trackingNumber}` (public, `[AllowAnonymous]`) endpoint দিয়ে এই পুরো timeline দেখতে পারে — কিন্তু PII-heavy fields (recipient phone, exact address, COD amount, proof-of-delivery) বাদ দিয়ে একটা "PII-light" DTO রিটার্ন হয়।

Warehouse side-এ একজন partner-এর একাধিক `Warehouse` থাকতে পারে, প্রতিটার ভিতরে `WarehouseZone` আর `WarehouseBin`। Stock movement সবসময় একটা **append-only ledger** (`WarehouseStockMovement`) হিসেবে রেকর্ড হয় — কখনোই শুধু একটা quantity column overwrite করা হয় না; প্রতিটা Receive/Issue/Transfer/Adjust/Reserve/Release অপারেশন একটা নতুন movement row লেখে আর সাথে সাথে `WarehouseStockItem.QuantityOnHand/Reserved/Available` আর bin/warehouse-এর occupancy roll-up sync রাখে, সব একই `SaveChangesAsync()` call-এ (EF Core-এর implicit transaction ব্যবহার করে atomicity নিশ্চিত হয়)। Return এর জীবনচক্রও একটা linear state machine: `Requested` → `Approved` / `Rejected` → (reverse pickup) `PickupScheduled` → `InTransit` → `Received` → `UnderInspection` → `Inspected` (per-item `Condition` + `Disposition` set হয়) → `Restocked` / `RefundPending` → `Refunded` → `Closed`। Interesting architectural note (explicitly commented in code): Restock ধাপে return items warehouse stock-এ **actually যুক্ত হয় না** — শুধু return item-এর `RestockedQuantity` field mark হয়; warehouse-stock integration একটা future TODO হিসেবে রয়ে গেছে। AI Logistics cluster (`AiLogisticsController`) হলো "delivery prediction / route optimization / demand forecast / smart warehouse allocation" — কিন্তু নামে "AI" থাকলেও বাস্তবে এগুলো সব `RuleBased*Provider` ক্লাস (namespace `ShilpoHubBD.Infrastructure.AILogistics`), কোনো ML model বা external LLM call নেই — যেমন route optimization আসলে nearest-neighbour + bounded 2-opt heuristic haversine distance দিয়ে (honest gotcha, নিচে বিস্তারিত)। এছাড়া `TraceabilityController`, `ImpactController`, `SustainabilityController` — এগুলো সরাসরি "Logistics Partner" ইউজার-এর নয়, বরং Producer/marketplace-side sustainability ও product-traceability এবং customer-side impact-summary ফিচার, কিন্তু namespace/domain থিম হিসেবে একই cluster-এ ধরা হয়েছে (supply-chain traceability ও sustainability ঘনিষ্ঠভাবে সম্পর্কিত)।

2. Deep Dive — 4 Representative Controllers

2.1 ShipmentsController — Delivery Tracking

File: `ShilpoHubBD.Api/Controllers/ShipmentsController.cs`
Service: `IDeliveryTrackingService` → `ShilpoHubBD.Application/Services/Logistics/DeliveryTrackingService.cs`
Route prefix: `api/logistics/shipments`
Class-level Auth: `[Authorize(Roles = "LogisticsPartner, SuperAdmin")]`

Verb	Route	Auth	Purpose
GET	<code>track/{trackingNumber}</code>	<code>[AllowAnonymous]</code> (overrides class attr)	Public PII-light tracking timeline
GET	<code>""</code> (paged)	Partner/Admin	List shipments (own scope unless admin)
GET	<code>{id:guid}</code>	Partner/Admin	Full shipment detail (owner-checked)

Verb	Route	Auth	Purpose
POST	""	Partner/Admin	Create shipment
PUT	{id:guid}	Partner/Admin	Edit shipment (only before pickup)
POST	{id:guid}/status	Partner/Admin	Move through status state machine
POST	{id:guid}/events	Partner/Admin	Add a free-form tracking event (scan/checkpoint)
POST	{id:guid}/location	Partner/Admin	Update live GPS/location checkpoint
POST	{id:guid}/delivery-attempts	Partner/Admin	Record an attempt (success or fail)
POST	{id:guid}/deliver	Partner/Admin	Directly mark Delivered (proof-of-delivery)
POST	{id:guid}/cancel	Partner/Admin	Cancel shipment
POST	{id:guid}/notes	Partner/Admin	Append an internal note event
DELETE	{id:guid}	Partner/Admin	Delete (only if Created/LabelCreated/Cancelled)

Key code — the public tracking endpoint (`ShipmentsController.cs:30-34`):

```
/// <summary>Public, PII-light tracking timeline for a tracking number.</summary>
[HttpGet("track/{trackingNumber}")]
[AllowAnonymous]
public async Task<ActionResult<ShipmentTrackingDto>> Track(string trackingNumber, CancellationToken cancellationToken)
    => Ok(await _service.TrackByNumberAsync(trackingNumber, cancellationToken));
```

Banglish: এই একটা method-ই class-level `[Authorize(...)]`-কে override করে `[AllowAnonymous]` দিয়ে — ASP.NET Core-এ method-level attribute সবসময় class-level-এর উপরে priority পায়। এখানে কোনো `currentUserId` / `isAdmin` param নেই, কারণ যে কেউ (login ছাড়া) tracking number জানলেই দেখতে পারবে — ঠিক যেমন Pathao/Sundarban courier-এর tracking page।

Service-side mapping (`DeliveryTrackingService.ToTrackingDto()` in `LogisticsMappings.cs:398-425`) শুধু এগুলো রিটার্ন করে: `TrackingNumber`, `Status`, `ServiceLevel`, `OriginCity` / `DestinationCity` (district/exact address না), `ParcelCount`, milestone timestamps, `CurrentLocationLabel`, আর `Checkpoints` list (event type, status, location label, district name, description, timestamp)। **বাদ যাওয়া** fields: `RecipientPhone`, `OriginPhone`, exact address lines, `Latitude/Longitude`, `CodAmount`, `ReceivedByName`, `SignatureImageUrl`, `CreatedByUserId`। এটিই "PII-light" design — anonymous user শুধু "কোথায় পৌঁছেছে" জানবে, কার কাছে পৌঁছাবে বা কত টাকা COD সেটা না।

Key code — status transition guard (`DeliveryTrackingService.cs:18-46`, `589-595`):

```
private static readonly Dictionary<ShipmentStatus, ShipmentStatus[]> Transitions = new()
{
    [ShipmentStatus.Created] = new[] { ShipmentStatus.LabelCreated, ShipmentStatus.PickedUp, ShipmentStatus.Cancelled },
    ...
    [ShipmentStatus.Delivered] = Array.Empty<ShipmentStatus>(),
    [ShipmentStatus.Returned] = Array.Empty<ShipmentStatus>(),
    [ShipmentStatus.Cancelled] = Array.Empty<ShipmentStatus>(),
};
...
private static void EnsureTransition(ShipmentStatus from, ShipmentStatus to)
{
    if (!Transitions.TryGetValue(from, out var allowed) || !allowed.Contains(to))
        throw new ConflictException($"Cannot move a shipment from {from} to {to}.");
}
```

Banglish: একটা static `Dictionary<Status, Status[]>` দিয়ে allowed-transition graph define করা — এটা একটা explicit finite-state-machine pattern। Delivered/Returned/Cancelled হলো terminal states (empty array মানে কোথাও যাওয়া যাবে না)। `UpdateStatusAsync` method এই dictionary check করে `ConflictException` (409) throw করে যদি invalid transition হয় (যেমন Delivered থেকে Created-এ ফেরানো)।

Key code — delivery attempt & COD capture (`DeliveryTrackingService.cs:413-467`): প্রতিটা attempt-এ `shipment.DeliveryAttemptCount` বাড়ে, একটা নতুন `DeliveryAttempt` child entity যোগ হয়। Outcome `Delivered` হলে status সরাসরি `Delivered` -এ চলে যায় আর `ApplyCod()` helper cash-on-delivery টাকা collect হয়েছে কিনা মার্ক করে (idempotent guard: `!shipment.CodCollected` চেক করে যাতে দুইবার true না হয়ে যায়)। Outcome ব্যর্থ হলে status `DeliveryFailed` -এ যায়, `FailureReason` সেট হয় — কিন্তু transition dictionary অনুযায়ী `DeliveryFailed` থেকে আবার retry (OutForDelivery) বা Returned/Cancelled/Delivered — সব সম্ভব, তাই একাধিক delivery attempt চলতেই পারে।

Business rules worth citing in viva:

- Tracking number generation is retried up to 5 times for uniqueness, then falls back to a GUID-suffixed one (`UniqueTrackingNumberAsync` , lines 661-673) — handles the astronomically rare collision case.
- `Shipment` details (address, weight, etc.) can only be edited while status is `Created` / `LabelCreated` — once picked up, immutable except through dedicated status/event endpoints (`ConflictException` guard, line 182-185).
- `EnsureNotClosed` blocks adding tracking events once a shipment reaches Delivered/Returned/Cancelled.
- Ownership check (`LoadOwnedAsync` , lines 553-571) throws `UnauthorizedAccessException` (→ 401) if a non-admin partner tries to touch another partner's shipment — this is a *service-layer* authorization check, separate from the controller's role-based `[Authorize]` .

2.2 `WarehouseStockController` — Warehouse Stock & Movement Ledger

File: `ShilpoHubBD.Api/Controllers/WarehouseStockController.cs`

Service: `IWarehouseStockService` → `WarehouseStockService.cs`

Route prefix: `api/logistics/warehouse-stock`

Class-level Auth: `[Authorize(Roles = "LogisticsPartner,SuperAdmin")]`

Verb	Route	Purpose
GET	<code>""</code>	Paged list of stock items (SKU holdings)
GET	<code>movements</code>	Paged list of ledger movements
GET	<code>{id:guid}</code>	Stock item detail + recent movements
POST	<code>receive</code>	Inbound stock (new or top-up existing item)
POST	<code>{id:guid}/issue</code>	Outbound (remove available stock)
POST	<code>{id:guid}/transfer</code>	Bin-to-bin move (full or partial split)
POST	<code>{id:guid}/adjust</code>	Manual correction / stock-count / damage / disposal
POST	<code>{id:guid}/reserve</code>	Reserve units (reduces available, not on-hand)
POST	<code>{id:guid}/release</code>	Release a reservation
DELETE	<code>{id:guid}</code>	Delete stock item (only if fully zeroed out)

Key code — Receive (inbound), creating the movement row (`WarehouseStockService.cs:89-109`):

```
item.QuantityOnHand += request.Quantity;
RecomputeAvailable(item);
item.ReceivedAt ??= occurredAt;
item.LastMovementAt = occurredAt;
item.UpdatedAt = now;

if (bin is not null) { bin.OccupiedUnits += request.Quantity; bin.UpdatedAt = now; }
warehouse.UsedCapacityUnits += request.Quantity;
warehouse.UpdatedAt = now;

await AddMovementAsync(warehouse.Id, item, WarehouseStockMovementType.Inbound, request.Quantity,
    currentUserId, occurredAt, now, fromBinId: null, toBinId: request.WarehouseBinId,
    request.ReferenceType, request.ReferenceId, request.Reason, request.Note, cancellationTokens);

await _repository.SaveChangesAsync(cancellationTokens);
```

Banglish: এখানে দুটো জিনিস একসাথে ঘটছে একই method-এ — (1) `WarehouseStockItem` -এর quantity mutate হচ্ছে (current state), আর (2) `AddMovementAsync` দিয়ে একটা নতুন **append-only** `WarehouseStockMovement` row তৈরি হচ্ছে (history)। এই দুটোই একই `SaveChangesAsync()` call-এ commit হয় — EF Core `SaveChangesAsync` by default একটা implicit database transaction-এ সব pending change wrap করে, তাই partial write (শুধু item update হলো কিন্তু movement row লেখা হলো না) হওয়ার সুযোগ নেই। কোনো explicit `BeginTransactionAsync()` / `CommitAsync()` কোডে নেই — কারণ single `DbContext.SaveChangesAsync()` call নিজেই atomic।

Key code — Movement ledger row shape (`AddMovementAsync` , lines 507-529):

```
⇒ await _repository.AddMovementAsync(new WarehouseStockMovement
{
    Id = Guid.NewGuid(),
    WarehouseId = warehouseId,
    WarehouseStockItemId = item.Id,
    Type = type,
    Quantity = quantity,
    QuantityOnHandAfter = item.QuantityOnHand,    // snapshot AFTER the mutation
    FromBinId = fromBinId,
    ToBinId = toBinId,
    Sku = item.Sku,
    ReferenceType = ..., ReferenceId = ...,        // links back to order/return/etc
    Reason = ..., Note = ...,
    PerformedByUserId = actorUserId,
    OccurredAt = occurredAt,
    CreatedAt = now,
}, cancellationToken);
```

Banglish: প্রতিটি movement row-এ `QuantityOnHandAfter` snapshot রাখা হয় — মানে ledger নিজেই self-auditing: কোনো point-in-time-এ কী ছিল তা পুরনো row রিপ্রেজেন্ট করেই জানা যায়। `Type` enum (`Inbound`, `Outbound`, `TransferIn`, `TransferOut`, `Adjustment`, `StockCount`, `Damage`, `Disposal`, `Reserve`, `ReleaseReservation`) দিয়ে movement category ধরা হয়।

Key code — Transfer, full vs. partial split (`TransferAsync`, lines 172-275): এখানে interesting business rule হলো — `isFullMove = request.Quantity == item.QuantityOnHand` চেক করে যদি পুরো quantity move হয়, তাহলে সরাসরি `item.WarehouseBinId` বদলে দেওয়া হয় (item টাই নতুন bin-এ চলে যায়, **দুইটা** movement row লেখা হয়: `TransferOut` + `TransferIn`, একই item-এর জন্য)। কিন্তু partial move হলে — original item থেকে quantity কমে, আর destination bin-এ matching item না পেলে **নতুন** `WarehouseStockItem` তৈরি হয় (`FindMatchingStockItemAsync` দিয়ে `warehouse+bin+SKU+batch` ম্যাচ করে দেখে, না পেলে clone করে নতুন row)। এখানেও guard আছে: partial split করা যাবে শুধু `unreserved (available) stock`-এর উপর — `if (!isFullMove && request.Quantity > item.QuantityAvailable)` throw করে, কারণ reserved stock অন্য কোথাও committed, bin split করলে reservation broken হয়ে যাবে।

Key code — Adjust, guarding against negative or under-reserved stock (`AdjustAsync`, lines 277-336):

```
if (newOnHand < 0)
    throw new ConflictException("An adjustment cannot take on-hand quantity below zero.");
if (newOnHand < item.QuantityReserved)
    throw new ConflictException($"On-hand cannot drop below the reserved quantity ({item.QuantityReserved}).");
```

Banglish: Business invariant — `QuantityOnHand ≥ QuantityReserved` সবসময় বজায় রাখতে হবে (নাহলে reserved stock physically না থেকেও reserved দেখাবে, inconsistent state)। `MovementType` চারটার একটায় সীমাবদ্ধ: Adjustment/StockCount/Damage/Disposal।

Business rules worth citing in viva:

- `QuantityAvailable = QuantityOnHand - QuantityReserved`, recomputed every mutation via `RecomputeAvailable()` — **never persisted independently as a raw input**, it's always derived.
- Delete only allowed when both `QuantityOnHand` and `QuantityReserved` are zero (lines 351-354) — can't delete a stock row with history/balance still outstanding.
- No optimistic concurrency token (no `RowVersion` / `[Timestamp]` on `WarehouseStockItem`) — under concurrent `Issue` calls on the same item, EF Core's default "last write wins" applies; there IS an application-level guard (`Quantity > item.QuantityAvailable` check) but it's checked against the in-memory value loaded at the start of the request, so a genuine race between two concurrent requests is still theoretically possible (**tricky/gotcha — see §5**).

2.3 ReturnsController — Return Handling (Inspection → Disposition → Restock/Refund)

File: `ShilpoHubBD.Api/Controllers/ReturnsController.cs`

Service: `IReturnHandlingService` → `ReturnHandlingService.cs`

Route prefix: `api/logistics/returns`

Class-level Auth: `[Authorize(Roles = "LogisticsPartner,SuperAdmin")]`

Verb	Route	Purpose
GET	<code>` / {id:guid}`</code>	List / detail
POST	<code>``</code>	Create return request (with line items)
PUT	<code>{id:guid}</code>	Edit (only while <code>Requested</code>)
POST	<code>{id:guid}/approve</code>	Approve → allows reverse pickup

Verb	Route	Purpose
POST	<code>{id:guid}/reject</code>	Reject → terminal-ish (only → Closed)
POST	<code>{id:guid}/schedule-pickup</code>	Schedule reverse pickup
POST	<code>{id:guid}/status</code>	Generic move to InTransit/Received/UnderInspection/Closed
POST	<code>{id:guid}/inspections</code>	Record inspection + per-item condition/disposition
POST	<code>{id:guid}/restock</code>	Mark inspected items as restocked
POST	<code>{id:guid}/refund</code>	Record refund (pending or paid)
POST	<code>{id:guid}/close</code>	Close the return
POST	<code>{id:guid}/cancel</code>	Cancel
POST	<code>{id:guid}/notes</code>	Append note
DELETE	<code>{id:guid}</code>	Delete (only Requested/Rejected/Cancelled)

Key code — the state machine (`ReturnHandlingService.cs:18-42`):

```
[ReturnStatus.Requested] = { Approved, Rejected, Cancelled }
[ReturnStatus.Approved] = { PickupScheduled, InTransit, Received, Cancelled }
[ReturnStatus.Received] = { UnderInspection, Inspected, Cancelled }
[ReturnStatus.Inspected] = { Restocked, RefundPending, Refunded, Closed }
[ReturnStatus.Restocked] = { RefundPending, Refunded, Closed }
[ReturnStatus.RefundPending] = { Refunded, Closed }
[ReturnStatus.Refunded] = { Closed }
```

Banglish: এই cluster-এর সবগুলো lifecycle-heavy entity (`Shipment` , `DeliveryRoute` , `ReturnRequest`) একই architectural pattern follow করে — static `Dictionary<Enum, Enum[]>` দিয়ে explicit allowed-transition graph, আর একটা common `EnsureTransition()` helper যা invalid move হলে `ConflictException` (409) ছোঁড়ে। এটা একটা reusable design decision যা পুরো Logistics module জুড়ে consistent।

Key code — Inspection, per-item condition/disposition (`RecordInspectionAsync` , lines 354-418):

```
foreach (var assessment in request.ItemAssessments)
{
    var item = ret.Items.FirstOrDefault(i => i.Id == assessment.ReturnItemId)
    ?? throw new ConflictException($"Return item {assessment.ReturnItemId} not found on this return.");

    if (assessment.QuantityReceived.HasValue)
        item.QuantityReceived = Math.Clamp(assessment.QuantityReceived.Value, 0, item.Quantity);
    if (!string.IsNullOrWhiteSpace(assessment.Condition))
        item.Condition = ParseEnum<ReturnItemCondition>(assessment.Condition, "Invalid item Condition.");
    if (!string.IsNullOrWhiteSpace(assessment.Disposition))
        item.Disposition = ParseEnum<ReturnDisposition>(assessment.Disposition, "Invalid item Disposition.");
    ...
}

ret.Inspections.Add(new ReturnInspection { ... OverallCondition = overall, RecommendedResolution = resolution, ... });
ret.Status = ReturnStatus.Inspected;
```

Banglish: একটা return-এ একাধিক item থাকতে পারে, আর inspection **item-level granularity**-তে হয় — প্রতিটা `ReturnItem` -এর নিজস্ব `Condition` (New/Used/Damaged/NotReceived ইত্যাদি) আর `Disposition` (Pending/Restock/Discard/... ইত্যাদি) সেট হয়, পাশাপাশি overall `ReturnInspection` একটা audit record (photos, summary, recommended resolution) হিসেবে append হয়। একই return-এ একাধিকবার inspection করা সম্ভব (status `Received` / `UnderInspection` / `Inspected` সবগুলো থেকেই আবার inspect করা যায়) — re-inspection allowed, যা বাস্তব warehouse workflow-এর সাথে মেলে (প্রথমবার miss হলে re-check)।

Key code — Restock, and the explicit "no real warehouse link yet" gotcha (`RestockAsync` , lines 420-465):

```

if (item.Disposition != ReturnDisposition.Restock)
    throw new ConflictException($"Item '{item.Description}' is dispositioned '{item.Disposition}', not Restock.");

var cap = item.QuantityReceived > 0 ? item.QuantityReceived : item.Quantity;
item.RestockedQuantity = Math.Clamp(line.RestockedQuantity, 0, cap);
...
ret.Status = ReturnStatus.Restocked;

```

Banglish: এখানে শুধু bookkeeping হচ্ছে — `item.RestockedQuantity` field-টা set হয় (কত ইউনিট restock হিসেবে count হলো)। কিন্তু কোনো `WarehouseStockItem.QuantityOnHand` বাড়ে না, কোনো `WarehouseStockMovement` তৈরি হয় না — service class-এর XML doc comment-এই explicitly লেখা আছে: "Restock does not (yet) write into warehouse stock — that link is a later integration item." এটা একটা honest, verified architectural gap — viva-তে খুব ভালো "explain the current limitation" question হতে পারে।

Key code — Refund, two-stage (`RecordRefundAsync`, lines 467-513): `request.MarkPaid` flag দিয়ে সিদ্ধান্ত হয় status `RefundPending` (initiated, not yet paid) নাকি সরাসরি `Refunded` (paid) হবে — একই endpoint দুই ধরনের transition handle করে, `EnsureTransition` দুইবার আলাদা target দিয়ে call হয় branch অনুযায়ী।

Business rules worth citing in viva:

- Approve can optionally re-target `DestinationWarehouseId` at approval time.
- `SchedulePickupAsync` only allowed from `Approved` or already `PickupScheduled` (idempotent reschedule).
- Delete only allowed pre-lifecycle (`Requested` / `Rejected` / `Cancelled`) — once physical goods move, no hard delete, only status transitions (audit trail preserved).
- `ReferenceCode` generated same pattern as shipment tracking number: `RTN-yyyyMM-####` with retry+GUID fallback.

2.4 AiLogisticsController — "AI" Logistics (Rule-Based, Not ML)

File: `ShilpoHubBD.Api/Controllers/AiLogisticsController.cs`

Service: `IAiLogisticsService` → `AiLogisticsService.cs`

Providers: `ShilpoHubBD.Infrastructure/AiLogistics/RuleBased*Provider.cs` (4 classes)

Route prefix: `api/logistics/ai`

Class-level Auth: `[Authorize(Roles = "LogisticsPartner,SuperAdmin")]`

Verb	Route	Purpose
POST/GET/GET-by-id/DELETE	<code>delivery-predictions[...]</code>	ETA prediction for a shipment
POST/apply/GET/GET-by-id/DELETE	<code>route-optimizations[...]</code>	Re-sequence a route's stops
POST/GET/GET-by-id/DELETE	<code>demand-forecasts[...]</code>	Forecast shipment/pickup/return volume
POST/GET/GET-by-id/DELETE	<code>warehouse-allocations[...]</code>	Recommend which warehouse to stock into

Each of the 4 "AI" features follows the **same shape**: gather signals from the partner's own historical data via the repository → build a plain-C# input record → call an injected provider interface (`IDeliveryPredictionProvider`, `IAiRouteOptimizationProvider`, `IDemandForecastProvider`, `IWarehouseAllocationProvider`) → map the result into a persistable entity → optionally persist (`request.Persist` flag — a caller can preview without saving) → return DTO.

Key code — Route optimization request/response wiring (`AiLogisticsService.cs:146-228`):

```

var input = new AiRouteOptimizationInput
{
    Objective = objective,
    AverageSpeedKmh = request.AverageSpeedKmh is > 0 ? request.AverageSpeedKmh!.Value : 25.0,
    StartLatitude = route.StartLatitude, StartLongitude = route.StartLongitude,
    Stops = route.Stops.OrderBy(s => s.Sequence).Select(s => new AiRouteStopInput( ... )).ToList(),
};
var result = _routeOptimizationProvider.Optimize(input);

```

Banglish: Controller → Service → Provider — এই তিন-স্তর indirection-টাই মূল design: service business-rule/ownership handle করে, **provider** interface-টা হলো "the algorithm" যেটা pluggable (আজকে rule-based, ভবিষ্যতে চাইলে একই interface implement করে একটা real ML/external API-backed provider দিয়ে swap করা যায় — Dependency Injection-এ শুধু `IAiRouteOptimizationProvider`-এর registration বদলালেই হবে, service/controller কোড touch করতে হবে না)।

Key code — the actual "algorithm" (honest look) (`RuleBasedAiRouteOptimizationProvider.cs:1-90`):

```
/// Rule-based stand-in for an AI route solver. Seeds a tour with nearest-neighbour from the route
/// start, then runs a bounded 2-opt improvement pass over haversine distances. ... No external calls.
public class RuleBasedAiRouteOptimizationProvider : IAIRouteOptimizationProvider
{
    public const string Name = "rule-based-route-2opt-v1";
    ...
    var optimised = NearestNeighbour(input.StartLatitude, input.StartLongitude, located);
    TwoOpt(input.StartLatitude, input.StartLongitude, optimised);
}
```

Banglish: এটাই সবচেয়ে বড় honest gotcha — কোনো neural network, কোনো LLM call, কোনো external AI service নেই।

"AI"-লেবেলড ফিচার আসলে classic operations-research heuristics: (1) **Nearest-Neighbour construction** — route start থেকে সবচেয়ে কাছের unvisited stop-এ যাওয়া, বারবার (greedy TSP approximation), তারপর (2) **2-opt local-search improvement** — pair of edges নিয়ে reverse করে দেখা distance কমে কিনা, না কমলে revert (up to 12 passes, `MaxTwoOptPasses`), আর দূরত্ব মাপা হয় **Haversine formula** দিয়ে (great-circle distance on Earth's sphere, `EarthRadiusKm = 6371.0`)। `Confidence` (High/Moderate/Low) শুধু কতগুলো stop-এর geo-coordinate ছিল তার উপর ভিত্তি করে একটি simple bucket, কোনো statistical model-output না। `Method` string ("`rule-based-route-2opt-v1`") সবসময় persisted result-এর সাথে সেভ থাকে — transparently marked as non-ML।

Key code — Apply, guarding against stale optimization runs (`ApplyRouteOptimizationAsync`, lines 230-286):

```
var runStopIds = run.Stops.Select(s => s.RouteStopId).OrderBy(x => x).ToList();
var routeStopIds = route.Stops.Select(s => s.Id).OrderBy(x => x).ToList();
if (!runStopIds.SequenceEqual(routeStopIds))
    throw new ConflictException("The route's stops have changed since this run was generated; re-run the optimiser.");
```

Banglish: একটি optimization "run" propose হবার পরও route-এ stop add/remove হতে পারে — তাই apply করার আগে set-equality চেক করা হয় (run-এর stop id set == route-এর current stop id set), নাহলে stale/mismatched optimization plan apply হয়ে যেতে পারত। Apply হলে route শুধু `Draft` / `Planned` অবস্থায় থাকতে পারে (already-dispatched route resequence করা যাবে না) এবং `route.OptimizationStrategy = "ai-2opt"` string tag করে রাখা হয় traceability-র জন্য।

Business rules worth citing in viva:

- `request.Persist == false` lets a caller preview a prediction/optimization/forecast/allocation without writing to DB — useful for "what-if" UI calls.
- Every artefact (`DeliveryPrediction`, `RouteOptimizationRun`, `DemandForecast`, `WarehouseAllocationRecommendation`) is scoped to `LogisticsPartnerProfileId` and ownership-checked identically to shipments/returns (`EnsureOwnedAsync` helper).
- Demand forecast validates `Metric` against an allow-list that differs by `Scope` (`WarehouseMetrics` = inbound/outbound vs `NetworkMetrics` = shipments/pickups/returns/weight_kg) — a `ConflictException` (400-mapped) guards against nonsensical metric/scope combos.

3. Reference Table — Remaining Controllers

Controller	Route prefix	Key endpoints	Auth/Roles	One-line purpose
PickupRequestsController	api/logistics/pickups	CRUD, /schedule, /assign, /status, /cancel, /notes	LogisticsPartner, SuperAdmin	Request → schedule → assign · pickup collection job
DeliveryRoutesController	api/logistics/routes	CRUD, /stops, /resequence, /optimize, /assign, /dispatch, /start, /complete, /cancel, /stops/{id}/arrive complete skip fail	LogisticsPartner, SuperAdmin	Plan a multi-stop delivery route, drive it through its lifecycle to stop
WarehousesController	api/logistics/warehouses	CRUD, /zones, /bins (add/update/remove)	LogisticsPartner, SuperAdmin	Manage physical warehouses & internal zone/bin layout
LogisticsPartnersController	api/logistics/partners	GET (admin-only paged list), GET me, GET {userId}, PUT {userId}, POST {userId}/verify (admin-only), service-area upsert/remove, DELETE	LogisticsPartner (own), SuperAdmin (all + verify)	Onboarding/profile/verification area management for a logistic partner account
TraceabilityController	api/traceability	GET products/{productId} (public), POST / PUT / DELETE	Public read; Producer/SuperAdmin write	Product provenance ("origin story" records tied to a marketplace F
ImpactController	api/impact	GET mine	Any authenticated user ([Authorize] , no role restriction)	Personal sustainability/impact for the logged-in user
SustainabilityController	api/sustainability	GET me (Producer), GET producers/{id} (public), POST materials, POST certifications (Producer), POST certifications/{id}/verify (SuperAdmin)	Mixed: Producer write, public read, SuperAdmin verify	Producer's sustainable-materials sourcing profile + third-party certification verification workfl

4. Viva Question Bank (20 Q&A)

Q1. Shipment তৈরি করার সময় TrackingNumber কীভাবে generate হয়, আর duplicate হবার সম্ভাবনা কীভাবে হ্যান্ডেল করা হয়?

- **Short answer:** SHP-yyyyMM-##### format-এ random 5-digit suffix, existence check করে retry (max 5 বার), শেষে fallback হিসেবে GUID-suffixed string।
- **Detailed explanation:** UniqueTrackingNumberAsync একটা loop-এ candidate বানায় (Random.Shared.Next(0, 100000):D5), প্রতিবার repository.TrackingNumberExistsAsync() দিয়ে DB-তে চেক করে। ৫ বার চেষ্টার পরও collision হলে (statistically প্রায় অসম্ভব) একটা GUID-based string বানিয়ে ২০ ক্যারেক্টারে truncate করে ব্যবহার করে।
- **Project example:** DeliveryTrackingService.cs, UniqueTrackingNumberAsync (lines 661-673); একইরকম pattern ReturnHandlingService.UniqueReferenceAsync ("RTN-yyyyMM-#####")-এও reuse হয়েছে।
- **Follow-up:** "এটা কি race-condition-safe?" → না পুরোপুরি — দুইটা concurrent request একই candidate পেতে পারে, existence check আর insert-এর মাঝে gap আছে (TOCTOU); সত্যিকারের safety নিশ্চিত করতে হলে DB-level unique constraint + catch on save লাগবে (থাকতে পারে, কিন্তু কোডে explicit try/catch নেই)।

Q2. track/{trackingNumber} endpoint public কেন, আর security-wise কী সাবধানতা নেওয়া হয়েছে?

- **Short answer:** Customer experience-এর জন্য login ছাড়া tracking দরকার (বাস্তব courier সাইটগুলোর মতো); সাবধানতা হলো একটা আলাদা "PII-light" DTO (ShipmentTrackingDto) যেখানে phone/address/COD/signature বাদ।
- **Detailed explanation:** Method-level [AllowAnonymous] class-level [Authorize] -কে override করে। Service-এ কোনো ownership check নেই কারণ input হলো tracking number নিজেই — এটা effectively "possession-based" access (যে number জানে সে-ই দেখতে পারবে), OTP-র মতো একটা low-entropy secret না হলেও practically enough কারণ number randomly generated এবং শুধু shipper+recipient জানে।
- **Project example:** ShipmentsController.cs:30-34, mapping LogisticsMappings.cs:398-425।
- **Follow-up:** "যদি কেউ tracking number brute-force করে?" → Format predictable (SHP-yyyyMM-#####, মাত্র ১ লাখ combination per month) — rate-limiting না থাকলে theoretically brute-forceable; project-wide global rate limiter policy (memory অনুযায়ী AddRateLimiter) থাকলে সেটা mitigate করে, কিন্তু controller নিজে কোনো extra throttle করে না — এটা একটা legitimate follow-up gotcha।

Q3. Warehouse stock movement-এর জন্য একটা quantity column-ই যথেষ্ট না কেন? Append-only ledger কেন দরকার?

- **Short answer:** Audit trail, point-in-time reconstruction, এবং dispute-resolution-এর জন্য — শুধু current quantity থাকলে "কীভাবে এই সংখ্যায় পৌঁছালাম" তার ইতিহাস হারিয়ে যায়।
- **Detailed explanation:** প্রতিটা stock-affecting action (Receive/Issue/Transfer/Adjust/Reserve/Release) একটা WarehouseStockMovement row লেখে যাতে QuantityOnHandAfter snapshot থাকে। এতে করে "গত মাসে damage কত হয়েছিল", "কোন reference (order/return) থেকে stock issue হয়েছিল" ইত্যাদি traceable থাকে। এটা accounting-এর double-entry ledger ধারণার সাথে সাদৃশ্যপূর্ণ।
- **Project example:** WarehouseStockService.AddMovementAsync (lines 507-529), entity WarehouseStockMovement.cs।
- **Follow-up:** "Ledger আর current-state row দুটোই কেন রাখা, ledger থেকে replay করলেই তো হয়?" → Performance — প্রতিবার current quantity জানতে পুরো movement history sum করা ব্যয়বহুল; তাই WarehouseStockItem -এ derived/cached current state রাখা হয়েছে, ledger শুধু audit+history-র জন্য (CQRS-এর মতো একটা lightweight pattern, যদিও formal CQRS না)।

Q4. QuantityAvailable = QuantityOnHand - QuantityReserved — এটা persisted না computed?

- **Short answer:** Persisted column, কিন্তু সবসময় RecomputeAvailable() helper দিয়ে on-hand/reserved বদলানোর সাথে সাথে sync করা হয়।
- **Detailed explanation:** DB-তে generated/computed column না — plain int column, কিন্তু service layer-এর discipline (protocol) হিসেবে প্রতিটা mutation-এর পরে recompute call করা হয়। এটা normalization-বিরোধী মনে হতে পারে, কিন্তু query-performance-এর জন্য derived value cache করা একটা সাধারণ trade-off।
- **Project example:** WarehouseStockService.cs:482-483, ব্যবহার হয় Receive/Issue/Transfer/Adjust/Reserve/Release সব জায়গায়।
- **Follow-up:** "এটা inconsistent হতে পারে কোনোভাবে?" → হ্যাঁ যদি কোনো নতুন mutation path (developer ভুল করে) RecomputeAvailable() call করতে ভুলে যায় — এটা একটা maintenance risk, DB-level computed column বা check constraint দিলে এই risk থাকত না।

Q5. Return-এর Restock action কি সত্যিই warehouse-এ stock যোগ করে?

- **Short answer:** না — শুধু ReturnItem.RestockedQuantity bookkeeping field সেট হয়, WarehouseStockItem / WarehouseStockMovement -এ কিছু লেখা হয় না।
- **Detailed explanation:** এটা service-এর XML doc comment-এ explicitly স্বীকার করা একটা gap: "Restock does not (yet) write into warehouse stock — that link is a later integration item." বাস্তবে production-ready system হলে RestockAsync -এর ভেতর থেকে WarehouseStockService.ReceiveAsync -এর মতো একটা movement তৈরি করা উচিত ছিল।
- **Project example:** ReturnHandlingService.cs:420-465।
- **Follow-up:** "কেন এমন রাখা হলো?" → সম্ভবত module boundary maintain করার জন্য (Return module warehouse module-কে সরাসরি না ঝুঁয়ে); কিন্তু এতে business-level data inconsistency তৈরি হয় (return "restocked" বলছে কিন্তু warehouse stock বাড়েনি)।

Q6. Return এবং Shipment দুটোরই status transition একই pattern দিয়ে কেন করা হয়েছে?

- **Short answer:** Consistency ও maintainability — একটা static Dictionary<Enum, Enum[]> + EnsureTransition() helper pattern পুরো Logistics module-জুড়ে reuse হয়েছে (Shipment, DeliveryRoute, Return সবগুলোতে)।
- **Detailed explanation:** এটা একটা lightweight, in-memory finite-state-machine — কোনো external state-machine library (যেমন Stateless) ব্যবহার হয়নি, শুধু plain C# dictionary।
- **Project example:** DeliveryTrackingService.cs:18-46, ReturnHandlingService.cs:18-42।
- **Follow-up:** "Library ব্যবহার না করার সুবিধা/অসুবিধা কী?" → সুবিধা: কোনো dependency নেই, simple, debug করা সহজ। অসুবিধা: complex guard condition (যেমন role-based transition, conditional branching) হলে dictionary approach আর যথেষ্ট expressive থাকে না।

Q7. LoadOwnedAsync / EnsureOwnedAsync pattern কী, আর কেন controller-level [Authorize(Roles=...)] যথেষ্ট না?

- **Short answer:** Role-based authorize শুধু "কে কোন role" চেক করে, কিন্তু "এই নির্দিষ্ট shipment/return/stock item কার" সেটা service-layer-এ resource-ownership check দরকার (multi-tenancy)।
- **Detailed explanation:** প্রতিটা service method প্রথমে entity load করে, তারপর if (!isAdmin) { ... if (entity.ProfileId != callerProfile.Id) throw UnauthorizedAccessException }। Controller role check করে "সে LogisticsPartner কিনা", কিন্তু "কোন partner-এর ডেটা" সেটা row-level authorization, যেটা business-logic layer-এই যুক্তিসঙ্গত।
- **Project example:** DeliveryTrackingService.LoadOwnedAsync (lines 553-571), WarehouseStockService.LoadOwnedItemAsync,

`ReturnHandlingService.LoadOwnedAsync`।

- **Follow-up:** "SuperAdmin সব দেখতে পারে কীভাবে guaranteed?" → `isAdmin` flag শুরুতেই controller-এ `User.IsInRole(RoleNames.SuperAdmin)` দিয়ে সেট হয়ে service-এ pass হয়; `isAdmin == true` হলে ownership-check block পুরোপুরি skip হয়।

Q8. `AiLogisticsController` -এর ফিচারগুলো আসলে কি real AI/ML?

- **Short answer:** না — সব `RuleBased*Provider` ক্লাস, deterministic heuristics/statistics, কোনো trained model বা external LLM call নেই।

- **Detailed explanation:** Route optimization = nearest-neighbour + 2-opt (classic TSP heuristic) + Haversine distance। বাকিগুলো (delivery prediction, demand forecast, warehouse allocation) না দেখলেও একই namespace/naming convention অনুসরণ করে — সবই "Rule-Based" prefix দিয়ে explicit।

- **Project example:** `RuleBasedAiRouteOptimizationProvider.cs` (`Method = "rule-based-route-2opt-v1"`)।

- **Follow-up:** "তাহলে 'AI' নাম কেন দেওয়া হলো?" → ইন্টারফেস abstraction (`IAiRouteOptimizationProvider`) ভবিষ্যতে real ML/API-backed provider দিয়ে swap করার জন্য design করা হয়েছে — নামটা "target capability" বোঝায়, বর্তমান implementation একটা honest placeholder।

Q9. `request.Persist` flag-এর উদ্দেশ্য কী?

- **Short answer:** Caller কে preview/what-if capability দেওয়া — DB-তে না লিখেই prediction/optimization দেখা যায়।

- **Detailed explanation:** `false` হলে entity বানানো হয় in-memory, তারপর DTO-তে map করে সরাসরি রিটার্ন করে দেওয়া হয় (`SaveChangesAsync` কখনো call হয় না)। `true` হলে repository-তে add + save + fresh read (navigation properties সহ) করে DTO রিটার্ন করে।

- **Project example:** `AiLogisticsService.PredictDeliveryAsync` lines 105-113।

- **Follow-up:** "Persist=false path-এ navigation properties (যেমন `prediction.Shipment`) manually কেন সেট করা হয়?" → কারণ DB round-trip না হলে EF Core lazy/eager-load করতে পারবে না, তাই `ToDto()` -তে যা লাগবে তা manually attach করে দেওয়া হয়।

Q10. `DeliveryTrackingService` -এ COD (Cash on Delivery) কীভাবে হ্যান্ডেল হয়?

- **Short answer:** `IsCashOnDelivery` true হলে `CodAmount` বাধ্যতামূলক (>0), আর delivery হলে `CodCollected` flag idempotent-ভাবে সেট হয়।

- **Detailed explanation:** Create/Update দুটোতেই validation: `if (request.IsCashOnDelivery && (request.CodAmount is null or ≤ 0)) throw ConflictException`। Delivery attempt বা mark-delivered-এ `ApplyCod()` helper `!shipment.CodCollected` guard দিয়ে দুইবার true সেট হওয়া বা `CodCollectedAt` overwrite হওয়া আটকায়।

- **Project example:** `DeliveryTrackingService.cs:88-91, 597-604`।

- **Follow-up:** "COD টাকা partner-এর wallet-এ কীভাবে যায়?" → এই service scope-এ নেই, সেটা payment/wallet module-এর দায়িত্ব — এই cluster শুধু "collected হয়েছে কিনা" রেকর্ড রাখে।

Q11. `WarehouseStockController` -এর `Transfer` action-এ full-move vs. partial-split আলাদা কেন হ্যান্ডেল করা হয়?

- **Short answer:** Full move হলে একই stock-item row-এর bin reference বদলে যায় (item identity অক্ষুণ্ণ); partial move হলে নতুন split stock-item তৈরি হতে পারে (দুই জায়গায় একই SKU-এর আলাদা holding)।

- **Detailed explanation:** `isFullMove = request.Quantity == item.QuantityOnHand`। Partial হলে destination-এ matching item খোঁজে (`FindMatchingStockItemAsync`, warehouse+bin+SKU+batch দিয়ে), না পেলে clone করে নতুন row বানায়। প্রতি ক্ষেত্রেই দুইটা movement (TransferOut+TransferIn) লেখা হয়।

- **Project example:** `WarehouseStockService.TransferAsync` lines 172-275।

- **Follow-up:** "Partial transfer-এ reserved stock কেন split করা যায় না?" → কারণ reservation একটা commitment (হয়তো কোনো order-এর জন্য); bin split করলে reservation কোন bin-এর সাথে bound সেটা ambiguous হয়ে যায়, তাই শুধু available (unreserved) অংশ split-transfer করা যায়।

Q12. `LogisticsPartnersController` -এ `Verify` endpoint কী করে, আর কেন এটা SuperAdmin-only?

- **Short answer:** একজন logistics partner-এর profile verify (trust-mark) করে — শুধু platform admin এই trust decision নিতে পারবে, তাই method-level

`[Authorize(Roles = RoleNames.SuperAdmin)]` অতিরিক্ত restriction যোগ করে।

- **Detailed explanation:** Class-level auth হলো "LogisticsPartner,SuperAdmin" উভয়ই access করতে পারবে, কিন্তু `GetPaged` আর `Verify` method-এ আলাদাভাবে `[Authorize(Roles = RoleNames.SuperAdmin)]` লাগানো আছে — এটা method-level attribute যেটা class-level-এর সাথে combine হয় (AND logic আসলে না, বরং override করে narrower করে)।

- **Project example:** `LogisticsPartnersController.cs:30-34, 57-61`।

- **Follow-up:** "একজন partner নিজের profile verify করতে চাইলে কী হবে?" → 403 Forbidden পাবে, কারণ method-level attribute তাকে exclude করে দেয়।

Q13. `TraceabilityController` আর `SustainabilityController` -এর GET endpoint public কেন, কিন্তু write endpoints role-restricted?

- **Short answer:** Product provenance/sustainability info হলো marketing/trust feature — যে কেউ (এমনকি না-লগইন কাস্টমার) দেখতে পারা উচিত, কিন্তু শুধু আসল Producer/verified admin লিখতে পারবে (ভুল/fraudulent দাবি ঠেকাতে)।

- **Detailed explanation:** `GetByProduct` / `GetByProducer` কোনো `[Authorize]` attribute ছাড়াই controller-এ আছে (default anonymous-allowed, যেহেতু কোনো global fallback policy override না থাকলে)। Write path-এ `[Authorize(Roles = "Producer,SuperAdmin")]`।

- **Project example:** `TraceabilityController.cs:28-33`, `SustainabilityController.cs:34-39`।

- **Follow-up:** "Certification verify কে করতে পারে?" → শুধু SuperAdmin (`VerifyCertification`, `[Authorize(Roles = RoleNames.SuperAdmin)]`) — third-party sustainability claim-এর trust-anchor platform-নিয়ন্ত্রিত।

Q14. Services-এ AutoMapper নেই কেন, তাহলে DTO mapping কীভাবে হয়?

- **Short answer:** Manual extension methods (`ToDto()`, `ToListItemDto()`, `ToTrackingDto()`) একটা static class-এ (`LogisticsMappings.cs`) লেখা — AutoMapper package project-এ referenced থাকলেও এই module actually ব্যবহার করে না।

- **Detailed explanation:** এটা একটা known project-wide pattern/gotcha — README stale থাকতে পারে (AutoMapper mention করে) কিন্তু বাস্তব কোড manual mapping করে, performance ও explicitness-এর জন্য (compile-time-checked, no reflection overhead)।

- **Project example:** `LogisticsMappings.cs` — পুরো cluster-এর সব ToDto/ToListItemDto method এখানে।

- **Follow-up:** "Manual mapping-এর downside কী?" → Boilerplate বেশি, নতুন field যোগ হলে প্রতিটা mapping method manually আপডেট করতে হয় (compile error না হলে silently miss হতে পারে যদি target property optional/nullable হয়)।

Q15. `GlobalExceptionHandler` কীভাবে এই cluster-এর exception গুলো handle করে?

- **Short answer:** `ConflictException` → 409, `NotFoundException` → 404, `UnauthorizedAccessException` → 401, বাকি সব → 500 (RFC7807 ProblemDetails ফরম্যাটে)।

- **Detailed explanation:** Service layer কখনো HTTP status code নিয়ে চিন্তা করে না — শুধু domain exception ছোঁড়ে (business-rule violation = `ConflictException`, entity না পেলে = `NotFoundException`, cross-tenant access = `UnauthorizedAccessException`)। Middleware centrally map করে।
- **Project example:** পুরো cluster জুড়ে — যেমন `ConflictException("Cannot move a shipment from ...")`, `NotFoundException("Shipment not found.")`, `UnauthorizedAccessException("This shipment belongs to another logistics partner.")`।
- **Follow-up:** "Validation error (FluentValidation) কীভাবে আলাদা?" → সেটা `ValidationFilter` দিয়ে action-execution-এর আগেই ধরা হয় (reflection দিয়ে প্রতিটা argument-এর validator চালিয়ে), 400 রিটার্ন করে — controller/service method-এর ভেতরেই পৌঁছায় না।

Q16. `DeliveryRoutesController` -এর stop-level actions (arrive, complete, skip, fail) কেন আলাদা endpoint?

- **Short answer:** প্রতিটা RESTful action আলাদা business-semantic বহন করে (arrival timestamp vs. completion vs. skip-reason vs. failure-reason), তাই আলাদা request DTO ও আলাদা validation দরকার — একটা generic "update stop status" endpoint দিয়ে সব case cleanly capture করা কঠিন হতো।
- **Detailed explanation:** এটা Shipment-এর dedicated status endpoints (deliver/cancel) থেকে একই design philosophy — generic PUT-status এর বদলে semantic-specific POST action endpoints, যেখানে প্রতিটার নিজস্ব pre-condition (guard) থাকে।
- **Project example:** `DeliveryRoutesController.cs:102-120`।
- **Follow-up:** "Skip আর Fail-এর মধ্যে পার্থক্য কী?" → (কোডে না দেখা গেলেও যুক্তিসঙ্গত অনুমান, honest বলা উচিত): Skip সম্ভবত planned bypass (recipient unavailable বলে জানা ছিল), Fail সম্ভবত attempted কিন্তু ব্যর্থ — উভয়ই একই `FailRouteStopRequest` DTO শেয়ার করে যা কোডে দেখা গেছে, তাই মূল পার্থক্য সম্ভবত শুধু resulting `DeliveryRouteStopStatus` enum ভ্যালুতে।

Q17. Pagination pattern কীভাবে কাজ করে এই cluster-এ?

- **Short answer:** প্রতিটা "GetPaged" method-এ `query.Page / query.PageSize` normalize হয় (`Page < 1 → 1`, `PageSize` 1-100 range না হলে default 20), তারপর repository `.Skip().Take()` + `.CountAsync()` দিয়ে `PagedResult<T>` রিটার্ন করে।
- **Detailed explanation:** Normalization service-layer-এ হয়, controller বা repository-তে না — এটা defensive coding, যাতে malicious/buggy client `PageSize=1000000` না পাঠাতে পারে।
- **Project example:** সব service-এর শুরুতে repeat হওয়া প্যাটার্ন: `query.Page = query.Page < 1 ? 1 : query.Page; query.PageSize = query.PageSize is < 1 or > 100 ? 20 : query.PageSize;`
- **Follow-up:** "এটা DRY violate করছে না (একই কোড বারবার)?" → হ্যাঁ, কিছুটা — `AiLogisticsService` -এ এটা `NormalisePaging()` static helper-এ extract করা হয়েছে, কিন্তু অন্য service গুলোতে (Shipment/Return/WarehouseStock) inline duplicate — refactor করার সুযোগ আছে।

Q18. `ImpactController` -এ শুধু `[Authorize]` আছে, কোনো role restriction নেই কেন?

- **Short answer:** কারণ impact summary personal — যেকোনো logged-in user (Customer, Producer, LogisticsPartner, যেই হোক) নিজের impact দেখতে পারবে; এটা role-specific ফিচার না।
- **Detailed explanation:** `CurrentUserId` দিয়েই scope হয় (`GetMyImpactAsync(CurrentUserId, ...)`), কোনো role-check দরকার নেই কারণ প্রতিটা user-ই "নিজের" ডেটা চাইছে।
- **Project example:** `ImpactController.cs:12-31`।
- **Follow-up:** "SuperAdmin কি অন্য কারো impact দেখতে পারে?" → এই endpoint দিয়ে না (`mine` route, শুধু self), সেটার জন্য আলাদা admin-facing endpoint লাগবে যেটা এই controller-এ নেই।

Q19. `WarehousesController` -এ zone/bin add/update/remove কেন warehouse-এর sub-resource হিসেবে nested route?

- **Short answer:** RESTful hierarchy বোঝাতে — Zone/Bin স্বাধীনভাবে exist করে না, সবসময় একটা Warehouse-এর owned child, তাই `warehouses/{id}/zones`, `warehouses/{id}/bins` route shape।
- **Detailed explanation:** এই nested route mutation methods (`AddZoneAsync`, `UpdateZoneAsync`) parent warehouse `id` -ও নেয় যাতে service ownership+parent-child consistency (zone/bin সঠিক warehouse-এর কিনা) verify করতে পারে।
- **Project example:** `WarehousesController.cs:59-87`।
- **Follow-up:** "Bin কোনো Zone-এর ভেতরে বাধ্যতামূলক কিনা?" → Route structure অনুযায়ী bin সরাসরি warehouse-এর sub-resource (`warehouses/{id}/bins`), zone-এর sub-resource না — সম্ভবত bin entity-তে একটা optional `WarehouseZoneId` FK আছে (entity file তালিকায় `WarehouseBin.cs` আলাদা দেখা গেছে যদিও এই research-এ পুরো entity content পড়া হয়নি, honestly এটা assumption হিসেবে বলা উচিত)।

Q20. `DeliveryPredictionDto / RouteOptimizationRunDto` -এর Confidence field আসলে কী represent করে?

- **Short answer:** একটা coarse bucket (High/Moderate/Low, enum `AiLogisticsConfidence`) যা input data-এর পরিমাণ/গুণমানের উপর ভিত্তি করে heuristically সেট হয় — কোনো statistical confidence-interval বা model-probability না।
- **Detailed explanation:** Route optimization-এ `located.Count ≥ 8 → High`, `≥ 4 → Moderate`, `else Low` — শুধু কতগুলো stop-এর geo-coordinate ছিল তার count-based bucket, কোনো error-margin calculation না।
- **Project example:** `RuleBasedAiRouteOptimizationProvider.cs:70-75`।
- **Follow-up:** "এটা misleading হতে পারে ইউজারদের কাছে?" → হ্যাঁ সম্ভাব্য — UI যদি "AI confidence: High" দেখায় তাহলে ইউজার ভাবতে পারে এটা statistical rigor বোঝায়, যেখানে বাস্তবে এটা শুধু "কতগুলো data point ছিল" গণনা। Honest engineering হলে UI-তে label স্পষ্ট করা উচিত (যেমন "based on N located stops")।

5. Tricky/Gotcha Questions (6)

G1. একটা quantity কলাম-এর বদলে append-only movement ledger কেন — এতে extra storage/complexity-র justification কী?

Append-only design deliberate — প্রতিটা movement (`Inbound`, `Outbound`, `TransferIn/Out`, `Adjustment`, `Damage`, `Disposal`, `Reserve`, `ReleaseReservation`) স্বতন্ত্র row হিসেবে থাকায় (ক) audit/compliance (কে, কবে, কেন quantity বদলানো), (খ) dispute resolution (customer complain করলে exact movement history দেখানো যায়), (গ) `ReferenceType / ReferenceId` দিয়ে resolution module (order, return) এর সাথে traceability সম্ভব হয়। Trade-off: বেশি row লেখা হয়, query-এর জন্য current-state আলাদা রাখতে হয় (denormalization) — যেটা `WarehouseStockItem` -এর দায়িত্ব।

G2. Stock mutation-এ কোনো optimistic concurrency (RowVersion) নেই কেন — এটা একটা real risk?

হ্যাঁ, এটা একটা genuine gap। `WarehouseStockItem` entity-তে কোনো `Timestamp / RowVersion` কলাম নেই (grep confirm করেছে)। দুইজন ওয়ারহাউজ ইউজার একই সময়ে একই item-এর উপর `Issue` করলে — উভয় request একই in-memory `item.QuantityAvailable` দেখবে, উভয়ই guard পাস করবে, শেষে EF Core "last write wins" আচরণ করবে, ফলে actual

over-issue হয়ে যেতে পারে (২ জন মিলিয়ে available-এর চেয়ে বেশি issue হয়ে গেলেও কোনোটাই ব্যর্থ হবে না)। এটা fix করতে দরকার: (i) EF Core `[ConcurrencyCheck] / RowVersion` টোকেন, অথবা (ii) DB-level row lock (`SELECT ... FOR UPDATE` সমতুল্য, PostgreSQL-এ), অথবা (iii) atomic SQL `UPDATE ... WHERE QuantityAvailable ≥ @qty` pattern।

G3. Public tracking endpoint (`track/{trackingNumber}`) সত্যিই কতটা secure?

Design intentional (courier industry standard), কিন্তু কয়েকটা layered concern: (ক) tracking number format predictable (`SHP-yyyyMM-####` , মাত্র ১,০০,০০০ combinations/month) — rate-limiting ছাড়া brute-force enumerable; (খ) response থেকে PII বাদ দেওয়া (recipient phone/address/COD/signature) সঠিক approach, কিন্তু `DestinationDistrictName` এখনো leak হয় — combined with public knowledge (যেমন কেউ জানে কার parcel এটা), সেটা কিছুটা inference risk তৈরি করতে পারে; (গ) এই endpoint-এ কোনো explicit rate-limit attribute দেখা যায়নি controller-এ, তাই project-wide global rate-limiter policy-র উপর নির্ভর করে (যা memory অনুযায়ী অন্য module-এ `AddRateLimiter` দিয়ে যোগ হয়েছে, কিন্তু এই controller specifically সেটা override/apply করে কিনা যাচাই করা হয়নি এই research-এ)।

G4. Route optimization-এর "AI" নামকরণ কি misleading marketing, নাকি legitimate architecture decision?

উভয় দিক থেকে দেখা যায়: Engineering দিক থেকে এটা একদম legitimate — `IAiRouteOptimizationProvider` interface একটা clean abstraction, আজকে rule-based (2-opt heuristic) কিন্তু আগামীকাল real solver (OR-Tools, external ML API) দিয়ে drop-in replace করা যাবে কোনো controller/service code না বদলিয়ে। কিন্তু user-facing দিক থেকে "AI" label একটু misleading হতে পারে যদি end-user statistical rigor আশা করে — বাস্তবে এটা deterministic, well-understood operations-research algorithm (NN + 2-opt), কোনো learning/training/model weight নেই। Viva-তে honest answer: "এটা algorithmic optimization, machine-learning না — কিন্তু naming এটাকে future-proof রাখার জন্য 'AI Logistics' module হিসেবে ধরা হয়েছে।"

G5. `SaveChangesAsync()` -ই কি সত্যিকারের "transaction" — নাকি explicit `BeginTransaction` দরকার ছিল?

`DbContext.SaveChangesAsync()` নিজেই — EF Core internally একটা implicit database transaction তৈরি করে সব pending `INSERT / UPDATE / DELETE` একসাথে commit করে (single call-এ multiple entity change থাকলেও)। তাই `WarehouseStockService.ReceiveAsync` -এ item update + bin update + warehouse update + নতুন movement row — সবগুলো একই `SaveChangesAsync()` -এ atomic। Explicit `BeginTransactionAsync()` / `CommitAsync()` তখনই দরকার হতো যদি একাধিক `SaveChangesAsync()` call-কে একটাই logical unit-এ wrap করতে হতো (যেমন cross-service call, বা কোনো external API call মাঝে থাকলে) — এই cluster-এ প্রতিটা service method single `SaveChangesAsync()` দিয়ে শেষ হয়, তাই implicit transaction যথেষ্ট।

G6. Return-এর `Restock` status কি misleading — actual stock না বাড়লেও status বলছে "Restocked"?

হ্যাঁ, এটা data-integrity-এর দৃষ্টিকোণ থেকে একটা সত্যিকারের সমস্যা। `ReturnRequest.Status = Restocked` হয়ে গেলে downstream কোনো report/dashboard যদি ধরে নেয় "warehouse-এ stock ব্যাক এসেছে", সেটা ভুল হবে — কারণ কোনো `WarehouseStockMovement` তৈরি হয়নি, `WarehouseStockItem.QuantityOnHand` বাড়েনি। Developer comment-এ এটা honestly acknowledged আছে ("a later integration item"), তাই viva-তে এটা "known limitation, not a bug I need to hide" হিসেবে confidently বলা উচিত — বরং এটা বোঝানো যে system design-এর boundary কোথায় আর কেন সেই gap আছে সেটা explain করাই ভালো answer।

Research grounded entirely in actual repository files under

`E:\Silpohub_bd\ShilpoHubBD\backend\src\ShilpoHubBD.Api\Controllers ,
ShilpoHubBD.Application\Services\Logistics , ShilpoHubBD.Domain\Entities\Logistics , and
ShilpoHubBD.Infrastructure\AILogistics` . No code was invented; line numbers refer to the files as read during this session.

Cluster: Business Partner & Research/Innovation — Viva Prep

Covers 23 controllers in `ShilpoHubBD.Api/Controllers` : `BusinessPartnersController`, `BusinessPartnerAnalyticsController`, `ContractsController`, `ProcurementsController`, `SupplierDiscoveryController`, `SupplierMatchingController`, `ManufacturingPartnershipsController`, `ProductDevelopmentController`, `InvestmentOpportunitiesController`, `ResearchProjectsController`, `ResearchPapersController`, `ResearchNotesController`, `ResearchMilestonesController`, `ResearchTasksController`, `ResearchPublicationsController`, `ResearchAiAssistantController`, `InnovationExperimentsController`, `InnovationPrototypesController`, `DesignCollaborationsController`, `CSRSponsorshipController`, `PublicationRepositoryController`, `AlBusinessAssistantController`, `AlIntelligenceController`. All 23 confirmed present by exact filename via Glob — no substitutions needed.

1. Cluster Overview

Ei cluster-ta ShilpoHubBD-er duita boro domain-ke ekshathe dhore rakhe: **B2B commerce** (BusinessPartner, Contracts, Procurement, Supplier Discovery/Matching, Manufacturing Partnership, Product Development, Investment, CSR Sponsorship, Design Collaboration) ebong **Research & Innovation** (Research Projects + Papers/Notes/Milestones/Tasks/Publications/AI Assistant, Innovation Experiments/Prototypes). Dutoi cluster-e ek common architectural pattern dekha jai: ekta "parent" entity (Contract, ProcurementRequest, Partnership, DevelopmentProject, InvestmentOpportunity, ResearchProject, InnovationExperiment/Prototype) thake, ar tar charidike child collections (Items, StatusHistory/StatusEvents, Documents, Milestones, Comments) build hoy — sob-i EF Core-er `ICollection<T>` navigation property hishebe.

Typical B2B request flow: ekjon `BusinessPartner` (role) prothome `SupplierDiscoveryController` -e producers search kore, ba `SupplierMatchingController` -e specific criteria (category, quantity, budget, district, material, certification, max delivery days) diye weighted-score match pay. Match-e best producer beechhe, তখন `ProcurementsController` diye ekta procurement request create kore (ba quotation theke convert kore), producer segula approve/reject kore, approved hole `ConvertToOrder` diye actual marketplace Order-e convert hoy. Longer-term relationship hole `ContractsController` diye formal contract banano hoy (items + delivery schedule shoho), producer accept/reject kore, active thakle delivery schedule update hoy, mead sesh hole `Renew` kora jay (notun Contract row create hoy, `PreviousContractId` diye chain track hoy — mutate na kore audit trail rakha hoy). Deeper collaboration hole `ManufacturingPartnershipsController` (long-term manufacturing deal + milestones), `ProductDevelopmentController` (new product design->prototype->convert-to-product pipeline), `DesignCollaborationsController` (design files + revision decision loop), `InvestmentOpportunitiesController` (producer investment ask -> BusinessPartner proposal -> milestones/documents), ebong `CSRSponsorshipController` (producer CSR opportunity -> BusinessPartner sponsorship proposal -> impact records) — egulo shob-i ekত "propose -> respond/accept-reject -> milestone tracking -> complete/cancel" state-machine pattern follow kore, shudhu domain vocabulary bhinno.

Research/Innovation side-e center-e ache `ResearchProject` (HeritageInnovationHub ba GovernmentNGO role create kore), jar charidike hang kore: `ResearchMilestone`, `ResearchTask` (Kanban-style priority/status), `ResearchNote` (visibility-controlled), `ResearchPaper` (draft->submitted->published lifecycle), `ResearchPublication` (cross-project public repository — `PublicationRepositoryController` diye browse kora jay), ebong `ResearchAiAssistantController` diye project-scoped AI analyses (insights/trends/correlations/report/citations — shobtai `ResearchAIAnalysis` entity hishebe persist hoy, history-shoho). `InnovationExperimentsController` ar `InnovationPrototypesController` alada — egulo ML/product experimentation track kore (versions, training runs, iterations, test cases, test runs, issues) — research project-er shathe directly linked na, kintu shei same "HeritageInnovationHub/GovernmentNGO researcher" role diye protected. Cross-cutting: `BusinessPartnerAnalyticsController` shob domain-er upor dashboard-style aggregation dey (market demand, spending, supplier performance), ar duita "AI"-named controller (`AlBusinessAssistantController` for Producer, `AlIntelligenceController` for BusinessPartner) actually rule-based heuristic engines — eta niche Section 5-e proof shoho dekhano hoyeche.

2. Deep Dive — 4 Representative Controllers

2.1 `AlIntelligenceController` — "AI" that is really weighted arithmetic

File: `ShilpoHubBD.Api/Controllers/AlIntelligenceController.cs`

Verb	Route	Auth/Roles	Purpose
POST	<code>api/ai-intelligence/supplier-ranking</code>	BusinessPartner, SuperAdmin	Rank candidate producers
POST	<code>api/ai-intelligence/quality-prediction</code>	BusinessPartner, SuperAdmin	Predict a producer's quality score
POST	<code>api/ai-intelligence/price-forecast</code>	BusinessPartner, SuperAdmin	Forecast category price trend
POST	<code>api/ai-intelligence/delivery-prediction</code>	BusinessPartner, SuperAdmin	Predict delivery days
POST	<code>api/ai-intelligence/risk-assessment</code>	BusinessPartner, SuperAdmin	Score supplier risk

Controller thin — DI-te `IBusinessPartnerAIService` (`_businessPartnerAIService`) inject kora, class-level `[Authorize(Roles = $"{RoleNames.BusinessPartner},{RoleNames.SuperAdmin}")]` . Prottekta method-e ekত pattern: request DTO nao, service-ke call koro, `Ok(result)` return koro — kono business logic controller-e nai.

Real work hoy `DummyBusinessPartnerAIProvider` -e (`ShilpoHubBD.Infrastructure/AlBusinessPartner/DummyBusinessPartnerAIProvider.cs`), jeta `IAIBusinessPartnerProvider` interface implement kore ebong `BusinessPartnerAIService` -er madhome call hoy. Class comment nijei shoccho likhe rekheche:

```
// Rule-based stand-in for a future AI/ML backend (Gemini, OpenAI, custom model, etc). Every method
// derives its answer from simple heuristics over the context it is given -- no external calls, no
// model weights. Swap this out for a real implementation of IAIBusinessPartnerProvider later;
// nothing in BusinessPartnerAIService or the controller needs to change.
```

RankSuppliersAsync — weighted sum diye score:

```
var ratingScore = Math.Min(100m, c.AverageRating / 5m * 100m);
var certificationScore = Math.Min(100m, c.CertificationCount * 25m);
var capacityScore = Math.Min(100m, c.EstimatedProductionCapacity / 10m);
var verifiedBonus = c.IsHandmadeVerified ? 100m : 0m;

var rankScore = Math.Round(
    (ratingScore * 0.4m) + (certificationScore * 0.2m) + (capacityScore * 0.2m) + (verifiedBonus * 0.1m) +
    (Math.Min(1m, c.ReviewCount / 20m) * 100m * 0.1m), 1);
```

Weight distribution: rating 40%, certification 20%, capacity 20%, handmade-verified 10%, review-volume 10% — mote 100%. `confidence` alada formula diye calculate hoy (base 0.3 + review-density bonus + certification bonus, capped at 1.0).

AssessRiskAsync — additive penalty model, high-risk factor-gulo `riskScore`-e jog hoy (rating<3.5 -> +25, cancellation rate>10% -> up to +25, quotation rejection rate>30% -> up to +20, procurement cancellation>10% -> up to +15, no verified certification -> +10, no history at all -> +15), তাঁরপৰ `Math.Min(100m, ...)` diye cap kora hoy, threshold diye Low/Medium/High tier deya hoy.

PredictDeliveryAsync — history na thakle generic 14-day fallback; thakle historical average nao, ebong requested quantity capacity-r beshi hole `Math.Ceiling(quantity/capacity)` cycles diye multiply kore — eta ekta simple production-cycle heuristic, kono ML model na.

Banglish summary: eita "AI" bola hoyeche marketing-er jonno, kintu code-e kono external API call nai, kono model weight load hoy na — shudhu `if/switch` ebong weighted arithmetic. DI registration-e eta proven (Section 5-e dekhano hocche).

2.2 `ResearchAiAssistantController` — persisted, project-scoped "AI" analyses

File: `ShilpoHubBD.Api/Controllers/ResearchAiAssistantController.cs`

Route prefix: `api/research/projects/{projectId:guid}/ai`, class-level `[Authorize]` (kono specific role na — service layer-e project membership check hoy).

Verb	Route	Purpose
POST	<code>... /ai/insights</code>	Descriptive-stats insight generate
POST	<code>... /ai/trends</code>	Time-series trend discovery (slope)
POST	<code>... /ai/correlations</code>	Pairwise Pearson correlation
POST	<code>... /ai/report</code>	5-section draft report assemble
POST	<code>... /ai/citations</code>	APA/MLA/Chicago/IEEE/BibTeX citation formatting
GET	<code>... /ai/analyses</code>	Paged list of past analyses (persisted!)
GET	<code>... /ai/analyses/{analysisId}</code>	Single analysis detail
DELETE	<code>... /ai/analyses/{analysisId}</code>	Delete an analysis

Interesting part: eta shudhu ekta stateless calculator na — result `ResearchAIAnalysis` entity hishebe DB-te save hoy (`IResearchAIAnalysisRepository`), tai history GET/DELETE endpoints ache — ei point-ta onno AI controller-gulo theke ekta design difference.

Provider: `DummyResearchAIProvider` (`ShilpoHubBD.Infrastructure/ResearchAI/DummyResearchAIProvider.cs`), `IResearchAIProvider` implement kore, `public const string Name = "dummy-rule-based-v1";` — nijei explicitly bole dicche eta "dummy" ebong "rule-based".

GenerateInsightsAsync — mean/min/max/stddev calculate kore selected numeric data theke:

```
var mean = allValues.Average();
var stdev = StandardDeviation(allValues, mean);
...
if (spread > 0 && stdev / (Math.Abs(mean) < 1e-9 ? 1 : Math.Abs(mean)) > 0.5)
{
    // "High variability detected" - coefficient of variation > 0.5
}
```

DetectCorrelationsAsync — real Pearson correlation coefficient formula manually implement kora:

```
private static double Pearson(IReadOnlyList<double> a, IReadOnlyList<double> b)
{
    ...
    double cov = 0, varA = 0, varB = 0;
    for (var i = 0; i < n; i++)
    {
        var da = a[i] - meanA;
        var db = b[i] - meanB;
        cov += da * db; varA += da * da; varB += db * db;
    }
    var denom = Math.Sqrt(varA * varB);
    return denom < 1e-12 ? double.NaN : cov / denom;
}
```

Eita statistics — legitimate math — kintu machine learning na, model training na.

GenerateCitationsAsync — pure string-formatting switch-expression per style (APA/MLA/Chicago/IEEE/BibTeX), kono NLP/LLM involve nai.

Banglish: Research AI Assistant-er naam "AI" holeo, eta আসলে একটা **rule-based statistics engine** — descriptive stats, least-squares slope, Pearson correlation, ar template-based report/citation generation. Comment-e nijei bole: *"These results come from a deterministic rule-based provider, not a trained model."*

2.3 **SupplierMatchingController** — weighted-criteria matching engine

File: `ShilpoHubBD.Api/Controllers/SupplierMatchingController.cs`

Verb	Route	Auth/Roles	Purpose
POST	<code>api/supplier-matching/match</code>	BusinessPartner, SuperAdmin	Score & rank producers against buyer criteria

Ekmatro endpoint, ekdom thin:

```
[HttpPost("match")]
public async Task<ActionResult<List<SupplierMatchResultDto>>> Match(SupplierMatchRequest request, CancellationToken cancellationToken)
{
    var result = await _supplierMatchingService.MatchAsync(request, cancellationToken);
    return Ok(result);
}
```

SupplierMatchingService.MatchAsync (`ShilpoHubBD.Application/Services/SupplierMatching/SupplierMatchingService.cs`) — class comment: *"Rule-based (not AI) matching engine"* — eta ekmatro jaiga jekhane code-e explicitly "not AI" lekha ache!

Weight constants:

```
private const decimal RatingWeight = 10m;
private const decimal CategoryWeight = 15m;
private const decimal KeywordWeight = 15m;
private const decimal QuantityWeight = 15m;
private const decimal BudgetWeight = 15m;
private const decimal LocationWeight = 10m;
private const decimal MaterialWeight = 10m;
private const decimal CertificationWeight = 10m;
private const decimal DeliveryWeight = 10m;
```

Core algorithm — **adaptive-denominator weighted scoring**: `RatingWeight` always counted, but baki shob criteria (`CategoryWeight`, `KeywordWeight`, etc.) shudhu tokhoni `totalWeight` -e jog hoy jokhon caller shei filter-ta specify koreche (`request.CategoryId.HasValue`, etc.):

```
if (request.Quantity.HasValue && request.Quantity.Value > 0)
{
    totalWeight += QuantityWeight;
    var fraction = Math.Min(1m, (decimal)candidate.EstimatedProductionCapacity / request.Quantity.Value);
    achievedWeight += QuantityWeight * fraction;
    ...
}
...
var score = totalWeight == 0m ? 0m : Math.Round(achievedWeight / totalWeight * 100m, 1);
```

Eta guruttopurno design decision — comment-e explain kora ache: “scores each producer against the criteria the caller actually specified, so an unspecified filter neither helps nor hurts a candidate’s score.” Mane, jodi user shudhu category diye search kore (quantity/budget na diye), tahole score $\frac{\text{rating} + \text{category-r}}{\text{rating} + \text{category-r} + \text{budget}}$ upor base kore, onno criteria-r absence-e kono penalty na — normalized percentage (0-100) always fair thake.

Partial-credit nuance (delivery days-e): jodi candidate-er delivery history achenaa (no track record), tahole neutral 50% credit; jodi thake but requirement-er cheye slow, tahole 25% partial credit — full zero na:

```
else
{
    // Has a track record, just slower than requested -- partial credit rather than zero.
    achievedWeight += DeliveryWeight * 0.25m;
}
```

Result `MatchScore` descending, tie-break `AverageRating` descending, `Take(request.MaxResults)` diye limit.

2.4 `ContractsController` — full contract lifecycle + renewal chain

File: `ShilpoHubBD.Api/Controllers/ContractsController.cs`

Route prefix: `api/contracts`, class-level `[Authorize]`, per-action role restriction:

Verb	Route	Roles	Purpose
POST	<code>api/contracts</code>	BusinessPartner, SuperAdmin	Create/propose contract
GET	<code>api/contracts</code>	BusinessPartner, SuperAdmin	List own (or all if admin) contracts
GET	<code>api/contracts/received</code>	Producer	List contracts sent to this producer
GET	<code>api/contracts/{id}</code>	BusinessPartner, Producer, SuperAdmin	Get one contract (party-only)
POST	<code>api/contracts/{id}/accept</code>	Producer	Accept pending contract
POST	<code>api/contracts/{id}/reject</code>	Producer	Reject pending contract
POST	<code>api/contracts/{id}/terminate</code>	BusinessPartner, Producer, SuperAdmin	Terminate active contract
POST	<code>api/contracts/{id}/renew</code>	BusinessPartner, SuperAdmin	Renew active/expired contract
POST	<code>api/contracts/{id}/documents</code>	BusinessPartner, Producer, SuperAdmin	Attach a document
POST	<code>api/contracts/{id}/delivery-schedule/{scheduleId}/status</code>	BusinessPartner, Producer, SuperAdmin	Update one delivery entry’s status

Status machine (`ContractStatus` enum): `PendingApproval(0) → Active(1) / Rejected(2)`, `Active → Terminated(3) / Expired(4)`. Prottek transition-e `ConflictException` guard ache — e.g. `AcceptAsync`:

```
if (Contract.Status != ContractStatus.PendingApproval)
{
    throw new ConflictException("Only a pending contract can be accepted.");
}
```

Renewal design — `Contract` entity nijei self-referencing:

```
// Self-reference: when a contract is renewed, a new Contract row is created linking back here
// rather than mutating dates in place, so the full renewal chain stays auditable.
public Guid? PreviousContractId { get; set; }
public Contract? PreviousContract { get; set; }
```

`RenewAsync` -e notun `Contract` row banano hoy (`Id = Guid.NewGuid()`), items either request-e deya othoba previous contract theke copy hoy, `PreviousContractId = previous.Id` set kore renewal-chain audit-safe rakha hoy — status abar `PendingApproval` (producer-ke abar accept korte hobe).

Authorization pattern — `CurrentUserId` computed property (JWT `sub` claim theke), `IsAdmin` computed property (`User.IsInRole(RoleNames.SuperAdmin)`). Service layer-e “party check” hoy — `GetPartyAsync` helper:

```
private async Task<Contract> GetPartyAsync(Guid id, Guid currentUserId, bool isAdmin, CancellationToken cancellationToken)
{
    var contract = await _contractRepository.GetByIdWithDetailsAsync(id, cancellationToken)
        ?? throw new NotFoundException("Contract not found.");
    if (!isAdmin && contract.BusinessPartnerId != currentUserId && contract.ProducerId != currentUserId)
    {
        throw new UnauthorizedAccessException("You do not have permission to access this contract.");
    }
    return contract;
}
```

Eta double-layer authorization dekhay: controller-level `[Authorize(Roles= ... )]` role check kore, service-level `GetPartyAsync` ownership check kore (data-level authorization) — role thakleo onno party-r contract dekha jabe na.

Data-integrity note (important gotcha): `ContractItem.ProductId → Product` FK-e `ContractItemConfiguration.cs` -e explicit `OnDelete(DeleteBehavior.Restrict)` — mane keu chaile chaileo ekta `Product` delete korte parbe na jodi kono `ContractItem` shei product-ke reference kore. Eta database-level integrity guard, EF exception throw korbe delete attempt-e (details Section 5-e).

3. Reference Table — Remaining Controllers

Controller	Route prefix	Key endpoints	Auth/Roles
BusinessPartnersController	api/business-partners	GET (paged, admin), GET {userId} , PUT {userId} (upsert), POST {userId}/verify , DELETE {userId}	Authorize (varies p
BusinessPartnerAnalyticsController	api/business-partner-analytics	GET market-demand, export-trends, production-forecast, industry-insights, procurement, supplier-performance, spending, order-trends	BusinessPartner, Si
ProcurementsController	api/procurements	POST, POST from-quotation/{id}, GET, GET {id} , POST {id}/approve , /reject , /convert-to-order , /cancel	BusinessPartner, Si
SupplierDiscoveryController	api/supplier-discovery	GET search, GET producers/{producerId}	BusinessPartner, Si
ManufacturingPartnershipsController	api/manufacturing-partnerships	POST, GET, GET received, GET {id} , POST respond, milestones (+status), complete, cancel	Authorize; create= respond=Produce
ProductDevelopmentController	api/product-development	POST, GET, GET received, GET {id} , respond, comments, milestones(+status), prototypes(+decision), convert-to-product, cancel	Authorize; per-acti
InvestmentOpportunitiesController	api/investment-opportunities	POST (Producer), GET, GET mine, GET {id} , close, cancel, proposals (CRUD+decision+milestones+documents+complete/cancel)	Authorize; Produci submits proposals
ResearchProjectsController	api/research/projects	GET, GET {id} , POST (HeritageInnovationHub/GovernmentNGO/SuperAdmin), PUT, PUT status, DELETE, members CRUD, GET activity	Authorize; create r
ResearchPapersController	... /projects/{projectId}/papers	GET, GET {paperId} , POST, PUT, DELETE	Authorize
ResearchNotesController	... /projects/{projectId}/notes	GET, POST, PUT {noteId} , DELETE	Authorize
ResearchMilestonesController	... /projects/{projectId}/milestones	GET, POST, PUT {milestoneId} , DELETE	Authorize
ResearchTasksController	... /projects/{projectId}/tasks	GET (paged), GET {taskId} , POST, PUT, PUT status, DELETE	Authorize
ResearchPublicationsController	... /projects/{projectId}/publications	GET, POST, PUT {publicationId} , DELETE	Authorize
PublicationRepositoryController	api/research/publications	GET (browse, paged), GET {id}	Authorize
InnovationExperimentsController	api/innovation-lab/experiments	GET, GET {id} , POST (researcher roles), PUT, DELETE, versions, runs (+update/delete)	Authorize; create=Heritageln

Controller	Route prefix	Key endpoints	Auth/Roles
InnovationPrototypesController	api/innovation-lab/prototypes	GET, GET {id}, POST, PUT, DELETE, iterations, test-cases (CRUD), test-runs (CRUD), issues (CRUD)	Authorize; create=
DesignCollaborationsController	api/design-collaborations	POST, GET, GET received, GET {id}, respond, comments, files, revisions (+decision), complete, cancel	Authorize; per-acti
CSRSponsorshipController	api/csr-sponsorship	opportunities (CRUD-ish, Producer-created), proposals (BusinessPartner submits, Producer decides), milestones, progress-updates, impact-records, complete/cancel	Authorize; per-acti
AIBusinessAssistantController	api/producer/ai-business	POST price-suggestion, description, translate, demand-forecast, production-plan, material-forecast, seasonal-prediction, sales-insights	Producer only

4. Viva Question Bank

Q1. SupplierDiscoveryController ar SupplierMatchingController -er moddhe parthokko ki?

- **Short answer:** Discovery = plain search/filter (no scoring); Matching = weighted scoring algorithm jeta ekta match percentage dey.
- **Detailed explanation:** SupplierDiscoveryController.Search shudhu SupplierSearchParameters diye filter kore paged list dey — kono score attach kore na. SupplierMatchingController.Match request-e specify kora প্রতিটি criteria-r against ekta 0-100 MatchScore calculate kore, MatchReasons explanation shoho.
- **Project example:** Buyer jodi "just browse producers in Dhamrai" korte chai, discovery use hobe; jodi "quantity 500, budget 200/unit, certification lagbe" — eirokom specific requirement thake, matching use hobe.
- **Follow-up:** Matching service-e query kothay run hoy? — ISupplierMatchingRepository.GetCandidatesAsync diye raw candidates ashe, তারপর in-memory Score() method-e calculation hoy (DB-te noy).

Q2. Ekta Contract-er ContractStatus enum-er shob states ki ki, ar prottekta transition kon role theke trigger hoy?

- **Short answer:** PendingApproval -> Active/Rejected (Producer decides), Active -> Terminated (any party) ba Expired (implicit, IsExpired computed flag), Active/Expired -> Renew (new PendingApproval contract chain-e).
- **Detailed:** ContractStatus enum-e 5 ta value ache (0-4). Accept/Reject shudhu Producer korte pare ([Authorize(Roles = RoleNames.Producer)]), karon contract offer-ta BusinessPartner theke Producer-ke pathano hoy. Terminate BusinessPartner/Producer/SuperAdmin — jekono party.
- **Project example:** ContractsController.Accept -> _contractService.AcceptAsync -> checks contract.ProducerId ≠ producerId (ownership) ar contract.Status ≠ ContractStatus.PendingApproval (state guard) -> ConflictException throw kore jodi invalid transition.
- **Follow-up:** Expired status kobe set hoy? — Code-e kono background job dekha jayni ei cluster-e explicit "mark expired" কোর্স; IsExpired DTO-te ekta computed property: contract.Status == ContractStatus.Active && contract.EndDate < DateTime.UtcNow — mane actual Status column change hoy na automatically, shudhu read-time-e flag calculate hoy. (Eta nijei ekta tricky gap — dekhun Section 5.)

Q3. Contract renewal-e notun row keno banano hoy, existing row update na kore?

- **Short answer:** Audit trail rakhar jonno — full renewal history/chain preserve korte.
- **Detailed:** RenewAsync -e notun Contract { Id = Guid.NewGuid(), PreviousContractId = previous.Id, ... } create hoy. Comment: "a new Contract row is created linking back here rather than mutating dates in place, so the full renewal chain stays auditable."
- **Project example:** 3-bar renew hole, DB-te 4 ta Contract row thakbe, linked-list style PreviousContractId diye — jeko-w row theke pura history trace kora jabe.
- **Follow-up:** Items copy na dile ki hoy? request.Items ?? previous.Items.Select(...) — jodi caller notun items na dey, previous contract-er items copy hoy.

Q4. SupplierMatchingService -er scoring algorithm-e "adaptive denominator" bolte ki bojhay?

- **Short answer:** Shudhu jei criteria caller specify koreche, shei criteria-r weight-i totalWeight -e jog hoy — unspecified filter-er weight totally baad thake, tai final percentage always fair thake.
- **Detailed:** RatingWeight (10) always counted; baki (Category 15, Keyword 15, Quantity 15, Budget 15, Location 10, Material 10, Certification 10, Delivery 10) shudhu if (request.XxxId.HasValue) { totalWeight += XxxWeight; ... } block-e conditionally jog hoy. Score = achievedWeight / totalWeight * 100.
- **Project example:** Jodi user shudhu CategoryId diye search kore, totalWeight = RatingWeight + CategoryWeight = 25, r shob criteria বাদ.
- **Follow-up:** Delivery criteria-te "partial credit" ki? — Slower-than-required delivery hole 25% credit, no-history hole 50% neutral credit — pura zero na diye graceful degrade.

Q5. Contract-er items delete korle ki Product delete kora jabe?

- **Short answer:** Na — ContractItemConfiguration -e OnDelete(DeleteBehavior.Restrict) set kora, tai kono Product delete attempt fail korbe jodi active ContractItem reference kore.
- **Detailed:** builder.HasOne(i => i.Product).WithMany().HasForeignKey(i => i.ProductId).OnDelete(DeleteBehavior.Restrict); — EF/Postgres-e ei FK-e delete restrict thake, database-level protection.
- **Project example:** Producer jodi ekta Product delete korte chai jeta kono signed contract-e item hishebe ache, EF DbUpdateException (Postgres FK violation) throw করবে.
- **Follow-up:** Global exception handler eta kivabe handle kore? — DbUpdateException specifically catch kora nai GlobalExceptionHandler -e (per confirmed pattern eta 500-e pore jabe unless kono specific mapping add kora hoy) — eta ekta improvement point interviewer highlight korte pare.

Q6. `AlBusinessAssistantController` ar `AIIntelligenceController` -er moddhe parthokko ki?

- **Short answer:** `AlBusinessAssistant` = Producer-facing tools (pricing/description/demand for own products); `AIIntelligence` = `BusinessPartner`-facing tools (ranking/predicting suppliers).
- **Detailed:** `AlBusinessAssistant` route `api/producer/ai-business`, `[Authorize(Roles = RoleNames.Producer)]`; `AIIntelligence` route `api/ai-intelligence`, `[Authorize(Roles = $"{RoleNames.BusinessPartner},{RoleNames.SuperAdmin}")]`. Duitai backend-e `Dummy*Provider` diye rule-based.
- **Project example:** Producer nijer product-er price suggest korte `AlBusinessAssistant` use korle; `BusinessPartner` ekjon supplier-ke rank/predict korte `AIIntelligence` use korle.
- **Follow-up:** *Duitai ki same provider share kore?* — Na, alada interfaces (`IAIBusinessProvider` vs `IAIBusinessPartnerProvider`), alada Dummy classes, alada namespace (`Infrastructure/AIBusiness` vs `Infrastructure/AIBusinessPartner`).

Q7. Ekjon researcher "ResearchProject" toiri korte parbe kina kivabe decide hoy?

- **Short answer:** Class constant `ProjectCreatorRoles = HeritageInnovationHub, GovernmentNGO, SuperAdmin` diye `[Authorize(Roles = ProjectCreatorRoles)]` attribute POST endpoint-e.
- **Detailed:** `ResearchProjectsController` -e GET, GetById shob-i shudhu `[Authorize]` (kono role restriction), kintu Create-e explicit role list. Update/Delete-e role attribute nai controller-e, tai service layer-e ownership/membership check hocche (thin controller pattern-er extension).
- **Project example:** Ekjon Customer role-er user shudhu GET korte parle (jodi member hoy), kintu POST korte parle na.
- **Follow-up:** *Membership check kothay hoy?* — Service-e (`IResearchProjectService`), controller-e dekha jai na, tai eta service-level authorization — thin-controller pattern-er evidence.

Q8. `ResearchAIAnalysis` result-gulo persist kora hoy keno, onno AI provider-gulor moto stateless na keno?

- **Short answer:** Research analyses history/audit hishebe rakhte hoy — researcher-der pore review korte hoy — tai GET (list+detail) ar DELETE endpoints deya ache.
- **Detailed:** `ResearchAiAssistantController` -e `GetAnalyses`, `GetAnalysis`, `DeleteAnalysis` ache — mane result database-e `ResearchAIAnalysis` entity hishebe save hoy `IResearchAIAnalysisRepository` -r madhome. `AlBusinessAssistant`/`AIIntelligence`-e eirokom persistence nai — shudhu on-demand calculate kore return kore.
- **Project example:** Ekjon researcher gotokal "trend discovery" run korle, aj `GET analyses` diye shei purano result abar dekhte pare, notun kore recalculate na kore.
- **Follow-up:** *Analysis delete korle underlying data (Notes/Papers) delete hoy?* — Na, `ResearchAIAnalysis` shudhu analysis result store kore, source data (notes/dataset) touch kore na.

Q9. Pearson correlation formula code-e kivabe implement kora hoyeche — ekta real statistical method na fake?

- **Short answer:** Real — `DummyResearchAIProvider.Pearson()` method-e standard covariance/std-dev formula manually likha.
- **Detailed:** `cov = sum(da*db)`, `varA = sum(da^2)`, `varB = sum(db^2)`, `r = cov / sqrt(varA*varB)` — eta textbook Pearson correlation coefficient, kono library call nai (manual loop diye), kintu mathematically correct.
- **Project example:** Duita numeric series (e.g. "sales" ar "rainfall") thakle system claim korle koto strong association ache — but interpret kore na causation, শুধু association.
- **Follow-up:** `double.NaN` kobe return hoy? — Jokhon `n < 2` (enough points nai) othoba `denom < 1e-12` (zero variance — constant series) tokhon `NaN`, ar caller shei item skip kore (`if (double.IsNaN(r)) continue;`).

Q10. Procurement-theke-Contract-e path ki, na duita completely separate?

- **Short answer:** Duita separate flow — Procurement chhoto/one-time purchase-er jonno (order-e convert hoy), Contract long-term recurring agreement-er jonno (delivery schedule-shoho). Kono direct code-level link nai duitar moddhe.
- **Detailed:** `ProcurementsController.ConvertToOrder` procurement-ke marketplace `Order` -e convert kore — eta `Order` domain-e jai, `Contract` domain-e na. `ContractsController.Create` completely independent — direct `CreateContractRequest` diye.
- **Project example:** Ekbar-er jonno 50 ta product kinte hole procurement use hobe; 6-mash dhore proti-mash 100 ta product supply-r deal hole Contract (with delivery schedule) use hobe.
- **Follow-up:** `CreateProcurementFromQuotationRequest` ki? — Quotation response theke directly procurement banano jai — mane supplier discovery/matching-e RFQ (quotation) system-o ache alada.

Q11. `InnovationExperimentsController` ar `ResearchProjectsController` -er moddhe relation ache kina?

- **Short answer:** Direct FK relation nai — duita alada domain, kintu same researcher roles (`HeritageInnovationHub`, `GovernmentNGO`, `SuperAdmin`) diye protected.
- **Detailed:** `InnovationExperimentsController.ResearcherRoles` constant `internal` — `InnovationPrototypesController` -o eta reuse kore (`InnovationExperimentsController.ResearcherRoles`), mane duita controller shared role-constant use kore, kintu `ResearchProject` -er shathe kono foreign key nai entity level-e.
- **Project example:** Ekta ML model experiment (`InnovationExperiment`) standalone thakle pare, kono `ResearchProject`-er shathe formally attach na kore.
- **Follow-up:** *Keno alada rakha holo?* — Likely karon `InnovationExperiments/Prototypes` ekta ML/product-engineering workflow (versions, training runs, test cases) — `ResearchProjects` ekta academic/policy research workflow (papers, publications, citations) — duitar shape onek different.

Q12. `BusinessPartnerAnalyticsController` -er shob endpoint GET keno, ar role check-e ki difference ache endpoints-er moddhe?

- **Short answer:** Shob-i read-only dashboard queries, tai GET. Class-level role `BusinessPartner`, `SuperAdmin` — kono per-action override nai, mane shob endpoint same role restriction.
- **Detailed:** `market-demand`, `export-trends`, `production-forecast`, `industry-insights` — market-wide data (kono specific user filter nai visible controller-e); `procurement`, `supplier-performance`, `spending`, `order-trends` — `CurrentUserId`, `IsAdmin` pass kore — mane user-specific data (nijer procurement/spending), admin hole shobar data.
- **Project example:** `GetSpendingAnalytics` call korle, non-admin `BusinessPartner` shudhu nijer spending dekhle, `SuperAdmin` shob business partner-der spending dekhle parle (service layer-e `isAdmin` flag diye).
- **Follow-up:** *Production forecast-e `categoryId`, `horizonMonths` default value ache keno query string-e?* — `[FromQuery] Guid categoryId`, `[FromQuery] int horizonMonths = 3` — optional parameter with sensible default, caller na dile 3-mash forecast dey.

Q13. `DesignCollaborationsController` -e revision "decision" flow ta kemon?

- **Short answer:** Producer `SubmitRevision` kore, BusinessPartner/SuperAdmin `DecideRevision` (accept/reject) kore.
- **Detailed:** `[Authorize(Roles = RoleNames.Producer)] POST {id}/revisions` — producer submit kore; `[Authorize(Roles = $"{RoleNames.BusinessPartner},{RoleNames.SuperAdmin}")] POST {id}/revisions/{revisionId}/decision` — client-side decide kore. Eta ekta approval-gate pattern jeta Contracts, Investment, CSR-e repeat hoy.
- **Project example:** Producer ekta logo design-er notun version upload korle, BusinessPartner shei version accept korle project "complete" phase-e jete pare, ba reject korle producer abar revise korle.
- **Follow-up:** `Complete` endpoint-e kono revision approved thaka mandatory kina? — Controller-level dekha jai na (service-level logic), kintu design pattern onujayi typically last accepted revision thakle-i complete kora uchit.

Q14. `InvestmentOpportunitiesController` -e Producer ar BusinessPartner-er role uita keno (onno controllers theke)?

- **Short answer:** Ekhane Producer "opportunity creator" (fund chai), BusinessPartner "investor/proposer" (fund dite chai) — onno flow-gulote (Contract, ProductDevelopment) BusinessPartner-i initiator thake.
- **Detailed:** `[Authorize(Roles = RoleNames.Producer)] POST (CreateOpportunity)` — Producer investment-er jonno "ask" post kore. `[Authorize(Roles = $"{RoleNames.BusinessPartner},{RoleNames.SuperAdmin}")] POST {id}/proposals` (SubmitProposal) — BusinessPartner shei opportunity-te proposal dey. Decision Producer-e fire jai (`DecideProposal` — Producer/SuperAdmin).
- **Project example:** Producer "Amar workshop expand korte 5 lakh taka lagbe" — opportunity post kore; BusinessPartner "Ami 3 lakh dite pari 20% equity-r bodole" — proposal submit kore; Producer accept/reject kore.
- **Follow-up:** `CSRSponsorship-o same pattern follow kore?` — Ha, ekdom same: Producer opportunity create kore, BusinessPartner proposal submit kore, Producer decide kore.

Q15. `ResearchNotesController` -e "visibility" ki bojhay?

- **Short answer:** `ResearchNoteVisibility` enum diye note-r access-scope control hoy (probably Private/Team/Public — exact values entity file-e).
- **Detailed:** `ResearchNote` entity-r shathe `ResearchNoteVisibility` enum attach — controller-level kono explicit filter dekha jai na, tai visibility enforcement service-layer-e (`IResearchNoteService.GetForProjectAsync`) hocche, jekhane current user-er access-level onujayi filter hoy.
- **Project example:** Ekjon researcher nijer personal note (draft idea) "Private" rakhle, onno project member shei note list-e dekhle na.
- **Follow-up:** `Eta backend-e query-level enforce hoy naki in-memory filter?` — Confirm korte hole `ResearchProjectRepository` / `ResearchNoteService` implementation dekhte hobe deeper — controller thekei ei detail spot kora jai na (thin controller pattern-er limitation).

Q16. `SupplierMatchingRequest` -e কেন `MaxResults` default 10?

- **Short answer:** Pagination na diye simple cap — `public int MaxResults { get; set; } = 10;` DTO-e default value.
- **Detailed:** Match endpoint paged result return kore na (`PagedResult<T>` na, shudhu `List<SupplierMatchResultDto>`), tai UI-friendly top-N result cap kora hoyeche instead of full pagination.
- **Project example:** BusinessPartner "match" click korle system top 10 best-matching producer dekha, shob thousands producer na.
- **Follow-up:** `Client 50 result chaille ki hoy?` — `request.MaxResults` override kora jai (client pathaite pare), kono upper-bound validation controller/DTO-te dekha jai nai explicitly — potential improvement point.

Q17. `ContractsController.GetById` -e keno `IsAdmin/CurrentUserId` duitai pass kora hoy service-e?

- **Short answer:** Non-admin user shudhu nijer contract dekhle parbe, admin shob dekhle parbe — double condition service-e enforce hoy.
- **Detailed:** `GetPartyAsync` helper-e: `if (!isAdmin && contract.BusinessPartnerId != currentUserId && contract.ProducerId != currentUserId) throw UnauthorizedAccessException`. Controller shudhu role check kore (`BusinessPartner`, `Producer`, `SuperAdmin` — kono authenticated user in these roles pass hoi), kintu **specific contract-er owner/party** kina — eta data-level (row-level) authorization, jeta service-e hoy.
- **Project example:** Business Partner A-r contract, Business Partner B (same role) try korle `GetById` call korle, controller-level role check pass hobe (un BusinessPartner) kintu service-e `UnauthorizedAccessException` (->401) throw hobe karon B, contract-er party na.
- **Follow-up:** `UnauthorizedAccessException-e kon HTTP status return hoy?` — Confirmed pattern onujayi `GlobalExceptionHandler` -e 401.

Q18. Contract-er "Documents" attach korle business rule ki?

- **Short answer:** Kono party (BusinessPartner/Producer/SuperAdmin) document add korte pare, kono status-restriction dekha jai na `AddDocumentAsync` -e.
- **Detailed:** `AddDocumentAsync` -e `GetPartyAsync` diye ownership check, kintu `contract.Status` -er upor kono condition nai — mane Terminated/Expired contract-e document add kora jabe (potentially odd business rule, viva-te challenge kora jete pare).
- **Project example:** Ekta terminated contract-e post-dispute evidence document upload korte pare user — eta allowed code-e kono status guard nai.
- **Follow-up:** `Eta ki intentional na bug/gap?` — Likely gap; interviewer jiggesh korle honestly bola uchit je "status guard nai, hoyto flexibility-r jonno rakha hoyeche, but audit-e eta tighten kora jete pare."

Q19. `ProcurementsController` ar `ContractsController` -er auth pattern-e ki difference?

- **Short answer:** Procurement-e class-level single role-set (`[Authorize(Roles = $"{RoleNames.BusinessPartner},{RoleNames.SuperAdmin}")]`), shob endpoint same), Contracts-e class-level `[Authorize]` (generic) + per-action specific role override.
- **Detailed:** Procurement approve/reject BusinessPartner/SuperAdmin-i kore (Producer na) — karon procurement ekta buyer-initiated internal-approval workflow. Contract accept/reject Producer kore — karon ota seller-side decision.
- **Project example:** Procurement request nijer company-r internal budget-approval-er moto (buyer-side approve), kintu Contract offer-ta seller (producer) accept/reject kore.
- **Follow-up:** `Keno eirokom design decision?` Domain semantics onujayi thik — procurement ekta internal purchase-request, contract ekta bilateral agreement.

Q20. `PublicationRepositoryController` ar `ResearchPublicationsController` -er moddhe parthokko ki, duitai `IResearchPublicationService` use kore?

- **Short answer:** Ha, duitai same service inject kore, kintu alada method call kore — `ResearchPublicationsController` project-scoped CRUD (`GetForProjectAsync`, Create, Update, Delete), `PublicationRepositoryController` cross-project read-only browse (`BrowseAsync`, `GetByIdAsync`).
- **Detailed:** `PublicationRepositoryController` -e class comment: *"Cross-project publication repository: public publications plus any belonging to projects the caller is a member of."* Mane visibility-aware cross-project search.

- **Project example:** Ekjon researcher nijer project-e "Create publication" kor'be `ResearchPublicationsController` diye, kintu onno shob public publications browse kor'be `PublicationRepositoryController` use kor'be.

- **Follow-up:** *Duitai same DTO (`ResearchPublicationDto`) return kore?* Ha — same underlying entity, different query scope.

Q21. `SupplierMatchingService` -e `KeywordWeight` ar `CategoryWeight` duitai 15 — same value keno?

- **Short answer:** Design choice — dutoi "product-level fit" signal, tai equal importance deya hoyeche; eta hardcoded magic number, kono config-driven tuning nai.

- **Detailed:** All 9 weight constants (10-15 range) hardcoded `private const decimal`. Kono admin-configurable weight system nai — production-e A/B test kor'be h'le code change lag'be.

- **Project example:** Jodi business want kore "quantity" beshi important hok certification-er cheye, developer-ke code edit kore redeploy korte hobe.

- **Follow-up:** *Eta improve kor'be kivabe kor'be?* Weight config DB/appsettings-e move kore runtime-tunable kora jete pare — legitimate improvement suggestion for viva.

Q22. `ContractDeliverySchedule` -r status update-e kono business validation ache kina (e.g., can't mark delivered before scheduled date)?

- **Short answer:** Na — `UpdateDeliveryStatusAsync` -e shudhu ownership check, `request.Status / ActualDeliveryDate` direct assign hoy, kono date-order ba sequential-status validation dekha jayni.

- **Detailed:**

```
schedule.Status = request.Status;
schedule.ActualDeliveryDate = request.ActualDeliveryDate;
schedule.Notes = ...;
```

Kono check nai je `Status` valid transition (e.g. Pending->Delivered direct jump allowed, ba future `ActualDeliveryDate` allowed kina).

- **Project example:** User accidentally `ActualDeliveryDate` future-e set kor'le system block kor'be na.

- **Follow-up:** *Eta validator diye fix kora jete pare?* Ha — `UpdateDeliveryStatusRequest` -er jonno `FluentValidation` validator add kore ba service-e business-rule check add kore.

5. Tricky/Gotcha Questions

G1. "Apnar project-e onek 'AI' controller ache — eta ki real AI/ML, naki shudhu if/else rule?"

- **Answer strategy:** Honestly bolben eta **rule-based heuristic**, real AI/ML na. Proof — DI registration-e (`ShilpoHubBD.Infrastructure/DependencyInjection.cs`):

```
services.AddScoped<IAIBusinessProvider, DummyAIBusinessProvider>();
services.AddScoped<IAIBusinessPartnerProvider, DummyBusinessPartnerAIProvider>();
services.AddScoped<IResearchAIProvider, DummyResearchAIProvider>();
```

Provider class-er naam nijei `Dummy*Provider`. Kono `HttpClient` `OpenAI/Anthropic/Gemini`-e call kore emon code nai ei classes-e — grep kore confirm kora, `Infrastructure/AIBusiness`, `Infrastructure/AIBusinessPartner`, `Infrastructure/ResearchAI` — tin'ni folder-e kono external API key/HttpClient registration nai. Prottek provider-er class-comment-e explicitly lekha: "*Rule-based stand-in for a future AI/ML backend ... no external calls, no model weights.*" `SupplierMatchingService` -o comment-e likhe: "*Rule-based (not AI) matching engine.*"

G2. "Matching/scoring algorithm-ta ekta weighted sum? Formula dekh'be."

- **Answer:** Ha, weighted-sum-of-fractions, normalized by achieved/total. `SupplierMatchingService.Score()`:

```
score = (achievedWeight / totalWeight) * 100
```

jekhane `achievedWeight` proti criterion-e `Weight * matchFraction` (e.g. quantity: `QuantityWeight * min(1, capacity/requestedQty)`). `totalWeight` dynamically build hoy — shudhu specified filter-r weight jog hoy (adaptive denominator — Q4 dekhun).

`AIIntelligenceController` -er `RankSuppliersAsync` -e onno formula: fixed weight percentages (rating 40%, cert 20%, capacity 20%, verified 10%, review-volume 10%) always applied, jekhane input na thakle 0 score dey shei component-e (na je excluded hoy) — eta `SupplierMatchingService` -er approach theke different — dhukhono duita alada algorithm, same "weighted scoring" concept.

G3. "Contract terminate hoye gele ba linked Product delete hoye gele, `ContractItem/Contract`-er data integrity ki thik thake?"

- **Answer:** Product delete case-e — `DeleteBehavior.Restrict` FK (`ContractItemConfiguration.cs`) thakar कारणे, database-level-e Product delete fail kor'be jodi kono `ContractItem` shei product reference kore — orphan data toiri hobe na, EF/Postgres FK violation exception throw kor'be. Kintu **code-level-e ei exception-er jonno specific handling dekha jai nai** (`GlobalExceptionHandler` shudhu

`ValidationException/ConflictException/NotFoundException/UnauthorizedAccessException` specifically map kore, baki shob 500-e pore — `DbUpdateException` explicitly handle na hole generic 500 `ProblemDetails` return kor'be, user-friendly message na diye). Eta ekta real gap — interviewer emphasize kor'be "yes, FK restrict database-level protect kore, but the API-level error message is not friendly — that's a genuine improvement area" bolben.

- Order/Producer delete related question hole similarly, `Contract.ProducerId → User` FK-r delete-behavior EF configuration file dekhe answer deben (ei research-e explicitly check kora hoyini — honestly bolben "let me check `ContractConfiguration.cs`" if asked live).

G4. "IsExpired flag database column na keno? Kivabe set hoy?"

- **Answer:** `ContractDto.IsExpired` ekta **computed property** — persist kora hoy na Database-e:

```
IsExpired = contract.Status == ContractStatus.Active && contract.EndDate < DateTime.UtcNow,
```

Mane `ContractStatus.Expired` enum value **থাকলেও**, code-e kono jaigay explicitly `Status = ContractStatus.Expired` set hote dekha jai nai visible controllers/services-e — **ওই RenewAsync** -e check hoy `previous.Status is not (ContractStatus.Active or ContractStatus.Expired)`. Mane হয়তো kono background job (scheduled task) ache jeta expired contracts-ke actual `Status = Expired` set kore, kintu eta ei cluster-er visible code-e paoya jai nai — eta live viva-te "I did not find an explicit background job for this in the files I reviewed — it may rely on the computed `IsExpired` flag at read-time, or there may be a scheduler elsewhere in the codebase I haven't checked" — erokom honest answer deya uchit.

G5. "SupplierMatching-e weight-gulo (RatingWeight=10, CategoryWeight=15, etc.) kothā theke ashche — configurable, na hardcoded?"

- **Answer:** Pura hardcoded `private const decimal` fields `SupplierMatchingService` class-er top-e. Kono `appsettings.json`, kono DB-driven config, kono admin-panel weight-tuning UI-r shathe connect kora nai. Deploy-time-e fixed, change kor^{তে} code-edit + redeploy lag^{বে}.

G6. "Research AI Assistant-er 'correlation' feature ki proves causation?"

- **Answer:** Na — code-e explicit caveat ache: *"This is an association only; it does not establish causation and is sensitive to outliers at this sample size."* — `DummyResearchAIProvider.DetectCorrelationsAsync` -e prottek correlation finding-r `Detail` text-e ei warning hardcoded thake. Statistically thik statement, ML/AI na hoyeও proper scientific caveat rakha hoyeche — eta positive design point hishebe mention kora jete pare viva-te.

End of cluster document. Total controllers covered: 23/23 (all confirmed present via exact filename match).

Part 9 — Master Question Bank (Core/Cross-Cutting)

এই অংশে core architecture, C#, ASP.NET Core, Async, EF Core, Auth নিয়ে সাধারণ/cross-cutting প্রশ্ন। Module-specific প্রশ্ন (Marketplace, Academy, Heritage, Government, Logistics, Business Partner) প্রতিটা নিজের Part 8-এর sub-section এ আছে।

Q1. তোমার প্রজেক্ট কী ধরনের architecture ব্যবহার করে, আর কেন?

- Short answer:** Clean/Layered Architecture — 5 layer: Domain, Application, Data, Infrastructure, Api — dependency সবসময় ভেতরের দিকে (Domain-এর দিকে) point করে।
- Detailed:** Domain layer-এ কোনো external dependency নেই (কোনো `ProjectReference` নেই তার `.csproj` -এ)। Application শুধু Domain-এর উপর নির্ভরশীল। Data/Infrastructure Domain+Application-এর উপর নির্ভরশীল। Api সবার উপরে (composition root)। এটাই Dependency Inversion Principle বাস্তবায়ন — high-level policy (business rule) low-level detail (database, framework) থেকে independent।
- Project example:** `ShilpoHubBD.Domain.csproj` -এ কোনো `<ProjectReference>` নেই; `IProductRepository` interface টা Application-এ কিন্তু implementation `ProductRepository` Data-তে।
- Follow-up:** "যদি তুমি Postgres থেকে MongoDB-তে সরতে চাইতে, কতটুকু কোড বদলাতে হতো?" → শুধু Data layer (Repository implementations + DbContext), Application/Domain/Api অপরিবর্তিত থাকতো কারণ তারা শুধু interface চেনে।

Q2. Controller এত "thin" কেন? সব logic Service-এ কেন রাখা হয়েছে?

- Short answer:** Separation of Concerns + Testability — Controller শুধু HTTP concern (routing, request/response shape) নিয়ে ভাবে, business logic Service-এ।
- Detailed:** Controller যদি business logic-ও রাখতো, তাহলে সেই logic টেস্ট করতে পুরো HTTP pipeline (ASP.NET Core test host) দরকার হতো। Service আলাদা হওয়ায় সরাসরি unit test করা যায় (mock repository দিয়ে), HTTP layer ছাড়াই। এছাড়া একই business logic একাধিক জায়গা থেকে (REST endpoint, background job, SignalR hub) reuse করা সহজ হয় যদি সেটা Controller-এ আটকে না থাকে।
- Project example:** `AuthController.Register` মাত্র ৪ লাইন — সব heavy lifting `_authService.RegisterAsync(...)` -এ।
- Follow-up:** "Controller-এ কি কোনো if/else business rule থাকা উচিত?" → না, Controller শুধু routing/orchestration; validation Filter-এ, business rule Service-এ।

Q3. তুমি কেন AutoMapper ব্যবহার করোনি, ম্যানুয়াল mapping কেন?

- Short answer:** প্রজেক্টে আসলে কোনো mapping library সেট-আপই করা হয়নি (Mappings/README stale) — mapping manual private method দিয়ে করা।
- Detailed:** Manual mapping-এর সুবিধা: explicit (কোন field কোথায় যাচ্ছে সরাসরি কোডে দেখা যায়, "magic" নেই), no reflection overhead runtime-এ, আর mapping-এর ভেতরে সহজে extra logic বসানো যায় (যেমন `product.ViewCount++` মত side-effect, বা conditional field)। অসুবিধা: বড় DTO-তে boilerplate বেশি, নতুন field যোগ করলে ম্যানুয়ালি mapping method-এও যোগ করতে মনে রাখতে হয় (কম্পাইলার সাহায্য করবে না automatic)।
- Project example:** `ProductService` -এর ভেতরে private `ToDto(Product product) / ToListItemDto(Product product)` মেথড।
- Follow-up:** "AutoMapper ব্যবহার করলে কী সুবিধা হতো?" → Convention-based auto mapping (একই নামের property automatic মিলে যায়), কম boilerplate — কিন্তু debugging কঠিন হতে পারে (কোন property কীভাবে ম্যাপ হচ্ছে runtime reflection-এ ঠিক হয়), performance সামান্য কম।

Q4. `IEnumerable`, `ICollection`, `IQueryable`, `List<T>` — পার্থক্য কী?

- Short answer:** `IEnumerable<T>` শুধু forward-iterate করা যায় (lazy, deferred); `IQueryable<T>` EF Core-এ SQL-এ translate হওয়া expression tree বহন করে; `ICollection<T>` add/remove/count সাপোর্ট করে; `List<T>` concrete, indexable, in-memory।
- Detailed:** Part 3.10-এ বিস্তারিত।
- Project example:** `ProductRepository.WithDetails()` রিটার্ন করে `IQueryable<Product>`; `GetFeaturedAsync` রিটার্ন করে `Task<List<Product>>` (কারণ শেষ পর্যন্ত পুরো result set দরকার, আর কোনো further filtering হবে না তাই materialize করে দেওয়া হয়েছে)।
- Follow-up:** "কেন Service method-এর return type `List<T>` কিন্তু Repository-র internal helper `IQueryable<T>`?" → Repository internally query build করে (deferred), কিন্তু শেষে caller-কে concrete data দিতে হয় (materialize) `ToListAsync`, যাতে caller accidentally আবার database এ hit না করে বা connection-এর বাইরে query iterate করার চেষ্টা না করে।

Q5. `[ApiController]` attribute কী কী automatic behavior দেয়?

- Short answer:** Automatic 400 on invalid model state, automatic binding-source inference, attribute-routing required, problem-details error format।
- Detailed:** Part 4.1-এ বিস্তারিত। এটা ছাড়া প্রতিটা action-এ ম্যানুয়ালি `if (!ModelState.IsValid) return BadRequest(ModelState);` লিখতে হতো — যদিও এই প্রজেক্টে যেভাবেই হোক `ValidationFilter` দিয়ে custom validation হচ্ছে, `[ApiController]` -এর automatic model-state validation malformed JSON/type-mismatch এর মতো basic case ধরে।
- Project example:** সব ১২৪টা controller-এ `[ApiController]` আছে।
- Follow-up:** "`[ApiController]` না থাকলে কী হতো?" → Model binding source ambiguous হতো (`[FromBody]` explicit লিখতে হতো সবখানে), automatic 400 হতো না, raw `ModelState` manual চেক করতে হতো।

Q6. Middleware pipeline-এ order কেন গুরুত্বপূর্ণ? একটা উদাহরণ দাও যেখানে order ভুল হলে bug হবে।

- Short answer:** Middleware sequential ভাবে চলে; পরের middleware আগেরটার উপর নির্ভর করতে পারে (যেমন Authorization নির্ভর করে Authentication-এর সেট করা `HttpContext.User` -এর উপর)।
- Detailed:** Part 2.1/4.4-এ বিস্তারিত।
- Project example:** `app.UseAuthentication()` অবশ্যই `app.UseAuthorization()` -এর আগে; `app.UseRateLimiter()` authentication-এর পরে (partition key-তে user id লাগে)।
- Follow-up:** "যদি `UseAuthorization()` কে `UseAuthentication()` -এর আগে বসানো হতো, কী হতো?" → প্রতিটা request 401/403 পেত, কারণ authorization চেকের সময় `HttpContext.User` তখনো populate হয়নি (anonymous হিসেবে দ্বিষ্ট হতো)।

Q7. Dependency Injection কী, আর Scoped / Transient / Singleton এর মধ্যে পার্থক্য কী?

- Short answer:** DI হলো একটা object-এর dependency বাইরে থেকে (constructor-এ) সরবরাহ করা, নিজে new না করে — loose coupling ও testability-র জন্য। Scoped = per-request instance, Transient = প্রতি injection-এ নতুন instance, Singleton = পুরো app lifetime-এ একটা instance।
- Detailed:** Part 2.4-এ বিস্তারিত — কেন সব Service/Repository Scoped (DbContext Scoped হওয়ায়)।
- Project example:** `services.AddScoped<IProductService, ProductService>()` (Application layer), `services.AddDbContext<ShilpoHubDbContext>(...)` (default Scoped)।
- Follow-up:** "যদি একটা Service ভুলবশত Singleton করা হতো, কী সমস্যা হতো?" → সেই Service-এর ভেতরে injected Scoped DbContext একটা "captivedependency" হয়ে যেত — প্রথম request-এর DbContext instance পরের সব request-এও ব্যবহার হতো, যেটা DbContext thread-safe না হওয়ায় data corruption/exception ঘটতো। .NET আসলে runtime-এ এই ভুল ধরে exception ছুঁড়ে দেয় ("Cannot consume scoped service from singleton")।

Q8. FluentValidation কীভাবে কাজ করে, ModelState.IsValid -এর চেয়ে আলাদা কীভাবে?

- Short answer:** FluentValidation একটা separate library, fluent C# API দিয়ে rule লেখা যায় (`RuleFor(x => x.Email).EmailAddress()`), Data Annotation attribute-এর বদলে।
- Detailed:** Part 2.2-এ বিস্তারিত — `AbstractValidator<T>`, `ValidationFilter` কীভাবে reflection দিয়ে resolve+invoke করে।
- Project example:** `BulkCreateProductsRequestValidator : AbstractValidator<BulkCreateProductsRequest>, RuleForEach(x => x.Products).SetValidator(new CreateProductRequestValidator())` (nested validator reuse)।
- Follow-up:** "যদি কোনো DTO-এর জন্য validator না থাকে, তখন কী হবে?" → `ValidationFilter` -এ `GetService` (nullable) ব্যবহার হয়েছে, তাই validator না পেলে সেই argument silently skip হয়ে যায়, কোনো error হয় না — validation optional, প্রতিটা DTO-এর validator লেখা বাধ্যতামূলক না।

Q9. Custom Exception (NotFoundException , ConflictException) কেন বানানো হয়েছে, generic Exception কেন যথেষ্ট না?

- Short answer:** যাতে `GlobalExceptionHandler` টাইপ দেখে সঠিক HTTP status code map করতে পারে।
- Detailed:** Generic `Exception` ছুঁড়লে সবকিছু 500 হয়ে যেত (কারণ handler-এর switch এ শুধু নির্দিষ্ট টাইপ চেনে)। Domain-specific exception টাইপ থাকায় Service layer-ই "কী ধরনের সমস্যা" সেটা express করতে পারে, HTTP status code নিয়ে না ভেবে (Separation of Concerns — business logic HTTP-agnostic থাকে)।
- Project example:** `throw new NotFoundException("Product not found.")` → 404; `throw new ConflictException("Email is already registered.")` → 409।
- Follow-up:** "কেন `UnauthorizedAccessException` একটা built-in .NET exception ব্যবহার হয়েছে, custom `UnauthorizedException` না বানিয়ে?" → .NET-এই একটা built-in আছে ঠিক এই purpose-এর জন্য (`System.UnauthorizedAccessException`), নতুন করে বানানোর দরকার নেই — reuse over reinvention।

Q10. Rate Limiting কীভাবে কাজ করে, কেন login endpoint-এ আলাদা কড়া limit?

- Short answer:** Fixed Window Rate Limiter, partition করা হয় user id (authenticated) অথবা IP address (anonymous) দিয়ে; login/register সবসময় IP-key কারণ user তখনো authenticated না।
- Detailed:** Part 2.1-এ বিস্তারিত — global (৩০০/৭৫ per min), "read" policy (২০০/৪০), "auth" policy (১০/min, IP-keyed) — brute-force/credential-stuffing/scraping প্রতিরোধ।
- Project example:** `[EnableRateLimiting("auth")] AuthController` -এ class-level।
- Follow-up:** "কেন IP-based rate limiting যথেষ্ট না always?" → NAT-এর পেছনে অনেক ব্যবহারকারী একই public IP শেয়ার করতে পারে (office/university network), legitimate user ভুলবশত rate-limited হতে পারে — trade-off, perfect solution না, কিন্তু basic protection।

Q11. Task আর Task<T> এর মধ্যে পার্থক্য? Task.Result কেন এড়ানো উচিত?

- Short answer:** `Task` = no return value async operation; `Task<T>` = future value। `.Result` / `.Wait()` synchronously block করে, deadlock/thread-starvation risk।
- Detailed:** Part 5.2, 5.4, 5.5-এ বিস্তারিত।
- Project example:** `Task LogoutAsync(...)` vs `Task<AuthResponse> LoginAsync(...)`; পুরো প্রজেক্টে কোথাও `.Result` / `.Wait()` ব্যবহার হয়নি — সবসময় `await`।
- Follow-up:** "যদি একটা synchronous method-এর ভেতর থেকে async method কল করতে হয়, কী করবে?" → পুরো chain-টাকে async বানানোই সঠিক পথ ("async all the way"); `.Result` / `GetAwaiter().GetResult()` last resort, avoid করা উচিত।

Q12. Guid কেন Primary Key হিসেবে ব্যবহার হয়েছে, auto-increment int না কেন?

- Short answer:** Distributed-friendly (client/multiple service নিজে থেকে ID বানাতে পারে, collision ছাড়া), আর ID guess/enumerate করা কঠিন (security)।
- Detailed:** `int` sequential ID হলে `/api/orders/101`, `/api/orders/102` — attacker সহজে অন্য user-দের অর্ডার নাম্বার guess/iterate করতে পারতো (IDOR-স্টাইল enumeration সহজ হতো, যদিও authorization check থাকলে সরাসরি ডেটা এক্সেস আটকাবে, কিন্তু existence/pattern leak হতে পারে)। `Guid` (১২৮ বিট, প্রায় unguessable) সেই ঝুঁকি কমায়।
- Project example:** প্রতিটা Entity-তে `Guid Id { get; set; }` (যেমন `User.Id`, `Product.Id`)।
- Follow-up:** "Guid Primary Key-এর downside কী?" → Index fragmentation (random Guid B-tree index-এ খারাপভাবে insert হয়, sequential int-এর মতো append-only না) — বড় স্কেলে performance-এর জন্য sequential/ordered GUID (যেমন UUIDv7) বিবেচনা করা হয়, কিন্তু এই প্রজেক্টে plain `Guid.NewGuid()` (random, v4) ব্যবহার হয়েছে।

Q13. appsettings.json -এ connection string ফাঁকা কেন, .env কীভাবে কাজ করে?

- Short answer:** Secret কখনো source control-এ commit করা হয় না; `.env` (local, git-ignored) অথবা প্রকৃত environment variable থেকে runtime-এ load হয়।
- Detailed:** Part 2.1-এ বিস্তারিত — `DotNetEnv.Env.Load()`, `___` (ডাবল আন্ডারস্কোর) দিয়ে nested config key (`ConnectionStrings___DefaultConnection` = `ConnectionStrings:DefaultConnection`)।
- Project example:** `appsettings.json` -এ `"DefaultConnection": ""` (ফাঁকা), production-এ platform-level env var সেট করা থাকবে।

- **Follow-up:** ".env ফাইলটা কি git এ commit করা উচিত?" → না, `.gitignore` -এ থাকা উচিত — committed হলে secret leak হয়ে যাবে।

Q14. `PagedResult<T>` জেনেরিক ক্লাস কীভাবে কাজ করে?

- **Short answer:** যেকোনো টাইপের জন্য reusable pagination wrapper — `Items`, `TotalCount`, `Page`, `PageSize`, computed `TotalPages`।
- **Detailed:** Part 3.7-এ বিস্তারিত।
- **Project example:** `PagedResult<ProductListItemDto>` রিটার্ন করে `ProductService.GetProductsAsync`।
- **Follow-up:** "যদি `PageSize` ০ হয়, `TotalPages` কত হবে?" → কোডে explicit চেক আছে (`PageSize ≤ 0 ? 0 : ...`), তাই ০ রিটার্ন করবে, division-by-zero exception হবে না।

Q15. তোমার প্রজেক্টে "AI" নাম দেওয়া features (যেমন `AIBusinessAssistant`, `ResearchAiAssistant`, `AILogistics`) কি সত্যিকারের Machine Learning?

- **Short answer:** না — verify করা হয়েছে যে এগুলো rule-based/heuristic logic (`Dummy*Provider` / `RuleBased*Provider` ক্লাস), কোনো external LLM API call বা ML model নেই।
- **Detailed:** Infrastructure-এর AI-নাম-দেওয়া ফোল্ডারগুলো (`AIBusiness`, `AIBusinessPartner`, `ResearchAI`, `GovForecasting`, `PolicySimulation`, `HeritageIntelligence`, `AILogistics`) খেঁটে দেখা গেছে — কোনো `HttpClient` কল OpenAI/Anthropic/Gemini-এর মতো external provider-এ নেই, বরং weighted-scoring formula, linear regression (OLS), nearest-neighbor + 2-opt heuristic ইত্যাদি — এগুলো classic algorithm/statistics, deep learning/LLM না।
- **Project example:** Business cluster গবেষণায় পাওয়া গেছে `DummyAIBusinessProvider`, `DummyBusinessPartnerAIProvider`, `DummyResearchAIProvider` ক্লাস নাম-ই স্বীকার করছে dummy/placeholder logic; Government cluster-এ `RuleBased*Provider` ক্লাসের doc-comment-এ literally লেখা "no external calls, no model weights"; Logistics cluster-এ AI route optimization আসলে nearest-neighbour + 2-opt (classic combinatorial optimization heuristic, ML না)।
- **Follow-up:** "তাহলে 'AI' নাম দেওয়া কি misleading?" — সৎ উত্তর দাও examiner কে: এগুলো "rule-based decision support" বা "algorithmic heuristics" — feature হিসেবে valid এবং কাজ করে, কিন্তু "AI" শব্দটা marketing/product-naming, প্রকৃত ML/LLM প্রযুক্তি না। ভবিষ্যতে চাইলে একটা real LLM API (`HttpClient` দিয়ে) integrate করে upgrade করা যায় — architecture তার জন্য ready (`IXxxProvider` interface abstraction থাকায় শুধু implementation বদলালেই হবে, বাকি কোড অক্ষত)।

Part 10 — Mock Viva কীভাবে চালাবে

এই PDF-টা static/read-only — কিন্তু original request-এ ছিল একটা **interactive mock viva** ("এক এক করে প্রশ্ন, উত্তর শুনে evaluate, correct, follow-up")। সেটা একটা **live conversation**-এ সম্ভব, PDF-এ না। কীভাবে করবে:

- এই PDF পুরোটা একবার পড়ে নাও (বিশেষ করে Part 1-9)।
- তারপর Claude-কে (বা তোমার examiner বন্ধুকে) বলো:

"আমাকে এই ShilpoHubBD প্রজেক্টের উপর mock viva নাও। প্রথমে architecture নিয়ে প্রশ্ন করো, তারপর ধাপে ধাপে গভীরে যাও। একটা একটা প্রশ্ন করো, আমার উত্তর শোনার পর evaluate করো, ভুল থাকলে ঠিক করে দাও, আর follow-up প্রশ্ন করো — ঠিক real viva examiner-এর মতো strict হও।"

- প্রতিটা রাউন্ডে honest ভাবে উত্তর দাও (না জানলে "জানি না" বলো — সেটাই ভালো, ভুল বলে ধরা পড়ার চেয়ে)। Claude তোমার উত্তর এই PDF-এর content-এর সাথে মিলিয়ে দেখবে এবং gap পূরণ করবে।
- প্রতিদিন একটা করে Part নিয়ে mock viva round করলে পুরো প্রস্তুতি সপ্তাহ-দুয়েকের মধ্যে ঠিকঠাক হয়ে যাবে।

Part 11 — Preparation Plan (কী আগে পড়বে, কীভাবে revise করবে)

ধাপ ১ (Day 1-2) — Big Picture আগে বোঝো

- Part 1 (Architecture Overview) পড়ো — 5 layer, dependency direction, request flow diagram মুখস্থের মতো বলতে পারা উচিত (কাগজে নিজে হাতে ঝঁকে দেখো)।
- Folder structure নিজে IDE খুলে explore করো — **Controllers/**, **Services/**, **Entities/**, **Repositories/** — একটা করে ফাইল খুলে মিলিয়ে দেখো এই PDF-এর সাথে।

ধাপ ২ (Day 2-4) — Core Building Blocks (সবচেয়ে গুরুত্বপূর্ণ অংশ)

- Part 2 পুরোটা — **Program.cs**, **ValidationFilter**, **GlobalExceptionHandler**, DI lifetimes, Options pattern, RBAC।
- এই অংশটা সবচেয়ে বেশি প্রশ্ন আসবে কারণ এটা পুরো প্রজেক্টের "spine" — একবার ভালো বুঝলে বাকি সব module সহজ লাগবে।
- নিজে নিজে **Program.cs** খুলে প্রতিটা লাইন উপরে-নিচে না দেখে ব্যাখ্যা করার চেষ্টা করো।

ধাপ ৩ (Day 4-6) — একটা end-to-end flow সম্পূর্ণ মুখস্থ করো

- AuthController.Login** → **AuthService.LoginAsync** → **JwtTokenService.GenerateAccessToken** — পুরো chain-টা লাইন বাই লাইন বলতে পারা উচিত, কোনো নোট না দেখে।
- একই ভাবে **ProductsController.GetById** → **ProductService.GetByIdAsync** → **ProductRepository.GetByIdAsync** — একটা simple CRUD flow।
- এই দুইটা flow ভালোভাবে জানলে, examiner অন্য যেকোনো controller দেখিয়ে জিজ্ঞেস করলেও তুমি একই যুক্তিতে ব্যাখ্যা করতে পারবে (pattern তো একই)।

ধাপ ৪ (Day 6-9) — Fundamentals (C#, Async, EF Core, Auth)

- Part 3 (C# fundamentals), Part 5 (Async/Await), Part 6 (EF Core), Part 7 (Auth) — প্রতিদিন একটা করে অংশ।
- প্রতিটা concept-এর পাশে যে code snippet আছে সেটা IDE-তে গিয়ে আসল ফাইলে দেখো, context বোঝো।
- সবচেয়ে বেশি **common trap**: IEnumerable vs IQueryable, async void কেন খারাপ, N+1 কী, AsSplitQuery কেন — এই ৪টা টপিক ১০০% নিশ্চিত করো।

ধাপ ৫ (Day 9-12) — নিজের Module + ২-৩টা অন্য Module

- Part 8-এর যে module(s) তোমার নিজের বানানো (backendsetup.md এ যেটা তোমার নামে), সেটা সবচেয়ে গভীরে পড়ো — প্রতিটা controller reference table থেকে অন্তত রুট আর purpose মুখস্থ করো, deep-dive অংশ পুরো বুঝে নাও।
- বাকি module গুলো থেকে অন্তত deep-dive অংশ (৪টা করে representative controller) পড়ে নাও, যাতে "এই অংশটা কে বানিয়েছে জানি না, কিন্তু architecture বুঝি" বলতে পারো — যেহেতু সব module একই pattern follow করে, examiner অন্য module থেকে প্রশ্ন করলেও তুমি general architecture দিয়ে উত্তর দিতে পারবে।

ধাপ ৬ (Day 12-14) — Question Bank + Mock Viva

- Part 9 (Master Q&A) আর প্রতিটা module-এর Q&A অংশ পড়ো, নিজে নিজে উত্তর দেওয়ার চেষ্টা করো (উত্তর না দেখে)।
- Part 10-এর নিয়ম অনুযায়ী কমপক্ষে ২-৩ বার mock viva করো।
- Viva-র আগের দিন: শুধু Part 2 (Core Building Blocks) আর Part 9 (Master Q&A) revise করো, নতুন কিছু পড়ার চেষ্টা করো না।

Revision Checklist (viva-র ঠিক আগে, দ্রুত দেখে নেওয়ার জন্য)

- [] 5 layer-এর নাম আর dependency direction বলতে পারি
- [] Middleware pipeline-এর সঠিক order বলতে পারি, আর কেন সেই order
- [] **Program.cs** -এর প্রতিটা বড় ব্লক (JWT setup, rate limiting, CORS, seeding) ব্যাখ্যা করতে পারি
- [] **ValidationFilter** আর **GlobalExceptionHandler** — কীভাবে কাজ করে, লাইন বাই লাইন
- [] JWT-তে কী কী claim থাকে, কেন; Access vs Refresh token পার্থক্য, rotation+reuse detection
- [] Controller → Service → Repository → DbContext flow একটা example দিয়ে সম্পূর্ণ বলতে পারি

- [] `IEnumerable` vs `IQueryable`, deferred execution
- [] `async` / `await` কীভাবে কাজ করে, blocking vs non-blocking, কেন `.Result` এড়ানো উচিত
- [] `Include` / `ThenInclude` / `AsSplitQuery` / `AsNoTracking` — প্রতিটির purpose
- [] N+1 problem কী, এই প্রজেক্টে কীভাবে এড়ানো হয়েছে
- [] DI lifetime (Scoped/Transient/Singleton) — কেন সব Scoped
- [] `[Authorize]` vs `[Authorize(Roles= ... )]` vs `[AllowAnonymous]`
- [] Custom exception → `GlobalExceptionHandler` → HTTP status code mapping
- [] নিজের module-এর প্রতিটি controller-এর route আর purpose
- [] "AI" ফিচারগুলো আসলে কী (rule-based, honest answer prepared)
- [] AutoMapper না থাকার গল্প (manual mapping কেন) মুখস্থ

শেষ পরামর্শ

- কখনো "সব মুখস্থ" করার চেষ্টা করো না — ১২৪টা controller-এর প্রতিটা লাইন মনে রাখা অসম্ভব এবং examiner সেটা আশাও করে না। বরং **pattern** বোঝো: একবার Controller→Service→Repository flow, JWT flow, EF Core query flow বুঝলে, যেকোনো নতুন controller দেখিয়ে দিলেও তুমি "এটা নিশ্চয়ই এই route এ আসবে, এই service call করবে, validation হবে এভাবে, exception হলে এভাবে handle হবে" — এই যুক্তিতে সব ব্যাখ্যা করতে পারবে।
- যেটা জানো না, honestly বলো — "এই নির্দিষ্ট controller-টা আমি ভালোভাবে পড়িনি, কিন্তু এটা নিশ্চয়ই [pattern] follow করছে কারণ..." — এটা examiner-কে দেখায় তুমি বুঝদার, মুখস্থবিদ্যা না।
- Live IDE খুলে viva দিতে পারলে, কোনো doubt হলে সরাসরি কোড দেখিয়ে বলো — এটা সবচেয়ে confident approach।